# Competitive Programming Notebook

ICPC team reference • August 13, 2026

**HOW TO FIND SOMETHING (in order of speed). 1.** You know the routine's name → the alphabetical *Index of every routine* at the very back gives the page. **2.** You know the topic → the running head on every page names the section (left) and the sub-topic (right); thumb until it matches. **3.** You know neither → section **12 Problem Solving** maps a problem's *shape* and its *constraints* onto the technique, and `docs/PLAYBOOK.md` does the same at more length.

**BEFORE YOU PASTE.** Line 1 of every file says what it does; a `// Needs:` line names the templates it depends on; a `// NOTE:` line warns when another file declares the same symbol. Vertices and array positions are **0-based** unless the header says otherwise. Geometry: `P<ll>` for every decision, `P<ld>` only when the answer is a real number.

## Contents



## 1 Miscellaneous

### 1.1 Template

```cpp
#include <bits/stdc++.h>
using namespace std;

//#define int long long
#define ll long long
#define ld long double
#define fi first
#define se second
#define pb push_back
#define eb emplace_back
#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define sz(x) (int)(x).size()
#define ones(n) __builtin_popcountll(n)
#define msb(n) (63 - __builtin_clzll(n))
#define lsb(n) __builtin_ctzll(n)
typedef vector<int> vi;
typedef pair<int, int> pii;

const int N = 2e5 + 5, M = 1e3 + 5, LOG = 31;
const int inf = 0x3f3f3f3f;
const ll llinf = 0x3f3f3f3f3f3f3f3f;
const int MOD = 1e9 + 7;
const ld eps = 1e-9, PI = acosl(-1.0L);        // ld, not double:
      geometry needs the digits

#ifdef LOCAL                                    // compile with -
     DLOCAL
template <class T> void _p(const T& x);
void _p(const string& s) { cerr << '"' << s << '"'; }
void _p(char c) { cerr << '\'' << c << '\''; }
template <class A, class B> void _p(const pair<A, B>& p) { cerr
     << '('; _p(p.fi); cerr << ", "; _p(p.se); cerr << ')'; }
template <class T> void _p(const T& x) {
    if constexpr (is_arithmetic_v<T>) cerr << x;
    else { cerr << '{'; int f = 1; for (auto& e : x) { if (!f)
     cerr << ", "; f = 0; _p(e); } cerr << '}'; }
}
void _pr() { cerr << "\n"; }
template <class H, class... T> void _pr(const H& h, const T&... t)
      { _p(h); if (sizeof...(t)) cerr << " | "; _pr(t...); }
#define dbg(...) (cerr << "[" << #__VA_ARGS__ << "] = ", _pr(
     __VA_ARGS__))
#else
#define dbg(...)
#endif

void testCase() {
}

void preCompute() {
}

int32_t main() {
    ios_base::sync_with_stdio(false), cin.tie(nullptr), cout.tie(
     nullptr);
    cout << fixed << setprecision(10);
    preCompute();

    int tc = 1;
    cin >> tc;                                  // DELETE THIS
     LINE for single-test problems
    for (int TC = 1; TC <= tc; TC++) {
        // cout << "Case #" << TC << ": ";
        testCase();
    }
}

/*



*/
```

## 1.2 fastIO

HAND-ROLLED FAST IO (fread/fwrite buffered). Use only when cin with sync off is measurably too

```
// slow: >1e6 numbers to read, or heavy output. Otherwise prefer
     the template's ios::sync_with_stdio.
const int MOD = 1e9 + 7;
const int BUF_SZ = 1 << 15;

inline namespace Input {
    char buf[BUF_SZ];
    int pos;
    int len;
    char next_char() {
        if (pos == len) {
            pos = 0;
            len = (int)fread(buf, 1, BUF_SZ, stdin);
            if (!len) { return EOF; }
        }
        return buf[pos++];
    }

    int read_int() {
        int x;
        char ch;
        int sgn = 1;
        while (!isdigit(ch = next_char())) {
            if (ch == '-') { sgn *= -1; }
        }
        x = ch - '0';
        while (isdigit(ch = next_char())) { x = x * 10 + (ch - '0
    '); }
        return x * sgn;
    }
}  // namespace Input
inline namespace Output {
    char buf[BUF_SZ];
    int pos;

    void flush_out() {
        fwrite(buf, 1, pos, stdout);
        pos = 0;
    }

    void write_char(char c) {
        if (pos == BUF_SZ) { flush_out(); }
        buf[pos++] = c;
    }

    void write_int(int x) {
        static char num_buf[100];
        if (x < 0) {
            write_char('-');
            x *= -1;
        }
        int len = 0;
        for (; x >= 10; x /= 10) { num_buf[len++] = (char)('0' +
    (x % 10)); }
        write_char((char)('0' + x));
        while (len) { write_char(num_buf[--len]); }
        write_char('\n');
    }

    // auto-flush output when program exits
    void init_output() { assert(atexit(flush_out) == 0); }
}  // namespace Output
```

## 1.3 grid

GRID DIRECTION VECTORS. First 4 entries = 4-neighbourhood (L,R,U,D), all 8 = king moves.

```
// Knight moves: dx {1,1,-1,-1,2,2,-2,-2}, dy
     {2,-2,2,-2,1,-1,1,-1}. Guard with 0<=x<n && 0<=y<m.
int dx[] = {0, 0, -1, 1, -1, -1, 1, 1};
int dy[] = {-1, 1, 0, 0, -1, 1, -1, 1};
char di[] = {'L', 'R', 'U', 'D'};
```

## 1.4 random

RANDOMNESS. Seed from the clock so tests cannot be prepared against you.

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
    count());
ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(
    rng); }   // inclusive

vector<int> randPerm(int n) {                          // random
     permutation of 0..n-1
    vector<int> v(n);
    iota(all(v), 0);
    shuffle(all(v), rng);
    return v;
}
// Shuffling the INPUT is a real algorithm, not just a utility:
//   - minimum enclosing circle is O(n) only after a shuffle
//   - quickselect / random pivots
//   - defeats anti-hash and anti-quicksort tests
//   - random rotation of points removes degenerate ties in a
     geometry sweep
```

## 1.5 coordinateCompression

Coordinate Compression - O(N log N)

```
template<typename T>
struct Compress {
    vector<T> v;

    Compress(vector<T>& a) : v(a) {
        sort(all(v));
        v.erase(unique(all(v)), v.end());
        for (auto &x : a) {
            x = lower_bound(all(v), x) - v.begin();
        }
    }

    T operator[](int i) const {
        return v[i];
    }
};
```

## 1.6 grayCode

GRAY CODE - orders 0..2^n-1 so consecutive values differ in exactly ONE bit. Use it to walk all

```
// subsets while changing one element at a time (incremental DP,
     XOR-basis sweeps, Tower of Hanoi).
// Gray Code Generator - O(2^N)
// g(n) = n ^ (n >> 1)
vector<int> grayCode(int n) {
    vector<int> res(1 << n);
    for (int i = 0; i < (1 << n); i++) {
        res[i] = i ^ (i >> 1);
    }
    return res;
}

int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

## 1.7 customHashHashTable

UNORDERED_MAP THAT CANNOT BE HACKED. The default std::hash for integers is the identity, so an

```
// adversary can force every key into one bucket and turn O(1)
     into O(n). splitmix64 + a clock seed
// fixes that. gp_hash_table is ~3x faster than unordered_map;
     reserve() and max_load_factor help more.
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

// Custom Hash to prevent anti-hash tests in unordered_map /
     gp_hash_table
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock
     ::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

// Usage: gp_hash_table<ll, int, custom_hash> table;
//        unordered_map<ll, int, custom_hash> safe_map;
```

## 1.8 pragmas

COMPILER PRAGMAS. O3 + unrolling, and AVX2 for auto-vectorised loops (bitset, simple array math).

```
// Put them at the very top, before any include. Risk: `target`
     crashes judges without those CPU
// features - never use avx2 on an unknown judge, and never on
     Codeforces problems with old machines.
// GCC Optimizations
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

## 1.9 int128

__int128 Stream Operators Helper

```
typedef __int128_t int128;
typedef __uint128_t uint128;

ostream &operator<<(ostream &os, int128 n) {
    if (n == 0) return os << "0";
    if (n < 0) { os << "-"; n = -n; }
    string s;
    while (n > 0) { s += (char)('0' + (n % 10)); n /= 10; }
    reverse(all(s));
    return os << s;
}

istream &operator>>(istream &is, int128 &n) {
    string s; is >> s;
    n = 0; int i = 0, sign = 1;
    if (!s.empty() && s[0] == '-') { sign = -1; i = 1; }
    for (; i < (int)s.size(); i++) n = n * 10 + (s[i] - '0');
    n *= sign;
    return is;
}
```

## 1.10 SearchIdioms

The four searches. Getting the boundary right here saves more contest time than any template.

```
// BINARY SEARCH ON A PREDICATE. pred must be monotone: F F F T T
     T.
// firstTrue returns the first x in [lo, hi] with pred(x), or hi+1
     if none.
template <class F> ll firstTrue(ll lo, ll hi, F pred) {
    hi++;
    while (lo < hi) { ll m = lo + (hi - lo) / 2; pred(m) ? hi = m
     : lo = m + 1; }
    return lo;
}
// lastTrue returns the last x in [lo, hi] with pred(x), or lo-1
     if none.  T T T F F F
template <class F> ll lastTrue(ll lo, ll hi, F pred) {
    lo--;
    while (lo < hi) { ll m = lo + (hi - lo + 1) / 2; pred(m) ? lo
     = m : hi = m - 1; }
    return lo;
}
// Reals: fixed iteration count beats an epsilon loop (no infinite
     loops, predictable time).
// 100 iterations halve the range 2^100 times - always enough.
// Returns the boundary: pred is false on [lo, x) and true on [x,
     hi]. (`lo` and `hi` converge.)
template <class F> ld binReal(ld lo, ld hi, F pred, int it = 100)
     {
    while (it--) { ld m = (lo + hi) / 2; pred(m) ? hi = m : lo =
     m; }
    return hi;                                                  //
     first TRUE, matching firstTrue
}

// TERNARY SEARCH on a UNIMODAL function (strictly decreasing then
     increasing => minimum).
// Integer version: stop when the window is tiny, then scan. Never
     use `l < r` alone.
template <class F> ll ternaryInt(ll lo, ll hi, F f) {          //
     returns argmin
    while (hi - lo > 2) {
        ll m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        f(m1) <= f(m2) ? hi = m2 : lo = m1;                     //
     <= keeps the LEFTmost min
    }
    ll best = lo;
    for (ll x = lo; x <= hi; x++) if (f(x) < f(best)) best = x;
    return best;
}
template <class F> ld ternaryReal(ld lo, ld hi, F f, int it =
     200) {
    while (it--) { ld m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo
     ) / 3; f(m1) < f(m2) ? hi = m2 : lo = m1; }
    return (lo + hi) / 2;
}
// If f is only WEAKLY unimodal (has flat regions), ternary search
     is WRONG: one comparison
// f(m1) == f(m2) inside a plateau discards the side holding the
     optimum. For any DISCRETELY
// CONVEX f (non-decreasing difference sequence, plateaus allowed)
     use the derivative instead -
// this is strictly safer AND uses half the evaluations:
template <class F> ll convexArgmin(ll lo, ll hi, F f) {
    return firstTrue(lo, hi - 1, [&](ll x) { return f(x + 1) >= f
     (x); });
}

// NEWTON'S METHOD for roots of a smooth f (quadratic convergence;
     needs a decent start).
template <class F, class DF> ld newton(ld x, F f, DF df, int it =
     60) {
    while (it--) { ld d = f(x) / df(x); x -= d; if (fabsl(d) < 1e
     -15L * max((ld)1, fabsl(x))) break; }
    return x;
}
// Integer sqrt / cbrt with correction - NEVER trust sqrt() alone
     near 1e18.
ll isqrt(ll n) { ll r = (ll)sqrtl((ld)n); while (r > 0 && r * r >
     n) r--; while ((r + 1) * (r + 1) <= n) r++; return r; }
ll icbrt(ll n) { ll r = (ll)cbrtl((ld)n); while (r > 0 && r * r *
     r > n) r--; while ((r + 1) * (r + 1) * (r + 1) <= n) r++;
    return r; }

// BINARY SEARCH ON THE ANSWER - the pattern, not the code:
//   "minimise the maximum X" / "maximise the minimum X" / "can we
     finish within T"
//   -> guess the answer, write a GREEDY feasibility check, binary
```

```
    search on it.
// PARALLEL BINARY SEARCH - when you need this for MANY queries at
    once: bucket the queries by
// their current midpoint, sweep the shared structure once per
    level, O(log) sweeps total.
```

## 1.11 BitTricks

BIT MANIPULATION CHEAT SHEET ==========

```
/* ========== BIT MANIPULATION CHEAT SHEET
    ==========
 BUILTINS (add ll suffix for 64-bit: __builtin_popcountll, ...)
   __builtin_popcount(x)  #set bits        __builtin_ctz(x)  #
     trailing zeros (x != 0)
   __builtin_clz(x)       #leading zeros   __builtin_parity(x)
     popcount & 1
   __lg(x) = 63 - clzll(x) = floor(log2 x)     (GCC only; write it
     yourself elsewhere)
   lowest set bit: x & -x        clear it: x &= x - 1        set/
     flip: x |= b, x ^= b
   is power of two: x && !(x & (x - 1))
   round UP to a power of two (x >= 1): x == 1 ? 1ULL : 2ULL <<
     (63 - __builtin_clzll(x - 1))
     - clzll(0) is UNDEFINED, so x = 1 must be special-cased, and
     the shift must be 64-bit

 SUBMASK ENUMERATION - iterate every subset of m:
   for (int s = m; ; s = (s - 1) & m) { ...; if (!s) break; }
   Over ALL masks this is O(3^n) total (each element is in/out/
     neither).

 SUPERMASK enumeration (every s with m subset of s, within n bits)
     :
   for (int s = m; s < (1 << n); s = (s + 1) | m) { ... }

 K-SUBSETS in increasing order (Gosper's hack):
   for (int s = (1 << k) - 1; s < (1 << n); ) { ...;
       int c = s & -s, r = s + c; s = (((r ^ s) >> 2) / c) | r; }

 GRAY CODE  g(i) = i ^ (i >> 1)     inverse: x ^= x >> 1
     repeatedly (see 06 - grayCode.cpp)
   consecutive gray codes differ in exactly one bit - use for "
     change one thing at a time".

 XOR FACTS
   x ^ x = 0, x ^ 0 = x, xor is its own inverse -> prefix xor
     answers range xor
   xor of 0..n = [n, 1, n+1, 0][n % 4]
   a + b = (a ^ b) + 2*(a & b)     a | b = (a ^ b) + (a & b)     a
      + b = (a | b) + (a & b)
   "OR of a range equals its SUM"  <=>  the values are pairwise
     BIT-DISJOINT
     (so at most 30 non-zero values fit in any such window -> two
     pointers)

 COUNTING PER BIT - the standard decomposition: handle each of the
      30/60 bits independently.
   sum over pairs of (a_i ^ a_j) = sum over bits b of (#with bit)
     * (#without bit) * 2^b

 BITSET
   std::bitset<N> b; b._Find_first(), b._Find_next(i)  (GCC) -
     iterate set bits fast
   b <<= k, b |= other  are O(N/64) - this is what makes bitset DP
      and reachability work.
================================================================
      */

// Iterate all submasks of m (including m and 0).
template <class F> void forEachSubmask(int m, F f) {
    for (int s = m;; s = (s - 1) & m) { f(s); if (!s) break; }
}
// All n-bit masks with exactly k bits set, in increasing order (
     Gosper's hack).
vector<int> kSubsets(int n, int k) {
    vector<int> r;
    if (k == 0) return {0};
    for (int s = (1 << k) - 1; s < (1 << n);) {
        r.push_back(s);
        int c = s & -s, x = s + c;
        s = (((x ^ s) >> 2) / c) | x;
    }
    return r;
}
// Sum over all pairs (i<j) of (a_i XOR a_j), by handling each bit
      independently.
ll pairXorSum(const vector<ll>& a, int B = 62) {
    ll n = a.size(), r = 0;
    for (int b = 0; b < B; b++) {
        ll c = 0;
        for (ll x : a) c += (x >> b) & 1;
        r += c * (n - c) % MOD * ((1LL << b) % MOD) % MOD;
    }
    return r % MOD;
}
```

## 1.12 StressTest

STRESS TESTING ==========

```
/* ================ STRESS TESTING =================
 The single highest-value 20 lines in this notebook. Use it the
     moment a solution WAs on a
 test you cannot see. Write gen.cpp + brute.cpp, then loop until
     they disagree.

 ---- run.sh
     -------------------------------------------------------------
 g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp && g++ -O2 -
     o gen gen.cpp
 for i in $(seq 1 100000); do
     ./gen $i > in.txt
     ./sol   < in.txt > out1.txt
     ./brute < in.txt > out2.txt
     if ! diff -qb out1.txt out2.txt > /dev/null; then echo "WA on
      test $i"; cat in.txt; break; fi
 done

     -------------------------------------------------------------

 Windows cmd: replace the loop with `for /L %%i in (1,1,100000) do
      ( ... fc out1.txt out2.txt )`

 RULES THAT MAKE IT ACTUALLY WORK
  1. Generate SMALL cases (n <= 8, values <= 5). Bugs hide in tiny
      corner cases, not big ones.
  2. Seed the generator from argv[1] so every iteration differs
     and is reproducible.
  3. If the answer is not unique, do not diff - write a CHECKER
     that validates sol's output.
  4. If there is no brute force, stress against yourself: compare
     two different solutions, or
     compare an O(n^2) version of your own idea against the O(n
     log n) one.
  5. For "find any valid X" problems, verify the constraints
     instead of comparing.
 ================================================ */

// ---- gen.cpp ----
// #include <bits/stdc++.h>
// using namespace std;
// int main(int argc, char** argv) {
//     mt19937 rng(atoi(argv[1]));
//     auto rnd = [&](int l, int r) { return
     uniform_int_distribution<int>(l, r)(rng); };
//     int n = rnd(1, 8);
//     printf("%d\n", n);
//     for (int i = 0; i < n; i++) printf("%d ", rnd(1, 5));
//     puts("");
// }

// ---- in-process version: no files, no scripts. Put brute + fast
      in one binary. ----
// Fastest way to start; switch to the shell loop only when you
     need the failing input on disk.
template <class Gen, class Fast, class Brute>
void stress(Gen gen, Fast fast, Brute brute, int iters = 100000)
     {
    for (int it = 1; it <= iters; it++) {
        auto in = gen(it);
        auto a = fast(in), b = brute(in);
        if (!(a == b)) {
            printf("MISMATCH on iteration %d: fast=", it);
            cout << a << " brute=" << b << "\n";
            return;                                    // print `in
     ` here too
        }
    }
    puts("all tests passed");
}
```

## 1.13 Fraction

Exact rational. Always normalised (gcd reduced, denominator > 0).

```
// Comparison uses __int128 so it is exact for |num|,|den| up to
     ~9e18.
// Use whenever comparing ratios: slopes, "maximum average",
     scheduling exchange arguments.
struct Frac {
    ll n = 0, d = 1;
    Frac(ll n = 0, ll d = 1) : n(n), d(d) { norm(); }
    void norm() {
        if (d < 0) n = -n, d = -d;
        ll g = gcd(n < 0 ? -n : n, d);
        if (g) n /= g, d /= g;
        if (!d) d = 1;                                 // 1/0 ->
     treat as 0/1; guard your input
    }
    Frac operator+(Frac o) const { return Frac(n * o.d + o.n * d,
      d * o.d); }
    Frac operator-(Frac o) const { return Frac(n * o.d - o.n * d,
      d * o.d); }
    Frac operator*(Frac o) const { return Frac(n * o.n, d * o.d);
      }
    Frac operator/(Frac o) const { return Frac(n * o.d, d * o.n);
      }
    bool operator<(Frac o) const { return (__int128)n * o.d < (
     __int128)o.n * d; }
    bool operator==(Frac o) const { return n == o.n && d == o.d;
     }
    bool operator<=(Frac o) const { return !(o < *this); }
    ld val() const { return (ld)n / d; }
    friend ostream& operator<<(ostream& os, Frac f) { return os
     << f.n << '/' << f.d; }
};
// OVERFLOW: + and * multiply denominators, so chains of
     operations blow up fast. If you only
// need COMPARISONS, keep the raw pair and cross-multiply in
     __int128 - do not normalise.
// For unbounded exactness use Python/BigInt, or the Stern-Brocot
     search (ContinuedFractions).
```

## 1.14 Inversions

INVERSIONS: #pairs i < j with a[i] > a[j]. O(n log n) by merge sort (also sorts a).

```
ll inversions(vector<int>& a) {
    int n = a.size();
    if (n < 2) return 0;
    vector<int> buf(n);
    function<ll(int, int)> rec = [&](int l, int r) -> ll {
        // [l, r)
        if (r - l < 2) return 0;
        int m = (l + r) / 2;
        ll res = rec(l, m) + rec(m, r);
        int i = l, j = m, k = l;
        while (i < m && j < r) {
            if (a[i] <= a[j]) buf[k++] = a[i++];
            else buf[k++] = a[j++], res += m - i;
        // a[i..m) all beat a[j]
        }
        while (i < m) buf[k++] = a[i++];
        while (j < r) buf[k++] = a[j++];
        copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
        return res;
    };
    return rec(0, n);
}
// WHY IT KEEPS APPEARING
//  - minimum number of ADJACENT SWAPS to sort = inversion count
//  - minimum adjacent swaps to turn a into b: relabel b's
     positions, then count inversions
//  - "how far is this permutation from sorted" / bubble-sort
     distance
//  - inversions modulo 2 = permutation PARITY (= sign, used in
     determinants and 15-puzzle)
// BIT VERSION (needed when you must also support updates, or
     count online):
//   compress values, sweep left to right, ans += (#inserted so
     far) - query(a[i]), then add a[i].
```

## 1.15 ExpressionParser

EXPRESSION PARSER (shunting-yard). Handles + - * / % ^, unary minus, parentheses.

```
// Left-associative except ^. Extend by editing prec() / applyOp().
struct Parser {
    static int prec(char op) {
        switch (op) {
            case '+': case '-': return 1;
            case '*': case '/': case '%': return 2;
            case '^': return 3;
            case '~': return 4;                          // unary minus (internal symbol)
            default: return -1;
        }
    }
    static bool rightAssoc(char op) { return op == '^' || op == '~'; }

    static ll apply(char op, ll a, ll b = 0) {
        switch (op) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/': return a / b;
            case '%': return a % b;
            case '~': return -a;
            case '^': { ll r = 1; while (b > 0) { if (b & 1) r *= a; a *= a, b >>= 1; } return r; }
        }
        return 0;
    }
    static ll eval(const string& s) {
        vector<ll> val;
        vector<char> ops;
        bool wantOperand = true;                         // distinguishes unary from binary minus
        auto pop = [&]() {
            char op = ops.back(); ops.pop_back();
            if (op == '~') { ll a = val.back(); val.pop_back(); val.push_back(apply(op, a)); }
            else { ll b = val.back(); val.pop_back(); ll a = val.back(); val.pop_back();
                   val.push_back(apply(op, a, b)); }
        };
        for (size_t i = 0; i < s.size(); i++) {
            char c = s[i];
            if (isspace((unsigned char)c)) continue;
            if (isdigit((unsigned char)c)) {
                ll x = 0;
                while (i < s.size() && isdigit((unsigned char)s[i])) x = x * 10 + (s[i++] - '0');
                i--;
                val.push_back(x), wantOperand = false;
            } else if (c == '(') ops.push_back(c), wantOperand = true;
            else if (c == ')') {
                while (!ops.empty() && ops.back() != '(') pop();
                if (!ops.empty()) ops.pop_back();
                wantOperand = false;
            } else {
                if (c == '-' && wantOperand) c = '~';  // unary
                while (!ops.empty() && ops.back() != '(' &&
                       (prec(ops.back()) > prec(c) || (prec(ops.back()) == prec(c) && !rightAssoc(c))))
                    pop();
                ops.push_back(c), wantOperand = (c != '~') || true;
            }
        }
        while (!ops.empty()) pop();
        return val.empty() ? 0 : val.back();
    }
};
// To build an AST instead of evaluating, push node indices instead of values.
// For boolean/comparison operators just add them to prec() with lower precedence.
```

## 1.16 CycleFinding

CYCLE FINDING in a FUNCTIONAL GRAPH x -> f(x). Every component is a "rho": a tail feeding

```
// into a cycle. Answers: where does x end up after k steps, cycle length, distance to cycle.

// Floyd (tortoise & hare), O(1) memory. Returns {start index of the cycle, cycle length}.
template <class F> pair<ll, ll> floydCycle(ll x0, F f) {
    ll t = f(x0), h = f(f(x0));
    while (t != h) t = f(t), h = f(f(h));
    ll mu = 0;                                           // steps from x0 to the cycle entry
    t = x0;
    while (t != h) t = f(t), h = f(h), mu++;
    ll lam = 1;                                          // cycle length
    h = f(t);
    while (t != h) h = f(h), lam++;
    return {mu, lam};
}
// f applied k times, using the rho structure (k can be 1e18).
template <class F> ll iterate(ll x0, ll k, F f) {
    auto [mu, lam] = floydCycle(x0, f);
    if (k < mu) { while (k--) x0 = f(x0); return x0; }
    for (ll i = 0; i < mu; i++) x0 = f(x0);
    ll rem = (k - mu) % lam;
    while (rem--) x0 = f(x0);
    return x0;
}
// WHOLE-GRAPH version when f is given as an array over 0..n-1: colour DFS, O(n).
// state: 0 unvisited, 1 on the current path, 2 done.  Fills cycId/cycPos/distToCycle.
struct FuncGraph {
    int n;
    vector<int> f, state, cyc, pos, dist, cycLen;        // cyc[v] = id of the cycle v reaches
    FuncGraph(vector<int> f) : n(f.size()), f(f), state(n, 0),
     cyc(n, -1), pos(n, -1), dist(n, 0) {
        for (int i = 0; i < n; i++) if (!state[i]) {
            vector<int> path;
            int v = i;
            while (!state[v]) state[v] = 1, path.push_back(v), v = f[v];
            if (state[v] == 1) {                         // found a NEW cycle, starting at v
                int id = cycLen.size(), len = 0;
                for (int u = v;; u = f[u]) { cyc[u] = id, pos[u] = len++; if (f[u] == v) break; }
                cycLen.push_back(len);
            }
            for (int j = path.size() - 1; j >= 0; j--) {
                int u = path[j];
                state[u] = 2;
                if (cyc[u] == -1) cyc[u] = cyc[f[u]], dist[u] = dist[f[u]] + 1;
            }
        }
    }
    int kth(int v, ll k) {                               // f^k(v)
        while (k > 0 && dist[v] > 0) v = f[v], k--;
        if (!k) return v;
        int L = cycLen[cyc[v]], p = (pos[v] + k % L) % L, u = v;
        for (int i = 0; i < (p - pos[v] + L) % L; i++) u = f[u];
        return u;
    }
};
// USES: "apply the permutation k times", Pollard's rho, "where does the ball stop",
// detecting a repeat in a modular sequence, linked-list loop detection.
```

## 1.17 ParallelBinarySearch

PARALLEL BINARY SEARCH - binary search the answer for MANY queries at once, sweeping the

```
// shared structure once per level instead of once per query.
// PRECONDITION: test(i) must be MONOTONE in time - once it is
     true it stays true. If updates
// can also undo things, parallel binary search does not apply;
     use offline divide & conquer over
// the timeline with a rollback DSU (02 - Data Structures/14)
     instead.
// Total: O(log(maxT) * (cost of one full sweep)).
// Shape: "after which update does query q first become
     satisfiable?"
//   lo[q], hi[q] track each query's remaining range; every round,
      bucket the queries by their
//   midpoint, replay the updates in order, and test each query
     when the sweep reaches its mid.
template <class Apply, class Reset, class Test>
vector<int> parallelBinarySearch(int q, int T, Apply applyUpdate,
      Reset reset, Test test) {
    // applyUpdate(t): apply update t to the shared structure
    // reset(): clear the structure
    // test(i): is query i satisfied by the current structure?
    vector<int> lo(q, 0), hi(q, T);                    // answer
     in [lo, hi]; hi == T means "never"
    while (true) {
        vector<vector<int>> at(T + 1);
        bool active = false;
        for (int i = 0; i < q; i++) if (lo[i] < hi[i]) at[(lo[i]
     + hi[i]) / 2].push_back(i), active = true;
        if (!active) break;
        reset();
        for (int t = 0; t <= T; t++) {
            if (t) applyUpdate(t - 1);                 // now
     updates 0..t-1 are applied
            for (int i : at[t]) { if (test(i)) hi[i] = t; else lo
     [i] = t + 1; }
        }
    }
    return lo;                                         // lo[i]
     == T  =>  never satisfied
}
// CONCRETE USES
//  - "for each query, the first moment a path/edge set connects u
      and v"  (structure = DSU)
//  - k-th smallest in a range, offline (structure = BIT over
     values, updates = insert a value)
//  - "smallest capacity so that a flow/matching of size k exists"
//  - anything where a single query's binary search would need its
      own O(T) replay.
// If the structure supports ROLLBACK, prefer divide-and-conquer
     over the timeline instead
// (see OfflineDynamicConnectivity) - it is one pass instead of
     log passes.
```

## 1.18 MeetInTheMiddle

MEET IN THE MIDDLE - n <= 40. Split into halves, enumerate 2^(n/2) subsets each side,

```
// then recombine by sorting + binary search / two pointers. O(2^(
     n/2) * n).

// All subset sums of a half, as a sorted vector.
vector<ll> subsetSums(const vector<ll>& a) {
    int n = a.size();
    vector<ll> s(1 << n, 0);
    for (int m = 1; m < (1 << n); m++) {
        int b = __builtin_ctz(m);
        s[m] = s[m ^ (1 << b)] + a[b];
    }
    return s;
}
// #subsets with sum <= S. Works with NEGATIVE values too (no pre-
     filter on the left half).
ll countAtMost(vector<ll> a, ll S) {
    int n = a.size(), h = n / 2;
    vector<ll> L(a.begin(), a.begin() + h), R(a.begin() + h, a.
     end());
    auto ls = subsetSums(L), rs = subsetSums(R);
    sort(all(rs));
    ll c = 0;
    for (ll x : ls) c += upper_bound(all(rs), S - x) - rs.begin()
     ;
    return c;
}
// Largest achievable sum <= S (the classic "closest subset sum
     from below").
ll bestAtMost(vector<ll> a, ll S) {
    int n = a.size(), h = n / 2;
    vector<ll> L(a.begin(), a.begin() + h), R(a.begin() + h, a.
     end());
    auto ls = subsetSums(L), rs = subsetSums(R);
    sort(all(rs));
    ll best = LLONG_MIN;
    for (ll x : ls) {                                  // no `x
     <= S` filter: x may be > S
        auto it = upper_bound(all(rs), S - x);         // and
     still pair with a negative right sum
        if (it != rs.begin()) best = max(best, x + *prev(it));
    }
    return best;
}
// OTHER SHAPES
//  - EXACT target: put one half in a hash map, look up (target -
      x) from the other.
//  - 4-SUM: split the four arrays 2+2, hash all pairwise sums.
//  - "choose exactly k items": bucket each half's subsets BY
     POPCOUNT, then pair bucket i with
//    bucket k-i (sort each bucket once).
//  - Optimising two objectives: sort one half by objective A and
     take a prefix maximum of
//    objective B, so each query is one binary search.
// If n <= 20 a plain 2^n loop is simpler; meet in the middle
     earns its keep from n = 30 upward.
```

## 1.19 Scheduling

SCHEDULING / EXCHANGE-ARGUMENT THEOREM CARD

```
/* ============== SCHEDULING / EXCHANGE-ARGUMENT THEOREM CARD
     ==============
 Each of these turns a hard-looking problem into a sort. Prove any
      new one by the EXCHANGE
 ARGUMENT: assume an optimal schedule, swap two ADJACENT jobs, and
      compare the two costs.

 SMITH'S RULE  (1 || sum w_j C_j : minimise the weighted sum of
     completion times)
   Sort by p_j / w_j ASCENDING. Adjacent swap of h,k improves iff
     w_k*p_h - w_h*p_k > 0,
   so ALWAYS compare by cross-multiplication (w_k*p_h vs w_h*p_k),
      never by float division.

 MOORE-HODGSON  (1 || sum U_j : minimise the NUMBER of late jobs),
      O(n log n)
   Sort by deadline ascending; keep a max-heap of the processing
     times taken so far.
   For each job: take it, push p_j, add p_j to the running time;
     if the running time exceeds
   this job's deadline, pop the largest p and subtract it. The
     heap size is the answer
   (the rejected jobs can go last in any order).

 EARLIEST DEADLINE FIRST  (preemptive, one machine)
   If ANY schedule meets all deadlines, EDF does. Feasibility
     check for non-preemptive
   unit-machine: sort by deadline and verify prefix sums of p_j
     never exceed d_j.

 JOHNSON'S RULE  (F2 || Cmax : two-machine flow shop, minimise
     makespan)
   Set A = jobs with p1 <  p2, sorted by p1 ASCENDING
   Set B = jobs with p1 >= p2, sorted by p2 DESCENDING
   Optimal order = A then B.

 LAWLER  (1 | prec | f_max : minimise the maximum penalty, with
     precedence), O(n^2)
   Build the schedule BACK TO FRONT. With t = total remaining time
     , among the jobs that have
   no unscheduled successor pick the one minimising f_j(t), and
     place it last.

 CLASSIC GREEDY COMPANIONS
   Interval scheduling (max #non-overlapping): sort by RIGHT
     endpoint, take greedily.
   Interval point cover (stab all intervals): sort by right
     endpoint, place a point at it.
   Minimum #machines for overlapping intervals: max overlap =
     sweep +1/-1 over endpoints.
   Deadline scheduling with profits (each job unit length): sort
     by profit DESC, place each
   job as late as possible before its deadline (DSU "next free
     slot" makes it near-linear).
   Fractional knapsack: sort by value/weight DESC.
   Minimise sum of |x - a_i|: x = the MEDIAN. Minimise sum of (x -
      a_i)^2: x = the MEAN.
================================================================================
      */

// Moore-Hodgson: maximum number of jobs finished on time. jobs =
     {processing time, deadline}.
int maxOnTime(vector<pair<ll, ll>> jobs) {
    sort(all(jobs), [](auto& a, auto& b) { return a.second < b.
     second; });
    priority_queue<ll> pq;
    ll t = 0;
    for (auto [p, d] : jobs) {
        t += p, pq.push(p);
        if (t > d) t -= pq.top(), pq.pop();
    }
    return pq.size();
}
// Johnson's rule: optimal two-machine flow-shop order (returns
     the job indices).
vector<int> johnson(const vector<pair<ll, ll>>& jobs) {     // {p
      on machine 1, p on machine 2}
    vector<int> A, B;
    for (int i = 0; i < (int)jobs.size(); i++) (jobs[i].first <
     jobs[i].second ? A : B).push_back(i);
    sort(all(A), [&](int x, int y) { return jobs[x].first < jobs[
     y].first; });
    sort(all(B), [&](int x, int y) { return jobs[x].second > jobs
     [y].second; });
    A.insert(A.end(), all(B));
    return A;
}
```

## 1.20 Josephus

JOSEPHUS PROBLEM. n people in a circle 0..n-1, every k-th is removed; who survives?

```
// Recurrence: J(1) = 0, J(n) = (J(n-1) + k) mod n. O(n).
ll josephus(ll n, ll k) { ll r = 0; for (ll i = 2; i <= n; i++) r
     = (r + k) % i; return r; }
// k = 2 closed form: write n = 2^m + l with 0 <= l < 2^m, the
     survivor is 2*l  (0-indexed).
ll josephus2(ll n) { return 2 * (n - (1LL << (63 -
     __builtin_clzll(n)))); }
// LARGE n, SMALL k: one full lap kills floor(n/k) people, so
     recurse on n - floor(n/k) and map the
// index back. O(k log n).
ll josephusFast(ll n, ll k) {
    if (n == 1) return 0;
    if (k == 1) return n - 1;
    if (k > n) return (josephusFast(n - 1, k) + k) % n;
    ll r = josephusFast(n - n / k, k) - n % k;
    return r < 0 ? r + n : r + r / (k - 1);
}
// ORDER OF ELIMINATION / the j-th person to die: keep a BIT over
     alive flags and binary-search the
// ((pos + k - 1) mod alive + 1)-th alive person each step. O(n
     log n) and handles "k changes".
// LAST TWO SURVIVORS: run the recurrence starting from J(2) = {0,
      1} instead of J(1) = 0.

// OTHER ONE-LINE CLASSICS worth having on paper:
// GRAY CODE:      g(i) = i ^ (i >> 1);   inverse: x ^= x >> 1, x
     >> 2, ... until 0.
// n-th PERMUTATION (factorial number system): digit i = k / (n-1-
     i)!, then k %= (n-1-i)!.
// 15-PUZZLE SOLVABLE iff (#inversions of the tiles, blank
     excluded) + (row of the blank counted
//   from the BOTTOM, 1-based) is EVEN, for a 4x4 board. For an
     odd width w, solvable iff the
//   inversion count is even. General w x h: inversions + (blank
     row from bottom) parity rule above
//   when w is even, plain inversion parity when w is odd.
// TOWER OF HANOI: 2^n - 1 moves; on move t (1-based) move disk (t
      & -t) counted by trailing zeros,
//   and for odd n the smallest disk cycles A->C->B->A, for even n
      A->B->C->A.
// BULLS AND COWS / stable-marriage style pairings: Gale-Shapley
     is O(n^2), always terminates, and
//   is optimal for the proposing side.
// CATALAN-STYLE BALLOT COUNTING: paths from (0,0) to (n,m)
     staying strictly above y = x are
//   (n-m)/(n+m) * C(n+m, m)  (Bertrand's ballot theorem).
```

## 1.21 BigInt

BIG INTEGERS, non-negative, base 1e9. Enough for the usual "the answer does not fit in 64 bits"

```
// problems: exact factorials / Catalan numbers / determinants,
     huge Fibonacci, exact comparisons.
// Add/sub O(n), schoolbook multiply O(n*m) (fine to ~1e4 digits),
      divide by a small int O(n).
// If you need big*big beyond that, convolve the limbs with FFT
     and carry.
struct Big {
    static const ll B = 1000000000;
    vector<ll> d;                                          //
      little-endian limbs, base 1e9
    Big(ll v = 0) { for (; v > 0; v /= B) d.push_back(v % B); }
    Big(const string& s) {
        for (int i = s.size(); i > 0; i -= 9)
            d.push_back(stoll(s.substr(max(0, i - 9), min(9, i)))
    );
        trim();
    }
    void trim() { while (!d.empty() && d.back() == 0) d.pop_back
    (); }
    bool operator<(const Big& o) const {
        if (d.size() != o.d.size()) return d.size() < o.d.size();
        for (int i = d.size() - 1; i >= 0; i--) if (d[i] != o.d[i
    ]) return d[i] < o.d[i];
        return false;
    }
    Big operator+(const Big& o) const {
        Big r; ll c = 0;
        for (size_t i = 0; i < max(d.size(), o.d.size()) || c; i
    ++) {
            if (i < d.size()) c += d[i];
            if (i < o.d.size()) c += o.d[i];
            r.d.push_back(c % B), c /= B;
        }
        return r;
    }
    Big operator-(const Big& o) const {                    //
      requires *this >= o
        Big r = *this; ll c = 0;
        for (size_t i = 0; i < r.d.size(); i++) {
            r.d[i] -= c + (i < o.d.size() ? o.d[i] : 0);
            if ((c = r.d[i] < 0)) r.d[i] += B;
        }
        r.trim();
        return r;
    }
    Big operator*(const Big& o) const {
        if (d.empty() || o.d.empty()) return Big();
        vector<lll> t(d.size() + o.d.size(), 0);
        for (size_t i = 0; i < d.size(); i++)
            for (size_t j = 0; j < o.d.size(); j++) t[i + j] += (
    lll)d[i] * o.d[j];
        Big r; lll c = 0;
        for (size_t i = 0; i < t.size(); i++) c += t[i], r.d.
    push_back((ll)(c % B)), c /= B;
        for (; c; c /= B) r.d.push_back((ll)(c % B));
        r.trim();
        return r;
    }
    Big operator*(ll v) const { return *this * Big(v); }
    Big operator/(ll v) const {                            //
      small divisor only
        Big r = *this; lll c = 0;
        for (int i = r.d.size() - 1; i >= 0; i--) c = c * B + r.d
    [i], r.d[i] = (ll)(c / v), c %= v;
        r.trim();
        return r;
    }
    ll operator%(ll v) const {
        lll c = 0;
        for (int i = d.size() - 1; i >= 0; i--) c = (c * B + d[i
    ]) % v;
        return (ll)c;
    }
    string str() const {
        if (d.empty()) return "0";
        string s = to_string(d.back());
        for (int i = d.size() - 2; i >= 0; i--) {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
};
// SIGNED: keep a separate sign flag and dispatch +/- to add/sub
     on magnitudes; comparison first.
// BASE 1e9 packs 9 decimal digits per limb, so printing is
     trivial - that is the whole point.
// FAST DIVISION of two Bigs is long and almost never needed; if a
      problem really wants it, binary
// search the quotient limb by limb, or move to Python-style
     thinking and avoid it.
// SQRT: binary search with the multiply above, or Newton on
     doubles then fix up by a few steps.
```

### 1.22 RankingAndDates

RANKING / UNRANKING COMBINATORIAL OBJECTS, and CALENDAR ARITHMETIC. Small, and each one costs

```
// 20 minutes to re-derive under pressure.

// --- FACTORIAL NUMBER SYSTEM: rank and unrank a PERMUTATION. O(n ^2) as written; use an
// order-statistic tree or a BIT for O(n log n) when n is large.
// order(p) = how many permutations of the same multiset are lexicographically smaller.
ll permRank(vector<int> p) {                                    // p is a permutation of 0..n-1
    int n = p.size();
    ll r = 0, f = 1;
    for (int i = n - 1; i >= 0; i--) {
        int c = 0;
        for (int j = i + 1; j < n; j++) if (p[j] < p[i]) c++;
     // the Lehmer code digit
        r += c * f, f *= (n - i);
    }
    return r;
}
vector<int> permUnrank(int n, ll k) {                           // k-th permutation, 0-indexed
    vector<int> avail(n), p(n);
    iota(all(avail), 0);
    vector<ll> f(n + 1, 1);
    for (int i = 1; i <= n; i++) f[i] = f[i - 1] * i;
    for (int i = 0; i < n; i++) {
        ll d = k / f[n - 1 - i];
        k %= f[n - 1 - i];
        p[i] = avail[d], avail.erase(avail.begin() + d);
    }
    return p;
}
// NEXT PERMUTATION in place is std::next_permutation, O(n) - use it unless you need the rank.
// COMBINATIONS: rank of a k-subset {c_1<...<c_k} of [n] is sum C(c_i, i); unrank greedily by
// finding the largest c with C(c, i) <= remaining. Same shape, C instead of factorials.
// BALANCED BRACKET SEQUENCES rank/unrank with the ballot numbers; GRAY CODE is g = i ^ (i >> 1).

// --- JULIAN DAY NUMBER: date <-> integer, closed form, no loops, correct for the Gregorian
// calendar including every leap rule. Date differences, "what weekday", and "add 1000 days" all
// become integer arithmetic once you are here.
ll dateToInt(int y, int m, int d) {                             // y-m-d Gregorian -> day number
    return 1461 * (y + 4800 + (m - 14) / 12) / 4
         + 367 * (m - 2 - (m - 14) / 12 * 12) / 12
         - 3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 + d -
     32075;
}
array<int, 3> intToDate(ll jd) {                                // inverse: {y, m, d}
    ll x, n, i, j, dd;
    x = jd + 68569, n = 4 * x / 146097, x -= (146097 * n + 3) /
     4;
    i = 4000 * (x + 1) / 1461001, x -= 1461 * i / 4 - 31;
    j = 80 * x / 2447, dd = x - 2447 * j / 80, x = j / 11;
    return {(int)(100 * (n - 49) + i + x), (int)(j + 2 - 12 * x),
      (int)dd};
}
int weekday(ll jd) { return (int)((jd + 1) % 7); }
    // 0 = Sunday
// LEAP YEAR: y % 4 == 0 && (y % 100 != 0 || y % 400 == 0).
// ZELLER'S CONGRUENCE gives the weekday directly, but the day number is strictly more useful.

// --- STERN-BROCOT SEARCH: the simplest fraction satisfying a MONOTONE predicate. ---
// Descend the mediant tree, but take many steps in one direction at once (double the step size)
// so a 1e18 denominator costs O(log^2) instead of O(answer).
// USES: "smallest denominator q with property P", recovering p/q from a decimal prefix, the
// simplest fraction strictly inside an interval, and best rational approximation with a bound.
// ok() must be MONOTONE in the value p/q: false on small fractions, true from some point on.
// Returns the SMALLEST such fraction with denominator <= N. Each iteration takes as many steps
// as possible in one direction (found by doubling, then binary search), so it is O(log^2 N)
// rather than O(answer) - the difference between instant and hopeless at N = 1e18.
template <class F>
pair<ll, ll> fracSearch(ll N, F ok) {
    ll lp = 0, lq = 1, rp = 1, rq = 0;
    // 0/1 (not ok) .. 1/0 (ok)
    for (;;) {
        ll moved = 0;
        auto walk = [&](ll& ap, ll& aq, ll bp, ll bq, bool want)
    {   // steps from a toward b
            ll k = 0, s = 1;
            auto test = [&](ll t) {
                return aq + t * bq <= N && ok(ap + t * bp, aq + t
     * bq) == want;
            };
            while (test(k + s)) k += s, s *= 2;
            for (s /= 2; s; s /= 2) if (test(k + s)) k += s;
            ap += k * bp, aq += k * bq, moved += k;
        };
        walk(lp, lq, rp, rq, false);
    // push the FALSE end up
        if (lq + rq > N) return {rp, rq};
        walk(rp, rq, lp, lq, true);
    // pull the TRUE end down
        if (lq + rq > N || !moved) return {rp, rq};
    // !moved: nothing left to refine
    }
}
// The Stern-Brocot tree IS the continued fraction expansion: going left/right k times is one
// partial quotient. See 04 - Number Theory/05 - Rational for the CF machinery and semiconvergents.
```

## 2 Data Structures (DS)

### 2.1 Segment Tree

### 2.1.1 SegmentTree

SEGMENT TREE, point update + range query, O(log n) each. Swap the merge in SEG to change the

```
// operation (sum/min/max/gcd/matrix); any associative monoid
     works. Size is rounded up to a power
// of two so the tree is perfect and indices are simple.
struct SEG {
    ll sum = 0;
    SEG() {
    }
    SEG(ll x){
        sum = x;
    }
};
struct segTree {
    vector<SEG> seg;
    int sz = 1,n;
    segTree(int nn){
        n = nn;
        while (sz < nn)
            sz *= 2;
        seg.assign(2 * sz , SEG());
    }

    SEG merge(SEG seg1, SEG seg2) {
        SEG ret;
        ret.sum = seg1.sum + seg2.sum;
        return ret;
    }

    void build(vector<int> &v,int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(v[lx]);
            return;
        }
        int mid = (lx + rx) / 2;
        int lf = 2 * x + 1, rt = 2 * x + 2;

        build(v,lf, lx, mid);
        build(v,rt, mid + 1, rx);
        seg[x] = merge(seg[lf], seg[rt]);
    }
    void build(vector<int> &v){
        build(v,0, 0, n-1);
    }

    void update(int i, ll val, int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(val);
            return;
        }
        int mid = (lx + rx) / 2,lf = 2*x+1,rt= 2*x+2;
        if(i <= mid)
            update(i, val, lf, lx, mid);
        else
            update(i, val, rt, mid + 1, rx);
        seg[x] = merge(seg[lf] , seg[rt]);
    }

    void update(int i, ll val) {
        update(i, val, 0, 0, n-1);
    }

    SEG query(int l, int r, int x, int lx, int rx) {
        if (l <= lx && r >= rx)
            return seg[x];
        if (l > rx || r < lx)
            return SEG();

        int mid = (lx + rx) / 2,lf = 2 * x + 1,rt = 2*x+2;

        return merge(query(l, r, lf, lx, mid) , query(l, r, rt,
     mid + 1, rx));
    }

    SEG query(int l, int r) {
        return query(l, r, 0, 0, n-1);
    }
};
```

### 2.1.2 LazySegmentTree

Lazy Segment Tree (range add, range sum) - build O(n), update/query O(log n)

```
// 1-based node indices, 0-based positions, ALL recursions on [0,
     n-1].
// To change the op: edit merge(), apply() and the identity in qry
     ().
struct LazySeg {
    int n;
    vector<ll> t, lz;

    LazySeg(int n = 0) : n(n), t(4 * n, 0), lz(4 * n, 0) {}

    ll merge(ll a, ll b) { return a + b; }
    void apply(int v, int l, int r, ll x) { t[v] += x * (r - l +
     1); lz[v] += x; }

    void push(int v, int l, int r) {
        if (!lz[v]) return;
        int m = (l + r) / 2;
        apply(2 * v, l, m, lz[v]);
        apply(2 * v + 1, m + 1, r, lz[v]);
        lz[v] = 0;
    }

    void build(const vector<ll>& a, int v, int l, int r) {
        if (l == r) { t[v] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * v, l, m), build(a, 2 * v + 1, m + 1, r);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void build(const vector<ll>& a) { if (n) build(a, 1, 0, n -
     1); }

    void upd(int ql, int qr, ll x, int v, int l, int r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) return apply(v, l, r, x);
        push(v, l, r);
        int m = (l + r) / 2;
        upd(ql, qr, x, 2 * v, l, m), upd(ql, qr, x, 2 * v + 1, m
     + 1, r);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void upd(int l, int r, ll x) { upd(l, r, x, 1, 0, n - 1); }

    ll qry(int ql, int qr, int v, int l, int r) {
        if (qr < l || r < ql) return 0;                 //
     identity
        if (ql <= l && r <= qr) return t[v];
        push(v, l, r);
        int m = (l + r) / 2;
        return merge(qry(ql, qr, 2 * v, l, m), qry(ql, qr, 2 * v
     + 1, m + 1, r));
    }
    ll qry(int l, int r) { return qry(l, r, 1, 0, n - 1); }
};
```

### 2.1.3 PersistentSegmentTree

PERSISTENT SEGMENT TREE - every update returns a NEW root and old versions stay queryable.

```
// O(log n) time and O(log n) new nodes per update. The standard use is offline k-th smallest in a
// range (build a version per prefix, then query version[r] minus version[l-1]).
struct Node {
    Node *l, *r;
    ll val = 0;

    Node(ll val) : l(NULL), r(NULL), val(val) {
    }

    Node() : l(NULL), r(NULL) {
    }

    Node(Node* l, Node* r) : l(l), r(r) {
        if (l != NULL) val += l->val;
        if (r != NULL) val += r->val;
    }

    void addChild() {
        l = new Node();
        r = new Node();
    }
};

struct PST {
    int n;

    PST(int n) : n(n + 1) {
    }

    Node merge(Node x, Node y) {
        Node ret;
        ret.val = x.val + y.val;
        return ret;
    }

    Node* update(Node* v, int i, ll val, int lx, int rx) {
        if (lx == rx) return new Node(v->val + val);
        int mid = (lx + rx) / 2;
        if (!v->l) v->addChild();
        if (i <= mid) return new Node(update(v->l, i, val, lx, mid), v->r);
        return new Node(v->l, update(v->r, i, val, mid + 1, rx));
    }

    Node* update(Node* v, int i, ll val) { return update(v, i, val, 0, n - 1); }

    Node query(Node* v, int l, int r, int lx, int rx) {
        if (l > rx || r < lx) return {};
        if (l <= lx && r >= rx) return *v;
        if (!v->l) v->addChild();
        int mid = (lx + rx) / 2;
        return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid + 1, rx));
    }

    Node query(Node* v, int l, int r) { return query(v, l, r, 0, n - 1); }

    int getKth(Node* a, Node* b, int k, int lx, int rx) {
        if (lx == rx) return lx;
        if (!a->l) a->addChild();
        if (!b->l) b->addChild();
        int rem = b->l->val - a->l->val;
        int mid = (lx + rx) / 2;
        if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
        return getKth(a->r, b->r, k - rem, mid + 1, rx);
    }

    int getKth(Node* a, Node* b, int k) { return getKth(a, b, k, 0, n - 1); }
};
```

### 2.1.4 SegmentTreeBeats

Rename one if you ever paste two of them together.

```
// NOTE: several segment-tree files here declare their own `struct Node`.
//       Rename one if you ever paste two of them together.
struct Node {
    ll mn;
    int mnCnt;
    ll secondMn;

    ll mx;
    int mxCnt;
    ll secondMx;

    ll sum;

    ll lazyAdd;
    ll lazyEqual;
    bool isLazyEqual;

    Node() {
        mx = -inf;
        secondMx = -inf;
        mxCnt = 0;

        mn = inf;
        secondMn = inf;
        mnCnt = 0;

        sum = 0;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    };

    Node(int x) {
        mx = x;
        secondMx = -inf;
        mxCnt = 1;

        mn = x;
        secondMn = inf;
        mnCnt = 1;

        sum = x;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    }
};

struct SegTree {
    int tree_size;
    vector<Node> seg_data;

    SegTree(int n) {
        tree_size = 1;
        while (tree_size < n) tree_size *= 2;
        seg_data.resize(2 * tree_size, Node());
    }

    Node merge(Node& lf, Node& ri) {
        Node ans = Node();

        ans.sum = lf.sum + ri.sum;

        ans.mx = max(lf.mx, ri.mx);
        ans.secondMx = max(lf.secondMx, ri.secondMx);
        ans.mxCnt = 0;
        if (lf.mx == ans.mx) ans.mxCnt += lf.mxCnt;
        else ans.secondMx = max(ans.secondMx, lf.mx);
        if (ri.mx == ans.mx) ans.mxCnt += ri.mxCnt;
        else ans.secondMx = max(ans.secondMx, ri.mx);

        ans.mn = min(lf.mn, ri.mn);
        ans.secondMn = min(lf.secondMn, ri.secondMn);
        ans.mnCnt = 0;
        if (lf.mn == ans.mn) ans.mnCnt += lf.mnCnt;
        else ans.secondMn = min(ans.secondMn, lf.mn);
        if (ri.mn == ans.mn) ans.mnCnt += ri.mnCnt;
        else ans.secondMn = min(ans.secondMn, ri.mn);

        return ans;
    }

    void init(vector<int>& nums, int ni, int lx, int rx) {
        if (rx - lx == 1) {
```

```
        if (lx < nums.size())
            seg_data[ni] = Node(nums[lx]);
        return;
    }

    int mid = lx + (rx - lx) / 2;
    init(nums, 2 * ni + 1, lx, mid);
    init(nums, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 *
 ni + 2]);
}

void init(vector<int>& nums) {
    init(nums, 0, 0, tree_size);
}

void doPushEq(int ni, int lx, int rx, int x) {
    seg_data[ni].mn = seg_data[ni].mx = seg_data[ni].
 lazyEqual = x;
    seg_data[ni].isLazyEqual = 1;
    seg_data[ni].mnCnt = seg_data[ni].mxCnt = rx - lx;
    seg_data[ni].secondMx = -inf;
    seg_data[ni].secondMn = inf;

    seg_data[ni].sum = (ll)x * (rx - lx);
    seg_data[ni].lazyAdd = 0;
}

void doPushMinEq(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mn >= x) doPushEq(ni, lx, rx, x);
    else if (seg_data[ni].mx > x) {
        if (seg_data[ni].secondMn == seg_data[ni].mx)
            seg_data[ni].secondMn = x;
        seg_data[ni].sum -= (ll)(seg_data[ni].mx - x) *
 seg_data[ni].mxCnt;
        seg_data[ni].mx = x;
    }
}

void doPushMaxEq(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mx <= x) doPushEq(ni, lx, rx, x);
    else if (seg_data[ni].mn < x) {
        if (seg_data[ni].secondMx == seg_data[ni].mn)
            seg_data[ni].secondMx = x;

        seg_data[ni].sum += (ll)(x - seg_data[ni].mn) *
 seg_data[ni].mnCnt;
        seg_data[ni].mn = x;
    }
}

void doPushSum(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mn == seg_data[ni].mx)
        doPushEq(ni, lx, rx, seg_data[ni].mn + x);
    else {
        seg_data[ni].mx += x;
        if (seg_data[ni].secondMx != -inf)
            seg_data[ni].secondMx += x;

        seg_data[ni].mn += x;
        if (seg_data[ni].secondMn != inf)
            seg_data[ni].secondMn += x;

        seg_data[ni].sum += (ll)(rx - lx) * x;
        seg_data[ni].lazyAdd += x;
    }
}

void propagete(int ni, int lx, int rx) {
    if (rx - lx == 1) return;

    int mid = lx + (rx - lx) / 2;
    if (seg_data[ni].isLazyEqual) {
        doPushEq(2 * ni + 1, lx, mid, seg_data[ni].lazyEqual)
 ;
        doPushEq(2 * ni + 2, mid, rx, seg_data[ni].lazyEqual)
 ;
        seg_data[ni].lazyEqual = seg_data[ni].isLazyEqual =
 0;
    }
    else {
        doPushSum(2 * ni + 1, lx, mid, seg_data[ni].lazyAdd);
        doPushSum(2 * ni + 2, mid, rx, seg_data[ni].lazyAdd);
        seg_data[ni].lazyAdd = 0;

        doPushMinEq(2 * ni + 1, lx, mid, seg_data[ni].mx);
        doPushMinEq(2 * ni + 2, mid, rx, seg_data[ni].mx);
        doPushMaxEq(2 * ni + 1, lx, mid, seg_data[ni].mn);
        doPushMaxEq(2 * ni + 2, mid, rx, seg_data[ni].mn);
    }
}

void minimize(int l, int r, int x, int ni, int lx, int rx) {
    if (rx <= l || lx >= r || seg_data[ni].mx <= x)
        return;
    if (lx >= l && rx <= r && seg_data[ni].secondMx < x) {
        doPushMinEq(ni, lx, rx, x);
        return;
    }

    propagete(ni, lx, rx);

    int mid = lx + (rx - lx) / 2;
    minimize(l, r, x, 2 * ni + 1, lx, mid);
    minimize(l, r, x, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 *
 ni + 2]);
}

void minimize(int l, int r, int x) {
    minimize(l, r, x, 0, 0, tree_size);
}

void maximize(int l, int r, int x, int ni, int lx, int rx) {
    if (rx <= l || lx >= r || seg_data[ni].mn >= x)
        return;
    if (lx >= l && rx <= r && seg_data[ni].secondMn > x) {
        doPushMaxEq(ni, lx, rx, x);
        return;
    }

    propagete(ni, lx, rx);

    int mid = lx + (rx - lx) / 2;
    maximize(l, r, x, 2 * ni + 1, lx, mid);
    maximize(l, r, x, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 *
 ni + 2]);
}

void maximize(int l, int r, int x) {
    maximize(l, r, x, 0, 0, tree_size);
}

void add(int l, int r, int x, int ni, int lx, int rx) {
    if (rx <= l || lx >= r)
        return;
    if (lx >= l && rx <= r) {
        doPushSum(ni, lx, rx, x);
        return;
    }

    propagete(ni, lx, rx);

    int mid = lx + (rx - lx) / 2;
    add(l, r, x, 2 * ni + 1, lx, mid);
    add(l, r, x, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 *
 ni + 2]);
}

void add(int l, int r, int x) {
    add(l, r, x, 0, 0, tree_size);
}

void set(int l, int r, int x, int ni, int lx, int rx) {
    if (rx <= l || lx >= r)
        return;
    if (lx >= l && rx <= r) {
        doPushEq(ni, lx, rx, x);
        return;
    }

    propagete(ni, lx, rx);

    int mid = lx + (rx - lx) / 2;
    set(l, r, x, 2 * ni + 1, lx, mid);
    set(l, r, x, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 *
 ni + 2]);
}
```

```
    void set(int l, int r, int x) {
        set(l, r, x, 0, 0, tree_size);
    }

    Node get_range(int l, int r, int ni, int lx, int rx) {
        propagete(ni, lx, rx);

        if (lx >= l && rx <= r)
            return seg_data[ni];
        if (rx <= l || lx >= r)
            return Node();

        int mid = (lx + rx) / 2;

        Node lf = get_range(l, r, 2 * ni + 1, lx, mid);
        Node ri = get_range(l, r, 2 * ni + 2, mid, rx);
        return merge(lf, ri);
    }

    ll get_sum(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).sum;
    }

    int get_mx(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mx;
    }

    int get_mn(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mn;
    }
};
```

### 2.1.5 DynamicSegTree

Dynamic (Implicit) Segment Tree - Space: O(Q log N) per query

```
// Allocates nodes on demand for ranges up to 1e9 or 1e18
template<typename T = ll>
struct DynamicSegTree {
    struct Node {
        T val = 0;
        int lc = -1, rc = -1;
    };

    vector<Node> tree;
    ll L, R;

    DynamicSegTree(ll L = 0, ll R = 1e9) : L(L), R(R) {
        add_node();
    }

    int add_node() {
        tree.pb(Node());
        return tree.size() - 1;
    }

    void update(int node, ll l, ll r, ll pos, T val) {
        tree[node].val += val;
        if (l == r) return;
        ll mid = l + (r - l) / 2;
        if (pos <= mid) {
            if (tree[node].lc == -1) {
                int nxt = add_node();
                tree[node].lc = nxt;
            }
            update(tree[node].lc, l, mid, pos, val);
        } else {
            if (tree[node].rc == -1) {
                int nxt = add_node();
                tree[node].rc = nxt;
            }
            update(tree[node].rc, mid + 1, r, pos, val);
        }
    }

    void update(ll pos, T val) {
        update(0, L, R, pos, val);
    }

    T query(int node, ll l, ll r, ll ql, ll qr) {
        if (node == -1 || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return tree[node].val;
        ll mid = l + (r - l) / 2;
        return query(tree[node].lc, l, mid, ql, qr) + query(tree[
     node].rc, mid + 1, r, ql, qr);
    }

    T query(ll ql, ll qr) {
        return query(0, L, R, ql, qr);
    }
};
```

### 2.1.6 MergeSortTree

Merge Sort Tree - Space: O(N log N), Build: O(N log N), Query: O(log^2 N)

```
// Stores sorted elements in each node to answer 2D range count
     queries
template<typename T = int>
struct MergeSortTree {
    int n;
    vector<vector<T>> tree;

    MergeSortTree(const vector<T>& a) : n(a.size()), tree(4 * n)
     {
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, const vector<T>& a) {
        if (l == r) {
            tree[node] = {a[l]};
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, a);
        build(2 * node + 1, mid + 1, r, a);
        merge(all(tree[2 * node]), all(tree[2 * node + 1]),
     back_inserter(tree[node]));
    }

    // Returns number of elements <= k in index range [ql, qr]
    int query_count_le(int node, int l, int r, int ql, int qr, T
     k) {
        if (ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) {
            return upper_bound(all(tree[node]), k) - tree[node].
     begin();
        }
        int mid = (l + r) / 2;
        return query_count_le(2 * node, l, mid, ql, qr, k) +
     query_count_le(2 * node + 1, mid + 1, r, ql, qr, k);
    }

    int query_count_le(int ql, int qr, T k) {
        return query_count_le(1, 0, n - 1, ql, qr, k);
    }
};
```

### 2.1.7 IterativeSegTree

Iterative (Non-Recursive) Segment Tree - Space: O(N), Update: O(log N), Query: O(log N)

```cpp
// Fast, cache-friendly implementation for point update and range query
template<typename T = ll>
struct IterativeSegTree {
    int n;
    vector<T> tree;
    T neutral;
    function<T(T, T)> combine;

    IterativeSegTree(int n = 0, T neutral = 0, function<T(T, T)> combine = [](T a, T b) { return a + b; })
        : n(n), tree(2 * n, neutral), neutral(neutral), combine(combine) {}

    IterativeSegTree(const vector<T>& a, T neutral = 0, function<T(T, T)> combine = [](T a, T b) { return a + b; })
        : n(a.size()), tree(2 * a.size(), neutral), neutral(neutral), combine(combine) {
        for (int i = 0; i < n; i++) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; i--) tree[i] = combine(tree[i << 1], tree[i << 1 | 1]);
    }

    void update(int p, T val) {
        // combine(LEFT, RIGHT) - do NOT write combine(tree[p], tree[p ^ 1]): when p is the right
        // child that reverses the arguments, which is wrong for any non-commutative combine
        // (matrix product, affine composition, "merge two structs").
        for (tree[p += n] = val; p > 1; p >>= 1)
            tree[p >> 1] = combine(tree[p & ~1], tree[p | 1]);
    }

    // Range query on [l, r] inclusive
    T query(int l, int r) {
        T resl = neutral, resr = neutral;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resl = combine(resl, tree[l++]);
            if (r & 1) resr = combine(tree[--r], resr);
        }
        return combine(resl, resr);
    }
};
```

### 2.1.8 SegTree2D

2D Segment Tree - Space: O(N * M), Update: O(log N * log M), Query: O(log N * log M)

```cpp
// Supports point updates and 2D subgrid range sum queries
template<typename T = ll>
struct SegTree2D {
    int n, m;
    vector<vector<T>> tree;

    SegTree2D(int n = 0, int m = 0) : n(n), m(m), tree(4 * n, vector<T>(4 * m, 0)) {}

    void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y, T val) {
        if (ly == ry) {
            if (lx == rx) tree[vx][vy] += val;
            else tree[vx][vy] = tree[2 * vx][vy] + tree[2 * vx + 1][vy];
            return;
        }
        int my = (ly + ry) / 2;
        if (y <= my) update_y(vx, lx, rx, 2 * vy, ly, my, x, y, val);
        else update_y(vx, lx, rx, 2 * vy + 1, my + 1, ry, x, y, val);
        tree[vx][vy] = tree[vx][2 * vy] + tree[vx][2 * vy + 1];
    }

    void update_x(int vx, int lx, int rx, int x, int y, T val) {
        if (lx != rx) {
            int mx = (lx + rx) / 2;
            if (x <= mx) update_x(2 * vx, lx, mx, x, y, val);
            else update_x(2 * vx + 1, mx + 1, rx, x, y, val);
        }
        update_y(vx, lx, rx, 1, 0, m - 1, x, y, val);
    }

    void add(int x, int y, T val) {
        update_x(1, 0, n - 1, x, y, val);
    }

    T query_y(int vx, int vy, int ly, int ry, int qy1, int qy2) {
        if (qy1 > ry || qy2 < ly) return 0;
        if (qy1 <= ly && ry <= qy2) return tree[vx][vy];
        int my = (ly + ry) / 2;
        return query_y(vx, 2 * vy, ly, my, qy1, qy2) + query_y(vx, 2 * vy + 1, my + 1, ry, qy1, qy2);
    }

    T query_x(int vx, int lx, int rx, int qx1, int qx2, int qy1, int qy2) {
        if (qx1 > rx || qx2 < lx) return 0;
        if (qx1 <= lx && rx <= qx2) return query_y(vx, 1, 0, m - 1, qy1, qy2);
        int mx = (lx + rx) / 2;
        return query_x(2 * vx, lx, mx, qx1, qx2, qy1, qy2) + query_x(2 * vx + 1, mx + 1, rx, qx1, qx2, qy1, qy2);
    }

    T query(int x1, int y1, int x2, int y2) {
        return query_x(1, 0, n - 1, x1, x2, y1, y2);
    }
};
```

### 2.1.9 MergeSortTreeFractional

MERGE SORT TREE WITH FRACTIONAL CASCADING - static "how many values <= x in [l,r]" in O(log n)

```
// instead of O(log^2 n): each node stores its sorted values plus
     pointers into the children, so the
// binary search happens ONCE at the root and is then followed
     down for free.
const int MAXN = 30005;
int a[MAXN];

struct Node {
    vector<int> vals;
    vector<int> to_left;
    vector<int> to_right;
} tree[4 * MAXN];

void build(int node, int start, int end) {
    if (start == end) {
        tree[node].vals.push_back(a[start]);
        tree[node].to_left.assign(2, 0);
        tree[node].to_right.assign(2, 0);
        return;
    }

    int mid = (start + end) / 2;
    int left_child = 2 * node;
    int right_child = 2 * node + 1;

    build(left_child, start, mid);
    build(right_child, mid + 1, end);

    int left_sz = tree[left_child].vals.size();
    int right_sz = tree[right_child].vals.size();
    int total_sz = left_sz + right_sz;

    tree[node].vals.resize(total_sz);
    tree[node].to_left.resize(total_sz + 1);
    tree[node].to_right.resize(total_sz + 1);

    int p1 = 0, p2 = 0;

    for (int i = 0; i < total_sz; i++) {
        tree[node].to_left[i] = p1;
        tree[node].to_right[i] = p2;

        if (p1 < left_sz && (p2 == right_sz || tree[left_child].
     vals[p1] <= tree[right_child].vals[p2])) {
            tree[node].vals[i] = tree[left_child].vals[p1++];
        } else {
            tree[node].vals[i] = tree[right_child].vals[p2++];
        }
    }

    tree[node].to_left[total_sz] = p1;
    tree[node].to_right[total_sz] = p2;
}

int query(int node, int start, int end, int l, int r, int idx) {
    if (l > end || r < start) {
        return 0;
    }

    if (l <= start && end <= r) {
        return idx;
    }

    int mid = (start + end) / 2;
    int next_idx_left = tree[node].to_left[idx];
    int next_idx_right = tree[node].to_right[idx];

    return query(2 * node, start, mid, l, r, next_idx_left) +
           query(2 * node + 1, mid + 1, end, l, r, next_idx_right
     );
}
```

### 2.1.10 RangeMajorityCandidates

```
const int MAXN = 200005;                              // array bound (
     Template.cpp calls this N)
int a[MAXN];
vector<int> pos[MAXN];
int K_LIMIT = 2;

struct Node {
    vector<pair<int, int>> cands;
};

Node tree_seg[4 * MAXN];

Node merge_nodes(const Node& left, const Node& right) {
    vector<pair<int, int>> combined;
    combined.reserve(left.cands.size() + right.cands.size());

    int i = 0, j = 0;
    while (i < left.cands.size() && j < right.cands.size()) {
        if (left.cands[i].first == right.cands[j].first) {
            combined.push_back({left.cands[i].first, left.cands[i
     ].second + right.cands[j].second});
            i++; j++;
        } else if (left.cands[i].first < right.cands[j].first) {
            combined.push_back(left.cands[i++]);
        } else {
            combined.push_back(right.cands[j++]);
        }
    }
    while (i < left.cands.size()) combined.push_back(left.cands[i
     ++]);
    while (j < right.cands.size()) combined.push_back(right.cands
     [j++]);

    if (combined.size() <= K_LIMIT - 1) {
        return {combined};
    }

    vector<int> counts;
    counts.reserve(combined.size());
    for (const auto& p : combined) {
        counts.push_back(p.second);
    }

    nth_element(counts.begin(), counts.begin() + K_LIMIT - 1,
     counts.end(), greater<int>());
    int penalty = counts[K_LIMIT - 1];

    vector<pair<int, int>> survived;
    survived.reserve(K_LIMIT - 1);
    for (const auto& p : combined) {
        if (p.second > penalty) {
            survived.push_back({p.first, p.second - penalty});
        }
    }

    return {survived};
}

void build(int node, int start, int end) {
    if (start == end) {
        tree_seg[node].cands.push_back({a[start], 1});
        return;
    }
    int mid = start + (end - start) / 2;
    build(2 * node, start, mid);
    build(2 * node + 1, mid + 1, end);
    tree_seg[node] = merge_nodes(tree_seg[2 * node], tree_seg[2 *
      node + 1]);
}

Node query_candidates(int node, int start, int end, int l, int r)
      {
    if (r < start || end < l) return {{}};
    if (l <= start && end <= r) return tree_seg[node];

    int mid = start + (end - start) / 2;
    Node left_res = query_candidates(2 * node, start, mid, l, r);
    Node right_res = query_candidates(2 * node + 1, mid + 1, end,
      l, r);

    if (left_res.cands.empty()) return right_res;
    if (right_res.cands.empty()) return left_res;

    return merge_nodes(left_res, right_res);
}

// O(log N) fast static frequency count
int get_frequency(int val, int l, int r) {
```

```
    auto& v = pos[val];
    auto right_it = upper_bound(v.begin(), v.end(), r);
    auto left_it = lower_bound(v.begin(), v.end(), l);
    return distance(left_it, right_it);
}

void solve() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        pos[a[i]].push_back(i);
    }

    build(1, 1, n);

    while (q--) {
        int l, r;
        cin >> l >> r;

        int len = r - l + 1;
        Node res = query_candidates(1, 1, n, l, r);

        int max_freq = 0;
        for (const auto& p : res.cands) {
            int cand = p.first;
            int freq = get_frequency(cand, l, r);
            max_freq = max(max_freq, freq);
        }

        if (max_freq > (len + 1) / 2) {
            cout << 2 * max_freq - len << "\n";
        } else {
            cout << 1 << "\n";
        }
    }
}
```

### 2.1.11 SegTreeDescent

Segment Tree DESCENT - answer "first / last index satisfying P" in O(log n), not O(log^2 n).

```
// Here: max-segtree. first_ge(l, x) = smallest i >= l with a[i]
     >= x, or n if none.
// Swap the stored aggregate + the two `good()` tests to get:
     first prefix-sum >= S,
// first zero, k-th set bit, "how far right can I extend". This is
     the workhorse behind
// greedy "extend as far as possible" and interval-cover problems.
struct SegMax {
    int n;
    vector<ll> t;

    SegMax(int n = 0) : n(n), t(4 * n, LLONG_MIN) {}

    void build(const vector<ll>& a, int v, int l, int r) {
        if (l == r) { t[v] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * v, l, m), build(a, 2 * v + 1, m + 1, r);
        t[v] = max(t[2 * v], t[2 * v + 1]);
    }
    void build(const vector<ll>& a) { if (n) build(a, 1, 0, n -
     1); }

    void set(int p, ll x, int v, int l, int r) {
        if (l == r) { t[v] = x; return; }
        int m = (l + r) / 2;
        p <= m ? set(p, x, 2 * v, l, m) : set(p, x, 2 * v + 1, m
     + 1, r);
        t[v] = max(t[2 * v], t[2 * v + 1]);
    }
    void set(int p, ll x) { set(p, x, 1, 0, n - 1); }

    ll qry(int ql, int qr, int v, int l, int r) {
        if (qr < l || r < ql) return LLONG_MIN;
        if (ql <= l && r <= qr) return t[v];
        int m = (l + r) / 2;
        return max(qry(ql, qr, 2 * v, l, m), qry(ql, qr, 2 * v +
     1, m + 1, r));
    }
    ll qry(int l, int r) { return qry(l, r, 1, 0, n - 1); }

    // smallest i in [ql, n) with a[i] >= x   (returns n if none)
    int first_ge(int ql, ll x, int v, int l, int r) {
        if (r < ql || t[v] < x) return n;
        if (l == r) return l;
        int m = (l + r) / 2, res = first_ge(ql, x, 2 * v, l, m);
        return res < n ? res : first_ge(ql, x, 2 * v + 1, m + 1,
     r);
    }
    int first_ge(int ql, ll x) { return first_ge(ql, x, 1, 0, n -
      1); }

    // largest i in [0, qr] with a[i] >= x    (returns -1 if none)
    int last_ge(int qr, ll x, int v, int l, int r) {
        if (l > qr || t[v] < x) return -1;
        if (l == r) return l;
        int m = (l + r) / 2, res = last_ge(qr, x, 2 * v + 1, m +
     1, r);
        return res >= 0 ? res : last_ge(qr, x, 2 * v, l, m);
    }
    int last_ge(int qr, ll x) { return last_ge(qr, x, 1, 0, n -
     1); }
};
```

## 2.2 Binary Indexed Tree (BIT)

### 2.2.1 FenwickTree

Fenwick Tree (Binary Indexed Tree) - 0-based indexing

```cpp
// Space: O(N), Time: O(log N) per update/query
template<typename T = ll>
struct FenwickTree {
    int n;
    vector<T> tree;

    FenwickTree(int n = 0) : n(n), tree(n + 1, 0) {}

    void add(int i, T delta) {
        for (i++; i <= n; i += i & -i) tree[i] += delta;
    }

    void set(int i, T val) {
        add(i, val - query(i, i));
    }

    T query(int i) {
        T sum = 0;
        for (i++; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }

    T query(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }

    // Returns first 0-based index where prefix sum >= val
    int lower_bound(T val) {
        int pos = 0;
        for (int sz = 1 << (n ? __lg(n) : 0); sz > 0; sz >>= 1) {
            if (pos + sz <= n && tree[pos + sz] < val) {
                val -= tree[pos + sz];
                pos += sz;
            }
        }
        return pos;
    }
};
```

### 2.2.2 FenwickTree2D

2D Fenwick Tree - 0-based indexing

```cpp
// Space: O(N * M), Time: O(log N * log M) per update/query
template<typename T = ll>
struct FenwickTree2D {
    int n, m;
    vector<vector<T>> tree;

    FenwickTree2D(int n = 0, int m = 0) : n(n), m(m), tree(n + 1,
      vector<T>(m + 1, 0)) {}

    void add(int x, int y, T val) {
        for (int i = x + 1; i <= n; i += i & -i) {
            for (int j = y + 1; j <= m; j += j & -j) {
                tree[i][j] += val;
            }
        }
    }

    void set(int x, int y, T val) {
        add(x, y, val - query(x, y, x, y));
    }

    T query(int x, int y) {
        T sum = 0;
        for (int i = min(x + 1, n); i > 0; i -= i & -i) {
            for (int j = min(y + 1, m); j > 0; j -= j & -j) {
                sum += tree[i][j];
            }
        }
        return sum;
    }

    // 2D Subgrid Range Sum from (x1, y1) to (x2, y2) inclusive
    T query(int x1, int y1, int x2, int y2) {
        if (x1 > x2 || y1 > y2) return 0;
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 -
      1) + query(x1 - 1, y1 - 1);
    }
};
```

### 2.2.3 Offline2DFenwickTree

Offline 2D Fenwick Tree (Compressed 2D BIT)

```cpp
// Space: O(N + Q log N), Time: O(log^2 N) per query
template<typename T = ll>
struct BIT2D {
    int n;
    vector<vector<int>> vals;
    vector<vector<T>> bit;

    int ind(const vector<int>& v, int x) {
        return upper_bound(all(v), x) - v.begin() - 1;
    }

    // mx: max row index, todo: all (r, c) update coordinates that
      will be called
    BIT2D(int mx, vector<array<int, 2>> todo) : n(mx), vals(mx +
     1), bit(mx + 1) {
        sort(all(todo), [](const auto& a, const auto& b) { return
      a[1] < b[1]; });

        for (int i = 0; i <= n; i++) vals[i].pb(INT_MIN);
        for (auto [r, c] : todo) {
            for (int x = r; x <= n; x |= x + 1) {
                if (vals[x].back() != c) vals[x].pb(c);
            }
        }
        for (int i = 0; i <= n; i++) bit[i].resize(vals[i].size()
     , 0);
    }

    void add(int r, int c, T val) {
        for (; r <= n; r |= r + 1) {
            for (int i = ind(vals[r], c); i < (int)bit[r].size();
      i |= i + 1) {
                bit[r][i] += val;
            }
        }
    }

    T query(int r, int c) {
        T res = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1) {
            for (int i = ind(vals[r], c); i >= 0; i = (i & (i +
     1)) - 1) {
                res += bit[r][i];
            }
        }
        return res;
    }

    T query(int r1, int c1, int r2, int c2) {
        return query(r2, c2) - query(r2, c1 - 1) - query(r1 - 1,
     c2) + query(r1 - 1, c1 - 1);
    }

    void reset() {
        for (auto& v : bit) fill(all(v), 0);
    }
};
```

### 2.2.4 BITRangeUpdateRangeQuery

BIT with RANGE UPDATE + RANGE QUERY (two BITs). 0-based, O(log n) per op.

```cpp
// pre(i) = sum_{j<=i} a[j] = S1(i)*(i+1) - S2(i), where a range-
     add of v on [l,r] does
//   S1: +v at l, -v at r+1        S2: +v*l at l, -v*(r+1) at r+1
template <typename T = ll>
struct BIT2 {
    int n;
    vector<T> b1, b2;

    BIT2(int n = 0) : n(n), b1(n + 1, 0), b2(n + 1, 0) {}
    void add(vector<T>& b, int i, T v) { for (i++; i <= n; i += i
      & -i) b[i] += v; }
    T sum(const vector<T>& b, int i) const { T s = 0; for (i++; i
      > 0; i -= i & -i) s += b[i]; return s; }

    void upd(int l, int r, T v) {               // add v to [l, r
     ]
        add(b1, l, v), add(b1, r + 1, -v);
        add(b2, l, v * l), add(b2, r + 1, -v * (r + 1));
    }
    T pre(int i) const { return i < 0 ? 0 : sum(b1, i) * (i + 1)
     - sum(b2, i); }
    T qry(int l, int r) const { return pre(r) - pre(l - 1); }
};
```

## 2.3 Sparse Table

### 2.3.1 SparseTable

SPARSE TABLE - O(1) range query after O(N log N) build, NO updates.

```cpp
// The operation must be IDEMPOTENT (min, max, gcd, and, or)
     because the two covering blocks
// OVERLAP. For sum/product/xor - anything not idempotent - use 12
      - DisjointSparseTable.cpp
// (still O(1) query) or a Fenwick tree. Guard l <= r: __lg(0) is
     undefined.
template<typename T, typename F = function<T(T, T)>>
struct SparseTable {
    int n;
    vector<vector<T>> mat;
    F func;

    SparseTable() = default;
    SparseTable(const vector<T>& a, F f = [](T x, T y) { return
     max(x, y); }) : n(a.size()), func(f) {
        int k = n ? __lg(n) + 1 : 0;
        mat.assign(k, a);
        for (int j = 1; j < k; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                mat[j][i] = func(mat[j - 1][i], mat[j - 1][i + (1
     << (j - 1))]);
            }
        }
    }

    T query(int l, int r) const {
        int j = __lg(r - l + 1);
        return func(mat[j][l], mat[j][r - (1 << j) + 1]);
    }
};
```

### 2.3.2 SparseTable2D

2D Sparse Table - O(N * M * log N * log M) build, O(1) query

```cpp
template<typename T = ll>
struct SparseTable2D {
    int n, m;
    // jmp[kr][kc][i][j] = max in 2^kr x 2^kc subgrid starting at
     (i, j)
    vector<vector<vector<vector<T>>>> jmp;

    SparseTable2D(const vector<vector<T>>& grid) {
        n = grid.size();
        if (!n) return;
        m = grid[0].size();
        if (!m) return;

        int log_n = __lg(n) + 1, log_m = __lg(m) + 1;
        jmp.assign(log_n, vector<vector<vector<T>>>(log_m, vector
     <vector<T>>(n, vector<T>(m))));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                jmp[0][0][i][j] = grid[i][j];
            }
        }

        for (int kc = 1; kc < log_m; kc++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j + (1 << kc) <= m; j++) {
                    jmp[0][kc][i][j] = max(jmp[0][kc - 1][i][j],
     jmp[0][kc - 1][i][j + (1 << (kc - 1))]);
                }
            }
        }

        for (int kr = 1; kr < log_n; kr++) {
            for (int kc = 0; kc < log_m; kc++) {
                for (int i = 0; i + (1 << kr) <= n; i++) {
                    for (int j = 0; j + (1 << kc) <= m; j++) {
                        jmp[kr][kc][i][j] = max(jmp[kr - 1][kc][i
     ][j], jmp[kr - 1][kc][i + (1 << (kr - 1))][j]);
                    }
                }
            }
        }
    }

    T query(int r1, int c1, int r2, int c2) const {
        int kr = __lg(r2 - r1 + 1), kc = __lg(c2 - c1 + 1);
        return max({jmp[kr][kc][r1][c1],
                    jmp[kr][kc][r1][c2 - (1 << kc) + 1],
                    jmp[kr][kc][r2 - (1 << kr) + 1][c1],
                    jmp[kr][kc][r2 - (1 << kr) + 1][c2 - (1 << kc
     ) + 1]});
    }
};
```

## 2.4 Trie

### 2.4.1 StringTrie

PREFIX TRIE over an alphabet. O(L) per insert/query. Reach for it for prefix counting, autocomplete,

```
// "is any inserted word a prefix of this one", and as the
     skeleton Aho-Corasick is built on.
// Prefix Trie for Strings
// Time: O(L) per insert/query, Space: O(N * L * Alphabet)
struct Trie {
    vector<vector<int>> nxt;
    vector<int> cnt, end_cnt;
    int alpha;

    Trie(int alpha = 26) : alpha(alpha) {
        add_node();
    }

    int add_node() {
        nxt.emplace_back(alpha, -1);
        cnt.emplace_back(0);
        end_cnt.emplace_back(0);
        return nxt.size() - 1;
    }

    void insert(const string& s, int val = 1) {
        int u = 0;
        cnt[u] += val;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) nxt[u][ch] = add_node();
            u = nxt[u][ch];
            cnt[u] += val;
        }
        end_cnt[u] += val;
    }

    int count_prefix(const string& s) {
        int u = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) return 0;
            u = nxt[u][ch];
        }
        return cnt[u];
    }
};
```

### 2.4.2 BinaryTrie

Binary trie over [0, 2^(B+1)). insert / ERASE / query all O(B).

```
// erase() is what lets you keep the trie equal to the current
     root->node path during a DFS
// ("min XOR with any ancestor"): add on entry, add(-1) on exit.
struct BTrie {
    static const int B = 30;
    vector<array<int, 2>> ch = {{-1, -1}};
    vector<int> cnt = {0};

    int node() { ch.push_back({-1, -1}); cnt.push_back(0); return
      ch.size() - 1; }
    void add(ll x, int d = 1) {                          // d = +1
      insert, -1 erase
        int u = 0; cnt[0] += d;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1;
            if (ch[u][b] < 0) ch[u][b] = node();
            u = ch[u][b], cnt[u] += d;
        }
    }
    int cn(int u, int b) const { return u < 0 || ch[u][b] < 0 ? 0
      : cnt[ch[u][b]]; }
    int size() const { return cnt[0]; }

    ll bestXor(ll x, bool mx) const {                    // mx=1
     -> max x^y, mx=0 -> min x^y
        if (!cnt[0]) return -1;
        int u = 0; ll r = 0;
        for (int i = B; i >= 0; i--) {
            int b = (x >> i & 1) ^ mx;                   // bit we
     'd like y to have
            if (!cn(u, b)) b ^= 1;
            r |= (ll)((x >> i & 1) ^ b) << i;
            u = ch[u][b];
        }
        return r;
    }
    ll maxXor(ll x) const { return bestXor(x, 1); }
    ll minXor(ll x) const { return bestXor(x, 0); }

    int cntLess(ll x, ll k) const {                      // #{y in
      trie : (x ^ y) < k}
        int u = 0, r = 0;
        for (int i = B; i >= 0 && u >= 0; i--) {
            int xb = x >> i & 1;
            if (k >> i & 1) r += cn(u, xb), u = ch[u][xb ^ 1];
     // xor-bit 0 branch is all < k
            else u = ch[u][xb];
        }
        return r;
    }
};
```

### 2.4.3 PersistentTrie

Persistent Binary Trie - Space: O(N * BITS), Time: O(BITS) per insert/query

```
// Supports versioning and range max XOR queries between versions
    [l_ver, r_ver]
struct PersistentTrie {
    struct Node {
        int cnt = 0;
        array<int, 2> nxt = {-1, -1};
    };

    vector<Node> tree;
    vector<int> roots;
    int max_bit;

    PersistentTrie(int max_bit = 30) : max_bit(max_bit) {
        roots.pb(add_node());
    }

    int add_node(int old_node = -1) {
        if (old_node != -1) {
            tree.pb(tree[old_node]);
        } else {
            tree.pb(Node());
        }
        return tree.size() - 1;
    }

    // Inserts x starting from prev_root and returns the new
     version root
    int insert(int prev_root, ll x) {
        int new_root = add_node(prev_root);
        int u = new_root, prev_u = prev_root;
        tree[u].cnt++;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int prev_child = (prev_u != -1) ? tree[prev_u].nxt[b]
     : -1;
            tree[u].nxt[b] = add_node(prev_child);
            u = tree[u].nxt[b];
            if (prev_u != -1) prev_u = tree[prev_u].nxt[b];
            tree[u].cnt++;
        }
        roots.pb(new_root);
        return new_root;
    }

    // Returns max (x ^ val) over elements inserted in version
     range [l_ver, r_ver]
    ll max_xor(int l_ver, int r_ver, ll x) const {
        int u_r = roots[r_ver], u_l = (l_ver > 0) ? roots[l_ver -
     1] : -1;
        ll res = 0;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int want = b ^ 1;

            int count_r = (u_r != -1 && tree[u_r].nxt[want] !=
    -1) ? tree[tree[u_r].nxt[want]].cnt : 0;
            int count_l = (u_l != -1 && tree[u_l].nxt[want] !=
    -1) ? tree[tree[u_l].nxt[want]].cnt : 0;

            if (count_r - count_l > 0) {
                res |= (1LL << i);
                u_r = tree[u_r].nxt[want];
                u_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ?
    tree[u_l].nxt[want] : -1;
            } else {
                u_r = (u_r != -1 && tree[u_r].nxt[b] != -1) ?
    tree[u_r].nxt[b] : -1;
                u_l = (u_l != -1 && tree[u_l].nxt[b] != -1) ?
    tree[u_l].nxt[b] : -1;
            }
        }
        return res;
    }
};
```

## 2.5 Disjoint Set Union (DSU)

### 2.5.1 DSU

Disjoint Set Union (DSU) - Path Compression & Union by Size

```
// Time: O(alpha(N)) per operation
struct DSU {
    vector<int> par, sz;

    DSU(int n = 0) : par(n), sz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : par[x] = find(par[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        sz[x] += sz[y];
        par[y] = x;
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }
};
```

### 2.5.2 DSURollback

DSU with Rollback - No path compression to allow backtracking

```
// Time: O(log N) per operation, O(1) rollback
struct DSURollback {
    vector<int> par, sz;
    vector<pair<int, int>> history; // {child_node,
     old_parent_size}

    DSURollback(int n = 0) : par(n), sz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : find(par[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            history.eb(-1, -1);
            return false;
        }
        if (sz[x] < sz[y]) swap(x, y);
        history.eb(y, sz[x]);
        sz[x] += sz[y];
        par[y] = x;
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }

    void rollback() {
        if (history.empty()) return;
        auto [y, old_sz_x] = history.back();
        history.pop_back();
        if (y != -1) {
            sz[par[y]] = old_sz_x;
            par[y] = y;
        }
    }

    int get_state() const {
        return history.size();
    }

    void rollback_to(int state) {
        while ((int)history.size() > state) rollback();
    }
};
```

### 2.5.3 WeightedDSU

DSU with POTENTIALS: maintains pot[v] = value(v) - value(root), so you can answer

```
// "what is value(u) - value(v)" and detect contradictions.
// Group is (Z, +) here; for parity/bipartiteness use XOR: change
     ALL FOUR of pot[x] += p -> ^=,
// pu - pv == w -> (pu ^ pv) == w, pot[rv] = pu - pv - w -> pu ^
     pv ^ w, and DELETE the w = -w line
// (negation is not the inverse in GF(2)), or Z_k for mod-k
     relations.
struct WDSU {
    vector<int> par;
    vector<ll> pot;                                  // potential
     relative to the parent
    vector<int> sz;

    WDSU(int n = 0) : par(n), pot(n, 0), sz(n, 1) { iota(all(par)
     , 0); }
    pair<int, ll> find(int x) {                      // {root, value(
     x) - value(root)}
        if (par[x] == x) return {x, 0};
        auto [r, p] = find(par[x]);
        par[x] = r, pot[x] += p;                     // path
     compression accumulates
        return {r, pot[x]};
    }
    // assert value(u) - value(v) == w. false = CONTRADICTION with
      what we already know.
    bool join(int u, int v, ll w) {
        auto [ru, pu] = find(u);
        auto [rv, pv] = find(v);
        if (ru == rv) return pu - pv == w;
        if (sz[ru] < sz[rv]) swap(ru, rv), swap(pu, pv), w = -w;
        par[rv] = ru, pot[rv] = pu - pv - w, sz[ru] += sz[rv];
        return true;
    }
    // value(u) - value(v), or nullopt if they are not related yet
    bool diff(int u, int v, ll& out) {
        auto [ru, pu] = find(u);
        auto [rv, pv] = find(v);
        if (ru != rv) return false;
        out = pu - pv;
        return true;
    }
};
// BIPARTITENESS under edge additions: use XOR potentials with w =
      1 per edge; join() returning
// false means an odd cycle appeared. Same structure answers "are
     u and v the same colour".
```

## 2.6 Square Root Decomposition

### 2.6.1 MoAlgorithm

MO'S ALGORITHM - answers m OFFLINE range queries in O((n + m) sqrt n) when you can move an endpoint

```
// by one in O(1) (add/remove an element and maintain the answer).
      Sort queries by block of l, then by
// r (alternating direction per block). Use it for distinct counts
     , frequency-of-frequency, XOR pairs.
class MoArray {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd)
                return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<int> a;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<int> freq;

    inline void add(int idx) {
        if (freq[a[idx]] == 0) ans++;
        freq[a[idx]]++;
    }

    inline void remove(int idx) {
        freq[a[idx]]--;
        if (freq[a[idx]] == 0) ans--;
    }

    inline long long get_answer() const {
        return ans;
    }
    // ------------------------------

public:
    MoArray(const vector<int>& arr) : n(arr.size()), a(arr) {
        int max_val = 0;
        for (int x : a) {
            max_val = max(max_val, x);
        }
        freq.assign(max_val + 1, 0);
        ans = 0;
    }

    vector<long long> solve(const vector<pair<int, int>>&
     raw_queries) {
        int q = raw_queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            queries[i].l = raw_queries[i].first;
            queries[i].r = raw_queries[i].second;
            queries[i].idx = i;
            queries[i].bnd = queries[i].l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(--cur_l);
            while (cur_r < qry.r) add(++cur_r);
            while (cur_l < qry.l) remove(cur_l++);
            while (cur_r > qry.r) remove(cur_r--);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

### 2.6.2 MoOnSubtrees

MO'S ON SUBTREES - flatten the tree with an Euler tour so every subtree is a contiguous range,

```
// then run plain Mo's on it. Same O((n + q) sqrt n). Use for "
     query over the subtree of v" when no
// online structure fits (distinct colours in a subtree, k-th most
      frequent, etc.).
class MoSubtree {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<vector<int>> adj;
    vector<int> in, out, flat, clr;

    // --- PROBLEM SPECIFIC STATE ---
    int mx_freq;
    vector<int> freq;
    vector<long long> sum_freq;

    inline void add(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]++;
        sum_freq[freq[c]] += c;
        if (freq[c] > mx_freq) mx_freq = freq[c];
    }

    inline void remove(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]--;
        sum_freq[freq[c]] += c;
        while (mx_freq > 0 && sum_freq[mx_freq] == 0) mx_freq--;
    }

    inline long long get_answer() const {
        return sum_freq[mx_freq];
    }
    // ------------------------------

    void dfs(int u, int p) {
        in[u] = flat.size();
        flat.push_back(u);
        for (int v : adj[u]) {
            if (v == p) continue;
            dfs(v, u);
        }
        out[u] = flat.size() - 1;
    }

public:
    MoSubtree(int n, int max_val, const vector<int>& colors)
        : n(n), adj(n + 1), in(n + 1), out(n + 1), clr(colors) {
        mx_freq = 0;
        freq.assign(max_val + 1, 0);
        sum_freq.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<long long> solve(int root, const vector<int>&
     query_nodes) {
        flat.reserve(n);
        dfs(root, 0);

        int q = query_nodes.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            int u = query_nodes[i];
            queries[i].l = in[u];
            queries[i].r = out[u];
            queries[i].idx = i;
```

```
            queries[i].bnd = queries[i].l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(flat[--cur_l]);
            while (cur_r < qry.r) add(flat[++cur_r]);
            while (cur_l < qry.l) remove(flat[cur_l++]);
            while (cur_r > qry.r) remove(flat[cur_r--]);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

### 2.6.3 MoOnPaths

MO'S ON TREE PATHS - the Euler tour trick where a vertex appearing TWICE in the range cancels out,

```
// plus the LCA added back by hand. Answers path queries offline
     in O((n + q) sqrt n) with no HLD.
class MoTreePath {
private:
    int n, LOG, block_size;
    vector<vector<int>> adj, anc;
    vector<int> depth, in, out, flat;
    vector<long long> up, a;
    vector<bool> exist;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<deque<int>> dep; // Tracks nodes by depth

    void add(int idx) {
        if (!dep[depth[idx]].empty()) {
            ans += 1ll * a[dep[depth[idx]].back()] * a[idx];
        }
        dep[depth[idx]].push_back(idx);
    }

    void remove(int idx) {
        if (dep[depth[idx]].size() == 2) {
            ans -= a[*dep[depth[idx]].begin()] * a[*next(dep[
     depth[idx]].begin())];
        }
        if (dep[depth[idx]].front() == idx) {
            dep[depth[idx]].pop_front();
        } else {
            dep[depth[idx]].pop_back();
        }
    }

    void update(int idx) {
        int node = flat[idx];
        if (exist[node]) {
            remove(node);
        } else {
            add(node);
        }
        exist[node] = !exist[node];
    }

    inline long long get_answer() const {
        return ans;
    }
    // ------------------------------

    void dfs(int u, int p = -1) {
        if (p != -1) {
            anc[u][0] = p;
            up[u] = a[u] * a[u] + up[p];
        } else {
            depth[u] = 0;
            up[u] = a[u] * a[u];
        }
        in[u] = flat.size();
        flat.push_back(u);

        for (int v : adj[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }

        out[u] = flat.size();
        flat.push_back(u); // Push again for Eulerian tour
    }

    void build_lca() {
        for (int k = 1; k < LOG; k++) {
            for (int i = 1; i <= n; i++) {
                anc[i][k] = anc[anc[i][k - 1]][k - 1];
            }
        }
    }

    int lift(int x, int k) {
        for (int bit = 0; bit < LOG; bit++) {
            if ((1 << bit) & k) x = anc[x][bit];
        }
        return x;
    }

    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
```

```
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int bit = LOG - 1; bit >= 0; bit--) {
            if (anc[u][bit] != anc[v][bit]) {
                u = anc[u][bit];
                v = anc[v][bit];
            }
        }
        return anc[u][0];
    }

public:
    struct Query {
        int l, r, idx, x = -1;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    MoTreePath(int n_nodes, const vector<long long>& node_values)
        : n(n_nodes), a(node_values), LOG(log2(n_nodes) + 2) {

        adj.assign(n + 1, vector<int>());
        anc.assign(n + 1, vector<int>(LOG, 0));
        depth.assign(n + 1, 0);
        in.assign(n + 1, 0);
        out.assign(n + 1, 0);
        up.assign(n + 1, 0);
        exist.assign(n + 1, false);
        dep.assign(n + 1, deque<int>());
        ans = 0;
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // PRIMARY ENTRY POINT: give it the paths as vertex pairs and
     it builds the queries itself.
    // (in[], out[] and lca() are only valid AFTER the dfs, so you
      cannot build them yourself.)
    vector<long long> solve(int root, const vector<pair<int, int
     >>& paths) {
        flat.clear();
        dfs(root);
        build_lca();
        vector<Query> queries;
        for (int i = 0; i < (int)paths.size(); i++) {
            int u = paths[i].first, v = paths[i].second;
            if (in[u] > in[v]) swap(u, v);
            int w = lca(u, v);
            // u inside v's subtree -> plain range; otherwise use
     out[u]..in[v] and add back the LCA
            if (w == u) queries.push_back({in[u], in[v], i, -1,
     0});
            else queries.push_back({out[u], in[v], i, w, 0});
        }
        return solveRanges(queries);
    }
    vector<long long> solveRanges(vector<Query>& queries) {
        int q = queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(flat.size() / sqrt(q)));

        for (auto& qry : queries) {
            qry.bnd = qry.l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) update(--cur_l);       // GROW
     first, then shrink - the other
            while (cur_r < qry.r) update(++cur_r);       // order
      can pass through an invalid window
            while (cur_l < qry.l) update(cur_l++);
            while (cur_r > qry.r) update(cur_r--);

            answers[qry.idx] = get_answer() + (qry.x != -1 ? up[
     qry.x] : 0);
        }
        return answers;
    }
};
```

### 2.6.4 BlockDecomposition

SQRT / BLOCK DECOMPOSITION. Split [0,n) into blocks of B ~ sqrt(n) and keep one aggregate per

```
// block. Point update O(1..B), range query O(sqrt n). A BIT/
     segtree beats it for sum, so reach for
// this when the aggregate is NOT a group / has no lazy tag: block
     -sorted arrays, block frequency
// tables, "count values > x in [l,r]", range-assign + weird query
     , or a per-block rebuild that is
// cheap. General shape below with sum; swap `agg` and the two
     combine lines for anything else.
struct Blocks {
    int n, B;
    vector<ll> a, agg;                                          //
      agg[b] = aggregate of block b
    Blocks(const vector<ll>& v) : n(v.size()), B(max(1, (int)sqrt
     (n) + 1)), a(v),
                                  agg((n + B - 1) / B, 0) {
        for (int i = 0; i < n; i++) agg[i / B] += a[i];
    }
    void set(int i, ll v) { agg[i / B] += v - a[i], a[i] = v; }
     // O(1); for non-group aggregates
                                                                //
      rebuild the block instead, O(B)
    ll query(int l, int r) {                                    //
      inclusive, O(n/B + B)
        ll s = 0;
        int bl = l / B, br = r / B;
        if (bl == br) { for (int i = l; i <= r; i++) s += a[i];
     return s; }
        for (int i = l; i < (bl + 1) * B; i++) s += a[i];
        for (int b = bl + 1; b < br; b++) s += agg[b];
        for (int i = br * B; i <= r; i++) s += a[i];
        return s;
    }
};
// RANGE UPDATE, RANGE QUERY without lazy propagation: keep add[b]
     per block. A partial block is
// updated element-wise and its aggregate rebuilt; a full block
     only bumps add[b]. Query adds
// add[b] * (elements taken from b).
// BLOCK-SORTED ARRAY ("sqrt tree"): store each block also as a
     sorted copy. Then
//   count of values <= x in [l,r] = partial blocks scanned +
     binary search inside full blocks,
//   O(sqrt(n) log n) per query, O(B log B) per point update (re-
     sort one block). This is the
//   offline-free alternative to a wavelet tree / merge-sort tree
     when updates are present.
// REBUILD TRICK: when an operation is expensive but rare (e.g.
     inserting into the middle), buffer
// changes and rebuild the whole decomposition every sqrt(q)
     operations -> O((n+q) sqrt q) total.
// CHOOSING B: query cost n/B + B*c is minimised at B = sqrt(n*c)
     where c is the per-element cost;
// if updates are much more frequent than queries, take bigger
     blocks, and vice versa.
```

### 2.6.5 MoWithUpdates

MO'S WITH UPDATES (3D Mo). Queries {l, r, t} where t = #updates applied before this query.

```
// Block size n^(2/3) gives O(n^(5/3)) total. Use when the array changes between queries and no
// online structure fits.
struct MoUpd {
    struct Q { int l, r, t, idx; };
    struct U { int pos, oldV, newV; };

    int n, B;
    vector<int> a, rec;                                  // a = state at time 0; rec = while recording
    vector<Q> qs;
    vector<U> us;

    // ---- problem state: #distinct in the window ----
    vector<int> cnt;
    int cur = 0;
    void add(int v) { if (cnt[v]++ == 0) cur++; }
    void rem(int v) { if (--cnt[v] == 0) cur--; }
    int answer() const { return cur; }
    // ------------------------------------------------

    MoUpd(vector<int> arr, int maxVal) : n(arr.size()), a(arr), rec(arr), cnt(maxVal + 1, 0) {
        B = max(1, (int)pow((double)max(n, 1), 2.0 / 3));
    }
    void addQuery(int l, int r) { qs.push_back({l, r, (int)us.size(), (int)qs.size()}); }
    void addUpdate(int pos, int newV) { us.push_back({pos, rec[pos], newV}); rec[pos] = newV; }

    void applyU(int i, int l, int r) {                   // step over update i, forwards or backwards
        auto& u = us[i];
        if (l <= u.pos && u.pos <= r) rem(a[u.pos]), add(u.newV);
        a[u.pos] = u.newV;
        swap(u.oldV, u.newV);                             // makes the operation its own inverse
    }
    vector<int> solve() {
        sort(all(qs), [&](const Q& x, const Q& y) {
            int bx = x.l / B, by = y.l / B;
            if (bx != by) return bx < by;
            int cx = x.r / B, cy = y.r / B;
            if (cx != cy) return (bx & 1) ? cx > cy : cx < cy;
            return (cx & 1) ? x.t > y.t : x.t < y.t;
        });
        vector<int> res(qs.size());
        int L = 0, R = -1, T = 0;
        for (auto& q : qs) {
            while (T < q.t) applyU(T++, L, R);
            while (T > q.t) applyU(--T, L, R);
            while (R < q.r) add(a[++R]);
            while (L > q.l) add(a[--L]);
            while (R > q.r) rem(a[R--]);
            while (L < q.l) rem(a[L++]);
            res[q.idx] = answer();
        }
        return res;
    }
};
// NOTE: `a` stays at time 0; `rec` tracks values while you record updates, so applyU() can
// step forwards AND backwards through time using the same swap-based inverse.
```

### 2.6.6 MoWithRollback

MO'S WITH ROLLBACK - for structures where ADD is easy but REMOVE is hard or impossible

```
// (running maximum, DSU, "best pair so far", knapsack-ish state).
// Per block of left endpoints: sort by r, keep a persistent right part that only grows, and
// rebuild the short left part from scratch for every query, rolling back afterwards.
// O((n + q) sqrt n) adds, ZERO removes.
struct MoRollback {
    struct Q { int l, r, idx; };
    int n, B;
    vector<ll> a;
    vector<Q> qs;

    // ---- problem state: max (value * count) seen; only add() and a snapshot/rollback ----
    vector<int> cnt;
    ll best = 0;
    vector<pair<int, ll>> hist;                          // {value index, previous best}
    void add(int v, bool rec) {
        if (rec) hist.push_back({v, best});
        cnt[v]++;
        best = max(best, (ll)cnt[v] * v);
    }
    void rollback(size_t to) {
        while (hist.size() > to) {
            auto [v, b] = hist.back(); hist.pop_back();
            cnt[v]--, best = b;
        }
    }
    void clearAll() { fill(all(cnt), 0), best = 0, hist.clear(); }
    ll answer() const { return best; }
    //
    ------------------------------------------------------------------------


    MoRollback(vector<ll> arr, int maxVal) : n(arr.size()), a(arr), cnt(maxVal + 1, 0) {
        B = max(1, (int)sqrtl((ld)max(1, n)));
    }
    void addQuery(int l, int r) { qs.push_back({l, r, (int)qs.size()}); }

    vector<ll> solve() {
        sort(all(qs), [&](const Q& x, const Q& y) {
            int bx = x.l / B, by = y.l / B;
            return bx != by ? bx < by : x.r < y.r;
        });
        vector<ll> res(qs.size());
        size_t i = 0;
        while (i < qs.size()) {
            int b = qs[i].l / B, rEnd = min(n - 1, (b + 1) * B - 1);
            size_t j = i;
            while (j < qs.size() && qs[j].l / B == b) j++;
            // pass 1: queries contained in this block - answer each from a CLEAN state
            for (size_t k2 = i; k2 < j; k2++) if (qs[k2].r <= rEnd) {
                clearAll();
                for (int k = qs[k2].l; k <= qs[k2].r; k++) add(a[k], false);
                res[qs[k2].idx] = answer();
            }
            // pass 2: long queries - right part grows monotonically, left part is rolled back
            clearAll();
            int R = rEnd;
            for (size_t k2 = i; k2 < j; k2++) if (qs[k2].r > rEnd) {
                while (R < qs[k2].r) add(a[++R], false);
                // grow-only, not recorded
                size_t snap = hist.size();
                for (int k = qs[k2].l; k <= rEnd; k++) add(a[k], true);   // short left part
                res[qs[k2].idx] = answer();
                rollback(snap);
            }
            i = j;
        }
        return res;
    }
};
```

## 2.7 Balanced Binary Search Tree (BBST)

### 2.7.1 Treap

TREAP (keyed) - a BST by key and a heap by random priority, so it is balanced in expectation.

```
// NOTE: 01 - Miscellaneous/04 - random.cpp declares the same 'rng'. Keep one copy.
// O(log n) insert/erase/find, plus split and merge, which is what makes it better than std::set:
// you can cut the tree at a key, operate on a whole range, and glue it back.
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template <typename T>
struct Treap {
    struct Node {
        T key, val, sum;
        T lazy = 0; // 1. Added lazy tag
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T k, T v = T()) : key(k), val(v), sum(v) {}
    }* root = nullptr;

    ~Treap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // 2. The Push Function: Propagates pending updates to children
    void push(Node* t) {
        if (!t || t->lazy == 0) return;

        if (t->l) {
            t->l->key += t->lazy; // Shift the key (critical for the DP solution)
            t->l->val += t->lazy; // Shift the value
            t->l->sum += t->lazy * sz(t->l); // Update subtree sum
            t->l->lazy += t->lazy; // Pass the tag down
        }
        if (t->r) {
            t->r->key += t->lazy;
            t->r->val += t->lazy;
            t->r->sum += t->lazy * sz(t->r);
            t->r->lazy += t->lazy;
        }
        t->lazy = 0; // Clear the tag on the current node
    }

    Node* update(Node* t) {
        if (t) {
            t->sz = 1 + sz(t->l) + sz(t->r);
            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
        }
        return t;
    }

    Node* merge(Node* l, Node* r) {
        push(l); // 3. Push before making routing decisions
        push(r);
        if (!l || !r) return l ? l : r;
        if (l->p > r->p) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }

    pair<Node*, Node*> split(Node* t, T k) {
        if (!t) return {nullptr, nullptr};
        push(t); // 3. Push before evaluating keys for splits
        if (t->key <= k) {
            auto [l, r] = split(t->r, k);
            t->r = l;
            return {update(t), r};
        }
        auto [l, r] = split(t->l, k);
        t->l = r;
        return {l, update(t)};
    }

    // --- Core Operations ---

    // Range Update Method
    void add_range(T l_val, T r_val, T add_val) {
        if (l_val > r_val) return;

        auto [left_mid, right] = split(root, r_val);
        auto [left, mid] = split(left_mid, l_val - 1);

        // Apply the lazy tag to the entire isolated segment
        if (mid) {
            mid->key += add_val;
            mid->val += add_val;
            mid->sum += add_val * sz(mid);
            mid->lazy += add_val;
        }

        root = merge(merge(left, mid), right);
    }

    bool search(T x) {
        Node* curr = root;
        while (curr) {
            push(curr); // Push during traversal
            if (curr->key == x) return true;
            curr = (x < curr->key) ? curr->l : curr->r;
        }
        return false;
    }

    void insert(T x, T val = T()) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        if (!r2) {
            r2 = new Node(x, val);
        } else {
            push(r2); // Push before modifying
            r2->val += val;
            update(r2);
        }
        root = merge(merge(l2, r2), r);
    }

    void erase(T x) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        destroy(r2);
        root = merge(l2, r);
    }

    // --- Utility Operations ---

    Node* find_by_order(Node* t, int k) {
        if (!t) return nullptr;
        push(t); // Push before accessing children sizes
        int left_sz = sz(t->l);
        if (k < left_sz) return find_by_order(t->l, k);
        if (k == left_sz) return t;
        return find_by_order(t->r, k - left_sz - 1);
    }
    Node* find_by_order(int k) { return find_by_order(root, k); }

    int order_of_key(Node* t, T k) {
        if (!t) return 0;
        push(t); // Push before comparing keys
        if (t->key >= k) return order_of_key(t->l, k);
        return sz(t->l) + 1 + order_of_key(t->r, k);
    }
    int order_of_key(T k) { return order_of_key(root, k); }

    void print(Node* t) {
        if (!t) return;
        push(t); // Push before printing to get true values
        print(t->l);
        cout << t->key << " ";
        print(t->r);
    }
    void print() {
        print(root);
        cout << '\n';
    }
};
```

### 2.7.2 ImplicitTreap

IMPLICIT TREAP - a treap keyed by POSITION (subtree size) instead of value, i.e. an array that

```
// NOTE: 01 - Miscellaneous/04 - random.cpp declares the same 'rng'. Keep one copy.
// supports insert/erase in the middle, reverse a range, and move a block, all in O(log n).
// This is the structure for "cut [l,r] out and paste it elsewhere" problems.
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template <typename T>
struct ImplicitTreap {
    struct Node {
        T val, sum;
        T lazy = 0;       // Lazy tag for range additions
        bool rev = false; // Lazy tag for range reversals
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T v = T()) : val(v), sum(v) {}
    }* root = nullptr;

    // Default Constructor
    ImplicitTreap() = default;

    // Vector Constructor - Builds treap in O(N) time
    ImplicitTreap(const vector<T>& a) {
        build(a);
    }

    ~ImplicitTreap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // Propagates pending lazy updates to children
    void push(Node* t) {
        if (!t) return;

        // 1. Process pending reversal
        if (t->rev) {
            swap(t->l, t->r);
            if (t->l) t->l->rev ^= true;
            if (t->r) t->r->rev ^= true;
            t->rev = false;
        }

        // 2. Process pending value addition
        if (t->lazy != 0) {
            if (t->l) {
                t->l->val += t->lazy;
                t->l->sum += t->lazy * sz(t->l);
                t->l->lazy += t->lazy;
            }
            if (t->r) {
                t->r->val += t->lazy;
                t->r->sum += t->lazy * sz(t->r);
                t->r->lazy += t->lazy;
            }
            t->lazy = 0; // Clear the tag
        }
    }

    Node* update(Node* t) {
        if (t) {
            t->sz = 1 + sz(t->l) + sz(t->r);
            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
        }
        return t;
    }

    // --- O(N) Linear Time Build ---
    void build(const vector<T>& a) {
        destroy(root);
        root = nullptr;
        if (a.empty()) return;

        vector<Node*> st;
        for (const T& val : a) {
            Node* curr = new Node(val);
            Node* last = nullptr;

            // Maintain max-heap property on random priority 'p'
            while (!st.empty() && st.back()->p < curr->p) {
                last = st.back();
                st.pop_back();
            }

            curr->l = last;
            if (!st.empty()) {
                st.back()->r = curr;
            }
            st.push_back(curr);
        }

        // Post-order pass to compute subtree sizes and sums in O(N)
        auto recompute = [&](auto& self, Node* t) -> void {
            if (!t) return;
            self(self, t->l);
            self(self, t->r);
            update(t);
        };

        root = st.front(); // Bottom of stack is the root
        recompute(recompute, root);
    }

    // --- Dynamic Array Operations ---

    Node* merge(Node* l, Node* r) {
        push(l);
        push(r);
        if (!l || !r) return l ? l : r;
        if (l->p > r->p) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }

    pair<Node*, Node*> split(Node* t, int k) {
        if (!t) return {nullptr, nullptr};
        push(t);

        int left_sz = sz(t->l);
        if (left_sz < k) {
            auto [l, r] = split(t->r, k - left_sz - 1);
            t->r = l;
            return {update(t), r};
        } else {
            auto [l, r] = split(t->l, k);
            t->l = r;
            return {l, update(t)};
        }
    }

    int size() { return sz(root); }

    void insert(int idx, T val) {
        auto [l, r] = split(root, idx);
        Node* node = new Node(val);
        root = merge(merge(l, node), r);
    }

    void erase(int idx) {
        if (idx < 0 || idx >= sz(root)) return;
        auto [left_mid, right] = split(root, idx + 1);
        auto [left, mid] = split(left_mid, idx);
        destroy(mid);
        root = merge(left, right);
    }

    void reverse_range(int l, int r) {
        if (l >= r || l < 0 || r >= sz(root)) return;

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        if (mid) mid->rev ^= true;

        root = merge(merge(left, mid), right);
    }

    void add_range(int l, int r, T add_val) {
        if (l > r || l < 0 || r >= sz(root)) return;
```

```
        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        if (mid) {
            mid->val += add_val;
            mid->sum += add_val * sz(mid);
            mid->lazy += add_val;
        }

        root = merge(merge(left, mid), right);
    }

    T query_range(int l, int r) {
        if (l > r || l < 0 || r >= sz(root)) return T();

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        T ans = sub_val(mid);

        root = merge(merge(left, mid), right);
        return ans;
    }

    T get(int idx) {
        return query_range(idx, idx);
    }

    void print(Node* t) {
        if (!t) return;
        push(t);
        print(t->l);
        cout << t->val << " ";
        print(t->r);
    }

    void print() {
        print(root);
        cout << '\n';
    }
};
```

## 2.8 Policy Based Data Structures

### 2.8.1 OrderedSet

Policy-Based Data Structure: ordered_set

```
// Time: O(log N) per operation
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
     tree_order_statistics_node_update>;

// Usage:
// st.find_by_order(k): Iterator to k-th element (0-based)
// st.order_of_key(x): Number of elements strictly smaller than x
```

## 2.9 Monotonic Data Structures

### 2.9.1 MonotonicStack

Monotonic Stack - Time: O(N), Space: O(N)

```
// Computes Next/Previous Smaller and Greater element indices for
     all positions
template<typename T = ll>
struct MonotonicStack {
    int n;
    vector<T> a;
    vector<int> prev_less, next_less;
    vector<int> prev_greater, next_greater;

    MonotonicStack(const vector<T>& arr) : n(arr.size()), a(arr),
        prev_less(n, -1), next_less(n, n),
        prev_greater(n, -1), next_greater(n, n) {

        vector<int> st;
        // Next Less & Previous Less
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] >= a[i]) {
                next_less[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_less[i] = st.back();
            st.pb(i);
        }

        st.clear();
        // Next Greater & Previous Greater
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] <= a[i]) {
                next_greater[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_greater[i] = st.back();
            st.pb(i);
        }
    }
};
```

### 2.9.2 TwoStackQueue

Monotonic Queue via Two Stacks (Sliding Window Min/Max)

```cpp
// Space: O(N), Time: O(1) amortized per push/pop/get
struct TwoStackQ {
    struct Node {
        ll mx = -llinf, mn = llinf, val;

        Node() : val(0) {}
        Node(ll x) : mx(x), mn(x), val(x) {}
    };

    stack<Node> a, b;

    int size() {
        return a.size() + b.size();
    }

    void mrg(Node &a, Node &b) {
        a.mn = min(a.mn, b.mn);
        a.mx = max(a.mx, b.mx);
    }

    // Pushes value into the queue
    void push(ll val) {
        auto nd = Node(val);
        if (!a.empty()) mrg(nd, a.top());
        a.push(nd);
    }

    // Transfers elements from stack a to stack b when b is empty
    void move() {
        while (!a.empty()) {
            auto nd = Node(a.top().val);
            if (!b.empty()) mrg(nd, b.top());
            b.push(nd);
            a.pop();
        }
    }

    // Returns min/max of all current elements in the queue in O
     (1)
    Node get() {
        Node res;
        if (!b.empty()) mrg(res, b.top());
        if (!a.empty()) mrg(res, a.top());
        return res;
    }

    // Removes the oldest element from the queue
    void pop() {
        if (b.empty()) move();
        if (!b.empty()) b.pop();
    }
};
```

### 2.9.3 MonotonicQueue2D

2D Monotonic Queue (Sliding Window 2D Min/Max) - Time: O(N * M), Space: O(N * M)

```cpp
// Computes min in every K x L subgrid of an N x M matrix
template<typename T = ll>
struct MonotonicQueue2D {
    int n, m;

    // Returns a matrix res where res[i][j] is the min in subgrid
     [i..i+K-1][j..j+L-1]
    vector<vector<T>> sliding_window_2d(const vector<vector<T>>&
     grid, int K, int L) {
        n = grid.size();
        m = grid[0].size();

        // 1. Sliding window min horizontally along each row
        vector<vector<T>> row_min(n, vector<T>(m - L + 1));
        for (int i = 0; i < n; i++) {
            deque<int> dq;
            for (int j = 0; j < m; j++) {
                while (!dq.empty() && grid[i][dq.back()] >= grid[
    i][j]) dq.pop_back();
                dq.pb(j);
                if (dq.front() <= j - L) dq.pop_front();
                if (j >= L - 1) row_min[i][j - L + 1] = grid[i][
    dq.front()];
            }
        }

        // 2. Sliding window min vertically down each column of
    row_min
        vector<vector<T>> res(n - K + 1, vector<T>(m - L + 1));
        for (int j = 0; j < m - L + 1; j++) {
            deque<int> dq;
            for (int i = 0; i < n; i++) {
                while (!dq.empty() && row_min[dq.back()][j] >=
    row_min[i][j]) dq.pop_back();
                dq.pb(i);
                if (dq.front() <= i - K) dq.pop_front();
                if (i >= K - 1) res[i - K + 1][j] = row_min[dq.
    front()][j];
            }
        }

        return res;
    }
};
```

## 2.10 DynamicMedian

RUNNING MEDIAN / k-th SMALLEST under insertions, via two multisets kept balanced around the split.

```
// O(log n) per insert, O(1) to read the median and either side's
     sum. The pattern generalises to
// "maintain the sum of the k smallest" - which is what cost-to-
     move-to-median problems need.
multiset<ll> L, R;
ll sum_L = 0, sum_R = 0;
int k;

// Rebalances the multisets so L always has the lower half/median
     of CURRENT elements
void balance() {
    int total_size = sz(L) + sz(R);
    int k_L = (total_size + 1) / 2; // Fixed: Target size based on
      actual current elements

    // If L is too big, move its largest element to R
    while (sz(L) > k_L) {
        auto it = prev(L.end());
        ll val = *it;

        sum_L -= val;
        sum_R += val;

        R.insert(val);
        L.erase(it);
    }

    // If L is too small, move the smallest element of R to L
    while (sz(L) < k_L) {
        auto it = R.begin();
        ll val = *it;

        sum_R -= val;
        sum_L += val;

        L.insert(val);
        R.erase(it);
    }
}

void add_val(ll val) {
    if (L.empty() || val <= *L.rbegin()) {
        L.insert(val);
        sum_L += val;
    } else {
        R.insert(val);
        sum_R += val;
    }
    balance();
}

void remove_val(ll val) {
    auto it = L.find(val);
    if (it != L.end()) {
        sum_L -= val;
        L.erase(it);
    } else {
        it = R.find(val);
        if (it == R.end()) return;                    // not
     present in either half: do nothing
        sum_R -= val;
        R.erase(it);
    }
    balance();
}

ll get_cost() {                                  // UB if empty -
      guard the caller
    ll median = *L.rbegin();
    return sum_R - sum_L + median * (sz(L) - sz(R));
}
```

## 2.11 IntervalSetODT

INTERVAL SET / "Chtholly tree" (ODT). Keeps the array as maximal runs of equal value.

```
// assign(l, r, v) is amortised near-O(log n) WHEN the data has
     many range-assigns (random or
// adversarial-but-assign-heavy); each assign destroys the
     intervals it covers, so the total
// number of intervals created is O(q).
// THIS is the standard way to support range bitwise-AND / OR /
     MOD / assign together with
// range sum, because those operations rapidly make long stretches
      constant.
struct ODT {
    map<int, pair<int, ll>> mp;                       // l -> {r,
     value}, covering [l, r]

    ODT(int n, ll v = 0) { mp[0] = {n - 1, v}; }

    // split so that an interval starts exactly at p; returns the
     iterator to it
    auto split(int p) {
        auto it = prev(mp.upper_bound(p));
        if (it->first == p) return it;
        auto [r, v] = it->second;
        it->second.first = p - 1;
        return mp.emplace(p, make_pair(r, v)).first;
    }
    void assign(int l, int r, ll v) {
        auto itr = split(r + 1), itl = split(l);
        mp.erase(itl, itr);
        mp[l] = {r, v};
    }
    // Generic scan over [l, r]: f(l, r, value) is called once per
      run. O(#runs touched).
    template <class F> void forEach(int l, int r, F f) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it) f(it->first, it->
     second.first, it->second.second);
    }
    ll sum(int l, int r) {
        ll s = 0;
        forEach(l, r, [&](int a, int b, ll v) { s += v * (b - a +
      1); });
        return s;
    }
    // RANGE ADD: touch each run once. Runs are not merged, so
     adds alone do NOT keep the
    // amortised bound - this is only cheap when assigns keep
     collapsing the structure.
    void add(int l, int r, ll x) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it) it->second.second +=
      x;
    }
    // RANGE AND / OR / MOD follow the same loop, applying the op
     to each run's value.
    // They are fast because every value can only decrease O(log V
     ) times before it stops
    // changing, at which point neighbouring runs become equal and
      can be merged.
    template <class Op> void apply(int l, int r, Op op) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it) it->second.second =
     op(it->second.second);
        merge(l, r);
    }
    void merge(int l, int r) {                        // glue
     neighbouring runs of equal value
        auto it = split(l);
        while (it != mp.end() && it->first <= r) {
            auto nx = next(it);
            if (nx != mp.end() && nx->second.second == it->second
     .second &&
                it->second.first + 1 == nx->first) {
                it->second.first = nx->second.first;
                mp.erase(nx);
            } else it = nx;
        }
    }
};
// Careful: worst case WITHOUT assigns is O(n) per operation. If
     the problem has only adds and
// queries, use a segment tree instead.
```

## 2.12 DisjointSparseTable

DISJOINT SPARSE TABLE - O(1) query for ANY associative operation (not just idempotent ones).

```
// Build O(n log n) time and memory. Use when a plain sparse table would double-count:
// sum, product, matrix product, hashing, "merge two structs", min-count pairs, ...
// (For min/max/gcd/or/and a normal sparse table is smaller - use that instead.)
template <typename T, class F = function<T(T, T)>>
struct DisjointST {
    int n, LOG;
    vector<vector<T>> t;
    F op;

    DisjointST(const vector<T>& a, F op) : n(a.size()), op(op) {
        LOG = 1;
        while ((1 << LOG) < n) LOG++;
        t.assign(LOG + 1, a);
        for (int lvl = 1; lvl <= LOG; lvl++) {
            int half = 1 << lvl;
            for (int mid = half; mid < n + half; mid += 2 * half) {
                if (mid - 1 < n) {                                  // build leftwards from the split
                    t[lvl][mid - 1] = a[mid - 1];
                    for (int i = mid - 2; i >= mid - half && i >= 0; i--)
                        t[lvl][i] = op(a[i], t[lvl][i + 1]);
                }
                if (mid < n) {                                      // and rightwards
                    t[lvl][mid] = a[mid];
                    for (int i = mid + 1; i < min(n, mid + half); i++)
                        t[lvl][i] = op(t[lvl][i - 1], a[i]);
                }
            }
        }
    }
    T query(int l, int r) const {                                   // inclusive [l, r]
        if (l == r) return t[0][l];
        int lvl = 63 - __builtin_clzll((unsigned long long)(l ^ r));   // highest differing bit
        return op(t[lvl][l], t[lvl][r]);
    }
};
// Every query splits at the unique power-of-two boundary between l and r, and both halves are
// already prefix/suffix-accumulated at that level - hence exactly one op() per query.
```

## 2.13 WaveletTree

WAVELET TREE over values in [lo, hi]. Build O(n log V).

```
// kth(l, r, k)      : k-th SMALLEST in a[l..r]
        O(log V)
// countLeq(l,r,x)   : how many a[i] <= x in a[l..r]
        O(log V)
// countEq(l,r,x)    : how many a[i] == x in a[l..r]
        O(log V)
// Online (no offline sorting, no persistence) - the practical alternative to a merge-sort tree.
struct Wavelet {
    int lo, hi;
    Wavelet *l = nullptr, *r = nullptr;
    vector<int> b;                                                  // b[i] = # elements going left among first i

    // build on a[from, to) - the array IS reordered in place (stable partition by midpoint)
    Wavelet(vector<int>::iterator from, vector<int>::iterator to,
      int x, int y) : lo(x), hi(y) {
        if (from >= to) return;
        if (lo == hi) { b.assign(to - from + 1, 0); return; }
        int mid = lo + (hi - lo) / 2;
        auto f = [mid](int v) { return v <= mid; };
        b.reserve(to - from + 1);
        b.push_back(0);
        for (auto it = from; it != to; ++it) b.push_back(b.back() + f(*it));
        auto pivot = stable_partition(from, to, f);
        l = new Wavelet(from, pivot, lo, mid);
        r = new Wavelet(pivot, to, mid + 1, hi);
    }
    int kth(int L, int R, int k) {                                  // 1-indexed k, inclusive [L, R] (1-based)
        if (L > R) return 0;
        if (lo == hi) return lo;
        int inLeft = b[R] - b[L - 1], lb = b[L - 1], rb = b[R];
        if (k <= inLeft) return l->kth(lb + 1, rb, k);
        return r->kth(L - lb, R - rb, k - inLeft);
    }
    int countLeq(int L, int R, int k) {                             // #{i in [L,R] : a[i] <= k}
        if (L > R || k < lo) return 0;
        if (hi <= k) return R - L + 1;
        int lb = b[L - 1], rb = b[R];
        return l->countLeq(lb + 1, rb, k) + r->countLeq(L - lb, R - rb, k);
    }
    int countEq(int L, int R, int k) {
        if (L > R || k < lo || k > hi) return 0;
        if (lo == hi) return R - L + 1;
        int lb = b[L - 1], rb = b[R], mid = lo + (hi - lo) / 2;
        if (k <= mid) return l->countEq(lb + 1, rb, k);
        return r->countEq(L - lb, R - rb, k);
    }
    ~Wavelet() { delete l; delete r; }
};
// Usage: compress values to [0, V), copy the array, then Wavelet w(a.begin(), a.end(), 0, V-1);
// queries are 1-BASED on the original positions.
```

## 2.14 OfflineDynamicConnectivity

OFFLINE DYNAMIC CONNECTIVITY - edges appear and disappear over time; answer queries at each

```
// Needs: DSURollback (02 - DSU/02 - DSURollback.cpp).
// moment. Segment tree over the TIMELINE + DSU with rollback. O(q
      log q * log n) - DSURollback has no path compression, so
     find is log n, not alpha.
// The same skeleton answers ANY "structure supports add +
     rollback, but not delete" problem:
// put each element on the time interval where it is alive, DFS
     the segment tree, add on entry,
// roll back on exit.  ("Deleting from a data structure in O(T log
      n)")
struct DynConn {
    int n, T;
    vector<vector<pair<int, int>>> seg;              // edges
     alive on each timeline node
    DSURollback d;
    vector<int> comps;                               // comps[t]
     = #components at time t

    DynConn(int n, int T) : n(n), T(T), seg(4 * max(1, T)), d(n),
      comps(T, 0) {}

    void addSeg(int v, int l, int r, int ql, int qr, pair<int,
     int> e) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { seg[v].push_back(e); return; }
        int m = (l + r) / 2;
        addSeg(2 * v, l, m, ql, qr, e), addSeg(2 * v + 1, m + 1,
     r, ql, qr, e);
    }
    // edge (u,v) exists during times [tl, tr] (inclusive)
    void addEdge(int u, int v, int tl, int tr) { if (tl <= tr)
     addSeg(1, 0, T - 1, tl, tr, {u, v}); }

    void dfs(int v, int l, int r, int cur) {
        int snap = d.get_state();
        for (auto [a, b] : seg[v]) cur -= d.join(a, b);
        if (l == r) comps[l] = cur;
        else {
            int m = (l + r) / 2;
            dfs(2 * v, l, m, cur), dfs(2 * v + 1, m + 1, r, cur);
        }
        d.rollback_to(snap);
    }
    void solve() { if (T) dfs(1, 0, T - 1, n); }
};
// USAGE: collect every edge's [birth, death] interval first (an
     edge that is never removed dies
// at T-1), then one solve() fills comps[]. To answer "are u and v
      connected at time t", instead
// record the queries per leaf and test d.same(u, v) inside the l
     == r branch.
```

## 2.15 CartesianTree

CARTESIAN TREE in O(n): heap-ordered by VALUE, BST-ordered by INDEX.

```
// The root of any range is its minimum, so "range minimum"
     structure becomes a TREE -
// which turns range problems into subtree problems (and enables
     divide & conquer on the min).
// Built with a monotonic stack; ties go left so the tree is
     unique.
struct CartTree {
    int n, root = -1;
    vector<int> l, r, par;

    CartTree(const vector<ll>& a) : n(a.size()), l(n, -1), r(n,
     -1), par(n, -1) {
        vector<int> st;
        for (int i = 0; i < n; i++) {
            int last = -1;
            while (!st.empty() && a[st.back()] > a[i]) last = st.
     back(), st.pop_back();
            l[i] = last;
            if (last != -1) par[last] = i;
            if (!st.empty()) r[st.back()] = i, par[i] = st.back()
     ;
            st.push_back(i);
        }
        root = st[0];
    }
};
// USES
//  - "for every element, the maximal range where it is the
     minimum" = its subtree range,
//    which is the contribution technique (sum over subarrays of
     the min) in tree form
//  - largest rectangle in a histogram = best over nodes of value
     * subtreeSize
//  - divide & conquer "solve(l, r) splits at the minimum" without
      recomputing RMQ
//  - counting subarrays whose minimum is a given element
//  - a Treap built on (index, random priority) is exactly a
     Cartesian tree - same code shape
```

## 2.16 SegTreeMerging

SEGMENT TREE MERGING - one dynamic segment tree per vertex over the VALUE domain, merged

```
// bottom-up along a tree. Total O(n log n) nodes touched over ALL merges (amortised), because
// each merge destroys as many nodes as it visits.
// Solves, per subtree and in one DFS: most frequent value, k-th value, count of a value,
// "sum of values in a range" - i.e. everything a merge-sort tree gives, but online per subtree.
struct SegMerge {
    struct Node { int l = 0, r = 0; ll sum = 0; int best = 0; ll bestCnt = 0; };
    vector<Node> t;
    int V;                                          // value domain is [0, V)

    SegMerge(int V, int reserve = 0) : V(V) { t.reserve(reserve), t.push_back(Node()); }  // 0 = null
    int newNode() { t.push_back(Node()); return t.size() - 1; }

    void pull(int v) {
        int L = t[v].l, R = t[v].r;
        t[v].sum = t[L].sum + t[R].sum;
        if (t[L].bestCnt >= t[R].bestCnt) t[v].best = t[L].best, t[v].bestCnt = t[L].bestCnt;
        else t[v].best = t[R].best, t[v].bestCnt = t[R].bestCnt;
    }
    // Returns the (possibly new) node index. NEVER take an `int&` into t here: newNode() can
    // reallocate the vector and invalidate the reference mid-recursion.
    int update(int v, int lo, int hi, int p, ll d) {
        if (!v) v = newNode();
        if (hi - lo == 1) { t[v].sum += d, t[v].best = lo, t[v].bestCnt = t[v].sum; return v; }
        int m = (lo + hi) / 2;
        if (p < m) { int c = update(t[v].l, lo, m, p, d); t[v].l = c; }
        else { int c = update(t[v].r, m, hi, p, d); t[v].r = c; }
        pull(v);
        return v;
    }
    int merge(int a, int b, int lo, int hi) {           // destroys b; returns the merged root
        if (!a || !b) return a | b;
        if (hi - lo == 1) { t[a].sum += t[b].sum, t[a].bestCnt = t[a].sum; return a; }
        int m = (lo + hi) / 2;
        int L = merge(t[a].l, t[b].l, lo, m), R = merge(t[a].r, t[b].r, m, hi);
        t[a].l = L, t[a].r = R;
        pull(a);
        return a;
    }
    ll query(int v, int lo, int hi, int ql, int qr) {  // sum of counts in [ql, qr)
        if (!v || qr <= lo || hi <= ql) return 0;
        if (ql <= lo && hi <= qr) return t[v].sum;
        int m = (lo + hi) / 2;
        return query(t[v].l, lo, m, ql, qr) + query(t[v].r, m, hi, ql, qr);
    }
    int kth(int v, int lo, int hi, ll k) {              // k-th smallest (1-indexed), -1 if k too big
        if (!v || k > t[v].sum) return -1;
        if (hi - lo == 1) return lo;
        int m = (lo + hi) / 2;
        return k <= t[t[v].l].sum ? kth(t[v].l, lo, m, k) : kth(t[v].r, m, hi, k - t[t[v].l].sum);
    }
    // wrappers
    void update(int& root, int p, ll d) { root = update(root, 0, V, p, d); }
    int merge(int a, int b) { return merge(a, b, 0, V); }
    ll query(int root, int ql, int qr) { return query(root, 0, V, ql, qr); }
    int kth(int root, ll k) { return kth(root, 0, V, k); }
};
// TREE USAGE: root[v] starts with just v's own value; after recursing, merge every child's root
// into v's. Answer v's query right there. Never reuse a root after merging it away.
```

## 2.17 BitsetTricks

BITSET SPEEDUPS - the /64 family. All of these are "obviously too slow" DPs that fit anyway.

```
// TRANSITIVE CLOSURE (reachability) in O(n^3 / 64). n up to ~5000.
template <int N>
void transitiveClosure(vector<bitset<N>>& g, int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++) if (g[i][k]) g[i] |= g[k];
}
// LCS in O(n*m/64) - bit-parallel (Allison-Dix). `a` is the string whose length bounds the
// bitset width; row is a bitmask over positions of `a`.
//   x    = row | match[b_i]
//   row' = x & ((x - (row << 1 | 1)) ^ x)
// The "| 1" is the carry-in that makes the shift borrow correctly.
typedef unsigned long long u64;
int bitLCS(const string& a, const string& b) {
    int n = a.size(), W = (n + 63) / 64;
    if (!n || b.empty()) return 0;
    vector<vector<u64>> match(256, vector<u64>(W, 0));
    for (int i = 0; i < n; i++) match[(unsigned char)a[i]][i >> 6] |= 1ULL << (i & 63);
    vector<u64> row(W, 0), x(W), sh(W);
    for (char c : b) {
        const auto& M = match[(unsigned char)c];
        for (int i = 0; i < W; i++) x[i] = row[i] | M[i];
        u64 carry = 1;                                  // sh = (row << 1) | 1
        for (int i = 0; i < W; i++) { u64 nc = row[i] >> 63; sh[i] = (row[i] << 1) | carry; carry = nc; }
        u64 borrow = 0;                                 // row = x & ((x - sh) ^ x)
        for (int i = 0; i < W; i++) {
            u64 t = sh[i] + borrow;
            borrow = (t < sh[i]) || (x[i] < t);         // t < sh[i] catches the carry overflow
            u64 d = x[i] - t;
            row[i] = x[i] & (d ^ x[i]);
        }
    }
    int r = 0;
    for (u64 v : row) r += __builtin_popcountll(v);
    return r;
}

// SUBSET SUM / KNAPSACK FEASIBILITY: dp |= dp << w   (see KnapsackFamily.cpp)
// BIPARTITE MATCHING on dense graphs: keep each left vertex's neighbourhood as a bitset and
//   pick the next free right vertex with (adj[u] & free). _Find_first().
// COUNTING PAIRS with a property: bitset per value class, then popcount of ANDs.
// GCC EXTRAS: b._Find_first(), b._Find_next(i) iterate set bits in O(N/64) total.
// RULE OF THUMB: bitset turns 1e9 "simple" operations into ~1.5e7 word operations - budget
// about n*m/64 <= 1e8 for a comfortable pass.
```

## 2.18 VeniceAndSWAG

VENICE TECHNIQUE - a multiset with an O(1) "add x to EVERY element".

```
// Keep a global offset; store values shifted by it. 15 lines, and it turns a whole family of
// "everyone loses X health, remove the dead" problems into a one-liner.
struct Venice {
    multiset<ll> s;
    ll water = 0;                                   // global offset

    void add(ll v) { s.insert(v + water); }         // insert the true value v
    void erase(ll v) { auto it = s.find(v + water); if (it != s.end()) s.erase(it); }
    void addAll(ll v) { water -= v; }               // O(1): add v to every element
    ll getMin() const { return *s.begin() - water; }
    ll getMax() const { return *s.rbegin() - water; }
    int size() const { return s.size(); }
    // Pop everything whose TRUE value is <= t (e.g. everyone who died).
    template <class F> void popLE(ll t, F onPop) {
        while (!s.empty() && *s.begin() - water <= t) { onPop(*s.begin() - water); s.erase(s.begin()); }
    }
};
// SAME IDEA elsewhere: a binary trie with a global XOR mask ("xor every element by x"),
// a BIT with a global additive offset, a lazy "add to all" on a heap.

// SWAG (Sliding Window Aggregation) for ANY monoid - non-commutative and non-invertible ops
// included: matrix product, affine composition, gcd, "merge two structs".
// Amortised O(1) push/pop/fold. This generalises 09/02 - TwoStackQueue.cpp, which is hardwired
// to min/max. Prefix-product-and-divide does NOT work without an inverse - this does.
template <class T, class Op>
struct SWAG {
    vector<pair<T, T>> front_, back_;               // {value, aggregate from here to the bottom}
    T id;
    Op op;
    SWAG(T id, Op op) : id(id), op(op) {}

    T foldFront() const { return front_.empty() ? id : front_.back().second; }
    T foldBack() const { return back_.empty() ? id : back_.back().second; }
    T fold() const { return op(foldFront(), foldBack()); }
      // front first: order matters
    void push(T v) { back_.push_back({v, op(foldBack(), v)}); }
    void pop() {
        if (front_.empty())
            while (!back_.empty()) {
                T v = back_.back().first; back_.pop_back();
                front_.push_back({v, op(v, foldFront())});
      // reversed accumulation
            }
        if (!front_.empty()) front_.pop_back();
    }
    int size() const { return front_.size() + back_.size(); }
};
// USAGE: SWAG<ll, function<ll(ll,ll)>> q(0, [](ll a, ll b){ return a + b; });
// For non-commutative ops keep the argument order exactly as written above, or the window
// product comes out reversed.
```

## 2.19 MergeableHeap

MERGEABLE (MELDABLE) HEAP - a priority queue that also supports MERGING two heaps in O(log n),

```
// which std::priority_queue cannot do. This is a skew heap: merge by walking the two right spines
// and swapping children, which keeps the amortised depth logarithmic with no stored rank.
// Supports a lazy "add v to every key in this heap" tag, because that is what the two algorithms
// that need a mergeable heap actually want.
// USES: Chu-Liu/Edmonds directed MST in O(E log V) (each contracted cycle merges its in-edge
// heaps and subtracts the chosen edge's weight from the whole heap); Dijkstra-style k-shortest
// paths (Eppstein); "merge the two children's candidate sets" DP on trees; leftist-heap solutions
// to scheduling problems (repeatedly merge and pop the largest, e.g. minimise total lateness).
struct MHeap {
    struct Node { ll key, lz = 0; Node *l = 0, *r = 0; };
    static void down(Node* t) {                                  //
      apply the lazy tag downwards
        if (!t || !t->lz) return;
        t->key += t->lz;
        if (t->l) t->l->lz += t->lz;
        if (t->r) t->r->lz += t->lz;
        t->lz = 0;
    }
    static Node* merge(Node* a, Node* b) {                       //
      O(log n) amortised, MIN-heap
        if (!a || !b) return a ? a : b;
        down(a), down(b);
        if (a->key > b->key) swap(a, b);
        a->r = merge(a->r, b);
        swap(a->l, a->r);                                        //
      the "skew" step
        return a;
    }
    static Node* push(Node* t, ll key) { return merge(t, new Node{key}); }
    static ll top(Node* t) { return down(t), t->key; }
    static Node* pop(Node* t) { down(t); return merge(t->l, t->r); }
    static void addAll(Node* t, ll v) { if (t) t->lz += v; }     //
      O(1) add to every key
};
// TO GET A MAX-HEAP: negate the keys, or flip the comparison in merge.
// LEFTIST HEAP is the same interface with a stored `dist` (null-path length) and the rule "keep the
// larger dist on the left"; it gives WORST-CASE O(log n) instead of amortised. Skew is shorter and
// fine unless you need the worst-case bound inside another amortised argument.
// PAIRING HEAP has O(1) merge and decrease-key in practice and is what you want if you need
// decrease-key (Dijkstra on very dense graphs); in contests a binary heap with lazy deletion wins.
// LAZY DELETION is usually enough: push (key, id), and pop until the top is still valid. That
// removes the need for decrease-key entirely and is 5 lines.
// SMALL-TO-LARGE with std::priority_queue also "merges" heaps: repeatedly pop from the smaller and
// push into the larger, O(n log^2 n) total. Shorter than this file - use it unless the log^2 hurts.
```

### 2.20 PersistentAndUndo

PERSISTENCE AND UNDO - three ways to travel in a structure's history, and when each applies.

```
//   ROLLBACK  (05 - DSU/02): undo the last op. Cheapest. Needs a
     stack discipline (LIFO).
//   OFFLINE DELETION (14 - OfflineDynamicConnectivity): segment
     tree over time + rollback.
//     Handles arbitrary insert/delete, but you must know every
     operation in advance.
//   PERSISTENCE (this file): branch the history. The only one
     that works ONLINE with deletions,
//     and the only one that lets you query a past version after
     moving on.

// --- PERSISTENT ARRAY: a segment tree over indices. get/set in O
     (log n), O(log n) new nodes. ---
struct PArray {
    struct Node { int l = 0, r = 0; ll v = 0; };
    vector<Node> t;
    int n;
    PArray(int n) : t(1), n(n) {}                                  //
      node 0 is the shared all-zero
    int build(const vector<ll>& a, int lo, int hi) {
        int id = t.size();
        t.push_back({});
        if (lo == hi) { t[id].v = a[lo]; return id; }
        int m = (lo + hi) / 2, L = build(a, lo, m), R = build(a,
     m + 1, hi);
        t[id].l = L, t[id].r = R;
        return id;
    }
    int build(const vector<ll>& a) { return build(a, 0, n - 1); }
    ll get(int u, int i, int lo, int hi) {
        if (lo == hi) return t[u].v;
        int m = (lo + hi) / 2;
        return i <= m ? get(t[u].l, i, lo, m) : get(t[u].r, i, m
     + 1, hi);
    }
    ll get(int u, int i) { return get(u, i, 0, n - 1); }
    int set(int u, int i, ll v, int lo, int hi) {
        int id = t.size();
        t.push_back(t[u]);
        if (lo == hi) { t[id].v = v; return id; }
        int m = (lo + hi) / 2;
        if (i <= m) t[id].l = set(t[u].l, i, v, lo, m);
        else t[id].r = set(t[u].r, i, v, m + 1, hi);
        return id;
    }
    int set(int u, int i, ll v) { return set(u, i, v, 0, n - 1);
     }
};
// --- PERSISTENT DSU on top of it. O(log^2 n) per operation. ---
// The catch that makes it work: NO PATH COMPRESSION (that would
     mutate an old version), so you
// MUST union by size/rank to keep the height O(log n).
struct PDSU {
    PArray par, sz;
    PDSU(int n) : par(n), sz(n) {}
    pair<int, int> init(int n) {                                   //
      returns {parRoot, szRoot}
        vector<ll> p(n), s(n, 1);
        iota(all(p), 0);
        return {par.build(p), sz.build(s)};
    }
    int find(int pr, int x) { ll p = par.get(pr, x); return p ==
     x ? x : find(pr, p); }
    pair<int, int> join(int pr, int sr, int a, int b) {
        a = find(pr, a), b = find(pr, b);
        if (a == b) return {pr, sr};
        if (sz.get(sr, a) < sz.get(sr, b)) swap(a, b);
        return {par.set(pr, b, a), sz.set(sr, a, sz.get(sr, a) +
     sz.get(sr, b))};
    }
};
/* THE OTHER TWO TIME-TRAVEL TRICKS, as recipes
 QUEUE UNDO. You have a structure that supports only "undo the
     last op" (rollback DSU), but the
   deletions arrive in FIFO order (a sliding window). Keep the
     undo stack partitioned so the
   OLDEST element is near the top; when it is not, pop a prefix
     and re-push it in the other order,
   always rebuilding the SMALLER half. O(log n) amortised extra.
     This is the online alternative to
   the offline segment-tree-on-time trick, and it is what "sliding
      window connectivity" needs.
 STATIC TO DYNAMIC (the logarithmic method). Any STATIC structure
     with O(B(n)) build and O(Q(n))
   query becomes insert-only: keep O(log n) instances of sizes
     2^0, 2^1, 2^2, ...; inserting is
   binary-counter carrying, so you rebuild an instance of size 2^k
      only every 2^k insertions.
   Insert becomes O(B(n)/n * log n) amortised and query O(Q(n) log
     n) (ask all instances).
   Use it to make a convex hull / a wavelet tree / an Aho-Corasick
      automaton accept insertions.
 PARTIALLY vs FULLY PERSISTENT: partially = you may query any past
     version but only modify the
   latest (much cheaper - a version-stamped array often suffices);
      fully = you may modify any
   version, branching the history into a tree. The code above is
     fully persistent.
 WHEN NOT TO BOTHER: if the problem is offline and only needs "
     delete", the segment tree over the
   timeline with a rollback DSU is shorter, faster, and has a
     smaller constant than any of this. */
```

# 3 Graph Theory

## 3.1 Graph Traversal

### 3.1.1 TopoSort

Topological Sort (Kahn's Algorithm) - O(V + E)

```
struct TopoSort {
    int n;
    vector<vector<int>> adj;
    vector<int> deg, order;

    TopoSort(int n = 0) : n(n), adj(n + 1), deg(n + 1, 0) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        deg[v]++;
    }

    bool solve() {
        queue<int> q;
        for (int i = 1; i <= n; i++) {
            if (!deg[i]) q.push(i);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.pb(u);
            for (int v : adj[u]) {
                if (!--deg[v]) q.push(v);
            }
        }
        return (int)order.size() == n; // false if graph has
     cycles
    }
};
```

### 3.1.2 BFS_DFS

BFS / DFS / 0-1 BFS / multi-source. 0-based. The bread and butter - keep them short.

```
// Unweighted shortest path + parent for reconstruction.
pair<vector<int>, vector<int>> bfs(int n, const vector<vector<int
    >>& adj, vector<int> src) {
    vector<int> d(n, -1), par(n, -1);
    queue<int> q;
    for (int s : src) d[s] = 0, q.push(s);       // MULTI-SOURCE:
     pass every source at once
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] < 0) d[v] = d[u] + 1, par[v
    ] = u, q.push(v);
    }
    return {d, par};
}
vector<int> pathTo(int t, const vector<int>& par) {
    vector<int> p;
    for (int v = t; v >= 0; v = par[v]) p.push_back(v);
    reverse(all(p));
    return p;
}

// 0-1 BFS: edge weights are only 0 or 1 -> deque instead of a
    heap, O(V + E).
// Use for "turning costs 1", "breaking one wall is free", "flip
    at most k edges".
vector<ll> bfs01(int n, const vector<vector<pair<int, int>>>& adj
    , int s) {
    vector<ll> d(n, llinf);
    deque<int> q;
    d[s] = 0, q.push_back(s);
    while (!q.empty()) {
        int u = q.front(); q.pop_front();
        for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            w ? q.push_back(v) : q.push_front(v);
        }
    }
    return d;
}

// Iterative DFS with entry/exit times (avoids stack overflow at n
     = 2e5).
void dfsIter(int n, const vector<vector<int>>& adj, int root,
    vector<int>& tin, vector<int>& tout) {
    tin.assign(n, -1), tout.assign(n, -1);
    vector<int> it(n, 0), st = {root};
    int timer = 0;
    tin[root] = timer++;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)adj[u].size()) {
            int v = adj[u][it[u]++];
            if (tin[v] < 0) tin[v] = timer++, st.push_back(v);
        } else tout[u] = timer++, st.pop_back();
    }
}

// GRID: 4- and 8-neighbour offsets (see 03 - grid.cpp). Encode a
    cell as r*m+c and reuse the
// graph routines above; add extra state dimensions (mask, parity,
     keys) by widening the index.
// CYCLE DETECTION, directed: colour DFS (0 white, 1 grey, 2 black
    ); a grey->grey edge closes a
// cycle - walk the recursion stack back to extract it. Undirected
    : any edge to an already-visited
// non-parent vertex closes one.
```

## 3.2 Shortest Paths

### 3.2.1 Dijkstra

Dijkstra's Shortest Path Algorithm - O((V + E) log V)

```
struct Dijkstra {
    int n;
    vector<vector<pair<int, ll>>> adj;
    vector<ll> dist;
    vector<int> par;

    Dijkstra(int n = 0) : n(n), adj(n + 1), dist(n + 1, llinf),
     par(n + 1, -1) {}

    void add_edge(int u, int v, ll w, bool directed = false) {
        adj[u].eb(v, w);
        if (!directed) adj[v].eb(u, w);
    }

    void solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
     greater<>> pq;

        dist[src] = 0;
        pq.emplace(0, src);

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d > dist[u]) continue;

            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    par[v] = u;
                    pq.emplace(dist[v], v);
                }
            }
        }
    }
};
```

### 3.2.2 FloydWarshall

Floyd-Warshall All-Pairs Shortest Path - O(V^3)

```
struct FloydWarshall {
    int n;
    vector<vector<ll>> dist;

    FloydWarshall(int n = 0) : n(n), dist(n + 1, vector<ll>(n +
     1, llinf)) {
        for (int i = 0; i <= n; i++) dist[i][i] = 0;
    }

    void add_edge(int u, int v, ll w, bool directed = false) {
        dist[u][v] = min(dist[u][v], w);
        if (!directed) dist[v][u] = min(dist[v][u], w);
    }

    void solve() {
        for (int k = 1; k <= n; k++) {
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    if (dist[i][k] < llinf && dist[k][j] < llinf)
      {
                        dist[i][j] = min(dist[i][j], dist[i][k] +
      dist[k][j]);
                    }
                }
            }
        }
    }
};
```

### 3.2.3 BellmanFord

If you paste both, keep only ONE copy of it.

```
// NOTE: 08 - Spanning Trees/01 - MST.cpp declares an identical `
    struct Edge`.
//       If you paste both, keep only ONE copy of it.
// Bellman-Ford Shortest Path with Negative Cycle Detection - O(V
    * E)
struct Edge {
    int u, v;
    ll w;
};

struct BellmanFord {
    int n;
    vector<Edge> edges;
    vector<ll> dist;
    vector<int> par;

    BellmanFord(int n = 0) : n(n), dist(n + 1, llinf), par(n + 1,
     -1) {}

    void add_edge(int u, int v, ll w) {
        edges.pb({u, v, w});
    }

    // Returns false if a negative cycle is reachable from src
    bool solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        dist[src] = 0;

        for (int i = 0; i < n - 1; i++) {
            bool relaxed = false;
            for (const auto& e : edges) {
                if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e
    .v]) {
                    dist[e.v] = dist[e.u] + e.w;
                    par[e.v] = e.u;
                    relaxed = true;
                }
            }
            if (!relaxed) break;
        }

        for (const auto& e : edges) {
            if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e.v])
    {
                return false; // Negative cycle detected
            }
        }
        return true;
    }
};
```

### 3.2.4 DominatorTree

```
/*
========================================================================

   CATALOGUE: Lengauer-Tarjan Dominator Tree (O((N + M) log N))
========================================================================

   DESCRIPTION:
   Finds the "bottlenecks" in any directed graph. A node 'u'
     dominates 'v' if
   EVERY path from the root to 'v' is forced to go through 'u'.

   USAGE:
   1. Build graph: Create a 2D vector `adj` (1-based) containing
     the directed edges.
   2. Initialize:  DominatorTree dt(n, root, adj);
                   (Automatically builds the tree on instantiation
     )

   EXTRACTING ANSWERS:
   - dt.dom[u]:       The Immediate Dominator of 'u' (the closest
     bottleneck to 'u').
   - dt.id[u]:        If dt.id[u] == 0 after building, 'u' is
     UNREACHABLE from the root.
========================================================================

*/
struct DominatorTree {
    int T = 0, n;
    vector<vector<int>> rg, bucket, adj;
    vector<int> dsu, par, sdom, idom, dom, label, id, rev;

    DominatorTree(int _n, int r, vector<vector<int>>& adj):
        n(_n + 1), rg(n), bucket(n), adj(adj), dsu(n),
        par(n), sdom(n), idom(n), dom(n), label(n), id(n), rev(n)
      { dfs(r); build(r); }

    int find(int u, int x = 0) {
        if (u == dsu[u]) return x ? -1 : u;
        int v = find(dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[label[dsu[u]]] < sdom[label[u]]) label[u] =
     label[dsu[u]];
        dsu[u] = v;
        return x ? v : label[u];
    }

    void dfs(int u) {
        id[u] = ++T, rev[T] = u;
        label[T] = sdom[T] = dsu[T] = T;
        for (int& w : adj[u]) {
            if (!id[w]) {
                dfs(w);
                par[id[w]] = id[u];
            }
            rg[id[w]].push_back(id[u]);
        }
    }

    void build(int r) {
        // FIXED: Loop MUST go down to i >= 1 to process the root'
     s bucket!
        for (int i = T; i >= 1; i--) {
            for (int& u : rg[i]) sdom[i] = min(sdom[i], sdom[find
     (u)]);
            if (i > 1) bucket[sdom[i]].push_back(i);

            for (int& w : bucket[i]) {
                int v = find(w);
                idom[w] = sdom[v] == sdom[w] ? sdom[w] : v;
            }

            if (i > 1) dsu[i] = par[i];
        }

        for (int i = 2; i <= T; i++) {
            if (idom[i] != sdom[i]) idom[i] = idom[idom[i]];
        }

        for (int u = 1; u <= T; ++u) {
            dom[rev[u]] = rev[idom[u]];
        }
    }
};

void testCase() {
    int n, m, s;
    cin >> n >> m >> s;
    s++; // Convert to 1-based indexing
```

```
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u + 1].push_back(v + 1);
    }

    // Automatically builds upon instantiation
    DominatorTree dt(n, s, adj);

    for (int i = 1; i <= n; i++) {
        if (i == s) {
            // pS is S
            cout << s - 1 << (i == n ? "" : " ");
        } else if (!dt.id[i]) {
            // If we can not reach i from S, print -1
            cout << -1 << (i == n ? "" : " ");
        } else {
            // Print the 0-based parent (which is now guaranteed
     to be calculated)
            cout << dt.dom[i] - 1 << (i == n ? "" : " ");
        }
    }
    cout << "\n";
}
```

### 3.2.5 ShortestPathEdgeModify

SHORTEST PATH WITH ONE EDGE CHANGED, answered OFFLINE in O(1) per query after O((n+m) log n).

```
// Query: "if edge id had weight x instead, what is dist(s,t)?" -
     each query is independent.
// IDEA. Let ds/dt be the distance arrays from s and to t, P the
     shortest path, base = ds[t].
//   * Edge NOT on P: the new path either ignores it (base) or
     crosses it once:
//         min(base, ds[u] + x + dt[v], ds[v] + x + dt[u]).
//   * Edge ON P at position i: either still use it (base - w + x)
     , or avoid it entirely, and
//     "the best s->t path avoiding path-edge i" is precomputed
     for every i at once:
//     for a non-path edge (u,v,w), the path ds[u] -> u -> v -> dt
     [v] skips exactly the path edges
//     in the window [L[u], R[v]), where L[x] = last path vertex
     on some shortest s->x path and
//     R[x] = first path vertex on some shortest x->t path. Sweep
     i from 0 to k-1 with a heap.
struct SPEdgeModify {
    struct Edge { int u, v; ll w; };
    int n, k;                                                    //
      k = #edges on the shortest path
    ll base;
    vector<Edge> E;
    vector<ll> ds, dt, best;                                     //
      best[i] = s->t avoiding path edge i
    vector<int> pos;                                             //
      pos[e] = index on P, or -1
    SPEdgeModify(int n, vector<Edge> E, int s, int t) : n(n), E(E
     ) {
        int m = E.size();
        vector<vector<pair<int, int>>> adj(n);
        for (int i = 0; i < m; i++)
            adj[E[i].u].push_back({E[i].v, i}), adj[E[i].v].
     push_back({E[i].u, i});
        vector<int> par(n, -1), pare(n, -1), ordS, ordT;
        auto dij = [&](int src, vector<ll>& d, vector<int>& ord,
     bool tree) {
            d.assign(n, llinf);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>,
     greater<>> pq;
            d[src] = 0, pq.push({0, src});
            while (!pq.empty()) {
                auto [dd, u] = pq.top(); pq.pop();
                if (dd > d[u]) continue;
                ord.push_back(u);
                for (auto [v, id] : adj[u]) if (d[u] + E[id].w <
     d[v]) {
                    d[v] = d[u] + E[id].w;
                    if (tree) par[v] = u, pare[v] = id;
                    pq.push({d[v], v});
                }
            }
        };
        dij(s, ds, ordS, true), dij(t, dt, ordT, false);
        base = ds[t];
        vector<int> pv, pe;                                      //
       the path, s ... t
        for (int c = t; c != s; c = par[c]) pv.push_back(c), pe.
     push_back(pare[c]);
        pv.push_back(s), reverse(all(pv)), reverse(all(pe));
        k = pe.size();
        pos.assign(m, -1);
        vector<char> onP(n, 0);
        for (int x : pv) onP[x] = 1;
        for (int i = 0; i < k; i++) pos[pe[i]] = i;
        vector<int> L(n, 0), R(n, k);
        for (int i = 0; i <= k; i++) L[pv[i]] = R[pv[i]] = i;
        for (int u : ordS) for (auto [v, id] : adj[u])
     // relaxed in shortest-path order
            if (!onP[v] && ds[v] == ds[u] + E[id].w) L[v] = max(L
     [v], L[u]);
        for (int u : ordT) for (auto [v, id] : adj[u])
            if (!onP[v] && dt[v] == dt[u] + E[id].w) R[v] = min(R
     [v], R[u]);
        vector<vector<pair<ll, int>>> ev(max(k, 1));
     // ev[start] = {cost, end}
        for (int i = 0; i < m; i++) {
            if (pos[i] >= 0) continue;
            auto [u, v, w] = E[i];
            if (L[u] < R[v]) ev[L[u]].push_back({ds[u] + w + dt[v
     ], R[v]});
            if (L[v] < R[u]) ev[L[v]].push_back({ds[v] + w + dt[u
     ], R[u]});
        }
        best.assign(max(k, 1), llinf);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
```

```
    greater<>> sw;
        for (int i = 0; i < k; i++) {
            for (auto& e : ev[i]) sw.push(e);
            while (!sw.empty() && sw.top().second <= i) sw.pop();
            if (!sw.empty()) best[i] = sw.top().first;
        }
    }
    ll query(int id, ll x) const {                                  //
      edge id gets weight x
        auto [u, v, w] = E[id];
        if (pos[id] < 0) return min({base, ds[u] + x + dt[v], ds[
     v] + x + dt[u]});
        return min(base - w + x, best[pos[id]]);
    }
};
// RELATED, same precomputation:
//   * "Does edge e lie on SOME shortest path?"  <=>  ds[u] + w +
     dt[v] == base (either direction).
//   * "Does e lie on EVERY shortest path?"      <=>  removing it
     increases the distance; the sweep
//     above answers this for path edges (best[i] > base means
     path edge i is essential).
//   * REPLACEMENT PATHS (delete one edge, ask the new distance) =
      query(id, +infinity).
//   * SECOND SHORTEST WALK (may reuse edges): min over edges of
     ds[u] + w + dt[v] strictly > base.
//   * If instead the whole graph changes per query, you need a
     fresh Dijkstra - this trick only
//     works because s, t and every other weight stay fixed.
```

### 3.2.6 CyclesAndConstraints

MINIMUM MEAN CYCLE, and SYSTEMS OF DIFFERENCE CONSTRAINTS - the two shortest-path tools that

```
// are not about shortest paths.

// --- KARP'S MINIMUM MEAN CYCLE, O(V*E). Returns the smallest
     possible (cycle weight)/(length).
// dp[k][v] = min weight of a walk of EXACTLY k edges ending at v
     (from a virtual source that
// reaches everything). Karp: the answer is  min over v of  max
     over k<n of (dp[n][v]-dp[k][v])/(n-k).
// Reach for it whenever the objective is an AVERAGE around a
     cycle: "minimum cost per unit time",
// "is there a cycle with negative average", profit-per-day loops.
ld minMeanCycle(int n, const vector<array<ll, 3>>& e) {         //
      e = {u, v, w}, directed
    vector<vector<ll>> dp(n + 1, vector<ll>(n, llinf));
    fill(all(dp[0]), 0);                                        //
      virtual source reaches every v
    for (int k = 1; k <= n; k++)
        for (auto [u, v, w] : e)
            if (dp[k - 1][u] < llinf) dp[k][v] = min(dp[k][v], dp
     [k - 1][u] + w);
    ld best = 1e18;
    for (int v = 0; v < n; v++) {
        if (dp[n][v] == llinf) continue;
        ld cur = -1e18;
        for (int k = 0; k < n; k++)
            if (dp[k][v] < llinf) cur = max(cur, (ld)(dp[n][v] -
     dp[k][v]) / (n - k));
        best = min(best, cur);
    }
    return best;                                                //
      1e18 if the graph is acyclic
}
// MINIMUM COST-TO-TIME RATIO CYCLE (minimise sum(cost)/sum(time),
      time > 0): binary search x and
// test for a negative cycle with weights cost - x*time. Same
     skeleton, one Bellman-Ford per step.
// MAXIMUM mean cycle: negate the weights.

// --- SYSTEM OF DIFFERENCE CONSTRAINTS. Every constraint x_j -
     x_i <= w becomes an edge i -> j of
// weight w; add a super-source with a 0-edge to every vertex and
      run Bellman-Ford.
// FEASIBLE iff there is no negative cycle. The distances are then
      a solution, and they are the
// POINTWISE MAXIMUM one (every other solution is <= it
     componentwise, after fixing x_source = 0).
// TRANSLATIONS you will need: x_j - x_i >= w  is  x_i - x_j <= -w
     .  x_j - x_i = w is both.
// x_i <= c is x_i - x_0 <= c against the source. Strict "<" on
     integers is "<= w-1".
// USES: scheduling with precedence and deadlines, "arrange these
     values so all the gaps hold",
// consistency of a set of relative measurements, and the dual of
     a min-cost-flow.
bool differenceConstraints(int n, const vector<array<ll, 3>>& c,
     vector<ll>& x) {
    vector<array<ll, 3>> e = c;                                 //
      c = {i, j, w} meaning x_j-x_i<=w
    for (int v = 0; v < n; v++) e.push_back({(ll)n, (ll)v, 0});
       // super-source n
    x.assign(n + 1, 0);
    for (int it = 0; it <= n; it++) {
        bool any = false;
        for (auto [u, v, w] : e)
            if (x[u] + w < x[v]) x[v] = x[u] + w, any = true;
        if (!any) { x.resize(n); return true; }
    }
    return false;                                               //
      negative cycle: infeasible
}
// --- BELLMAN-FORD EXTRAS worth remembering
// NEGATIVE CYCLE RECOVERY: after n rounds, take a vertex that
     still relaxed, walk n parent
// pointers back (this lands you ON the cycle - the relaxed vertex
      may only be reachable from it),
// then follow parents until a vertex repeats.
// SPFA is Bellman-Ford with a queue; it is O(VE) worst case and
     killable, but the SLF heuristic
// (push to the front if the new distance beats the front) makes
     it fast in practice.
// ONLY SHORTEST PATHS WITH AT MOST K EDGES: run exactly k rounds,
      updating from a SNAPSHOT of the
// previous round (otherwise one round can propagate along many
     edges).
// JOHNSON'S APSP for sparse graphs with negative edges: Bellman-
     Ford from a super-source gives
```

```
// potentials h[], reweight w'(u,v) = w + h[u] - h[v] >= 0, then
     run n Dijkstras. O(VE + V E log V).
```

### 3.2.7 SegmentTreeGraph

SEGMENT-TREE GRAPH - add "u -> every vertex in [l, r]" (or the reverse) with O(log n) edges

```
// instead of O(n). Then run Dijkstra/BFS as usual. Total O((n + q
      log n) log n).
// Two trees: OUT (parent -> children, cost 0) to broadcast INTO a
     range,
//            IN  (children -> parent, cost 0) to collect FROM a
     range.
// Leaves of both trees are the original vertices.
struct SegGraph {
    int n, cnt;
    vector<int> outId, inId;                              // node ids
      of the two trees
    vector<vector<pair<int, ll>>> g;

    SegGraph(int n) : n(n) {
        cnt = n;
        outId.assign(4 * n, -1), inId.assign(4 * n, -1);
        g.assign(n + 8 * n, {});
        buildOut(1, 0, n - 1), buildIn(1, 0, n - 1);
    }
    int node() { return cnt++; }
    void buildOut(int v, int l, int r) {
        if (l == r) { outId[v] = l; return; }
        outId[v] = node();
        int m = (l + r) / 2;
        buildOut(2 * v, l, m), buildOut(2 * v + 1, m + 1, r);
        g[outId[v]].push_back({outId[2 * v], 0});
        g[outId[v]].push_back({outId[2 * v + 1], 0});
    }
    void buildIn(int v, int l, int r) {
        if (l == r) { inId[v] = l; return; }
        inId[v] = node();
        int m = (l + r) / 2;
        buildIn(2 * v, l, m), buildIn(2 * v + 1, m + 1, r);
        g[inId[2 * v]].push_back({inId[v], 0});
        g[inId[2 * v + 1]].push_back({inId[v], 0});
    }
    void addRange(int v, int l, int r, int ql, int qr, int u, ll
     w, bool toRange) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            if (toRange) g[u].push_back({outId[v], w});
     // u -> everything in [ql,qr]
            else g[inId[v]].push_back({u, w});
     // everything in [ql,qr] -> u
            return;
        }
        int m = (l + r) / 2;
        addRange(2 * v, l, m, ql, qr, u, w, toRange);
        addRange(2 * v + 1, m + 1, r, ql, qr, u, w, toRange);
    }
    void edgeToRange(int u, int l, int r, ll w) { addRange(1, 0,
     n - 1, l, r, u, w, true); }
    void edgeFromRange(int l, int r, int u, ll w) { addRange(1,
     0, n - 1, l, r, u, w, false); }
    void edge(int u, int v, ll w) { g[u].push_back({v, w}); }

    vector<ll> dijkstra(int s) {
        vector<ll> d(cnt, llinf);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
     greater<>> pq;
        d[s] = 0, pq.push({0, s});
        while (!pq.empty()) {
            auto [dd, u] = pq.top(); pq.pop();
            if (dd > d[u]) continue;
            for (auto [v, w] : g[u]) if (dd + w < d[v]) d[v] = dd
      + w, pq.push({d[v], v});
        }
        d.resize(n);                                      // only the
      original vertices matter
        return d;
    }
};
// The same construction gives 2-SAT range clauses and "range ->
     range" edges (route both
// through one fresh auxiliary node).
```

## 3.3 Graph Connectivity

### 3.3.1 LowLink

LowLink: BRIDGES + ARTICULATION POINTS + 2-EDGE-CONNECTED COMPONENTS - O(V + E)

```
// Skips the parent by EDGE ID, so MULTI-EDGES are handled
     correctly
// (a doubled edge is never a bridge). 0-based vertices.
// comp[v] = id of v's 2-edge-connected component; contracting
     those gives the BRIDGE TREE,
// on which "number of bridges between u and v" is just a tree
     path length.
struct LowLink {
    int n, timer = 0, nc = 0;
    vector<vector<pair<int, int>>> adj;                   // {to,
     edge id}
    vector<int> tin, low, comp;
    vector<bool> isCut;
    vector<pair<int, int>> bridges;
    vector<int> st;

    LowLink(int n = 0) : n(n), adj(n), tin(n, -1), low(n), comp(n
     , -1), isCut(n, false) {}
    void add_edge(int u, int v, int id) { adj[u].push_back({v, id
     }), adj[v].push_back({u, id}); }

    void dfs(int u, int pe) {
        tin[u] = low[u] = timer++;
        st.push_back(u);
        int kids = 0;
        for (auto [v, id] : adj[u]) {
            if (id == pe) continue;                       // skip
     the edge we came in on, not the vertex
            if (tin[v] != -1) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, id), kids++;
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.push_back({u, v});
                if (low[v] >= tin[u] && pe != -1) isCut[u] = true
     ;
            }
        }
        if (pe == -1 && kids > 1) isCut[u] = true;
        if (low[u] == tin[u]) {                           // u is
     the top of a 2-edge-connected comp
            while (true) {
                int v = st.back(); st.pop_back();
                comp[v] = nc;
                if (v == u) break;
            }
            nc++;
        }
    }
    void solve() { for (int i = 0; i < n; i++) if (tin[i] == -1)
     dfs(i, -1); }

    // Bridge tree: nc nodes, one edge per bridge.
    vector<vector<int>> bridgeTree() {
        vector<vector<int>> t(nc);
        for (auto [u, v] : bridges) t[comp[u]].push_back(comp[v])
     , t[comp[v]].push_back(comp[u]);
        return t;
    }
};
```

### 3.3.2 SCC

Strongly Connected Components (Tarjan's Algorithm) - O(V + E)

```
// IMPORTANT: component ids come out in REVERSE topological order
     - for every edge u->v with
// comp[u] != comp[v] we have comp[u] > comp[v]. Iterate ids
     DOWNWARD to walk the condensation
// forwards. build_dag() may emit duplicate arcs (fine for
     reachability, wrong for counting).
struct SCC {
    int n, timer = 0, num_comps = 0;
    vector<vector<int>> adj;
    vector<int> tin, low, comp;
    vector<bool> in_st;
    stack<int> st;
    vector<vector<int>> comps;

    SCC(int n = 0) : n(n), adj(n + 1), tin(n + 1, 0), low(n + 1,
     0), comp(n + 1, -1), in_st(n + 1, false) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
    }

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u);
        in_st[u] = true;

        for (int v : adj[u]) {
            if (!tin[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }

        if (low[u] == tin[u]) {
            comps.pb({});
            while (true) {
                int v = st.top();
                st.pop();
                in_st[v] = false;
                comp[v] = num_comps;
                comps.back().pb(v);
                if (u == v) break;
            }
            num_comps++;
        }
    }

    void solve() {
        for (int i = 1; i <= n; i++) {
            if (!tin[i]) dfs(i);
        }
    }

    vector<vector<int>> build_dag() {
        vector<vector<int>> dag(num_comps);
        for (int u = 1; u <= n; u++) {
            for (int v : adj[u]) {
                if (comp[u] != comp[v]) dag[comp[u]].pb(comp[v]);
            }
        }
        return dag;
    }
};
```

### 3.3.3 BlockCutTree

BLOCK-CUT TREE (vertex biconnected components). O(V + E).

```
// Builds a forest whose nodes are: the original CUT VERTICES,
     plus one node per BICONNECTED
// COMPONENT ("block"). Every cut vertex is joined to each block
     containing it.
// Answers "is there a path u -> v avoiding vertex w", "which
     blocks does a path cross",
// "articulation structure of the graph" - all become tree queries
     .
struct BlockCut {
    int n, timer = 0;
    vector<vector<int>> adj, comps, tree;            // comps[i]
     = vertices of block i
    vector<int> tin, low, stk, id;                   // id[v] =
     tree node of original vertex v
    vector<bool> isCut;

    BlockCut(int n = 0) : n(n), adj(n), tin(n, -1), low(n), id(n,
      -1), isCut(n, false) {}
    void add_edge(int u, int v) { adj[u].push_back(v), adj[v].
     push_back(u); }

    void dfs(int u, int p) {
        tin[u] = low[u] = timer++;
        stk.push_back(u);
        int kids = 0;
        for (int v : adj[u]) {
            if (v == p) { p = -1; continue; }         // skip ONE
     parent edge (multi-edge safe)
            if (tin[v] != -1) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u), kids++;
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u]) {               // u closes
     a block
                    isCut[u] = (tin[u] > 0 || kids > 1);
                    comps.push_back({u});
                    while (true) {
                        int w = stk.back(); stk.pop_back();
                        comps.back().push_back(w);
                        if (w == v) break;
                    }
                }
            }
        }
    }
    void build() {
        for (int v = 0; v < n; v++) if (adj[v].empty()) comps.
     push_back({v});   // isolated vertex
        for (int i = 0; i < n; i++) if (tin[i] == -1) { timer =
     0; dfs(i, -1); stk.clear(); }
        tree.assign(comps.size(), {});
        for (int v = 0; v < n; v++) if (isCut[v]) id[v] = tree.
     size(), tree.push_back({});
        for (int i = 0; i < (int)comps.size(); i++)
            for (int v : comps[i]) {
                if (!isCut[v]) id[v] = i;
                else tree[i].push_back(id[v]), tree[id[v]].
     push_back(i);
            }
    }
};
// A vertex that is NOT a cut vertex lies in exactly one block ->
     id[v] is that block.
// An edge (u,v) always lies in exactly one block. Leaves of the
     tree are blocks; a graph is
// biconnected iff the tree has a single node.
```

### 3.3.4 TriangleAndCycleCounting

COUNTING SMALL SUBGRAPHS in O(m sqrt m) - the "degree ordering" trick. Orient every edge from

```
// the lower (degree, index) endpoint to the higher one. In that
     DAG every out-degree is O(sqrt m),
// because a vertex with out-degree d has d neighbours of degree
     >= d, so d^2 <= 2m.
// Undirected simple graph, 0-indexed, edges given as pairs.

// --- TRIANGLES: enumerate each once. O(m sqrt m). ---
ll countTriangles(int n, const vector<pair<int, int>>& es) {
    vector<int> deg(n, 0);
    for (auto [u, v] : es) deg[u]++, deg[v]++;
    auto better = [&](int a, int b) { return deg[a] != deg[b] ?
     deg[a] < deg[b] : a < b; };
    vector<vector<int>> out(n);
    for (auto [u, v] : es) better(u, v) ? out[u].push_back(v) :
     out[v].push_back(u);
    vector<char> mark(n, 0);
    ll cnt = 0;
    for (int u = 0; u < n; u++) {
        for (int v : out[u]) mark[v] = 1;
        for (int v : out[u]) for (int w : out[v]) cnt += mark[w];
        for (int v : out[u]) mark[v] = 0;
    }
    return cnt;                                                 //
      each triangle counted exactly once
}
// LISTING them: inside the inner loop, (u,v,w) with mark[w] IS a
     triangle.
// PER-VERTEX / PER-EDGE triangle counts: add 1 to each of u,v,w (
     or to each of the three edges)
// at that moment - this is what "local clustering coefficient"
     and many CF problems want.

// --- 4-CYCLES: O(m sqrt m) with the same orientation. ---
// For each u, walk two steps (u -> v -> w) in the oriented graph
     and count how many times each w
// is reached; a pair of distinct paths u->w gives a 4-cycle. Sum
     C(cnt[w], 2).
ll countFourCycles(int n, const vector<pair<int, int>>& es) {
    vector<int> deg(n, 0);
    for (auto [u, v] : es) deg[u]++, deg[v]++;
    auto better = [&](int a, int b) { return deg[a] != deg[b] ?
     deg[a] < deg[b] : a < b; };
    vector<vector<int>> adj(n), out(n);
    for (auto [u, v] : es) {
        adj[u].push_back(v), adj[v].push_back(u);
        better(u, v) ? out[u].push_back(v) : out[v].push_back(u);
    }
    vector<ll> cnt(n, 0);
    ll r = 0;
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) for (int w : out[v]) if (better(u, w
     )) r += cnt[w]++;
        for (int v : adj[u]) for (int w : out[v]) if (better(u, w
     )) cnt[w] = 0;
    }
    return r;
}
// WHY IT IS O(m sqrt m): the inner loop runs sum over v of deg(v)
      * outdeg(v) <= O(m sqrt m).
//
// OTHER COUNTING FACTS WORTH HAVING:
//   * #triangles = trace(A^3) / 6 for the adjacency matrix A;
     with bitsets that is O(n^3 / 64) and
//     is often the shorter answer when n <= 2000 (also gives per-
     pair common-neighbour counts).
//   * #paths of length 2 = sum over v of C(deg v, 2). Subtract 3
     * #triangles for "paths that are
//     not part of a triangle".
//   * A graph is TRIANGLE-FREE  =>  m <= n^2/4 (Mantel / Turan
     for K3). More generally Turan:
//     a K_{r+1}-free graph has at most (1 - 1/r) n^2 / 2 edges.
//   * #closed walks of length k = trace(A^k) - use matrix power
     for k up to 1e18 on small n.
//   * COUNTING CYCLES OF LENGTH <= 5 is polynomial by similar
     tricks; length >= 6 is as hard as
//     matrix multiplication in general. For counting ALL simple
     cycles use inclusion-exclusion
//     over subsets (2^n) or DP over bitmask start-vertex.
//   * GIRTH (shortest cycle) of an unweighted graph: BFS from
     every vertex, O(nm). For the
//     shortest cycle THROUGH a given edge, delete it and run BFS.
```

## 3.4 Satisfiability

### 3.4.1 TwoSat

2-SAT via Kosaraju - O(n + m). Self-contained, 0-based variables.

```
// Literal encoding: 2*i = (x_i false), 2*i+1 = (x_i true).
// Satisfiable iff x and !x are never in the same SCC; then take
     the literal whose SCC is LATER
// in topological order.
struct TwoSat {
    int n;
    vector<vector<int>> adj, radj;
    vector<int> ord, comp;
    vector<bool> used, val;

    TwoSat(int n = 0) : n(n), adj(2 * n), radj(2 * n) {}
    int lit(int i, bool t) { return 2 * i + t; }
    int add_var() { n++, adj.resize(2 * n), radj.resize(2 * n);
     return n - 1; }

    void imply(int a, int b) {                                  //
     literal a  =>  literal b  (and contrapositive)
        adj[a].push_back(b), radj[b].push_back(a);
        adj[b ^ 1].push_back(a ^ 1), radj[a ^ 1].push_back(b ^ 1)
     ;
    }
    void either(int a, int b) { imply(a ^ 1, b); }      // (a OR
     b)
    void force(int a) { imply(a ^ 1, a); }              // a must
      be true
    void nand_(int a, int b) { either(a ^ 1, b ^ 1); }  // not
     both
    void eq(int a, int b) { imply(a, b), imply(b, a); } // a <=>
     b
    void xor_(int a, int b) { eq(a, b ^ 1); }

    // AT MOST ONE of `lits` is true, with O(k) edges instead of O
     (k^2) (prefix/"lightbulb" trick):
    // aux_i means "someone at index <= i was chosen".
    void at_most_one(const vector<int>& lits) {
        int prev = -1;
        for (int c : lits) {
            int cur = lit(add_var(), true);
            imply(c, cur);
            if (prev != -1) imply(prev, cur), imply(c, prev ^ 1);
            prev = cur;
        }
    }

    void dfs1(int u) { used[u] = true; for (int v : adj[u]) if (!
     used[v]) dfs1(v); ord.push_back(u); }
    void dfs2(int u, int c) { comp[u] = c; for (int v : radj[u])
     if (comp[v] < 0) dfs2(v, c); }

    bool solve() {
        used.assign(2 * n, false), comp.assign(2 * n, -1), val.
     assign(n, false), ord.clear();
        for (int i = 0; i < 2 * n; i++) if (!used[i]) dfs1(i);
        int c = 0;
        for (int i = 2 * n - 1; i >= 0; i--) if (comp[ord[i]] <
     0) dfs2(ord[i], c++);
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1]) return false;
            val[i] = comp[2 * i + 1] > comp[2 * i];
        }
        return true;                                            //
     read val[0..base_n-1]; aux vars are padding
    }
};
// RANGE CLAUSE ("if x then ALL of [l,r]") in O(log n) edges:
     build a segment tree over the
// variables, give every internal node an aux variable, add parent
      => both children, then
// imply(x, node) for the O(log n) covering nodes. Mirror the tree
      (child => parent) for
// "if any of [l,r] then x".
```

## 3.5 Eulerian Path

### 3.5.1 EulerianPath

Hierholzer's Algorithm for Eulerian Path / Circuit - O(V + E)

```
// Works for both Directed and Undirected graphs
struct EulerianPath {
    int n, m;
    vector<vector<pair<int, int>>> adj; // {neighbor, edge_id}
    vector<bool> used_edge;
    vector<int> in_deg, out_deg, path;

    EulerianPath(int n = 0, int m = 0) : n(n), m(m), adj(n + 1),
     used_edge(m, false), in_deg(n + 1, 0), out_deg(n + 1, 0) {}

    void add_edge(int u, int v, int edge_id, bool directed = true
     ) {
        adj[u].pb({v, edge_id});
        out_deg[u]++;
        in_deg[v]++;
        if (!directed) {
            adj[v].pb({u, edge_id});
            out_deg[v]++;
            in_deg[u]++;
        }
    }

    // Finds Eulerian path starting at start_node. Returns false
     if graph is not Eulerian
    bool solve(int start_node) {
        vector<int> ptr(n + 1, 0);
        stack<int> st;
        st.push(start_node);

        while (!st.empty()) {
            int u = st.top();
            if (ptr[u] < adj[u].size()) {
                auto [v, id] = adj[u][ptr[u]++];
                if (!used_edge[id]) {
                    used_edge[id] = true;
                    st.push(v);
                }
            } else {
                path.pb(u);
                st.pop();
            }
        }
        reverse(all(path));

        // Verify that every edge was traversed
        for (bool used : used_edge) {
            if (!used) return false;
        }
        return true;
    }
};
```

## 3.6 Flow and Min Cut

### 3.6.1 Dinic

DINIC MAX FLOW. O(V^2 E) in general, O(E sqrt(V)) on unit-capacity graphs, O(V^2 E) worst case but

```
// fast in practice. After maxflow, vertices reachable from s in
     the residual graph form the min cut.
// Scaling (start with big capacities) helps on dense graphs with
     large capacities.
// DINIC MAX FLOW. O(V^2 E) general, O(E sqrt(E)) with unit
     capacities, O(E sqrt(V)) for bipartite
// matching. This is the SCALING variant (big capacities first),
     so it stays fast on wide ranges.
// addEdge(a, b, cap, rcap): pass rcap = cap for an UNDIRECTED
     edge. Edge::flow() recovers the flow
// on each arc. After calc(s,t), leftOfMinCut(v) is true exactly
     for the SOURCE side of a minimum
// cut - the cut edges are those from a left vertex to a right
     vertex. 0-indexed.
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need
     flows
    };

    vector<int> lvl, ptr, q;
    vector<vector<Edge> > adj;

    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {
    }

    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, (int) adj[b].size(), c, c});
        adj[b].push_back({a, (int) adj[a].size() - 1, rcap, rcap
     });
    }

    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < (int) adj[v].size(); i++) {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }

    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        for (int L = 0; L < 31; L++)
            do {
                // 'int L=30' maybe faster for random data
                lvl = ptr = vector<int>((int) q.size());
                int qi = 0, qe = lvl[s] = 1;
                while (qi < qe && !lvl[t]) {
                    int v = q[qi++];
                    for (Edge e: adj[v])
                        if (!lvl[e.to] && e.c >> (30 - L))
                            q[qe++] = e.to, lvl[e.to] = lvl[v] +
     1;
                }
                while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
            } while (lvl[t]);
        return flow;
    }

    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

### 3.6.2 MCMF

MIN-COST MAX-FLOW with JOHNSON POTENTIALS (Dijkstra instead of SPFA on every augmentation).

```
// O(F * E log V) where F = number of augmentations. Much faster than the SPFA version and the
// one to use by default. Negative edge costs are allowed at construction PROVIDED there is no negative-cost CYCLE:
// run one Bellman-Ford
// to initialise the potentials (setPotentialsBF), after that every reduced cost is >= 0.
struct MCMF {
    struct E { int to, rev; ll cap, cost, flow = 0; };
    int n;
    vector<vector<E>> g;
    vector<ll> pot, dist;
    vector<int> pv, pe;

    MCMF(int n) : n(n), g(n), pot(n, 0), dist(n), pv(n), pe(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    void setPotentialsBF(int s) {                       // only needed if some cost < 0
        fill(all(pot), llinf);
        pot[s] = 0;
        for (int it = 0; it < n; it++) {
            bool ch = false;
            for (int u = 0; u < n; u++) if (pot[u] < llinf)
                for (auto& e : g[u]) if (e.cap - e.flow > 0 && pot[u] + e.cost < pot[e.to])
                    pot[e.to] = pot[u] + e.cost, ch = true;
            if (!ch) break;
        }
        for (auto& x : pot) if (x == llinf) x = 0;
    }
    bool dijkstra(int s, int t) {
        fill(all(dist), llinf);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        dist[s] = 0, pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (int i = 0; i < (int)g[u].size(); i++) {
                auto& e = g[u][i];
                if (e.cap - e.flow <= 0) continue;
                ll nd = d + e.cost + pot[u] - pot[e.to];      // reduced cost, always >= 0
                if (nd < dist[e.to]) dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        return dist[t] < llinf;
    }
    // Returns {flow, cost}. Pass maxf to cap the flow (e.g. for min-cost k-flow).
    pair<ll, ll> run(int s, int t, ll maxf = llinf) {
        ll flow = 0, cost = 0;
        while (flow < maxf && dijkstra(s, t)) {
            for (int i = 0; i < n; i++) if (dist[i] < llinf) pot[i] += dist[i];
            ll aug = maxf - flow;
            for (int v = t; v != s; v = pv[v]) aug = min(aug, g[pv[v]][pe[v]].cap - g[pv[v]][pe[v]].flow);
            for (int v = t; v != s; v = pv[v]) {
                auto& e = g[pv[v]][pe[v]];
                e.flow += aug, g[v][e.rev].flow -= aug;
                cost += aug * e.cost;
            }
            flow += aug;
        }
        return {flow, cost};
    }
    // MIN-COST flow of exactly K units: run(s, t, K) and check flow == K.
    // MIN-COST (not max) flow: stop as soon as dist[t] + pot[t] - pot[s] > 0 (no profitable path).
};
// MODELLING: assignment on a SPARSE graph; for a dense/complete one use 07 - Matching/03 - Hungarian * transportation * "k disjoint paths of minimum total cost" *
// min-cost bipartite matching * scheduling with penalties * MCMF is also how you solve
// "choose k items with a matroid-ish constraint minimising cost".
```

### 3.6.3 FlowWithLowerBounds

FLOWS WITH LOWER BOUNDS (circulation with demands). Every edge needs d(e) <= f(e) <= c(e).

```
// Build an auxiliary network, run one max flow; feasible iff every edge out of s' saturates.
//   excess(v) = (sum of d over edges INTO v) - (sum of d over edges OUT of v)
//   excess(v) > 0 -> edge s'->v of capacity excess(v);  < 0 -> edge v->t' of capacity -excess
//   every original edge (u,v) keeps capacity c - d
// For an s-t flow, add t->s with capacity INF first, which turns it into a circulation.
// Needs Dinic (01 - Dinic.cpp) with addEdge(u, v, cap) and calc(s, t).
struct LowerBoundFlow {
    int n, S, T;                                        // S, T are the auxiliary super source/sink
    Dinic din;
    vector<ll> excess, low;
    vector<array<int, 2>> eid;                          // per original edge: {u, index in din.adj[u]}

    LowerBoundFlow(int n) : n(n), S(n), T(n + 1), din(n + 2), excess(n + 2, 0) {}
    void addEdge(int u, int v, ll lo, ll hi) {
        low.push_back(lo);
        eid.push_back({u, (int)din.adj[u].size()});
        din.addEdge(u, v, hi - lo);
        excess[v] += lo, excess[u] -= lo;
    }
    bool feasible() {                                   // circulation only (no s, t)
        ll need = 0;
        for (int v = 0; v < n; v++) {
            if (excess[v] > 0) din.addEdge(S, v, excess[v]), need += excess[v];
            else if (excess[v] < 0) din.addEdge(v, T, -excess[v]);
        }
        return din.calc(S, T) == need;
    }
    // Min / max s-t flow subject to the bounds - see the RECIPES block below for the ONLY
    // correct order of operations. You must DELETE the t->s edge (zero both residual directions)
    // before the second max flow, otherwise Dinic's first augmenting path runs s -> t backwards
    // along that arc and CANCELS the feasible flow instead of extending it.
    // feasible() appends the S/T edges every time it is called - call it exactly once.
    ll flowOn(int i) { return low[i] + din.adj[eid[i][0]][eid[i][1]].flow(); }
};
/* RECIPES
   "each edge must be used at least once"          -> d(e) = 1
   "each vertex must be visited between L and R"   -> split v into v_in -> v_out with [L, R]
   "exact degree subgraph"                         -> d = c = required degree on the split edge
   MAX s-t FLOW WITH BOUNDS: add t->s with capacity INF, run feasible(); then delete that edge
   and run one more max flow from s to t on the residual graph. The answer is the flow that
   was on t->s plus the extra augmentation.
   MIN s-t FLOW WITH BOUNDS: same, but augment from t to s to cancel as much as possible. */
```

### 3.6.4 StoerWagner

STOER-WAGNER GLOBAL MIN CUT - minimum cut over ALL vertex pairs, undirected weighted, O(V^3).

```
// No source/sink. Returns {cut weight, one side of the cut}.
// Key lemma: in a maximum-adjacency order, if s is the second-to-
     last and t the last vertex
// added, then the minimum s-t cut is exactly {t}. Merge s and t
     and repeat n-1 times.
pair<ll, vector<int>> globalMinCut(vector<vector<ll>> w) {
    int n = w.size();
    if (n < 2) return {0, {}};
    vector<char> gone(n, 0);
    vector<vector<int>> grp(n);
    for (int i = 0; i < n; i++) grp[i] = {i};
    ll bestW = llinf;
    vector<int> best;
    for (int phase = 0; phase + 1 < n; phase++) {
        vector<ll> ww(n, 0);
        vector<char> added(n, 0);
        int prev = -1, last = -1;
        for (int it = 0; it < n - phase; it++) {       // maximum
    -adjacency order
            int sel = -1;
            for (int j = 0; j < n; j++)
                if (!gone[j] && !added[j] && (sel < 0 || ww[j] >
    ww[sel])) sel = j;
            added[sel] = 1, prev = last, last = sel;
            for (int j = 0; j < n; j++) if (!gone[j] && !added[j
    ]) ww[j] += w[sel][j];
        }
        ll cut = 0;                                    // cut-of-
    the-phase separates {last}
        for (int j = 0; j < n; j++) if (!gone[j] && j != last)
    cut += w[last][j];
        if (cut < bestW) bestW = cut, best = grp[last];
        gone[last] = 1;                                // merge
    last into prev
        grp[prev].insert(grp[prev].end(), all(grp[last]));
        for (int j = 0; j < n; j++) if (!gone[j] && j != prev)
            w[prev][j] += w[last][j], w[j][prev] = w[prev][j];
    }
    return {bestW, best};
}
// USES: "split the network into two non-empty parts as cheaply as
      possible", edge connectivity
// of a weighted undirected graph, "minimum number of edges to
     delete to disconnect the graph"
// (unit weights). Doing this with n-1 max-flows is both slower
     and longer to write.
// KARGER (randomized contraction) finds a min cut with
     probability >= 1/C(n,2) per run; the
// useful corollary is that a graph has AT MOST C(n,2) distinct
     minimum cuts.
```

### 3.6.5 MinCutModelling

MIN-CUT MODELLING - the reason flow appears at ICPC. The code is never the hard part; choosing

```
// Needs: Dinic (01 - Dinic.cpp).
// what the vertices and the infinite capacities mean is. Rule of
     thumb: if the objective contains
// an "infinity", you are CUTTING, not flowing.

// --- MAXIMUM WEIGHT CLOSURE / PROJECT SELECTION ---
// Choose a set S closed under the dependency arcs (if v in S and
     v -> u is a dependency, u in S)
// maximising the total weight. Build: s -> v with capacity w(v)
     for every w(v) > 0;
// v -> t with capacity -w(v) for every w(v) < 0; u -> v with INF
     for every dependency u needs v.
// Answer = (sum of positive weights) - maxflow.  S = the source
     side of the min cut.
// USES: "each project earns p but needs these machines that cost
     c"; "pick cells maximising
// profit but a chosen cell forces its neighbours"; densest-
     subgraph style selections.
ll maxClosure(int n, const vector<ll>& w, const vector<pair<int,
     int>>& need, vector<int>* chosen = nullptr) {
    Dinic din(n + 2);
    int s = n, t = n + 1;
    ll pos = 0;
    for (int v = 0; v < n; v++)
        if (w[v] > 0) pos += w[v], din.addEdge(s, v, w[v]);
        else if (w[v] < 0) din.addEdge(v, t, -w[v]);
    for (auto [u, v] : need) din.addEdge(u, v, llinf);         //
      choosing u forces choosing v
    ll cut = din.calc(s, t);
    if (chosen) for (int v = 0; v < n; v++) if (din.leftOfMinCut(
     v)) chosen->push_back(v);
    return pos - cut;
}
/* THE OTHER STANDARD REDUCTIONS
 TWO LABELS WITH PAIR PENALTIES ("cats or dogs", image
     segmentation)
   Every vertex gets label A or B. Cost a_v for A, b_v for B, and
     c_uv extra if u and v differ.
   Build s -> v with b_v, v -> t with a_v, and u <-> v with c_uv
     in both directions.
   The min cut is the minimum total cost; the source side takes
     label A.
   PRECONDITION: the pair penalty must be SUBMODULAR (cost(A,A) +
     cost(B,B) <= cost(A,B) +
   cost(B,A)). A penalty that rewards disagreement cannot be
     modelled by a cut - it is NP-hard.

 VERTEX CAPACITIES / VERTEX-DISJOINT PATHS
   Split v into v_in -> v_out with the vertex capacity. Vertex-
     disjoint s-t paths use capacity 1.

 BIPARTITE FAMILY (see 07 - Matching and 10 - Theorems)
   max matching        = max flow with unit capacities
   min vertex cover    = max matching (Konig)          -> recover
     via the alternating-BFS set
   max independent set = n - max matching
   min path cover of a DAG (vertex-disjoint) = n - max matching of
      the split graph

 "MUST BE ON THE SOURCE SIDE" -> add s -> v with INF.   "MUST BE
     ON THE SINK SIDE" -> v -> t INF.
 That is how you force decisions inside an otherwise free min cut.

 EDGE / VERTEX DISJOINT PATHS = Menger. Recover the paths by
     decomposing the integral flow.

 LOWER BOUNDS ON EDGES -> 03 - FlowWithLowerBounds.cpp.

 MAXIMUM DENSITY SUBGRAPH (maximise |E(S)| / |S|): binary search
     lambda, then min cut with
   s -> v capacity m, v -> t capacity m + 2*lambda - deg(v), u <->
      v capacity 1. Feasible iff
   the min cut is < n*m. O(log(n^2) * maxflow) since the density
     is a ratio of integers <= n^2.

 MINIMUM CUT WITHOUT s AND t (global) -> 04 - StoerWagner.cpp, O(V
     ^3), or n-1 max flows.

 ALL-PAIRS MIN CUT -> the Gomory-Hu tree: n-1 max flows build a
     weighted tree in which the min
   u-v cut equals the MINIMUM EDGE on the u-v tree path. Use it
     when many pairs are asked.

 COMPLEXITY TABLE - read this at minute 3, not minute 90
   Dinic                  O(V^2 E) general | O(E sqrt E) unit caps
      | O(E sqrt V) bipartite
```

```
Edmonds-Karp            O(V E^2)                -- do not
Push-relabel / HLPP     O(V^2 sqrt E)           -- dense
  graphs, n ~ 1000, m ~ n^2/2
MCMF (SSP + potentials) O(F * E log V)          -- F = total
   flow, so bound F first
Hungarian               O(n^2 m)                -- dense
  assignment, n <= ~1000
Hopcroft-Karp           O(E sqrt V)             -- bipartite
   matching, n >= few thousand
Kuhn                    O(V E)                  -- shorter,
  fine to ~2000 vertices
Blossom (general)       O(V^3)
Stoer-Wagner            O(V^3)
Gomory-Hu               O(V * maxflow)
If the flow value itself can be huge (capacities up to 1e9), an O
   (F ...) bound is not a bound -
scale the capacities or use a strongly polynomial algorithm. */
```

### 3.6.6 GomoryHu

GOMORY-HU TREE - ALL-PAIRS MINIMUM CUT from only n-1 max-flow runs.

```
// Needs: Dinic (01 - Dinic.cpp).
// The result is a weighted tree on the same vertex set with the
   property that, for EVERY pair
// (u, v), the minimum u-v cut in the original graph equals the
   MINIMUM EDGE WEIGHT on the u-v
// path in the tree. (There are at most n-1 distinct min-cut
   values in any graph - that is the
// whole content of the theorem.)
// Reach for it when a problem asks about min cuts between many
   pairs: "sum/max/k-th of all
// pairwise min cuts", "order the vertices to maximise the sum of
   consecutive cuts", "how many
// pairs have min cut >= k". Undirected graphs only.
// Complexity: (n-1) x Dinic. The tree is returned as parent +
   weight arrays (par[0] = -1).
struct GomoryHu {
    int n;
    vector<array<ll, 3>> edges;                                 //
      {u, v, capacity}, undirected
    vector<int> par;
    vector<ll> w;
    GomoryHu(int n) : n(n), par(n, 0), w(n, 0) {}
    void addEdge(int u, int v, ll c) { edges.push_back({(ll)u, (
     ll)v, c}); }
    void build() {
        par.assign(n, 0), w.assign(n, 0), par[0] = -1;
        for (int i = 1; i < n; i++) {
            Dinic din(n);
            for (auto [u, v, c] : edges) din.addEdge(u, v, c, c);
        // undirected: both directions
            w[i] = din.calc(i, par[i]);
            for (int j = i + 1; j < n; j++)
     // re-parent the same-side vertices
                if (par[j] == par[i] && din.leftOfMinCut(j)) par[
     j] = i;
        }
    }
    ll minCut(int u, int v) const {                             //
      min edge on the tree path
        ll best = llinf;
        vector<int> du(n, 0);
        for (int x = u; x != -1; x = par[x]) du[x] = 1;
        vector<int> pv;
        int m = v;
        while (!du[m]) pv.push_back(m), m = par[m];
     // m = the meeting vertex
        for (int x = u; x != m; x = par[x]) best = min(best, w[x
     ]);
        for (int x : pv) best = min(best, w[x]);
        return best;
    }
};
// EQUIVALENT-FLOW TREE (Gusfield's variant) is the same n-1 flows
      without the re-parenting step;
// it gets the min-cut VALUES right but not the actual cuts. The
   version above is the real
// Gomory-Hu, so the tree edges are genuine cuts.
// GLOBAL MIN CUT is the minimum edge of the whole tree - but
   Stoer-Wagner (04) is O(V^3) and
// needs no flow at all, so prefer it when that is all you want.
// DIRECTED graphs have no Gomory-Hu tree; you genuinely need O(n
   ^2) flows there.
```

## 3.7 Matching

### 3.7.1 HopcroftKarp

HOPCROFT-KARP - maximum bipartite matching in O(E sqrt(V)), the fast alternative to Kuhn's O(VE).

```
// Use it when n is above a few thousand; below that Kuhn is
   shorter and fast enough.
// HOPCROFT-KARP - maximum bipartite matching in O(E sqrt(V)), the
    fast alternative to Kuhn's
// O(VE). Use above a few thousand vertices; below that Kuhn (02)
   is shorter and fast enough.
// INDEXING TRAP: leftMatch has size m and is indexed by a RIGHT
   vertex; rightMatch has size n and
// is indexed by a LEFT vertex. Reading the names the obvious way
   is the default way to get this
// wrong. For the minimum vertex cover / maximum independent set (
   Konig), see 02 - KuhnAndKonig.
struct HopcroftKarp {
    vector<int> leftMatch, rightMatch, dist, cur;
    vector<vector<int> > a;
    int n, m;

    HopcroftKarp() {}

    HopcroftKarp(int n, int m) {
        this->n = n;
        this->m = m;
        a = vector<vector<int> >(n);
        leftMatch = vector<int>(m, -1);
        rightMatch = vector<int>(n, -1);
        dist = vector<int>(n, -1);
        cur = vector<int>(n, -1);
    }

    void addEdge(int x, int y) {
        a[x].push_back(y);
    }

    int bfs() {
        int found = 0;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (rightMatch[i] < 0) dist[i] = 0, q.push(i);
            else dist[i] = -1;

        while (!q.empty()) {
            int x = q.front();
            q.pop();
            for (int i = 0; i < int(a[x].size()); i++) {
                int y = a[x][i];
                if (leftMatch[y] < 0) found = 1;
                else if (dist[leftMatch[y]] < 0)
                    dist[leftMatch[y]] = dist[x] + 1, q.push(
     leftMatch[y]);
            }
        }

        return found;
    }

    int dfs(int x) {
        for (; cur[x] < int(a[x].size()); cur[x]++) {
            int y = a[x][cur[x]];
            if (leftMatch[y] < 0 || (dist[leftMatch[y]] == dist[x
     ] + 1 && dfs(leftMatch[y]))) {
                leftMatch[y] = x;
                rightMatch[x] = y;
                return 1;
            }
        }
        return 0;
    }

    int maxMatching() {
        int match = 0;
        while (bfs()) {
            for (int i = 0; i < n; i++) cur[i] = 0;
            for (int i = 0; i < n; i++)
                if (rightMatch[i] < 0) match += dfs(i);
        }
        return match;
    }
};
```

### 3.7.2 KuhnAndKonig

Kuhn's bipartite matching, O(V*E) but tiny and easy to modify. Left size n, right size m.

```
// In practice fast; randomise the adjacency order + greedy pre-match if you need more speed.
struct Kuhn {
    int n, m;
    vector<vector<int>> adj;                    // left -> right
    vector<int> mt;                             // mt[right] = matched left, or -1
    vector<bool> used;

    Kuhn(int n = 0, int m = 0) : n(n), m(m), adj(n), mt(m, -1) {}
    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool try_(int u) {
        for (int v : adj[u]) if (!used[v]) {
            used[v] = true;
            if (mt[v] < 0 || try_(mt[v])) { mt[v] = u; return true; }
        }
        return false;
    }
    int maxMatching() {
        int r = 0;
        vector<int> ord(n);
        iota(all(ord), 0);
        shuffle(all(ord), rng);
        for (int u : ord) { used.assign(m, false); r += try_(u); }
        return r;
    }
    // KONIG: minimum VERTEX COVER = maximum matching. Recover it:
    //   alternate from every UNMATCHED left vertex; let Z = reached set.
    //   cover = (left \ Z) union (right and Z);  max independent set = complement of the cover.
    pair<vector<int>, vector<int>> minVertexCover() {
        vector<int> matchL(n, -1);
        for (int v = 0; v < m; v++) if (mt[v] >= 0) matchL[mt[v]] = v;
        vector<bool> zl(n, false), zr(m, false);
        vector<int> st;
        for (int u = 0; u < n; u++) if (matchL[u] < 0) zl[u] = true, st.push_back(u);
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : adj[u]) if (!zr[v]) {
                zr[v] = true;
                if (mt[v] >= 0 && !zl[mt[v]]) zl[mt[v]] = true, st.push_back(mt[v]);
            }
        }
        vector<int> L, R;
        for (int u = 0; u < n; u++) if (!zl[u]) L.push_back(u);
        for (int v = 0; v < m; v++) if (zr[v]) R.push_back(v);
        return {L, R};                          // |L| + |R| == maxMatching()
    }
};
/* THEOREMS you actually quote (bipartite graphs):
   Konig:    max matching = min vertex cover
   max independent set = V - min vertex cover
   MIN PATH COVER of a DAG (VERTEX-DISJOINT paths covering all vertices)
             = n - max matching of the split graph (u_out -> v_in for every edge)
             CAREFUL: this is the vertex-disjoint cover. To get Dilworth (FINITE poset)'s width (minimum
             number of CHAINS, i.e. paths allowed to share vertices) run it on the
             TRANSITIVE CLOSURE of the DAG first.
   Hall:     a perfect matching on the left exists iff |N(S)| >= |S| for every subset S of the LEFT part X
   Dilworth: min chain cover of a poset = longest antichain (build the DAG, TRANSITIVELY CLOSE
             it, then use the path cover above)
   Mirsky:   the dual - longest chain = minimum number of antichains partitioning the poset
   Gallai:   alpha + tau = n always;  nu + rho = n only if there are NO ISOLATED VERTICES
             (alpha = max independent set, tau = min vertex cover, nu = max matching,
              rho = min edge cover)
   Hall defect: nu(G) = |X| - max over W subset of X of ( |W| - |N(W)| )
   Tutte:    G has a perfect matching iff odd(G - U) <= |U| for every U
   Tutte-Berge: nu(G) = (1/2) * min over U of ( |V| + |U| - odd(G - U) )
   Menger:   min edge cut(s,t) = max edge-disjoint s-t paths;
             min vertex cut(s,t) = max internally-vertex-disjoint s-t paths (s,t non-adjacent) */
```

### 3.7.3 Hungarian

HUNGARIAN ALGORITHM - minimum-cost perfect matching on a DENSE bipartite graph, O(n^2 m).

```
// a is 1-indexed: a[1..n][1..m] with n <= m. Returns the cost; ans[j] = row matched to column j.
// For MAXIMUM cost, negate the matrix.
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a) {
    int n = a.size() - 1, m = a[0].size() - 1;
    vector<ll> u(n + 1, 0), v(m + 1, 0);
    vector<int> p(m + 1, 0), way(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> minv(m + 1, llinf);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            ll delta = llinf;
            for (int j = 1; j <= m; j++) if (!used[j]) {
                ll cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            for (int j = 0; j <= m; j++)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do { int j1 = way[j0]; p[j0] = p[j1], j0 = j1; } while (j0);
    }
    vector<int> ans(m + 1);
    for (int j = 1; j <= m; j++) ans[j] = p[j];
    return {-v[0], ans};                        // -v[0] is the optimal total cost
}
// Use it for: assignment problems, "match every task to a worker minimising total time",
// minimum-cost perfect matching in a complete bipartite graph.
// FORBIDDEN PAIRS: use a large finite cost (1e15), NOT llinf - the reduced cost a[i][j]-u[i]-v[j]
// would overflow.  ans[j] == 0 means column j is unmatched ( possible only when n < m).
// WHICH ONE: on a DENSE / complete graph this beats MCMF - n^2*m here versus n augmentations of
// Dijkstra over n^2 arcs (n^3 log n) there. Use MCMF only when the graph is genuinely SPARSE.
```

### 3.7.4 Blossom

MAXIMUM MATCHING IN A GENERAL (NON-BIPARTITE) GRAPH - Edmonds' blossom algorithm, O(V^3).

```
// Everything else in this folder assumes bipartite. Reach for this the moment the two sides are
// not fixed: "pair up people who are compatible", "tile a board where the colouring argument
// fails", "maximum set of disjoint edges" on an arbitrary graph.
// The trick it exists for: an ODD cycle can be traversed in two directions, so an alternating
// search can enter and leave it on the same side. Edmonds CONTRACTS such a cycle (a "blossom")
// into a single vertex, searches the contracted graph, and expands afterwards.
// Certificate: the answer equals the Tutte-Berge bound (see 10 - Theorems/01 - GraphTheorems).
struct Blossom {
    int n;
    vector<vector<char>> g;                                          // adjacency MATRIX
    vector<int> match, p, base;
    vector<char> used, inBlossom;
    Blossom(int n) : n(n), g(n, vector<char>(n, 0)), match(n, -1), p(n), base(n),
                     used(n), inBlossom(n) {}
    void addEdge(int u, int v) { g[u][v] = g[v][u] = 1; }
    int lca(int a, int b) {
        vector<char> seen(n, 0);
        for (;;) { a = base[a], seen[a] = 1; if (match[a] < 0) break; a = p[match[a]]; }
        for (;;) { b = base[b]; if (seen[b]) return b; b = p[match[b]]; }
    }
    void mark(int v, int b, int child) {                             // walk v up to the blossom base
        while (base[v] != b) {
            inBlossom[base[v]] = inBlossom[base[match[v]]] = 1;
            p[v] = child, child = match[v], v = p[match[v]];
        }
    }
    bool augment(int root) {                                         // one BFS for an augmenting path
        fill(all(used), 0), fill(all(p), -1);
        iota(all(base), 0);
        used[root] = 1;
        queue<int> q({root});
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int to = 0; to < n; to++) {
                if (!g[v][to] || base[v] == base[to] || match[v] == to) continue;
                if (to == root || (match[to] >= 0 && p[match[to]] >= 0)) {   // odd cycle -> contract
                    int b = lca(v, to);
                    fill(all(inBlossom), 0);
                    mark(v, b, to), mark(to, b, v);
                    for (int i = 0; i < n; i++) if (inBlossom[base[i]]) {
                        base[i] = b;
                        if (!used[i]) used[i] = 1, q.push(i);
                    }
                } else if (p[to] < 0) {
                    p[to] = v;
                    if (match[to] < 0) {
    // free vertex: flip the path
                        for (int u = to, pv, nv; u >= 0; u = nv)
                            pv = p[u], nv = match[pv], match[u] = pv, match[pv] = u;
                        return true;
                    }
                    used[match[to]] = 1, q.push(match[to]);
                }
            }
        }
        return false;
    }
    int solve() {
        int r = 0;
        for (int v = 0; v < n; v++)                                  // greedy start: fewer BFS rounds
            if (match[v] < 0)
                for (int u = 0; u < n; u++)
                    if (g[v][u] && match[u] < 0) { match[v] = u, match[u] = v, r++; break; }
        for (int v = 0; v < n; v++) if (match[v] < 0) r += augment(v);
        return r;                                                    // match[v] = partner, or -1
    }
};
// PERFECT MATCHING EXISTS iff solve() * 2 == n. Tutte's theorem gives the certificate when it
// does not: some U with odd(G - U) > |U|.
// MINIMUM EDGE COVER (no isolated vertices) = n - maximum matching (Gallai).
// MAXIMUM INDEPENDENT SET is still NP-hard here - Konig's alpha = n - nu is BIPARTITE ONLY.
// WEIGHTED general matching needs the dual-variable version (~150 lines); if the graph is small
// (n <= 20) a bitmask DP over matched vertices is far shorter and usually enough.
// CHINESE POSTMAN (undirected): make every degree even by duplicating a minimum-weight perfect
// matching on the ODD-degree vertices, using all-pairs shortest paths as the edge weights, then
// take an Eulerian circuit. The MIXED version (some edges directed) is NP-hard.
// THE MATCHING GAME (a token moves along edges to unvisited vertices; stuck loses): the first
// player wins iff the start vertex is covered by EVERY maximum matching, i.e. iff deleting it
// decreases the matching size. With this file that works on a general graph, not just bipartite.
```

## 3.8 Spanning Trees

### 3.8.1 MST

MINIMUM SPANNING TREE. Kruskal O(m log m) - use on sparse graphs / when you also want the

```
// Needs: DSU (02 - Data Structures/05 - DSU/01).
// KRUSKAL RECONSTRUCTION TREE. Prim O(n^2) - use on dense or IMPLICIT complete graphs.
struct Edge { int u, v; ll w; bool operator<(const Edge& o) const { return w < o.w; } };

ll kruskal(int n, vector<Edge> e, vector<Edge>* used = nullptr) {
    sort(all(e));
    DSU d(n);
    ll tot = 0;
    for (auto& x : e) if (d.join(x.u, x.v)) { tot += x.w; if (used) used->push_back(x); }
    return tot;                                   // check d.size(0) == n for connectivity
}
// Dense / implicit graphs: cost(i,j) given by a function. O(n^2), no edge list needed.
template <class F>
ll prim(int n, F cost) {
    vector<ll> best(n, llinf);
    vector<bool> in(n, false);
    best[0] = 0;
    ll tot = 0;
    for (int it = 0; it < n; it++) {
        int u = -1;
        for (int i = 0; i < n; i++) if (!in[i] && (u < 0 || best[i] < best[u])) u = i;
        in[u] = true, tot += best[u];
        for (int v = 0; v < n; v++) if (!in[v]) best[v] = min(best[v], cost(u, v));
    }
    return tot;
}
// KRUSKAL RECONSTRUCTION TREE: binary tree with 2n-1 nodes; internal node = merging edge weight.
// Then min possible MAX edge on a path u->v = val[lca(u,v)], and "all vertices reachable using
// edges of weight <= w" is exactly a subtree. Turns "binary search the threshold + DSU" into O(1).
struct KRT {
    int n, cnt;
    vector<ll> val;
    vector<array<int, 2>> ch;
    KRT(int n, vector<Edge> e) : n(n), cnt(n), val(2 * n, 0), ch(2 * n, {-1, -1}) {
        sort(all(e));
        DSU d(2 * n);
        vector<int> top(2 * n);
        iota(all(top), 0);
        for (auto& x : e) {
            int a = top[d.find(x.u)], b = top[d.find(x.v)];
            if (a == b) continue;
            val[cnt] = x.w, ch[cnt] = {a, b};
            d.join(x.u, x.v);
            top[d.find(x.u)] = cnt++;
        }
    }                                             // root = cnt-1 (if connected); then build LCA on it
};
// SECOND-BEST MST: build the MST, then for each non-tree edge (u,v,w) the answer candidate is
// mst - maxEdgeOnPath(u,v) + w. Get maxEdgeOnPath with binary lifting or the KRT.
```

### 3.8.2 DirectedMSTandExtras

DIRECTED MST (Chu-Liu / Edmonds) - minimum spanning ARBORESCENCE rooted at r, O(V*E).

```
// Needs: struct Edge {int u,v; ll w;} and DSU (both from 01 - MST.cpp).
// Every non-root vertex takes its cheapest incoming edge; if that creates a cycle, contract it,
// subtract the chosen weights, and repeat. Returns -1 if some vertex is unreachable from r.
ll directedMST(int n, int r, vector<Edge> e) {          // Edge = {u, v, w} meaning u -> v
    ll total = 0;
    vector<int> id(n), par(n);
    while (true) {
        vector<ll> best(n, llinf);
        vector<int> from(n, -1);
        for (auto& x : e) if (x.u != x.v && x.w < best[x.v]) best[x.v] = x.w, from[x.v] = x.u;
        best[r] = 0;
        for (int i = 0; i < n; i++) if (i != r && from[i] < 0) return -1;   // unreachable
        int cnt = 0;
        fill(all(id), -1), fill(all(par), -1);
        for (int i = 0; i < n; i++) {
            if (i != r) total += best[i];
            int v = i;
            while (v != r && par[v] == -1 && id[v] == -1) par[v] = i, v = from[v];
            if (v != r && id[v] == -1 && par[v] == i) {  // found a new cycle
                for (int u = from[v]; u != v; u = from[u]) id[u] = cnt;
                id[v] = cnt++;
            }
        }
        if (!cnt) break;                                // no cycle -> done
        for (int i = 0; i < n; i++) if (id[i] == -1) id[i] = cnt++;
        for (auto& x : e) {
            int u = x.u, v = x.v;
            x.u = id[u], x.v = id[v];
            if (x.u != x.v) x.w -= best[v];              // reduced cost inside the contraction
        }
        n = cnt, r = id[r];
    }
    return total;
}

// SECOND-BEST MST, O(E log V + E * log V). The second-best tree differs from the MST by exactly
// ONE edge swap. For each non-tree edge (u,v,w) the candidate is mst - maxEdgeOnPath(u,v) + w.
// You must track BOTH the maximum and the strict second maximum on the path, because if the
// maximum equals w the swap is not a real change - use the second maximum instead.
// With binary lifting: up[k][v], mx1[k][v], mx2[k][v] merged pairwise while lifting.
// (The Kruskal Reconstruction Tree in 01 - MST.cpp gives maxEdgeOnPath directly as val[lca].)

// BORUVKA - MST in O(E log V) by phases; each phase every component picks its cheapest outgoing
// edge and they are all merged at once. Use it when the edge set is IMPLICIT:
//   XOR-MST      : cheapest cross edge per component via a binary trie
//   Manhattan MST: cheapest cross edge per octant (see Transforms.cpp)
ll boruvka(int n, const vector<Edge>& e) {
    DSU d(n);
    ll total = 0;
    int comps = n;
    while (comps > 1) {
        vector<int> best(n, -1);
        for (int i = 0; i < (int)e.size(); i++) {
            int a = d.find(e[i].u), b = d.find(e[i].v);
            if (a == b) continue;
            if (best[a] < 0 || e[i].w < e[best[a]].w) best[a] = i;
            if (best[b] < 0 || e[i].w < e[best[b]].w) best[b] = i;
        }
        bool any = false;
        for (int i = 0; i < n; i++) if (best[i] >= 0) {
            auto& x = e[best[i]];
            if (d.join(x.u, x.v)) total += x.w, comps--, any =
```

```
            true;
        }
        if (!any) break;                                    //
        disconnected
    }
    return comps == 1 ? total : -1;
}
```

### 3.8.3 SecondBestMST

SECOND-BEST SPANNING TREE - cheapest spanning tree whose weight is STRICTLY greater than the MST.

```
// Needs: Edge, kruskal (01 - MST.cpp).
// It differs from the MST in exactly one edge: add a non-tree
     edge (u,v,w), drop the heaviest edge
// on the tree path u->v whose weight is < w. By the cycle
     property every edge on that path has
// weight <= w, so "heaviest edge < w" is either the path maximum
     (if it is < w) or the second
// LARGEST DISTINCT value on the path. Binary lifting carries both
     . O(m log n).
ll secondBestMST(int n, const vector<Edge>& e) {
    vector<Edge> used;
    ll mst = kruskal(n, e, &used);
    if ((int)used.size() != n - 1) return -1;                   //
      disconnected
    vector<vector<pair<int, ll>>> g(n);
    for (auto& x : used) g[x.u].push_back({x.v, x.w}), g[x.v].
     push_back({x.u, x.w});
    typedef array<ll, 2> T;                                     //
      {largest, 2nd largest DISTINCT}
    const T NONE = {LLONG_MIN, LLONG_MIN};
    auto join = [&](T a, T b) {
        array<ll, 4> v = {a[0], a[1], b[0], b[1]};
        sort(v.rbegin(), v.rend());
        T r = {v[0], LLONG_MIN};
        for (ll x : v) if (x < v[0]) { r[1] = x; break; }
        return r;
    };
    int L = 1;
    while ((1 << L) < n) L++;
    vector<vector<int>> up(L + 1, vector<int>(n, 0));
    vector<vector<T>> mx(L + 1, vector<T>(n, NONE));
    vector<int> dep(n, 0), st{0};
    vector<char> vis(n, 0);
    vis[0] = 1;
    while (!st.empty()) {                                       //
      iterative DFS from 0
        int u = st.back(); st.pop_back();
        for (auto [v, w] : g[u]) if (!vis[v])
            vis[v] = 1, dep[v] = dep[u] + 1, up[0][v] = u, mx[0][
     v] = {w, LLONG_MIN}, st.push_back(v);
    }
    for (int k = 1; k <= L; k++)
        for (int v = 0; v < n; v++) {
            int m = up[k - 1][v];
            up[k][v] = up[k - 1][m], mx[k][v] = join(mx[k - 1][v
     ], mx[k - 1][m]);
        }
    auto path = [&](int u, int v) {                             //
      top-2 distinct weights on u->v
        T r = NONE;
        if (dep[u] < dep[v]) swap(u, v);
        for (int k = L; k >= 0; k--) if (dep[up[k][u]] >= dep[v])
      r = join(r, mx[k][u]), u = up[k][u];
        if (u == v) return r;
        for (int k = L; k >= 0; k--) if (up[k][u] != up[k][v])
            r = join(join(r, mx[k][u]), mx[k][v]), u = up[k][u],
     v = up[k][v];
        return join(join(r, mx[0][u]), mx[0][v]);
    };
    ll best = LLONG_MAX;
    for (auto& x : e) {                                         //
      tree edges give p = {w, MIN} and
        T p = path(x.u, x.v);                                   //
      fall through to the MIN branch
        ll rem = p[0] < x.w ? p[0] : p[1];
        if (rem != LLONG_MIN) best = min(best, mst - rem + x.w);
    }
    return best == LLONG_MAX ? -1 : best;
     // -1: no second tree exists
}
// K-TH BEST SPANNING TREE: repeat the swap argument with a
     priority queue over (tree, forced-in /
// forced-out edge sets) - Gabow's algorithm, O(k m log n). Rare;
     the second-best case is the one
// that actually shows up.
// MINIMUM BOTTLENECK SPANNING TREE = any MST (minimising the
     maximum edge is implied by Kruskal).
// MST OF A GRAPH PLUS ONE NEW EDGE: add it, find the cycle, drop
     the cycle maximum. O(n).
// MINIMUM SPANNING TREE WITH DEGREE CONSTRAINT at one vertex r:
     build the MST of G - r, then add
// the cheapest edge from r into each component, then improve by
     swaps while degree < k.
```

## 3.9 Exact Exponential

### 3.9.1 CliqueColouringMIS

EXACT ALGORITHMS FOR NP-HARD GRAPH PROBLEMS ON SMALL n. The constraint IS the hint:

```
//   n <= 20-24  -> plain subset DP, O(2^n * n)
//   n <= 40-50  -> meet in the middle, or Bron-Kerbosch (fast in
    practice on sparse graphs)
//   n <= 100+   -> only if the graph is bipartite/perfect/a tree
    - then it is polynomial (Konig)
// adj[] is an ADJACENCY BITMASK: bit j of adj[i] is set iff i-j
    is an edge. No self loops.

// --- MAXIMUM CLIQUE, Bron-Kerbosch with PIVOTING. Worst case O
    (3^(n/3)), very fast in practice. ---
// R = clique built so far, P = candidates, X = already-explored
    vertices (avoids duplicates).
// The pivot u in P|X is chosen to maximise |P & adj[u]|; only P \
     adj[u] needs branching, because
// any maximal clique either omits u or contains a non-neighbour
    of u.
int bestClique;
unsigned bestSet;
void bk(unsigned R, unsigned P, unsigned X, const vector<unsigned
    >& adj) {
    if (!P && !X) {
        if (__builtin_popcount(R) > bestClique) bestClique =
    __builtin_popcount(R), bestSet = R;
        return;
    }
    int u = __builtin_ctz(P | X), best = -1;
    for (unsigned t = P | X; t; t &= t - 1) {
        int v = __builtin_ctz(t), c = __builtin_popcount(P & adj[
    v]);
        if (c > best) best = c, u = v;
    }
    for (unsigned cand = P & ~adj[u]; cand; cand &= cand - 1) {
        int v = __builtin_ctz(cand);
        bk(R | 1u << v, P & adj[v], X & adj[v], adj);
        P &= ~(1u << v), X |= 1u << v;
    }
}
int maxClique(const vector<unsigned>& adj) {                    //
     n <= 32
    bestClique = 0, bestSet = 0;
    bk(0, (1u << adj.size()) - 1, 0, adj);
    return bestClique;                                          //
     the clique itself is in bestSet
}
// MAXIMUM INDEPENDENT SET = maximum clique of the COMPLEMENT
    graph. Just flip adj and mask off i.
// MINIMUM VERTEX COVER = n - MIS. On BIPARTITE graphs both are
    polynomial: Konig says
//   min vertex cover = maximum matching, and MIS = n - matching (
    see 07 - Matching).

// --- CHROMATIC NUMBER, O(2^n * n) by inclusion-exclusion (
    Bjorklund-Husfeldt-Koivisto). ---
// Let ind[S] = 1 if S is independent. The number of ways to cover
     V with k ORDERED independent
// sets is  sum_S (-1)^(n-|S|) * I(S)^k,  where I(S) = #
    independent subsets of S. The chromatic
// number is the smallest k making that non-zero. Compute I over
    subsets with one SOS pass.
int chromatic(const vector<unsigned>& adj) {                    //
     n <= 20 with this modulus
    int n = adj.size(), N = 1 << n;
    if (!n) return 0;
    const ll M = (1LL << 61) - 1;                               //
     any large prime works
    vector<ll> I(N), pw(N, 1);
    I[0] = 1;
    for (int S = 1; S < N; S++) {                               //
     I[S] = #independent subsets of S
        int v = __builtin_ctz(S), rest = S ^ 1 << v;            //
     (v in it, or not)
        I[S] = (I[rest] + I[rest & ~adj[v]]) % M;
    }
    for (int k = 1; k <= n; k++) {
        ll tot = 0;
        for (int S = 0; S < N; S++) {
            pw[S] = (__int128)pw[S] * I[S] % M;                 //
     I(S)^k
            ll t = (n - __builtin_popcount(S)) & 1 ? M - pw[S] :
    pw[S];
            tot = (tot + t) % M;
        }
        if (tot % M) return k;
    }
    return n;
}
```

```
// The count is done MODULO a prime, so a zero could in principle
    be a false negative; with a
// 61-bit modulus that never happens on contest inputs. Check
    bipartiteness first - k <= 2 is O(n+m).

// --- MINIMUM STEINER TREE (connect a set of k terminals in a
    weighted graph), O(3^k n + 2^k n^2).
// dp[S][v] = cheapest tree containing terminal set S and touching
     vertex v.
//   merge:  dp[S][v] = min over submasks T of dp[T][v] + dp[S\T][
    v]
//   grow:   dp[S][v] = min over edges (u,v) of dp[S][u] + w
        (run Dijkstra per S)
ll steinerTree(int n, const vector<vector<pair<int, ll>>>& g,
    const vector<int>& term) {
    int k = term.size();
    if (!k) return 0;
    vector<vector<ll>> dp(1 << k, vector<ll>(n, llinf));
    for (int i = 0; i < k; i++) dp[1 << i][term[i]] = 0;
    for (int S = 1; S < 1 << k; S++) {
        for (int T = (S - 1) & S; T; T = (T - 1) & S)
            for (int v = 0; v < n; v++)
                if (dp[T][v] < llinf && dp[S ^ T][v] < llinf)
                    dp[S][v] = min(dp[S][v], dp[T][v] + dp[S ^ T
    ][v]);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
    greater<>> pq;
        for (int v = 0; v < n; v++) if (dp[S][v] < llinf) pq.push
    ({dp[S][v], v});
        while (!pq.empty()) {                                   //
     Dijkstra to relax the tree outward
            auto [d, u] = pq.top(); pq.pop();
            if (d > dp[S][u]) continue;
            for (auto [v, w] : g[u]) if (dp[S][u] + w < dp[S][v])
                dp[S][v] = dp[S][u] + w, pq.push({dp[S][v], v});
        }
    }
    return *min_element(all(dp[(1 << k) - 1]));
}
// STEINER TREE IN A GRID / with k up to ~10 is the usual contest
    size. If k == 2 it is a shortest
// path; if k == n it is the MST; the DP interpolates between them
    .
// RELATED EXACT-EXPONENTIAL TOOLS:
//   * HAMILTONIAN PATH / TSP: dp[mask][v], O(2^n n^2). See 08 -
    DP/02 - BitmaskDP.
//   * SET COVER: dp over covered elements, O(2^n * #sets), or IE
    like the chromatic number.
//   * GRAPH COLOURING with k fixed: try to 3-colour by 2-SAT? No
    - 3-colouring is NP-hard; use
//     the IE formula above, or branch on the highest-degree
    vertex with memoisation.
//   * MAXIMUM INDEPENDENT SET for n <= 40-50: meet in the middle
    - split the vertices in half,
//     enumerate independent subsets of the left half, and for
    each store the best independent
//     subset of the right half compatible with its neighbourhood
    (SOS over the right mask).
//   * COUNTING INDEPENDENT SETS / colourings on a graph of small
    TREEWIDTH: DP over a tree
//     decomposition, O(2^tw * n) - usually a grid problem in
    disguise (see broken-profile DP).
```

## 3.10 Theorems

### 3.10.1 GraphTheorems

GRAPH THEOREM SHEET ==========

```
/* ==================== GRAPH THEOREM SHEET ====================
 Every hypothesis here matters. A theorem quoted without its
    precondition is a wrong answer.

 MATCHING AND COVERING
   KONIG (BIPARTITE only):  max matching = min vertex cover.
     Constructive: Z = vertices reachable by alternating paths
     from the UNMATCHED left vertices;
     the cover is (X \ Z) union (Y and Z).
   GALLAI: for EVERY graph on n vertices,  alpha + tau = n   (
     independent set + vertex cover).
           for every graph WITH NO ISOLATED VERTEX,  nu + rho = n
       (matching + edge cover).
           Bipartite + Konig gives  alpha = n - nu.
   HALL (bipartite): a matching saturating every vertex of the
     LEFT part X exists iff
     |N(S)| >= |S| for every S contained in X.   (Not every S of V
      - the side matters.)
   ORE'S DEFECT FORM:  nu(G) = |X| - max over S contained in X of
     ( |S| - |N(S)| ).
   HALL FOR REGULAR BIPARTITE: every k-regular bipartite graph (k
     >= 1) has a perfect matching,
     so its edges split into exactly k perfect matchings.
     Equivalently KONIG'S EDGE COLOURING
     THEOREM: chi'(G) = Delta(G) for every bipartite MULTIGRAPH. (
     Timetabling, "split into
     Delta rounds", Latin-square completion.)
   BERGE: a matching is maximum iff there is no augmenting path. (
     Why Kuhn / HK / Blossom stop.)
   TUTTE (general graphs): G has a PERFECT matching iff odd(G - U)
      <= |U| for every U contained
     in V, where odd(H) = the number of components of H with an
     ODD number of vertices.
     U = empty forces n even.
   TUTTE-BERGE:  nu(G) = (1/2) * min over U of ( |V| + |U| - odd(G
      - U) ).
   DILWORTH (FINITE poset): min number of chains covering P = size
      of the maximum antichain.
   MIRSKY (dual): min number of antichains partitioning P = length
      of the longest chain.
   SPERNER: the largest antichain in the subset lattice of [n] has
      size C(n, floor(n/2)).
   PATH COVERS - the distinction that most notebooks get wrong:
     min VERTEX-DISJOINT path cover of a DAG = n - (max matching
     of the split graph, no closure).
     min chain cover for Dilworth = n - (max matching of the
     TRANSITIVE CLOSURE's split graph).

 CONNECTIVITY AND FLOW
   MENGER: for NON-ADJACENT s, t: min s-t VERTEX cut = max
     internally-vertex-disjoint s-t paths.
     For any s != t: min s-t EDGE cut = max edge-disjoint s-t
     paths.
   WHITNEY: kappa(G) <= lambda(G) <= delta(G)  (vertex conn <=
     edge conn <= min degree).
     A graph on >= 3 vertices is 2-connected iff every two
     vertices lie on a common cycle.
   MAX-FLOW MIN-CUT, with INTEGRALITY: integer capacities => some
     maximum flow is integral (and
     Dinic produces one). Corollary used constantly: a 0/1-
     capacity max flow decomposes into that
     many edge-disjoint s-t paths.
   ROBBINS: an undirected graph has a STRONG orientation iff it is
      connected and BRIDGELESS.
     Constructive in O(V+E): DFS, orient tree edges away from the
     root and back edges toward it.
   ESWARAN-TARJAN (augmentation):
     directed - min arcs to make a digraph strongly connected =
     max(#sources, #sinks) of the
       condensation (0 if the condensation is a single node).
     undirected - for a CONNECTED graph, min edges to make it 2-
     edge-connected = ceil(L/2) where
       L is the number of leaves of the bridge tree (0 if the
     bridge tree is one node).

 DEGREE SEQUENCES
   ERDOS-GALLAI: d1 >= ... >= dn >= 0 is the degree sequence of a
     SIMPLE graph iff sum d_i is
     even and for every k in [1, n]:
       sum_{i<=k} d_i  <=  k(k-1) + sum_{i>k} min(d_i, k).
     Only k up to the largest index with d_k >= k needs checking (
     the Durfee square), so it is
     O(n) after sorting with prefix sums.
   HAVEL-HAKIMI: graphical iff d1 <= n-1 and, after deleting d1
     and subtracting 1 from the next
     d1 entries (re-sorted), the rest is graphical. Gives a
     CONSTRUCTION, which Erdos-Gallai
     does not. O(n^2) naive.
   LANDAU (tournaments): s1 <= ... <= sn is a score sequence iff
     sum_{i<=k} s_i >= C(k,2) for
     every k, WITH EQUALITY at k = n.

 TOURNAMENTS
   REDEI: every finite tournament has a Hamiltonian PATH (
     insertion sort, O(n log n)).
   CAMION: every STRONGLY CONNECTED tournament has a Hamiltonian
     CYCLE.
   MOON: every strongly connected tournament is vertex-pancyclic -
      for every vertex v and every
     k in [3, n] there is a k-cycle through v.

 COUNTING
   MATRIX-TREE (Kirchhoff), undirected: #spanning trees (or the
     sum over trees of the product of
     weights) = ANY cofactor of L = D - A, loops excluded.
   MATRIX-TREE, DIRECTED (Tutte), with A_ij = #arcs i -> j:
     L_out = D_out - A, delete row AND column r  ->  arborescences
      CONVERGING TO r (in-trees).
     L_in  = D_in  - A, delete row AND column r  ->  arborescences
      DIVERGING FROM r (out-trees).
     These two are the pair everyone swaps. Verify on the 2-vertex
      graph 1 -> 2.
   BEST THEOREM: for a CONNECTED digraph with indeg(v) = outdeg(v)
      everywhere, the number of
     Eulerian circuits up to starting point is  ec(G) = t_w(G) *
     prod_v (outdeg(v) - 1)!,
     where t_w is the number of arborescences CONVERGING TO any
     fixed w (the L_out form above);
     t_w is the same for every w in such a graph. Multiply by
     outdeg(v) to fix a start vertex.
   CAYLEY: n^(n-2) labelled trees; with prescribed degrees, (n-2)!
      / prod (d_i - 1)!;
     labelled forests with k trees whose roots are k specified
     vertices: k * n^(n-k-1).

 EULERIAN CONDITIONS  (the check EulerianPath.cpp needs before it
     starts)
   UNDIRECTED: a circuit exists iff every degree is even and all
     EDGES lie in one component.
     A trail exists iff exactly 0 or 2 vertices have odd degree (
     start at an odd one), same
     connectivity condition.
   DIRECTED: a circuit exists iff indeg = outdeg at every vertex
     and all vertices of nonzero
     degree lie in one component of the underlying undirected
     graph. A trail exists iff exactly
     one vertex has out - in = 1 (the start), exactly one has in -
      out = 1 (the end), the rest
     are balanced, same connectivity condition.

 COLOURING
   BROOKS: for connected G, chi(G) <= Delta(G) UNLESS G is
     complete or an odd cycle, where
     chi = Delta + 1. Greedy on any order gives Delta+1; greedy on
      a degeneracy order gives d+1.
   VIZING (SIMPLE graphs): Delta <= chi' <= Delta + 1. Multigraphs
     : chi' <= Delta + mu, and
     Shannon's chi' <= floor(3 Delta / 2). Deciding class 1 vs
     class 2 is NP-complete - do not
     look for a polynomial algorithm mid-contest. Bipartite is the
      exception (Konig above).
   GALLAI-HASSE-ROY-VITAVER: chi(G) = 1 + min over orientations of
      (longest directed path length).
     Equivalently every orientation contains a directed path on
     chi(G) vertices.

 EXTREMAL
   TURAN: a K_{r+1}-free graph has e <= (1 - 1/r) n^2 / 2, with
     equality only for the balanced
     complete r-partite Turan graph. r = 2 is MANTEL: triangle-
     free => e <= floor(n^2/4).
   Every graph has a bipartite subgraph with >= m/2 edges (greedy
     max-cut).
   A graph is bipartite iff it has no odd cycle.
   RAMSEY, the known small values: R(3,3)=6, R(3,4)=9, R(3,5)=14,
     R(3,6)=18, R(3,7)=23,
     R(3,8)=28, R(3,9)=36, R(4,4)=18, R(4,5)=25. R(4,6) in [36,40]
      and R(5,5) in [43,46] are
     OPEN. Bound: R(s,t) <= C(s+t-2, s-1).
   NASH-WILLIAMS ARBORICITY: a(G) = max over subgraphs S with >= 2
      vertices of
     ceil( m_S / (n_S - 1) ). Consequences: degeneracy d satisfies
      a <= d <= 2a - 1; m <= a(n-1);
     triangle enumeration is O(m * a).
   TUTTE / NASH-WILLIAMS TREE PACKING: G has t edge-disjoint
     spanning trees iff every partition
```

```
    of V into k non-empty parts has at least t(k-1) crossing
    edges.
  PETERSEN: every BRIDGELESS CUBIC graph has a perfect matching.
    Every 2k-regular graph splits
    into k edge-disjoint 2-factors, so it has an Eulerian circuit
     and an orientation with
    indeg = outdeg = k.

PLANARITY
  EULER: connected plane graph V - E + F = 2; with c components,
    V - E + F = 1 + c.
  Simple planar with V >= 3: E <= 3V - 6. Triangle-free (girth >=
     4): E <= 2V - 4.
  Every planar graph has a vertex of degree <= 5, i.e. it is 5-
    degenerate.
  KURATOWSKI / WAGNER: planar iff no K5 and no K_{3,3}
    subdivision (minor).
  FOUR COLOUR: every planar graph is 4-colourable - but there is
    no usable algorithm, so never
    plan around it. FIVE COLOUR is constructive in O(n) from the
    degree-<=5 vertex.
  K5, K_{3,3} and the Petersen graph are non-planar. In a planar
    graph min cut = shortest path
    in the dual.

HAMILTONICITY (sufficient conditions only - the general problem
    is NP-complete)
  DIRAC: n >= 3 and min degree >= n/2  =>  Hamiltonian.
  ORE: deg(u) + deg(v) >= n for every non-adjacent pair  =>
    Hamiltonian.
  Both are constructive by rotation-extension.

TREE FACTS
  JORDAN: every tree has 1 or 2 CENTRES; if 2 they are adjacent;
    they are the middle of any
    diameter path.
  CENTROID: every tree has 1 or 2 centroids; c is a centroid iff
    every component of T - c has
    <= n/2 vertices iff c minimises the sum of distances. If 2
    they are adjacent and n is even.
    That <= n/2 bound is exactly what makes centroid
    decomposition O(log n) deep.
  DIAMETER: the greedy two-BFS is valid on trees and on
    positively weighted trees, NOT on
    general graphs. Every longest path passes through a centre.
    Joining two trees by an edge
    gives diameter max(d1, d2, r1 + r2 + 1), minimised by joining
     centre to centre.
  A tree is bipartite, has >= 2 leaves, and sum of degrees = 2(n
    -1).

WHAT IS NP-HARD, AND WHAT ONLY LOOKS IT
  NP-hard: max clique, chromatic number (>= 3 colours),
    Hamiltonian path/cycle, TSP, MIXED
    Chinese postman, vertex cover / independent set on general
    graphs, max cut (general),
    Steiner tree with unbounded terminals, longest path,
    dominating set, feedback vertex/arc
    set, edge-colouring class decision, bandwidth.
  Polynomial despite appearances: vertex cover and independent
    set on BIPARTITE graphs (Konig)
    and on trees / interval / chordal graphs; 2-colouring; 2-SAT;
     max cut on PLANAR graphs;
    min mean cycle; min-cost flow; directed MST; maximum matching
     in a GENERAL graph (Blossom);
    min path cover of a DAG; Steiner tree with k <= ~15 terminals
    ; TSP with n <= ~20.
============================================================ */
```

## 3.11 Special Structures

### 3.11.1 StableChordalComplement

THREE GRAPH TOOLS THAT ARE NOT ABOUT PATHS OR FLOW.

```
// --- GALE-SHAPLEY STABLE MARRIAGE, O(n^2). ---
// n proposers, n receivers, each with a full preference ranking.
     A matching is STABLE if no pair
// mutually prefer each other to their assigned partners. One
     always exists, and this finds the
// PROPOSER-OPTIMAL one: every proposer gets the best partner they
      could have in ANY stable
// matching, and simultaneously every receiver gets their worst. (
     Swap the roles to flip that.)
// pref[i] = the proposers' preference lists; rank[j][i] =
     receiver j's ranking of proposer i.
vector<int> stableMarriage(const vector<vector<int>>& pref, const
      vector<vector<int>>& rank) {
    int n = pref.size();
    vector<int> match(n, -1), nxt(n, 0);                      //
      match[receiver] = proposer
    vector<int> free_(n);
    iota(all(free_), 0);
    while (!free_.empty()) {
        int m = free_.back(); free_.pop_back();
        int w = pref[m][nxt[m]++];                            //
      propose down the list
        if (match[w] < 0) match[w] = m;
        else if (rank[w][m] < rank[w][match[w]]) free_.push_back(
     match[w]), match[w] = m;
        else free_.push_back(m);
    }
    return match;
}
// HOSPITALS/RESIDENTS (one side has capacity c) is the same loop
     with a heap of accepted
// proposers per hospital. STABLE ROOMMATES (one pool, no sides)
     may have NO stable matching -
// Irving's algorithm decides it, and it is a different, longer
     algorithm.

// --- TRAVERSAL OF THE COMPLEMENT GRAPH in O(n + m), never
     materialising the O(n^2) edges. ---
// Keep the not-yet-visited vertices in a linked structure; from u
     , walk that set and step to
// every w that is NOT a neighbour of u. Each vertex leaves the
     set once, and each REAL edge is
// inspected once, so the total is O(n + m).
// Reach for it whenever the input describes the NON-edges: "
     connected components of the graph of
// pairs that do NOT conflict", complement colouring, complement
     BFS distances.
vector<int> complementComponents(int n, const vector<vector<int
     >>& adj) {
    vector<char> isAdj(n, 0);
    set<int> unvis;
    for (int i = 0; i < n; i++) unvis.insert(i);
    vector<int> comp(n, -1);
    int c = 0;
    while (!unvis.empty()) {
        int s = *unvis.begin();
        unvis.erase(unvis.begin());
        vector<int> st{s};
        comp[s] = c;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : adj[u]) isAdj[v] = 1;
            vector<int> take;
            for (int w : unvis) if (!isAdj[w]) take.push_back(w);
            for (int w : take) unvis.erase(w), comp[w] = c, st.
     push_back(w);
            for (int v : adj[u]) isAdj[v] = 0;
        }
        c++;
    }
    return comp;                                              //
      comp[u] = component id
}
// The complement of a DISCONNECTED graph is connected, so a
     complement graph almost always has
// very few components - that is why this is fast in practice as
     well as in theory.

// --- CHORDAL GRAPHS: maximum cardinality search gives a PERFECT
     ELIMINATION ORDERING. ---
// A graph is CHORDAL if every cycle of length >= 4 has a chord.
     Equivalently it has a PEO: an
// order in which every vertex's LATER neighbours form a clique.
// Why you care: on a chordal graph, MAXIMUM CLIQUE, CHROMATIC
     NUMBER, MAXIMUM INDEPENDENT SET
// and MINIMUM CLIQUE COVER are all POLYNOMIAL - the four problems
```

```
      that are NP-hard in general.
// Interval graphs and trees are chordal, so "these intervals
     conflict" problems land here.
vector<int> mcsOrder(int n, const vector<vector<int>>& adj) {
     // reverse of a PEO if chordal
    vector<int> w(n, 0), ord;
    vector<char> used(n, 0);
    for (int it = 0; it < n; it++) {
        int b = -1;
        for (int v = 0; v < n; v++) if (!used[v] && (b < 0 || w[v
     ] > w[b])) b = v;
        used[b] = 1, ord.push_back(b);
        for (int u : adj[b]) if (!used[u]) w[u]++;
    }
    reverse(all(ord));
    return ord;                                                 //
      this IS a PEO iff chordal
}
bool isChordal(int n, const vector<vector<int>>& adj) {
    auto ord = mcsOrder(n, adj);
    vector<int> pos(n);
    for (int i = 0; i < n; i++) pos[ord[i]] = i;
    vector<char> mark(n, 0);
    for (int i = 0; i < n; i++) {                               //
      v's later neighbours must be
        int v = ord[i], par = -1;                               //
      a clique; it is enough to check
        for (int u : adj[v]) if (pos[u] > i && (par < 0 || pos[u]
      < pos[par])) par = u;
        if (par < 0) continue;
        for (int u : adj[par]) mark[u] = 1;
        for (int u : adj[v]) if (pos[u] > i && u != par && !mark[
     u]) {
            for (int x : adj[par]) mark[x] = 0;
            return false;
        }
        for (int u : adj[par]) mark[u] = 0;
    }
    return true;
}
// GIVEN A PEO: greedy colouring in PEO order uses exactly omega
     colours (so chi = omega); the
// maximum clique is the largest "v plus its later neighbours";
     the maximum independent set is the
// greedy "take v, delete its later neighbours" over the PEO.
```

# 4 Number Theory

## 4.1 Euclidean Algorithm

### 4.1.1 ExtendedEuclid

EXTENDED EUCLID - returns g = gcd(a,b) and x, y with a*x + b*y = g. This is the one routine that

```
// gives you modular inverses for a NON-prime modulus, solves
     linear Diophantine equations, and
// underpins CRT. |x| <= b/(2g) and |y| <= a/(2g), so no overflow
     for 64-bit inputs.
long long extGCD(long long a, long long b, long long &x, long long
      &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = extGCD(a, m, x, y);
    if (g != 1) return -1; // Inverse doesn't exist
    return (x % m + m) % m;
}
```

### 4.1.2 LinearDiophantine

Linear Diophantine Equations (2 Variables & N Variables)

```
// 1. Solves a * x + b * y = c (Particular solution & GCD check)
// 2. Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N variables
struct LinearDiophantine {
    static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
        if (b == 0) {
            x = 1; y = 0;
            return a;
        }
        ll x1, y1;
        ll d = ext_gcd(b, a % b, x1, y1);
        x = y1;
        y = x1 - y1 * (a / b);
        return d;
    }

    // Solves a * x + b * y = c. Returns false if no solution
     exists
    static bool solve_2var(ll a, ll b, ll c, ll &x, ll &y, ll &g)
      {
        if (a == 0 && b == 0) {
            if (c == 0) { x = y = g = 0; return true; }
            return false;
        }
        g = ext_gcd(abs(a), abs(b), x, y);
        if (c % g != 0) return false;
        x *= c / g;
        y *= c / g;
        if (a < 0) x = -x;
        if (b < 0) y = -y;
        return true;
    }

    // Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N
     variables.
    // Returns true if a solution exists and populates x vector
    static bool solve_nvar(const vector<ll>& a, ll c, vector<ll>&
      x) {
        int n = a.size();
        if (n == 0) return c == 0;
        x.assign(n, 0);

        vector<ll> prefix_g(n);
        prefix_g[0] = abs(a[0]);
        for (int i = 1; i < n; i++) prefix_g[i] = std::gcd(
     prefix_g[i - 1], a[i]);

        if (prefix_g.back() == 0) return c == 0;
        if (c % prefix_g.back() != 0) return false;

        ll cur_c = c;
        for (int i = n - 1; i > 0; i--) {
            ll x_prev, y_curr, g;
            solve_2var(prefix_g[i - 1], a[i], cur_c, x_prev,
     y_curr, g);
            x[i] = y_curr;
            cur_c = prefix_g[i - 1] * x_prev;
        }
        x[0] = cur_c / a[0];
        return true;
    }
};
```

### 4.1.3 FloorSum

EUCLIDEAN-LIKE ALGORITHM (AtCoder Library 'floor_sum').

```
//   floorSum(n, m, a, b) = sum_{i=0}^{n-1} floor((a*i + b) / m)
       in O(log m).
// This is THE tool for "sum over a lattice line": counting
    lattice points under a line, sums of
// floor/mod expressions, and anything of the shape sum floor((a i
     + b) / m) that a loop cannot do
// because n is up to 1e9 or more. Works like the Euclidean
    algorithm: reduce a mod m, then SWAP
// the roles of the axes (count by rows instead of by columns) and
     recurse.
// Requires n >= 0, m >= 1, a >= 0, b >= 0. Overflow: a*n + b must
     fit in ll (use __int128 if not).
ll floorSum(ll n, ll m, ll a, ll b) {
    ll ans = 0;
    if (a >= m) ans += (n - 1) * n / 2 * (a / m), a %= m;
    if (b >= m) ans += n * (b / m), b %= m;
    ll ymax = (a * n + b) / m;
    if (ymax == 0) return ans;
    ll xmax = ymax * m - b;
    ans += ymax * (n - (xmax + a - 1) / a);
    return ans + floorSum(ymax, a, m, (a - xmax % a) % a);
}
// NEGATIVE a or b: shift them into range first.
//   a < 0: let a2 = a mod m in [0,m); floorSum(n,m,a,b) =
    floorSum(n,m,a2,b) - (a2-a)/m * (n-1)n/2
//   b < 0: let b2 = b mod m in [0,m); subtract n * (b2-b)/m.  (
    Same trick, easier: use __int128.)
//
// WHAT IT SOLVES, directly:
//   * #lattice points (x,y) with 1 <= x <= n and 1 <= y <= (a x +
     b)/m       -> floorSum itself
//   * sum_{i=0}^{n-1} ((a i + b) mod m) = (a * n(n-1)/2 + b*n) -
    m * floorSum(n, m, a, b)
//   * sum_{i=1}^{n} floor(n / i) in O(sqrt n) instead - use
    DIVISOR BLOCKS (below), not this.
//   * "how many multiples of q lie in [l, r]" = floor(r/q) -
    floor((l-1)/q); the basic case.
//
// THE OTHER TWO SUMS of the same family (needed together often
    enough to be worth knowing they
// exist): S1 = sum i * floor((a i + b)/m) and S2 = sum floor((a i
     + b)/m)^2 obey the same
// recursion with a 3-tuple state - the "universal Euclidean
    algorithm" generalises
// this to any monoid: you supply the two operators U ("move right
    ") and R ("move up") and it
// composes the word for the line in O(log) monoid multiplications
    .
//
// DIVISOR BLOCKS - the other floor trick, O(sqrt n):
//   floor(n / i) is constant on maximal blocks; for a block
    starting at l the block ends at
//   r = n / (n / l). Use it for sum_{i=1}^{n} floor(n/i), sum of
    tau/sigma, Du sieve, Mobius sums.
//     for (ll l = 1, r; l <= n; l = r + 1) { r = n / (n / l); /*
    value n/l on [l, r] */ }
//   Two-argument version (blocks constant for BOTH n/i and m/i):
    r = min(n/(n/l), m/(m/l)).
```

## 4.2 Primes and Divisors

### 4.2.1 Sieve

Linear Sieve & Smallest Prime Factor (SPF) - Time: O(N), Space: O(N)

```
// Computes primes list, SPF array, primality test, and O(log N)
    prime factorization
struct Sieve {
    int n;
    vector<int> primes;
    vector<int> spf;

    Sieve(int n = 1e7) : n(n), spf(n + 1, 0) {
        for (int i = 2; i <= n; i++) {
            if (spf[i] == 0) {
                spf[i] = i;
                primes.pb(i);
            }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n) break;
                spf[i * p] = p;
            }
        }
    }

    bool is_prime(int x) const {
        return x >= 2 && x <= n && spf[x] == x;
    }

    // O(log N) prime factorization using precomputed SPF array
    vector<int> get_prime_factors(int x) const {
        vector<int> factors;
        while (x > 1) {
            factors.pb(spf[x]);
            x /= spf[x];
        }
        return factors;
    }
};
```

### 4.2.2 Factorization

Trial Division Prime Factorization - Time: O(sqrt(N))

```
template <typename T = ll>
struct Factorization {
    static vector<T> get_prime_factors(T n) {
        vector<T> factors;
        for (T i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                factors.pb(i);
                n /= i;
            }
        }
        if (n > 1) factors.pb(n);
        return factors;
    }
};
```

### 4.2.3 Divisors

Trial Division Divisor Generation - Time: O(sqrt(N))

```
template <typename T = ll>
struct Divisors {
    static vector<T> get_divisors(T n) {
        vector<T> divs;
        for (T i = 1; i * i <= n; i++) {
            if (n % i == 0) {
                divs.pb(i);
                if (n / i != i) divs.pb(n / i);
            }
        }
        return divs;
    }
};
```

### 4.2.4 Sieve1e9

Fast Bitset Sieve up to N = 10^9 - Time: ~0.3s for 10^9, Space: N / 16 bytes (~60MB)

```
// Stores odd numbers only (index i represents number 2*i + 1)
// WARNING: the bitset is ~60 MB - the instance MUST be global/
    static, never a local.
template <int MAXPR = (int)1e9>
struct Sieve1e9 {
    bitset<MAXPR / 2> is_prime_odd;

    Sieve1e9(int n = MAXPR) {
        is_prime_odd.set();
        is_prime_odd[0] = 0; // 1 is not prime
        for (int i = 1; (2 * i + 1) * (2 * i + 1) <= n; i++) {
            if (is_prime_odd[i]) {
                int p = 2 * i + 1;
                // bound is (n-1)/2, NOT n/2: n/2 writes one index
     past the end of the bitset
                for (int j = 2 * i * (i + 1); j <= (n - 1) / 2; j
     += p) {
                    is_prime_odd[j] = 0;
                }
            }
        }
    }

    bool is_prime(int x) const {
        if (x == 2) return true;
        if (x < 2 || x % 2 == 0) return false;
        return is_prime_odd[x / 2];
    }
};
```

### 4.2.5 SegmentedSieve

Segmented Sieve for Range [L, R] - Time: O(sqrt(R) + (R - L) log log R), Space: O(sqrt(R) + R - L)

```
// Computes all prime numbers in range [L, R] where R <= 1e12 and
    (R - L) <= 1e7
struct SegmentedSieve {
    static vector<ll> get_primes_in_range(ll L, ll R) {
        ll limit = sqrtl((ld)R);
        while (limit * limit > R) limit--;
        while ((limit + 1) * (limit + 1) <= R) limit++;
        vector<bool> is_prime_small(limit + 1, true);
        vector<ll> primes;
        for (ll i = 2; i <= limit; i++) {
            if (is_prime_small[i]) {
                primes.pb(i);
                for (ll j = i * i; j <= limit; j += i)
    is_prime_small[j] = false;
            }
        }

        vector<bool> is_prime_range(R - L + 1, true);
        for (ll p : primes) {
            ll start = max(p * p, (L + p - 1) / p * p);
            for (ll j = start; j <= R; j += p) {
                is_prime_range[j - L] = false;
            }
        }
        for (ll x = L; x <= min(R, 1LL); x++) is_prime_range[x -
    L] = false;   // 0 and 1

        vector<ll> result;
        for (ll i = 0; i <= R - L; i++) {
            if (is_prime_range[i]) result.pb(L + i);
        }
        return result;
    }
};
```

### 4.2.6 MillerRabinPollard

Primality + factorization for n up to 1e18. isPrime O(log^3 n), factor O(n^(1/4)).

```
// THE gate for any problem with a_i up to 1e18 - divisors, phi,
    tau, sigma all follow from factor().
typedef unsigned long long u64;
u64 mulmod(u64 a, u64 b, u64 m) { return (__int128)a * b % m; }
u64 powmod(u64 a, u64 e, u64 m) {
    u64 r = 1; a %= m;
    for (; e; e >>= 1, a = mulmod(a, a, m)) if (e & 1) r = mulmod
    (r, a, m);
    return r;
}
// Deterministic for all n < 3.3e24 with these 7 bases (Miller-
    Rabin).
bool isPrime(u64 n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    u64 d = n - 1; int s = __builtin_ctzll(d); d >>= s;
    for (u64 a : {2, 325, 9375, 28178, 450775, 9780504,
    1795265022}) {
        u64 p = powmod(a % n, d, n); int i = s;
        while (p != 1 && p != n - 1 && a % n && i--) p = mulmod(p
    , p, n);
        if (p != n - 1 && i != s) return false;
    }
    return true;
}
u64 pollard(u64 n) {                                    //
    returns a non-trivial divisor (Brent)
    auto f = [n](u64 x) { return mulmod(x, x, n) + 1; };
    u64 x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 || gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = mulmod(prd, x > y ? x - y : y - x, n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return gcd(prd, n);
}
vector<u64> factor(u64 n) {                             //
    unsorted multiset of primes
    if (n <= 1) return {};                              // guard:
     factor(0) would loop forever
    if (isPrime(n)) return {n};
    u64 x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
// {prime, exponent} pairs, sorted
vector<pair<u64, int>> factorize(u64 n) {
    auto f = factor(n); sort(all(f));
    vector<pair<u64, int>> r;
    for (u64 p : f) (!r.empty() && r.back().first == p) ? r.back
    ().second++ : (r.push_back({p, 1}), 0);
    return r;
}
vector<u64> divisors(u64 n) {                           // all
    divisors, unsorted
    vector<u64> d = {1};
    for (auto [p, e] : factorize(n)) {
        int sz = d.size();
        for (u64 pe = 1, k = 0; k < (u64)e; k++) {
            pe *= p;
            for (int i = 0; i < sz; i++) d.push_back(d[i] * pe);
        }
    }
    return d;
}
u64 phi(u64 n) { u64 r = n; for (auto [p, e] : factorize(n)) r -=
     r / p; return r; }      // Euler totient
u64 tau(u64 n) { u64 r = 1; for (auto [p, e] : factorize(n)) r *=
     e + 1; return r; }      // #divisors
u64 sigma(u64 n) {
                            // sum of divisors
    u64 r = 1;
    for (auto [p, e] : factorize(n)) { u64 t = 1, pe = 1; for (
    int i = 0; i < e; i++) t += (pe *= p); r *= t; }
    return r;
}
```

### 4.2.7 MultiplicativeSieve

Linear sieve computing ANY multiplicative function for all i <= n in O(N).

```
// Shipped: spf, phi, mu, tau (#divisors), sigma (sum of divisors)
    .
// To add your own f: you need f(p), f(p^k) and f(a*b)=f(a)f(b)
    for coprime a,b.
struct MultSieve {
    int n;
    vector<int> spf, primes, phi, mu, tau, cnt;          // cnt[i]
      = exponent of spf[i] in i
    vector<ll> sigma, pw;                                // pw[i]
     = spf[i]^cnt[i]

    MultSieve(int n) : n(n), spf(n + 1), phi(n + 1), mu(n + 1),
     tau(n + 1), cnt(n + 1),
                       sigma(n + 1), pw(n + 1) {
        phi[1] = mu[1] = tau[1] = sigma[1] = pw[1] = 1;
        for (int i = 2; i <= n; i++) {
            if (!spf[i]) {
                spf[i] = i, primes.push_back(i);
                phi[i] = i - 1, mu[i] = -1, tau[i] = 2, cnt[i] =
    1, pw[i] = i, sigma[i] = i + 1LL;
            }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n) break;
                int j = i * p;
                spf[j] = p;
                if (p == spf[i]) {                       // p
    divides i: same prime, exponent grows
                    cnt[j] = cnt[i] + 1, pw[j] = pw[i] * p;
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    tau[j] = tau[i] / (cnt[i] + 1) * (cnt[j] + 1)
    ;
                    sigma[j] = sigma[i] / ((pw[i] * p - 1) / (p -
     1)) * ((pw[j] * p - 1) / (p - 1));
                } else {                                 //
    coprime: multiply
                    cnt[j] = 1, pw[j] = p;
                    phi[j] = phi[i] * (p - 1);
                    mu[j] = -mu[i];
                    tau[j] = tau[i] * 2;
                    sigma[j] = sigma[i] * (p + 1);
                }
            }
        }
    }
    vector<int> factors(int x) const {                   // O(log
    x) via spf
        vector<int> f;
        while (x > 1) f.push_back(spf[x]), x /= spf[x];
        return f;
    }
};
```

## 4.3 Modular Arithmetic

### 4.3.1 CRT

```
// NOTE: extGCD below is the same one as in 01 - Euclidean
    Algorithm. Paste only one copy.
long long extGCD(long long a, long long b, long long &x, long long
     &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Solves x = a_i (mod m_i). Returns {x, lcm(m_1, ..., m_k)}.
pair<long long, long long> CRT(const vector<long long>& a, const
     vector<long long>& m) {
    long long res = a[0], lcm = m[0];
    for (int i = 1; i < a.size(); i++) {
        long long x, y;
        long long g = extGCD(lcm, m[i], x, y);
        if ((a[i] - res) % g != 0) return {-1, -1}; // No
    solution

        long long step = m[i] / g;
        long long k = ((a[i] - res) / g) % step * (x % step) %
    step;
        res = (long long)(((__int128)k * lcm + res) % ((__int128)
    lcm * step));
        lcm *= step;
        res = (res % lcm + lcm) % lcm;
    }
    return {res, lcm};
}
```

### 4.3.2 DiscreteLogAndRoots

Discrete log, modular square root, primitive root, discrete k-th root, multiplicative order.

```
// Needs: powmod (02 - Primes and Divisors/06 - MillerRabinPollard
    .cpp).
// All need mulmod/powmod from MillerRabinPollard.

// BSGS: smallest x >= 0 with a^x = b (mod m). Works for ANY m (
    handles gcd(a,m) != 1). O(sqrt m).
ll discreteLog(ll a, ll b, ll m) {
    a %= m, b %= m;
    ll k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {                       // strip
     common factors
        if (b == k) return add;
        if (b % g) return -1;
        b /= g, m /= g, add++, k = k * (a / g) % m;
    }
    ll n = (ll)sqrtl((ld)m) + 1, an = 1;
    for (ll i = 0; i < n; i++) an = an * a % m;
    unordered_map<ll, ll> vals;
    for (ll q = 0, cur = b; q <= n; q++) vals[cur] = q, cur = cur
      * a % m;
    for (ll p = 1, cur = k; p <= n; p++) {
        cur = cur * an % m;
        if (vals.count(cur)) { ll ans = n * p - vals[cur] + add;
     if (ans >= 0) return ans; }
    }
    return -1;
}

// Tonelli-Shanks: x with x^2 = a (mod p), p odd prime. Returns -1
     if a is not a QR.
ll sqrtMod(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    if (powmod(a, (p - 1) / 2, p) != 1) return -1;      // Euler'
     s criterion
    if (p % 4 == 3) return powmod(a, (p + 1) / 4, p);
    ll s = p - 1, n = 2; int r = 0;
    while (s % 2 == 0) s /= 2, r++;
    while (powmod(n, (p - 1) / 2, p) != p - 1) n++;     // find a
      non-residue
    ll x = powmod(a, (s + 1) / 2, p), b = powmod(a, s, p), g =
     powmod(n, s, p);
    for (int m = r;; m = r) {
        ll t = b;
        for (r = 0; r < m && t != 1; r++) t = t * t % p;
        if (r == 0) return x;
        ll gs = powmod(g, 1LL << (m - r - 1), p);
        g = gs * gs % p, x = x * gs % p, b = b * g % p;
    }
}

// Smallest primitive root of a PRIME p (generator of Z_p^*).
ll primitiveRoot(ll p) {
    if (p == 2) return 1;
    auto f = factorize(p - 1);
    for (ll g = 2; g < p; g++) {
        bool ok = true;
        for (auto [q, e] : f) if (powmod(g, (p - 1) / q, p) == 1)
      { ok = false; break; }
        if (ok) return g;
    }
    return -1;
}

// Multiplicative order of a mod m (smallest e > 0 with a^e = 1).
    Requires gcd(a, m) = 1.
ll multOrder(ll a, ll m) {
    ll e = phi(m), r = e;
    for (auto [q, k] : factorize(e))
        while (r % q == 0 && powmod(a, r / q, m) == 1) r /= q;
    return r;
}

// All x with x^k = a (mod p), p prime. Reduce to a discrete log
    in base g.
vector<ll> discreteRoot(ll k, ll a, ll p) {
    if (a == 0) return {0};
    ll g = primitiveRoot(p), y = discreteLog(powmod(g, k, p), a,
     p);
    if (y < 0) return {};
    ll x0 = powmod(g, y, p), d = gcd(k, p - 1), step = powmod(g,
     (p - 1) / d, p);
    vector<ll> r;
    for (ll i = 0, cur = x0; i < d; i++, cur = cur * step % p) r.
     push_back(cur);
    sort(all(r)), r.erase(unique(all(r)), r.end());
    return r;
}
```

### 4.3.3 LucasAndBinomialMod

C(n, r) mod m for HUGE n. Pick by what m is:

```
// Needs: Mint (06 - Math/01), and factorize (02 - Primes and
    Divisors/06) for the last section.
//   m prime and small           -> lucas
//   m = p^e                     -> binomPrimePower (Andrew
    Granville / Wilson generalisation)
//   m arbitrary (squarefree-ish)-> binomAnyMod (CRT over prime
    powers)

// LUCAS. p prime, needs fact/invFact built up to p. C(n,r) = prod
     C(n_i, r_i) over base-p digits.
Mint lucas(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    Mint res = 1;
    while (n || r) {
        ll a = n % MOD, b = r % MOD;
        if (b > a) return 0;
        res *= fact[a] * invFact[b] * invFact[a - b];
        n /= MOD, r /= MOD;
    }
    return res;
}

// --- arbitrary modulus ---
ll pw(ll a, ll e, ll m) { ll r = 1; a %= m; for (; e; e >>= 1, a
    = a * a % m) if (e & 1) r = r * a % m; return r; }
ll inv_mod(ll a, ll m) {                                    //
    requires gcd(a, m) = 1
    ll g = m, x = 0, x1 = 1, a1 = a % m;
    while (a1) { ll q = g / a1; g -= q * a1, swap(g, a1); x -= q
     * x1, swap(x, x1); }
    return (x % m + m) % m;
}
// n! with all factors of p removed, mod p^e.  O(p^e + log n).
ll factNoP(ll n, ll p, ll pe) {
    if (n == 0) return 1;
    vector<ll> f(pe);
    f[0] = 1;
    for (ll i = 1; i < pe; i++) f[i] = (i % p ? f[i - 1] * i : f[
     i - 1]) % pe;
    ll r = 1;
    for (ll m = n; m; m /= p) {
        r = r * pw(f[pe - 1], m / pe, pe) % pe;
        r = r * f[m % pe] % pe;
    }
    return r;
}
// C(n, r) mod p^e
ll binomPrimePower(ll n, ll r, ll p, ll pe) {
    if (r < 0 || r > n) return 0;
    ll k = 0;                                               //
     exponent of p in C(n,r) (Kummer's theorem:
    for (ll m = n / p, a = r / p, b = (n - r) / p; m; m /= p, a
     /= p, b /= p) k += m - a - b;
    if (k >= 60 || pw(p, k, pe * p) % pe == 0) { ll t = pw(p, k,
     pe); if (t == 0) return 0; }
    ll num = factNoP(n, p, pe), d1 = factNoP(r, p, pe), d2 =
     factNoP(n - r, p, pe);
    ll res = num * inv_mod(d1, pe) % pe * inv_mod(d2, pe) % pe;
    return res * pw(p, k, pe) % pe;
}
// C(n, r) mod m for arbitrary m: factor m, solve mod each p^e,
     CRT.
ll binomAnyMod(ll n, ll r, ll m) {
    ll res = 0, M = m;
    for (auto [p, e] : factorize(m)) {
        ll pe = 1; for (int i = 0; i < e; i++) pe *= p;
        ll c = binomPrimePower(n, r, p, pe), t = M / pe;
        res = (res + c % pe * t % M * inv_mod(t % pe, pe)) % M;
    }
    return res;
}
// KUMMER'S THEOREM: the exponent of prime p in C(n,r) = number of
      CARRIES when adding
//                  r and n-r in base p.  (Used above to pull out
     the p-part.)
// WILSON: (p-1)! = -1 (mod p) for prime p.
```

### 4.3.4 CongruencesAndTowers

Linear congruences and power towers. Needs powmod / phi / gcd.

```
// Needs: powmod, phi (04 - NT/02 - Primes and Divisors/06 - MillerRabinPollard.cpp).

// Solve a*x = b (mod m). Returns all solutions in [0, m) (there are gcd(a,m) of them, or none).
vector<ll> linCongruence(ll a, ll b, ll m) {
    a = ((a % m) + m) % m, b = ((b % m) + m) % m;
    ll g = gcd(a, m);
    if (b % g) return {};
    ll m2 = m / g, a2 = a / g, b2 = b / g;
    // a2 is invertible mod m2
    ll x0 = 0, x1 = 1, g2 = m2, a3 = a2 % m2;
    while (a3) { ll q = g2 / a3; g2 -= q * a3, swap(g2, a3); x0 -= q * x1, swap(x0, x1); }
    ll inv = ((x0 % m2) + m2) % m2, x = b2 % m2 * inv % m2;
    vector<ll> r;
    for (ll i = 0; i < g; i++) r.push_back((x + i * m2) % m);
    return r;
}

// GENERALISED EULER:  a^k = a^(k mod phi(m) + phi(m))  (mod m)  whenever k >= log2(m).
// This is what makes power towers computable even when gcd(a, m) != 1.
ll capTower(const vector<ll>& a, int i, ll CAP = 64) {   // min( true tower value, CAP)
    if (i == (int)a.size()) return 1;
    if (a[i] == 0) return capTower(a, i + 1, CAP) == 0 ? 1 : 0;
      // 0^0 = 1, 0^(k>0) = 0
    if (a[i] == 1) return 1;
    ll e = capTower(a, i + 1, CAP), r = 1;
    for (ll k = 0; k < e; k++) { r *= a[i]; if (r >= CAP) return CAP; }
    return r;
}
// a[i] ^ a[i+1] ^ ... ^ a[last]   (mod m), right-associative.
ll powTower(const vector<ll>& a, int i, ll m) {
    if (m == 1) return 0;
    if (i == (int)a.size()) return 1 % m;
    ll ph = phi(m), t = capTower(a, i + 1);
    ll e = t < 64 ? t : powTower(a, i + 1, ph) + ph;      // exact when tiny, +phi when large
    return powmod(a[i] % m, e, m);
}

// Extended CRT for NON-coprime moduli: x = a1 (mod m1), x = a2 (mod m2).
// Returns {x, lcm} or {-1, -1}. Chain it to merge many congruences.
pair<ll, ll> crt2(ll a1, ll m1, ll a2, ll m2) {
    ll g = gcd(m1, m2), l = m1 / g * m2;
    if ((a2 - a1) % g) return {-1, -1};
    ll m2g = m2 / g;
    ll x0 = 0, x1 = 1, gg = m2g, aa = (m1 / g) % m2g;
    while (aa) { ll q = gg / aa; gg -= q * aa, swap(gg, aa); x0 -= q * x1, swap(x0, x1); }
    ll inv = ((x0 % m2g) + m2g) % m2g;
    __int128 k = (__int128)((a2 - a1) / g % m2g) * inv % m2g;
    ll x = (ll)(((__int128)k * m1 + a1) % l);
    return {(x % l + l) % l, l};
}
```

### 4.3.5 Garner

GARNER'S ALGORITHM - reconstruct x from residues modulo PAIRWISE COPRIME m_i, in mixed-radix

```
// form, so you can emit x modulo a target MOD without ever building prod(m_i).
// This is what CRT cannot do once the product overflows 64 bits.
//   x = x1 + x2*m1 + x3*m1*m2 + ... ,   0 <= xi < mi
ll garner(vector<ll> a, vector<ll> m, ll MODOUT) {
    int k = a.size();
    vector<ll> x(k);
    auto inv = [](ll a, ll M) {                          // requires gcd(a, M) = 1
        ll g = M, X = 0, x1 = 1, A = ((a % M) + M) % M;
        while (A) { ll q = g / A; g -= q * A, swap(g, A); X -= q * x1, swap(X, x1); }
        return (X % M + M) % M;
    };
    for (int i = 0; i < k; i++) {
        x[i] = ((a[i] % m[i]) + m[i]) % m[i];
        ll mul = 1;
        for (int j = 0; j < i; j++) {
            x[i] = (x[i] - x[j] % m[i] * mul) % m[i];
            mul = mul * (m[j] % m[i]) % m[i];
        }
        x[i] = (x[i] % m[i] + m[i]) % m[i] * inv(mul, m[i]) % m[i];
    }
    ll res = 0, mul = 1;
    for (int i = 0; i < k; i++) {
        res = (res + x[i] % MODOUT * mul) % MODOUT;
        mul = mul * (m[i] % MODOUT) % MODOUT;
    }
    return (res % MODOUT + MODOUT) % MODOUT;
}
// MAIN USE: arbitrary-modulus convolution. Run NTT under three NTT primes, then Garner the
// three residues down to the target modulus.
//   NTT PRIMES:  998244353 = 119*2^23 + 1  (g=3)     167772161 = 5*2^25 + 1  (g=3)
//                469762049 =   7*2^26 + 1  (g=3)    1004535809 = 479*2^21 + 1  (g=3)
//                754974721 =  45*2^24 + 1  (g=11)   <- note: NOT 9*2^23+1, a common misprint
// The 3-prime product 5.95e25 covers n * (MOD-1)^2 for any MOD < 2^30 and n up to ~5e7.
```

## 4.4 Theorems

### 4.4.1 NumberTheoryFormulas

NUMBER THEORY - FORMULA SHEET ==========

```
/* ================= NUMBER THEORY - FORMULA SHEET
    =================
 MULTIPLICATIVE FUNCTIONS  (f(ab)=f(a)f(b) for gcd(a,b)=1)
   tau(n)=#divisors=prod(e_i+1)   sigma(n)=sum of divisors=prod (p
     ^(e+1)-1)/(p-1)
   phi(n)=n*prod(1-1/p)           mu(n)= 0 if NOT SQUAREFREE (some
      p^2 | n), else (-1)^#primes
   sum_{d|n} phi(d) = n           sum_{d|n} mu(d) = [n==1]
     (this is the key identity)
   sum_{d|n} mu(d)/d = phi(n)/n   sum_{d|n} mu(d)*tau(n/d) = 1

 DIRICHLET CONVOLUTION  (f*g)(n) = sum_{d|n} f(d) g(n/d)
   1*1 = tau      Id*1 = sigma      phi*1 = Id      mu*1 = eps(=[n
     ==1])
   mu is the inverse of 1, so f = g*1  <=>  g = f*mu.

 MOBIUS INVERSION
   g(n) = sum_{d|n} f(d)        <=>  f(n) = sum_{d|n} mu(d) g(n/d)
   g(n) = sum_{n|d, d<=N} f(d)  <=>  f(n) = sum_{n|d, d<=N} mu(d/n
     ) g(d)      (the "upward" form)
   Standard use: [gcd(i,j)==1] = sum_{d | gcd(i,j)} mu(d).

 GCD / LCM SUMS  (all O(N log N) or better with a mu/phi sieve)
   #coprime pairs (i,j)<=n        = sum_{d=1..n} mu(d) * floor(n/d
     )^2
   sum_{i,j<=n} gcd(i,j)          = sum_{d=1..n} phi(d) * floor(n/
     d)^2
   sum_{d|n} d*phi(n/d)           = sum_{i=1..n} gcd(i,n)
   sum_{i=1..n} lcm(i,n)          = n/2 * (1 + sum_{d|n} d*phi(d))
   sum_{i,j<=n} lcm(i,j)          = sum_g g * sum_{a,b<=n/g, gcd(a
     ,b)=1} a*b
   general: sum_{i,j} f(gcd(i,j)) = sum_d (f*mu)(d) * floor(n/d)^2

 DIVISOR BLOCKING  - evaluates sum_{i=1..n} F(floor(n/i)) in O(
     sqrt n):
   for (ll l = 1, r; l <= n; l = r + 1) { r = n / (n / l); }    //
      floor(n/i) == n/l on [l, r]
   Gives sum tau(i), sum sigma(i), sum floor(n/i), and Min_25 /
     Lucy-style prefix sums.

 MODULAR
   Fermat:   a^(p-1) = 1 (mod p), p prime, p !| a     => a^-1 = a
     ^(p-2)
   Euler:    a^phi(m) = 1 (mod m) if gcd(a,m)=1
   GENERALISED EULER (a and m NOT coprime), for n >= log2(m):
             a^n = a^(n mod phi(m) + phi(m))  (mod m)          <-
     power towers
   Wilson:   (p-1)! = -1 (mod p)  iff p prime
   CRT:      x = a_i (mod m_i) has a unique solution mod lcm(m_i)
     iff a_i = a_j (mod gcd(m_i,m_j))
   Legendre: exponent of p in n!  = sum_{i>=1} floor(n / p^i)  = (
     n - s_p(n)) / (p - 1)
   Kummer:   exponent of p in C(n,r) = #carries when adding r and
     (n-r) in base p
   LTE (p odd, p | a-b, p !| a,b):  v_p(a^n - b^n) = v_p(a-b) +
     v_p(n)
       p=2 (a, b both ODD):
         n odd  : v_2(a^n - b^n) = v_2(a-b)          <- the case
     everyone forgets
         n even : v_2(a^n - b^n) = v_2(a-b) + v_2(a+b) + v_2(n) -
     1
         n odd  : v_2(a^n + b^n) = v_2(a+b)
         n even : v_2(a^n + b^n) = 1
   LTE for SUMS (p odd, p | a+b, p !| a,b):
             n odd  -> v_p(a^n + b^n) = v_p(a+b) + v_p(n)
             n even -> p does not divide a^n + b^n at all

 QUADRATIC RESIDUES (p odd prime)
   Euler's criterion: a^((p-1)/2) = +1 if a is a QR, -1 if not
   #QRs = (p-1)/2.  x^2=a has 0 or 2 roots.  Use Tonelli-Shanks.
   -1 is a QR iff p = 1 (mod 4);  2 is a QR iff p = +-1 (mod 8)

 PRIMITIVE ROOTS exist exactly for m = 1, 2, 4, p^k, 2p^k (p odd
     prime). Count = phi(phi(m)).

 REPRESENTATIONS
   n is a sum of two squares  iff every prime p = 3 (mod 4) has an
      EVEN exponent
   n is a sum of three squares iff n != 4^a (8b+7)      every n is
      a sum of four squares
   Primitive Pythagorean triples: (m^2-k^2, 2mk, m^2+k^2), m>k>0,
     gcd(m,k)=1, m-k odd

 SIZE FACTS (for choosing an algorithm)
   max tau(n): 1344 for n<=1e9, 6720 for n<=1e12, 103680 for n<=1
     e18
   sum_{i<=n} tau(i) ~ n ln n ;  sum_{i<=n} phi(i) ~ 3n^2/pi^2
    pi(n) ~ n/ln n ;  n-th prime ~ n ln n ;  Bertrand: a prime
      always lies in (n, 2n)
    gaps: for n<=1e18 the max prime gap is < 1500
    Pisano period pi(m) | ... : F(n) mod m repeats with period <= 6
      m
================================================================
      */
```

### 4.4.2 MobiusInversion

Working code for the identities on the formula sheet. Needs MultSieve (mu, phi).

```
// Needs: MultSieve (04 - NT/02 - Primes and Divisors/07 -
     MultiplicativeSieve.cpp) for mu/phi.

// #pairs (i,j) with 1<=i,j<=n and gcd(i,j)=1        O(n)
ll coprimePairs(int n, const MultSieve& S) {
    ll r = 0;
    for (int d = 1; d <= n; d++) r += (ll)S.mu[d] * (n / d) * (n
     / d);
    return r;
}
// sum over all pairs of gcd(i,j)                    O(n)
ll gcdSum(int n, const MultSieve& S) {
    ll r = 0;
    for (int d = 1; d <= n; d++) r += (ll)S.phi[d] * (n / d) * (n
      / d);
    return r;
}
// General:  sum_{i,j<=n} f(gcd(i,j))  =  sum_d (f*mu)(d) * floor(
     n/d)^2
// Build g = f*mu once by sieving over multiples:  for d: for m=d
     ,2d,...: g[m] += mu[m/d]*f[d]
vector<ll> dirichletMu(const vector<ll>& f, const MultSieve& S) {
        // g = f * mu   O(n log n)
    int n = f.size() - 1;
    vector<ll> g(n + 1, 0);
    for (int d = 1; d <= n; d++) if (f[d])
        for (int m = d; m <= n; m += d) g[m] += (ll)S.mu[m / d] *
      f[d];
    return g;
}

// DIVISOR BLOCKING: sum_{i=1..n} floor(n/i) in O(sqrt n). Same
     skeleton for sum tau / sum sigma.
ll sumFloorDiv(ll n) {
    ll r = 0;
    for (ll l = 1, rr; l <= n; l = rr + 1) { rr = n / (n / l); r
     += (n / l) * (rr - l + 1); }
    return r;
}
// sum_{i=1..n} tau(i) = sum_{i=1..n} floor(n/i)      (count
     multiples)
// sum_{i=1..n} sigma(i) = sum_{i=1..n} i * floor(n/i)

// COUNT MULTIPLES / INCLUSION-EXCLUSION OVER DIVISORS:
// "how many i<=n are divisible by at least one of p1..pk" -> IE
     over the 2^k squarefree products,
// sign = mu of the product. For k up to ~20 enumerate submasks;
     beyond that, sieve mu.

// sum_{i=1..n} gcd(i, n) = sum_{d|n} d * phi(n/d)
ll gcdSumWithN(ll n) {
    ll r = 0;
    for (auto d : divisors(n)) r += (ll)d * phi(n / d);
    return r;
}
// sum_{i=1..n} lcm(i, n) = n/2 * (1 + sum_{d|n} d*phi(d))
ll lcmSumWithN(ll n) {
    ll s = 1;
    for (auto d : divisors(n)) s += (ll)d * phi(d);
    return n / 2 * s + (n % 2 ? s / 2 : 0);
}
```

### 4.4.3 PrimeCounting

LUCY_HEDGEHOG: pi(n) = #primes <= n, and the SUM of primes <= n, in O(n^(3/4)) time and

```
// O(sqrt n) memory. Handles n up to ~1e12 in well under a second; 1e13 in a few seconds.
// Idea: S(v, p) = (count or sum) of the integers in [2, v] left after sieving by all primes < p.
// Sieving by p only touches v >= p^2, so the whole table lives on the O(sqrt n) distinct
// values of floor(n / i).
ll primePi(ll n) {
    if (n < 2) return 0;
    ll r = (ll)sqrtl((ld)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    vector<ll> S(r + 1), L(r + 1);                          // S[v] for v <= r, L[i] for v = n/i
    for (ll i = 1; i <= r; i++) S[i] = i - 1, L[i] = n / i - 1;
    for (ll p = 2; p <= r; p++) {
        if (S[p] == S[p - 1]) continue;                     // p is not prime
        ll sp = S[p - 1], p2 = p * p;
        for (ll i = 1; i <= min(r, n / p2); i++)
            L[i] -= (i * p <= r ? L[i * p] : S[n / (i * p)]) - sp;
        for (ll v = r; v >= p2; v--) S[v] -= S[v / p] - sp;
    }
    return L[1];
}
// Sum of all primes <= n. Same recurrence with f(x) = x(x+1)/2 - 1 and a factor of p.
ll primeSum(ll n) {
    if (n < 2) return 0;
    ll r = (ll)sqrtl((ld)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    // __int128: n(n+1)/2 wraps for n > 4.29e9, and the ANSWER (~n^2/(2 ln n)) wraps at n ~ 3e10.
    auto f = [](ll x) { return x % 2 ? (lll)(x + 1) / 2 * x - 1 : (lll)x / 2 * (x + 1) - 1; };
    vector<ll> S(r + 1), L(r + 1);
    for (ll i = 1; i <= r; i++) S[i] = f(i), L[i] = f(n / i);
    for (ll p = 2; p <= r; p++) {
        if (S[p] == S[p - 1]) continue;
        ll sp = S[p - 1], p2 = p * p;
        for (ll i = 1; i <= min(r, n / p2); i++)
            L[i] -= p * ((i * p <= r ? L[i * p] : S[n / (i * p)]) - sp);
        for (ll v = r; v >= p2; v--) S[v] -= p * (S[v / p] - sp);
    }
    return L[1];
}
// SAME SKELETON computes the prefix sum of ANY completely-multiplicative f (change the two
// initialisations and the multiplier). For general multiplicative f use the Min_25 sieve, which
// runs this as its first phase and then re-adds the prime-power contributions.
// APPROXIMATIONS for sanity checks: pi(n) ~ n/ln n, the k-th prime ~ k ln k.
```

### 4.4.4 Representations

Representing integers: sums of two squares, Pythagorean triples, Legendre symbol.

```
// Needs: powmod, factorize (02 - Primes and Divisors/06 - MillerRabinPollard.cpp).
// Needs factorize / sqrtMod / isPrime.

int legendre(ll a, ll p) {                                  // 1 QR, -1 non-residue, 0 if p | a
    a %= p; if (a < 0) a += p;
    if (!a) return 0;
    return powmod(a, (p - 1) / 2, p) == 1 ? 1 : -1;
}
// n is a sum of two squares  <=>  every prime p = 3 (mod 4) appears to an EVEN power.
bool isSumOfTwoSquares(ll n) {
    if (n <= 0) return n == 0;
    for (auto [p, e] : factorize(n)) if (p % 4 == 3 && (e & 1)) return false;
    return true;
}
// One representation n = x^2 + y^2 (Cornacchia). Returns {-1,-1} if none exists.
pair<ll, ll> sumOfTwoSquares(ll n) {
    if (n == 0) return {0, 0};
    ll t = n, pw2 = 1;
    while (t % 4 == 0) t /= 4, pw2 *= 2;                    // pull out squares of 2
    if (!isSumOfTwoSquares(t)) return {-1, -1};
    // Gaussian-integer product over the prime factorisation
    ll xr = 1, yr = 0;                                      // current value as xr + yr*i
    for (auto [p, e] : factorize(t)) {
        if (p == 2) { for (int i = 0; i < e; i++) { ll nx = xr - yr, ny = xr + yr; xr = nx, yr = ny; } continue; }
        if (p % 4 == 3) { ll k = 1; for (int i = 0; i < e / 2; i++) k *= p; xr *= k, yr *= k; continue; }
        ll r = sqrtMod(p - 1, p);                           // r^2 = -1 (mod p)
        ll a = p, b = r;
        while (b * b > p) { ll c = a % b; a = b, b = c; }   // Cornacchia descent: p = b^2 + c^2
        ll c = (ll)sqrtl((ld)(p - b * b));
        while (c * c < p - b * b) c++;
        while (c * c > p - b * b) c--;
        for (int i = 0; i < e; i++) { ll nx = xr * b - yr * c, ny = xr * c + yr * b; xr = nx, yr = ny; }
    }
    return {llabs(xr) * pw2, llabs(yr) * pw2};
}
// #representations of n as an ORDERED pair of squares (with signs) = 4 * (d_1(n) - d_3(n)),
// where d_i counts divisors = i (mod 4).   [Jacobi's two-square theorem]

// PRIMITIVE Pythagorean triples with hypotenuse <= L:  (m^2-k^2, 2mk, m^2+k^2),
// m > k > 0, gcd(m,k) = 1, m-k odd. Every triple is a multiple of exactly one primitive triple.
vector<array<ll, 3>> pythagorean(ll L) {
    vector<array<ll, 3>> r;
    for (ll m = 2; m * m <= L; m++)
        for (ll k = 1 + (m % 2); k < m; k += 2)             // m-k must be ODD => opposite parity
            if (gcd(m, k) == 1 && m * m + k * k <= L)
                r.push_back({m * m - k * k, 2 * m * k, m * m + k * k});
    return r;
}
// n is a sum of THREE squares  <=>  n != 4^a * (8b + 7).   Every n is a sum of FOUR squares.
```

### 4.4.5 DuSieve

DU SIEVE (Dirichlet prefix sums) - prefix sums of a multiplicative f in SUBLINEAR time.

```
// Pick g with an easy prefix sum G and an easy (f*g) prefix sum;
     then
//     g(1)*S(n) = sum_{i=1..n} (f*g)(i)  -  sum_{i=2..n} g(i) * S
     (floor(n/i))
// Sieve S(x) directly for x <= LIM (take LIM ~ n^(2/3)); recurse
     with memoisation above that.
// Total O(n^(2/3)) with the sieve, O(n^(3/4)) without.
//   mu * 1  = eps  ->  M(n)   = 1 - sum_{i=2..n} M(n/i)
//   phi * 1 = Id   ->  Phi(n) = n(n+1)/2 - sum_{i=2..n} Phi(n/i)
struct DuSieve {
    ll LIM;
    vector<ll> mu, phi;                                  // prefix
     sums for x <= LIM
    unordered_map<ll, ll> memoM, memoP;

    DuSieve(ll n) {
        LIM = max((ll)1000, (ll)powl((ld)n, 2.0L / 3));
        vector<int> spf(LIM + 1, 0), primes, m(LIM + 1, 0), p(LIM
     + 1, 0);
        m[1] = p[1] = 1;
        for (ll i = 2; i <= LIM; i++) {
            if (!spf[i]) spf[i] = i, primes.push_back(i), m[i] =
    -1, p[i] = i - 1;
            for (int q : primes) {
                if (q > spf[i] || i * q > LIM) break;
                spf[i * q] = q;
                if (q == spf[i]) m[i * q] = 0, p[i * q] = p[i] *
    q;
                else m[i * q] = -m[i], p[i * q] = p[i] * (q - 1);
            }
        }
        mu.assign(LIM + 1, 0), phi.assign(LIM + 1, 0);
        for (ll i = 1; i <= LIM; i++) mu[i] = mu[i - 1] + m[i],
    phi[i] = phi[i - 1] + p[i];
    }
    ll M(ll n) {                                         // sum_{i
     =1..n} mu(i)   (Mertens)
        if (n <= LIM) return mu[n];
        auto it = memoM.find(n);
        if (it != memoM.end()) return it->second;
        ll r = 1;
        for (ll l = 2, rr; l <= n; l = rr + 1) { rr = n / (n / l)
    ; r -= (rr - l + 1) * M(n / l); }
        return memoM[n] = r;
    }
    ll P(ll n) {                                         // sum_{i
     =1..n} phi(i)
        if (n <= LIM) return phi[n];
        auto it = memoP.find(n);
        if (it != memoP.end()) return it->second;
        ll r = n % 2 ? (n + 1) / 2 * n : n / 2 * (n + 1);
        for (ll l = 2, rr; l <= n; l = rr + 1) { rr = n / (n / l)
    ; r -= (rr - l + 1) * P(n / l); }
        return memoP[n] = r;
    }
};
// USES: squarefree count up to n = sum_{d<=sqrt n} mu(d) * floor(
    n/d^2);
//       #coprime pairs up to n = 2*Phi(n) - 1;  sum of gcd/lcm
    over pairs at n ~ 1e10;
//       any "sum over 1..n of a multiplicative function" that a
    linear sieve cannot reach.
```

### 4.4.6 MoreFormulaCards

NUMBER THEORY, SECOND FORMULA SHEET ==========

```
/* ============ NUMBER THEORY, SECOND FORMULA SHEET ============
 Everything here was machine-verified against brute force / sympy.

 QUADRATIC RECIPROCITY.  p, q distinct ODD primes.
   (p/q)(q/p) = (-1)^( (p-1)/2 * (q-1)/2 )
     => (p/q) = (q/p), UNLESS p = q = 3 (mod 4), where (p/q) = -(q
     /p)
   SUPPLEMENTS: (-1/p) = 1 iff p = 1 (mod 4);  (2/p) = 1 iff p =
     +-1 (mod 8);
                (3/p) = 1 iff p = +-1 (mod 12)
   EULER'S CRITERION (p odd prime AND p !| a):  a^((p-1)/2) = (a/p
     ) (mod p).
     If p | a the value is 0, not +-1 - state the hypothesis.
   #k-th ROOTS of a != 0 mod p:  d = gcd(k, p-1); the count is d
     if a^((p-1)/d) = 1, else 0.
 JACOBI SYMBOL (a/n) for ODD n > 0, n NOT necessarily prime - O(
     log n), no factoring: */
int jacobi(ll a, ll n) {                                      //
     n odd, n > 0
    int r = 1;
    a = ((a % n) + n) % n;                                    //
     (a/n) is periodic in a mod n
    while (a) {
        while (a % 2 == 0) { a /= 2; if (n % 8 == 3 || n % 8 ==
     5) r = -r; }
        swap(a, n);
        if (a % 4 == 3 && n % 4 == 3) r = -r;
        a %= n;
    }
    return n == 1 ? r : 0;
}
/* CAUTION: (a/n) = 1 does NOT prove a is a QR mod a composite n.
     (a/n) = -1 DOES prove it is
   not. Use it as a cheap filter, and to find a non-residue fast
     for Tonelli-Shanks.

 FIBONACCI  (F_0 = 0, F_1 = 1)
   FAST DOUBLING, O(log n), no matrices:
     F(2k) = F(k) * (2 F(k+1) - F(k)),   F(2k+1) = F(k)^2 + F(k+1)
     ^2
   gcd(F_m, F_n) = F_gcd(m,n)   =>   F_m | F_n iff m | n  (m >= 3)
   Cassini:  F_{n-1} F_{n+1} - F_n^2 = (-1)^n
   d'Ocagne: F_m F_{n+1} - F_{m+1} F_n = (-1)^n F_{m-n}
   sum_{i<=n} F_i = F_{n+2} - 1        sum_{i<=n} F_i^2 = F_n F_{n
     +1}
   F_92 is the last Fibonacci number fitting in a signed 64-bit
     integer.
 PISANO PERIOD pi(m) = the period of F(n) mod m
   pi(1)=1, pi(2)=3, and pi(m) is EVEN for every m > 2
   pi(lcm(a,b)) = lcm(pi(a), pi(b))          pi(p^k) = p^(k-1) *
     pi(p)
   pi(m) <= 6m, with EQUALITY exactly at m = 2 * 5^k   (verified:
     m = 10, 50, 250)
   p = +-1 (mod 5) => pi(p) | p-1;   p = +-2 (mod 5) => pi(p) | 2(
     p+1)
 ZECKENDORF: every n >= 1 is a UNIQUE sum of NON-CONSECUTIVE
     Fibonacci numbers, found
   greedily from the top. The basis of "Fibonacci base" digit DP
     and of Wythoff's game.

 PELL:  x^2 - D y^2 = 1, D > 0 non-square. Always infinitely many
     solutions.
   sqrt(D) = [a0; a1, ..., a_L] is periodic with a_L = 2 a0 and a
     palindromic middle.
   FUNDAMENTAL SOLUTION = the convergent h_k / k_k with
       k = L - 1   if the period L is EVEN,      k = 2L - 1  if L
     is ODD.
   ALL solutions: x_m + y_m sqrt(D) = (x1 + y1 sqrt(D))^m, i.e.
       x_{m+1} = x1 x_m + D y1 y_m,   y_{m+1} = x1 y_m + y1 x_m
   NEGATIVE PELL x^2 - D y^2 = -1 is solvable IFF the period L is
     ODD; then the fundamental
   solution is the convergent at k = L - 1.
   SIZE WARNING: solutions explode. D = 61 gives x = 1766319049,
     but D = 661 gives a
   39-digit x. `ll` is useless past D ~ 150 - use bignum, or work
     modulo the answer.
   CF MACHINERY (a0 = floor(sqrt D); m = 0, d = 1, a = a0), all
     exact integers:
       m' = d*a - m,    d' = (D - m'^2) / d,    a' = floor((a0 + m
     ') / d')
   BEST RATIONAL APPROXIMATION: the closest p/q to x with q <= N
     is a convergent OR a
   semiconvergent (t h1 + h2)/(t k1 + k2) with 1 <= t <= a_next.
     Check both.
   FAREY MEDIANT: for neighbours a/b < c/d (bc - ad = 1), (a+c)/(b
     +d) is the unique fraction
   with the SMALLEST denominator strictly between them.
```

```
SUMS OF SQUARES
  TWO:   n = x^2 + y^2 possible IFF every prime p = 3 (mod 4) has
    an EVEN exponent.
         r_2(n) = 4 (d_1(n) - d_3(n)), d_i = #divisors congruent
    to i mod 4.
  THREE: n = x^2 + y^2 + z^2 possible IFF n is NOT of the form 4^
    a (8b + 7).
  FOUR:  ALWAYS possible (Lagrange).  r_4(n) = 8 * sum over d | n
    with 4 !| d of d.
  PRIMITIVE PYTHAGOREAN TRIPLES: (m^2 - k^2, 2mk, m^2 + k^2), m >
    k > 0, gcd(m,k) = 1,
    m - k odd.
  GAUSSIAN INTEGERS Z[i]: the norm N(a+bi) = a^2 + b^2 is
    multiplicative. Primes of Z[i]:
    1+i (over 2); p = 1 (mod 4) SPLITS as pi * conj(pi); p = 3 (
    mod 4) stays prime.
    gcd by rounded division: q = round(a conj(b) / N(b)), r = a -
     qb, then Euclid on norms.

FROBENIUS / CHICKEN MCNUGGET.  a, b >= 2, gcd(a,b) = 1:
  largest non-representable  g(a,b) = ab - a - b;   #non-
    representable = (a-1)(b-1)/2.
  For k >= 3 coins there is NO closed form: run Dijkstra on
    residues mod a_1 (d[r] = the
  smallest representable value congruent to r), then g = max_r d[
    r] - a_1. O(a_1 k log a_1).
BEATTY / RAYLEIGH.  r, s > 1 IRRATIONAL with 1/r + 1/s = 1  =>
    floor(n r) and floor(n s),
  n >= 1, PARTITION the positive integers exactly. (Wythoff's
    losing positions are exactly
  (floor(n phi), floor(n phi^2)).)
THREE-DISTANCE (Steinhaus). The points {a}, {2a}, ..., {na} cut
    [0,1) into intervals of AT
  MOST 3 distinct lengths, and when there are 3 the largest is
    the sum of the other two.
  Use: min over k <= n of (k*a mod m) in O(log) via the continued
     fraction of a/m.

DETERMINISTIC MILLER-RABIN BASE SETS (all machine-verified)
  n < 2^32              : {2, 7, 61}                          (
    valid to 4759123141)
  n < 3.2e18            : the first 12 primes {2..37}
  n < 3.317e24 > 2^64   : {2, 325, 9375, 28178, 450775, 9780504,
    1795265022}
  {2,3,5,7} is NOT enough even for 32 bits: 3215031751 =
    151*751*28351 passes all four.
  Skip a base a when n divides a, or small primes are reported
    composite.

NTT PRIMES (form, primality and primitive root all machine-
    verified)
  p                   = c * 2^k + 1      g     max transform
    length
  998244353           = 119 * 2^23 + 1    3      2^23     <- the
     default
  167772161           =   5 * 2^25 + 1    3      2^25
  469762049           =   7 * 2^26 + 1    3      2^26
  754974721           =  45 * 2^24 + 1   11      2^24     <- NOT
    9*2^23+1 (common misprint)
  1004535809          = 479 * 2^21 + 1    3      2^21
  2013265921          =  15 * 2^27 + 1   31      2^27
  1224736769          =  73 * 2^24 + 1    3      2^24
  3221225473          =   3 * 2^30 + 1    5      2^30
  9223372036737335297  = 549755813881 * 2^24 + 1, g = 3   (64-bit
    , needs __int128 mulmod)
  THE 3-PRIME TRIPLE for an arbitrary modulus: 167772161 *
    469762049 * 754974721 = 5.95e25.
  THE BINDING CONSTRAINT IS LENGTH, NOT MAGNITUDE: 754974721 caps
     the transform at 2^24, so
  the input length is capped at 2^23. At that length the
    magnitude headroom is still 6x.

64-BIT CEILINGS - memorise these
  a*b fits in signed ll        iff a, b < 3037000499  ( = floor(
    sqrt(2^63 - 1)) )
  n(n+1)/2 fits in ll          iff n <= 4294967295
  sum of primes <= n fits      iff n <~ 3e10          (the sum is
    ~ n^2 / (2 ln n))
  sum of phi(i), i <= n, fits  iff n <= 5508509576    (the sum is
    ~ 0.3040 n^2)
  max tau(n): 1344 for n <= 1e9,  6720 for 1e12,  103680 for 1e18
  NEED __int128 FOR: any modmul with modulus > 3.04e9 (Pollard,
    BSGS, Tonelli, CRT, Garner);
    floorSum with n > 1e9; Cornacchia's b*b > p test; prime/
    totient SUMS at n >= 1e10.
  FFT PRECISION, the real criterion (not "|result| < 1e15"):
    safe iff (sum a_i^2 + sum b_i^2) * log2(N) < 9e14.
    Splitting at sqrt(M) additionally needs M^1.5 < 9.2e18, i.e.
     M <~ 4.4e12, with the
     inputs already reduced into [0, M).
   FWHT: the forward transform inflates magnitudes by up to N, so
     the pointwise product is
     bounded by (N * max|a|)^2. Under a modulus the XOR inverse
     must MULTIPLY BY inv(N) mod p -
     an integer `x /= n` is exact over Z and WRONG mod p.
============================================================ */
```

## 4.5 Rational

### 4.5.1 ContinuedFractions

Continued fractions, convergents, best rational approximation, Stern-Brocot search.

```
// x = [a0; a1, a2, ...]. For a rational p/q this is just the
     Euclidean algorithm's quotients.
vector<ll> cfrac(ll p, ll q) {
    vector<ll> a;
    // FLOOR division: C++ truncates toward zero, so cfrac(-7,2)
     would give [-3,-2] instead of
    // the correct [-4,2] and every convergent would be wrong.
    while (q) { ll d = p / q - ((p % q) && ((p < 0) ^ (q < 0)));
     a.push_back(d); ll r = p - d * q; p = q, q = r; }
    return a;
}
// Convergents h[i]/k[i] of a continued fraction: h = a*h1 + h2, k
     = a*k1 + k2.
// Every convergent is a BEST approximation: no fraction with a
     smaller denominator is closer.
vector<pair<ll, ll>> convergents(const vector<ll>& a) {
    vector<pair<ll, ll>> c;
    ll h1 = 1, h2 = 0, k1 = 0, k2 = 1;
    for (ll x : a) {
        ll h = x * h1 + h2, k = x * k1 + k2;
        c.push_back({h, k});
        h2 = h1, h1 = h, k2 = k1, k1 = k;
    }
    return c;
}
// Best rational approximation to p/q with denominator <= N.
// The answer is a convergent OR a SEMICONVERGENT (t*h1+h2)/(t*k1+
     k2) - you must compare both.
pair<ll, ll> bestApprox(ll p, ll q, ll N) {
    ll h1 = 1, h2 = 0, k1 = 0, k2 = 1, P = p, Q = q;
    pair<ll, ll> best = {p >= 0 ? 0 : -((-p + q - 1) / q), 1};
        // 0/1 (or floor) is legal
    auto better = [&](pair<ll, ll> a, pair<ll, ll> b) {
        // is a closer to p/q than b?
        __int128 da = (__int128)llabs(p * a.second - a.first * q)
      * b.second;
        __int128 db = (__int128)llabs(p * b.second - b.first * q)
      * a.second;
        return da != db ? da < db : a.second < b.second;
    };
    while (Q) {
        ll a = P / Q, r = P % Q;
        if (k1 && a * k1 + k2 > N) {
        // cannot take the full step
            ll t = (N - k2) / k1;
            if (t > 0) { pair<ll, ll> c = {t * h1 + h2, t * k1 +
     k2}; if (better(c, best)) best = c; }
            break;
        }
        ll h = a * h1 + h2, k = a * k1 + k2;
        if (k > N) break;
        h2 = h1, h1 = h, k2 = k1, k1 = k;
        if (better({h1, k1}, best)) best = {h1, k1};
        P = Q, Q = r;
    }
    return best;
}
// STERN-BROCOT SEARCH: find the simplest fraction p/q with f(p/q)
      true, when f is monotone.
// Walk the mediant tree; take exponential jumps in each direction
      so it is O(log^2).
// Classic uses: "smallest denominator between two reals",
     recovering a fraction from a decimal,
// and answering "is the answer rational with small denominator"
     style problems.
// FAREY / mediant fact: if a/b < c/d are neighbours (bc - ad = 1)
      then their mediant
// (a+c)/(b+d) is the unique simplest fraction strictly between
     them.
```

## 4.6 Sequences

### 4.6.1 FibonacciAndPisano

FIBONACCI, FAST. The notebook had the identities on a formula sheet and no routine; this is it.

```
// F(0)=0, F(1)=1. Fast doubling is O(log n) with two
     multiplications per bit and NO matrices -
// shorter and ~2x faster than 2x2 matrix exponentiation.
//   F(2k)   = F(k) * (2*F(k+1) - F(k))
//   F(2k+1) = F(k)^2 + F(k+1)^2
pair<ll, ll> fibPair(ll n, ll mod) {                              //
      returns {F(n), F(n+1)} mod
    if (!n) return {0, 1 % mod};
    auto [a, b] = fibPair(n >> 1, mod);
    ll c = a * ((2 * b % mod - a % mod + mod) % mod) % mod;
     // F(2k)
    ll d = (a * a + b * b) % mod;
     // F(2k+1)
    return (n & 1) ? make_pair(d, (c + d) % mod) : make_pair(c, d
     );
}
ll fib(ll n, ll mod) { return fibPair(n, mod).first; }
// LUCAS NUMBERS L(n) = F(n-1) + F(n+1) = 2F(n+1) - F(n).
// NEGATIVE index: F(-n) = (-1)^(n+1) F(n).
// The mod must be < ~3e9 for the ll products above, or use
     __int128 / Mint.

// --- PISANO PERIOD pi(m): the period of F(n) mod m. Needed to
     reduce a huge index. ---
// pi(1) = 1, pi(2) = 3, and pi(m) is EVEN for every m > 2.
// pi(lcm(a,b)) = lcm(pi(a), pi(b))      pi(p^k) = p^(k-1) * pi(p)
// pi(m) <= 6m, with equality exactly at m = 2 * 5^k.
// p = +-1 (mod 5) => pi(p) divides p-1;   p = +-2 (mod 5) => pi(p
     ) divides 2(p+1).
// So: factor m, get pi for each prime power by testing the
     divisors of that bound, then lcm.
ll pisanoPrime(ll p) {                                            //
      p prime
    if (p == 2) return 3;
    if (p == 5) return 20;
    ll cand = (p % 5 == 1 || p % 5 == 4) ? p - 1 : 2 * (p + 1);
    vector<ll> divs;
    for (ll d = 1; d * d <= cand; d++)
        if (cand % d == 0) divs.push_back(d), divs.push_back(cand
      / d);
    sort(all(divs));
    for (ll d : divs) {
        auto [a, b] = fibPair(d, p);
        if (a == 0 && b == 1 % p) return d;
    }
    return cand;
}
// Full pi(m): factorise m (Pollard rho), take pisanoPrime(p) * p
     ^(e-1) per prime power, lcm them.
/* MORE FIBONACCI FACTS worth the page
   gcd(F_m, F_n) = F_gcd(m,n)      =>   F_m | F_n  iff  m | n   (m
      >= 3)
   Cassini:   F_{n-1} F_{n+1} - F_n^2 = (-1)^n
   Catalan:   F_n^2 - F_{n-r} F_{n+r} = (-1)^{n-r} F_r^2
   d'Ocagne:  F_m F_{n+1} - F_{m+1} F_n = (-1)^n F_{m-n}
   sum_{i<=n} F_i = F_{n+2} - 1        sum_{i<=n} F_i^2 = F_n F_{n
     +1}
   sum_{i<=n} F_{2i-1} = F_{2n}        sum_{i<=n} F_{2i} = F_{2n
     +1} - 1
   F_n = (phi^n - psi^n)/sqrt5;  F_92 is the last one fitting in a
      signed 64-bit integer.
   ZECKENDORF: every n >= 1 is a UNIQUE sum of NON-CONSECUTIVE
     Fibonacci numbers, found greedily
     from the largest. This is the "Fibonacci base", and it is
     what makes Fibonacci-nim work.
   LCM of F_1..F_n and "which F are divisible by k" both follow
     from the gcd identity.
   Fibonacci mod p when 5 is a QR mod p: sqrt(5) exists, so the
     closed form works in F_p directly.
     When it is not, work in F_p[x]/(x^2-5) (a "Phi field") - two
     components, same O(log n). */

// --- TRIBONACCI / any linear recurrence: matrix exponentiation
     (06 - Math/03), or Kitamasa /
// Bostan-Mori for a long recurrence. Berlekamp-Massey RECOVERS
     the recurrence from ~2k terms.
// --- LUCAS SEQUENCES U_n, V_n with U_{n+1} = P U_n - Q U_{n-1}
     generalise all of this and have
// the same doubling identities; they are what Baillie-PSW
     primality is built on.
```

# 5 Combinatorics

## 5.1 StarsAndBars

Stars and Bars (Number of Solutions to x_1 + x_2 + ... + x_k = n) - O(1)

```
// 1. Non-negative integer solutions (x_i >= 0): C(n + k - 1, k -
     1)
// 2. Positive integer solutions (x_i >= 1): C(n - 1, k - 1)
struct StarsAndBars {
    // Requires precomputed Combination C(n, k)
    static ll count_non_negative(ll n, ll k, function<ll(ll, ll)>
      nCr) {
        if (n < 0 || k <= 0) return 0;
        return nCr(n + k - 1, k - 1);
    }

    static ll count_positive(ll n, ll k, function<ll(ll, ll)> nCr
     ) {
        if (n < k || k <= 0) return 0;
        return nCr(n - 1, k - 1);
    }
};
```

## 5.2 BurnsidePolya

Burnside / Polya - count colourings up to symmetry. #orbits = (1/|G|) sum_g |fix(g)|

```
// Needs: Mint (06 - Math/01); phi (02 - Primes and Divisors/06)
     for the necklace formulas.
// Needs Mint and phi() (single-value Euler totient).

// Necklaces: n beads in a circle, k colours, ROTATIONS only.
// A rotation by r has gcd(r,n) cycles -> k^gcd(r,n) fixed
     colourings; group by d = gcd.
Mint necklaces(ll n, ll k) {
    Mint s = 0;
    for (ll d = 1; d * d <= n; d++) if (n % d == 0) {
        s += Mint(phi(n / d)) * Mint(k).pow(d);
        if (d != n / d) s += Mint(phi(d)) * Mint(k).pow(n / d);
    }
    return s * Mint(n).inv();
}
// Bracelets: rotations AND reflections (dihedral group, |G| = 2n)
     .
Mint bracelets(ll n, ll k) {
    Mint s = necklaces(n, k) * Mint(n);                   // = sum
     over the n rotations
    if (n & 1) s += Mint(n) * Mint(k).pow((n + 1) / 2);
    else s += Mint(n / 2) * (Mint(k).pow(n / 2) + Mint(k).pow(n /
      2 + 1));
    return s * Mint(2 * n).inv();
}
// GENERAL RECIPE: for each group element g, count its CYCLES c(g)
      on the objects being
// coloured; it fixes k^c(g) colourings. Answer = (1/|G|) * sum_g
     k^c(g).
//   square grid n x n under the 4 rotations: cycles = n^2, ceil(n
     ^2/4)... (work them out per problem)
//   cube faces under 24 rotations: cycle counts 6,3,3,4,2,2,...
     -> (k^6+3k^4+12k^3+8k^2)/24
// POLYA (colours have weights): replace k^c by prod over cycles
     of (sum of x_i^len).
Mint burnside(const vector<ll>& cyclesPerElement, ll k) {
    Mint s = 0;
    for (ll c : cyclesPerElement) s += Mint(k).pow(c);
    return s * Mint((ll)cyclesPerElement.size()).inv();
}
```

## 5.3 CombinatoricsFormulas

COMBINATORICS - FORMULA SHEET ==========

```
/* ================= COMBINATORICS - FORMULA SHEET
   ================
BASICS   C(n,k)=n!/(k!(n-k)!)   P(n,k)=n!/(n-k)!   multinomial n
   !/(k1!..km!)
  STARS AND BARS:  x1+..+xk = n, xi>=0 -> C(n+k-1, k-1);  xi>=1
   -> C(n-1, k-1)
                   with upper bounds xi<=c: IE over which
   variables overflow
  Compositions of n into k parts C(n-1,k-1); into any parts 2^(n
   -1)

BINOMIAL IDENTITIES  (memorise these - recognising one is worth
   30 minutes)
  Pascal        C(n,k) = C(n-1,k-1) + C(n-1,k)
  Symmetry      C(n,k) = C(n,n-k)
  Absorption    k*C(n,k) = n*C(n-1,k-1)
  Hockey stick  sum_{i=r..n} C(i,r) = C(n+1, r+1)
  Vandermonde   sum_k C(m,k) C(n,p-k) = C(m+n, p)
  sum_k C(n,k) = 2^n            sum_k (-1)^k C(n,k) = [n==0]
  sum_k k*C(n,k) = n*2^(n-1)    sum_k C(n,k)^2 = C(2n,n)
  sum_k C(k,r) x^k ... upper negation: C(-n,k) = (-1)^k C(n+k-1,
   k)
  sum_{k} C(n,k) C(k,m) = C(n,m) 2^(n-m)
  Binomial inversion: f(n)=sum_k C(n,k) g(k)  <=>  g(n)=sum_k
   (-1)^(n-k) C(n,k) f(k)

INCLUSION-EXCLUSION
  |A1 u..u An| = sum |Ai| - sum |Ai n Aj| + ...
  #surjections [n]->[k] = sum_{i=0..k} (-1)^i C(k,i) (k-i)^n = k!
    * S2(n,k)
  "exactly m" from "at least m":  E(m) = sum_{j>=m} (-1)^(j-m) C(
   j,m) A(j)
  derangements D(n) = n! sum_{k=0..n} (-1)^k/k! = (n-1)(D(n-1)+D(
   n-2)), D0=1,D1=0
  permutations with exactly k fixed points = C(n,k) * D(n-k)

CATALAN  Cat(n) = C(2n,n)/(n+1) = C(2n,n) - C(2n,n+1),  Cat(n+1)
   = sum Cat(i)Cat(n-i)
  1,1,2,5,14,42,132,429,1430,4862  - counts: balanced bracket
   strings of length 2n;
  binary trees with n nodes; triangulations of an (n+2)-gon;
   monotone lattice paths under
  the diagonal; stack-sortable permutations; non-crossing
   partitions/chords.
  BALLOT / CYCLE LEMMA: paths from 0 to k in n steps staying > 0
   = (k/n) C(n, (n+k)/2)
  REFLECTION: #paths (0,0)->(a,b) touching the line y=x+c  =  #
   paths to the reflected point

STIRLING
  2nd kind S2(n,k) = #partitions of n LABELLED items into k non-
   empty UNLABELLED blocks
    S2(n,k) = k*S2(n-1,k) + S2(n-1,k-1);  S2(n,k) = (1/k!) sum_i
    (-1)^i C(k,i)(k-i)^n
    x^n = sum_k S2(n,k) * x_falling(k)          surjections = k!
    S2(n,k)
  1st kind (unsigned) c(n,k) = #permutations of n with exactly k
   cycles
    c(n,k) = c(n-1,k-1) + (n-1) c(n-1,k);   sum_k c(n,k) x^k = x
   (x+1)..(x+n-1)
  BELL B(n) = sum_k S2(n,k) = #set partitions.  B(n+1) = sum_k C(
   n,k) B(k).  Bell triangle.

PARTITIONS (unordered, into positive parts)
  p(n) via pentagonal number theorem, O(n sqrt n):
   p(n) = sum_{k>=1} (-1)^(k+1) [ p(n - k(3k-1)/2) + p(n - k(3k
   +1)/2) ]
  partitions into <= k parts = partitions into parts <= k (
   conjugate)

TWELVEFOLD WAY  (n balls into k boxes)
  balls  boxes  any               injective       surjective
  dist.  dist.  k^n               k!/(k-n)!       k! S2(n,k)
  ident. dist.  C(n+k-1, n)       C(k,n)          C(n-1, n-k)
  dist.  ident. sum_{i<=k} S2(n,i)  [n<=k]        S2(n,k)
  ident. ident. p_k(n)            [n<=k]          p(n into
   exactly k parts)

BURNSIDE / POLYA   #orbits = (1/|G|) sum_{g in G} |fix(g)|
  necklaces (rotations), n beads, k colours:  (1/n) sum_{d|n} phi
   (n/d) k^d
  bracelets (+ reflections): (necklace sum + reflection sum) / (2
   n),
   reflection sum = n*k^((n+1)/2) if n odd, (n/2)(k^(n/2) + k^(n
   /2+1)) if n even

TREES / GRAPHS
  Cayley: n^(n-2) labelled trees on n vertices; with given
   degrees d_i: (n-2)! / prod (d_i-1)!
  Prufer code <-> labelled tree, bijective
  Matrix-Tree: #spanning trees = any cofactor of the Laplacian (D
    - A)
  LGV: #non-intersecting path systems = det( M ) with M_ij = #
   paths a_i -> b_j

GENERATING FUNCTIONS
  OGF   1/(1-x) = sum x^n            1/(1-x)^k = sum C(n+k-1, k
   -1) x^n
       x/(1-x-x^2) = Fibonacci       (1-sqrt(1-4x))/(2x) =
   Catalan
       prod 1/(1-x^i) = partitions  prod (1+x^i) = distinct-part
    partitions
  EGF   e^x = sum x^n/n!             EGF of set partitions = e^(e
   ^x - 1)
  EXPONENTIAL FORMULA: if C(x) is the EGF of connected structures
   , e^C(x) is the EGF of all
  Coefficient extraction: [x^n] A(x)B(x) = sum_k a_k b_(n-k)  (a
   convolution -> NTT)
  Lagrange inversion, for F(0)=0, F'(0)!=0:  [x^n] G(F^-1)(x) =
   (1/n)[x^(n-1)] G'(x)(x/F(x))^n

MISC
  hook length: #standard Young tableaux of shape lambda = n! /
   prod(hook lengths)
  Erdos-Ko-Rado, Dilworth (min chain cover = max antichain),
   Sperner (max antichain C(n,n/2))
====================================================================
    */
```

## 5.4 CatalanStirlingBell

Catalan, Stirling (both kinds), Bell, derangements, partitions. Needs Mint + buildFact().

```
Mint catalan(int n) { return C(2 * n, n) * inv_[n + 1]; }
// paths from 0 to k in n steps (+-1) that stay strictly positive
    - ballot / cycle lemma
Mint ballot(int n, int k) {
    if (n == 0) return k == 0;                          // the
     empty path
    if (k <= 0 || k > n || (n + k) % 2) return 0;       // k <= 0
     was unguarded: Mint(-2) wraps
    return Mint(k) * inv_[n] * C(n, (n + k) / 2);
}

Mint derange(int n) {                                    // D(n) =
     (n-1)(D(n-1)+D(n-2))
    Mint a = 1, b = 0;                                   // D0=1,
     D1=0
    for (int i = 2; i <= n; i++) { Mint c = Mint(i - 1) * (a + b)
     ; a = b, b = c; }
    return n ? b : a;
}
// permutations of n with exactly k fixed points
Mint fixedPoints(int n, int k) { return C(n, k) * derange(n - k);
     }

// Stirling 2nd kind, single value: S2(n,k) = (1/k!) sum_i (-1)^i
    C(k,i) (k-i)^n     O(k log n)
Mint stirling2(ll n, int k) {
    Mint r = 0;
    for (int i = 0; i <= k; i++) {
        Mint t = C(k, i) * Mint(k - i).pow(n);
        (i & 1) ? r -= t : r += t;
    }
    return r * invFact[k];
}
Mint surjections(ll n, int k) { return stirling2(n, k) * fact[k];
     }

// Whole ROW of Stirling 2nd (all k for fixed n) in O(n^2), or O(n
     log n) as a convolution of
// a_i = (-1)^i/i! and b_i = i^n/i!  (NTT).
vector<Mint> stirling2Row(int n) {
    vector<Mint> a(n + 1), b(n + 1), r(n + 1, 0);
    for (int i = 0; i <= n; i++) a[i] = (i & 1 ? Mint(0) -
     invFact[i] : invFact[i]), b[i] = Mint(i).pow(n) * invFact[i
     ];
    for (int k = 0; k <= n; k++) for (int i = 0; i <= k; i++) r[k
     ] += a[i] * b[k - i];
    return r;
}
// Stirling 1st kind (unsigned), whole row: prod_{i=0..n-1} (x + i
    )   O(n^2)
vector<Mint> stirling1Row(int n) {
    vector<Mint> p = {1};
    for (int i = 0; i < n; i++) {
        vector<Mint> q(p.size() + 1, 0);
        for (size_t j = 0; j < p.size(); j++) q[j + 1] += p[j], q
     [j] += p[j] * Mint(i);
        p = q;
    }
    return p;                                            // p[k] =
      c(n, k)
}
// Bell numbers B(0..n) via the Bell triangle, O(n^2)
vector<Mint> bell(int n) {
    vector<Mint> B(n + 1), row = {1};
    B[0] = 1;
    for (int i = 1; i <= n; i++) {
        vector<Mint> nr = {row.back()};
        for (Mint x : row) nr.push_back(nr.back() + x);
        row = nr, B[i] = row[0];
    }
    return B;
}
// Integer partitions p(0..n) via the pentagonal number theorem, O
    (n sqrt n)
vector<Mint> partitions(int n) {
    vector<Mint> p(n + 1, 0);
    p[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int k = 1;; k++) {
            ll g1 = (ll)k * (3 * k - 1) / 2, g2 = (ll)k * (3 * k
     + 1) / 2;
            if (g1 > i) break;
            Mint t = p[i - g1] + (g2 <= i ? p[i - g2] : Mint(0));
            (k & 1) ? p[i] += t : p[i] -= t;
        }
    return p;
}
```

## 5.5 InclusionExclusion

INCLUSION-EXCLUSION recipes. Needs Mint + buildFact().

```
// x1+..+xk = n with 0 <= xi <= ub[i].  IE over which variables
    overflow their bound.
// O(2^k) - fine for k <= ~20. With equal bounds use the closed
    form below instead.
Mint boundedStarsBars(ll n, const vector<ll>& ub) {
    int k = ub.size();
    Mint r = 0;
    for (int m = 0; m < (1 << k); m++) {
        ll rem = n;
        for (int i = 0; i < k; i++) if (m >> i & 1) rem -= ub[i]
     + 1;
        if (rem < 0) continue;
        Mint t = C(rem + k - 1, k - 1);
        (__builtin_popcount(m) & 1) ? r -= t : r += t;
    }
    return r;
}
// Same with a COMMON bound c: sum_j (-1)^j C(k,j) C(n - j(c+1) +
    k-1, k-1).   O(k)
Mint boundedStarsBarsEqual(ll n, ll k, ll c) {
    Mint r = 0;
    for (ll j = 0; j <= k && j * (c + 1) <= n; j++) {
        Mint t = C(k, j) * C(n - j * (c + 1) + k - 1, k - 1);
        (j & 1) ? r -= t : r += t;
    }
    return r;
}
// #surjections [n] -> [k]  (equivalently k! * S2(n,k))
Mint surj(ll n, ll k) {
    Mint r = 0;
    for (ll i = 0; i <= k; i++) {
        Mint t = C(k, i) * Mint(k - i).pow(n);
        (i & 1) ? r -= t : r += t;
    }
    return r;
}
// "AT LEAST" -> "EXACTLY" (binomial / Mobius inversion over the
    subset lattice):
//   if A(j) = #configurations having at least a chosen set of j
    properties (summed over sets),
//   then E(m) = sum_{j>=m} (-1)^(j-m) C(j,m) A(j) counts those
    with EXACTLY m.
Mint exactlyFromAtLeast(const vector<Mint>& A, int m) {
    Mint r = 0;
    for (int j = m; j < (int)A.size(); j++) {
        Mint t = C(j, m) * A[j];
        ((j - m) & 1) ? r -= t : r += t;
    }
    return r;
}
// BINOMIAL INVERSION:  f(n) = sum_k C(n,k) g(k)   <=>   g(n) =
    sum_k (-1)^(n-k) C(n,k) f(k)
vector<Mint> binomialInvert(const vector<Mint>& f) {
    int n = f.size();
    vector<Mint> g(n, 0);
    for (int i = 0; i < n; i++)
        for (int k = 0; k <= i; k++) {
            Mint t = C(i, k) * f[k];
            ((i - k) & 1) ? g[i] -= t : g[i] += t;
        }
    return g;
}
// COUNTING COPRIME PAIRS / "gcd is exactly g": count multiples
    then Mobius-invert downwards:
//   exact[g] = cntMultiples[g] - sum_{g | d, d > g} exact[d]
     (iterate g downwards)
vector<ll> exactGcdCounts(const vector<ll>& cntMultiples) {
    int n = cntMultiples.size() - 1;
    vector<ll> e(n + 1);
    for (int g = n; g >= 1; g--) {
        e[g] = cntMultiples[g];
        for (int d = 2 * g; d <= n; d += g) e[g] -= e[d];
    }
    return e;
}
```

## 5.6 TreeCounting

COUNTING TREES AND GRAPHS.

```
// Needs: Mint (06 - Math/01) and determinant (06 - Math/04 - Linear Algebra/02).

// PRUFER CODE: bijection between labelled trees on n>=2 vertices and sequences in [0,n)^(n-2).
// Immediate corollaries: Cayley n^(n-2) labelled trees; with prescribed degrees d_i the count is
// (n-2)! / prod (d_i - 1)!  (because vertex i appears exactly d_i - 1 times in the code).
vector<int> toPrufer(const vector<vector<int>>& adj) {      // repeatedly strip the SMALLEST leaf
    int n = adj.size();
    if (n <= 2) return {};
    vector<int> deg(n), code;
    vector<bool> gone(n, false);
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 0; i < n; i++) if ((deg[i] = adj[i].size()) == 1) q.push(i);
    for (int it = 0; it < n - 2; it++) {
        int leaf = q.top(); q.pop();
        gone[leaf] = true;
        int nb = -1;
        for (int v : adj[leaf]) if (!gone[v]) nb = v;
        code.push_back(nb);
        if (--deg[nb] == 1) q.push(nb);
    }
    return code;
}
vector<pair<int, int>> fromPrufer(vector<int> code) {
    int n = code.size() + 2;
    vector<int> deg(n, 1);
    for (int x : code) deg[x]++;
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 0; i < n; i++) if (deg[i] == 1) q.push(i);
    vector<pair<int, int>> e;
    for (int x : code) {
        int leaf = q.top(); q.pop();
        e.push_back({leaf, x});
        if (--deg[x] == 1) q.push(x);
    }
    int a = q.top(); q.pop();
    e.push_back({a, q.top()});
    return e;
}

// MATRIX-TREE THEOREM: #spanning trees of a graph = any cofactor of the Laplacian L = D - A.
// Multigraphs work (put the multiplicity in A). Needs determinant() from GaussianElimination.
Mint spanningTrees(int n, const vector<pair<int, int>>& edges) {
    if (n == 1) return 1;
    vector<vector<Mint>> L(n, vector<Mint>(n, 0));
    for (auto [u, v] : edges) {
        L[u][u] += 1, L[v][v] += 1;
        L[u][v] -= 1, L[v][u] -= 1;
    }
    vector<vector<Mint>> M(n - 1, vector<Mint>(n - 1));     // delete row/col 0
    for (int i = 1; i < n; i++) for (int j = 1; j < n; j++) M[i - 1][j - 1] = L[i][j];
    return determinant(M);
}
// DIRECTED version (Tutte): #arborescences rooted at r = cofactor of (Dout - A) deleting row/col r
// for in-trees, or (Din - A) for out-trees.
// WEIGHTED version: put edge weights in A and the weighted degree in D -> sum over spanning
// trees of the product of weights (useful mod p for "sum over all spanning trees" problems).

// EXPONENTIAL FORMULA: if C(x) is the EGF of CONNECTED structures then e^(C(x)) is the EGF of
// all structures.  Concretely, with g_n = #labelled graphs on n nodes = 2^C(n,2):
//   c_n = g_n - sum_{k=1}^{n-1} C(n-1, k-1) * c_k * g_{n-k}
//   (connected labelled graphs)
vector<Mint> connectedLabelledGraphs(int n) {
    vector<Mint> g(n + 1), c(n + 1, 0);
    for (int i = 0; i <= n; i++) g[i] = Mint(2).pow((ll)i * (i - 1) / 2);
    for (int i = 1; i <= n; i++) {
        c[i] = g[i];
        for (int k = 1; k < i; k++) c[i] -= C(i - 1, k - 1) * c[k] * g[i - k];
    }
    return c;
}
```

## 5.7 SubsetConvolution

TRANSFORMS OVER THE SUBSET LATTICE. n bits, arrays of size 2^n.

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01 - Mint.cpp).

// ZETA (sum over subsets) and its inverse MOBIUS. O(n 2^n). This is SOS DP.
void zetaSub(vector<Mint>& f, int n) {                    // f[m] := sum_{s subset of m} f[s]
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (m >> i & 1) f[m] += f[m ^ (1 << i)];
}
void mobiusSub(vector<Mint>& f, int n) {                  // exact inverse of zetaSub
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (m >> i & 1) f[m] -= f[m ^ (1 << i)];
}
void zetaSuper(vector<Mint>& f, int n) {                  // f[m] := sum_{m subset of s} f[s]
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (!(m >> i & 1)) f[m] += f[m | (1 << i)];
}
void mobiusSuper(vector<Mint>& f, int n) {
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (!(m >> i & 1)) f[m] -= f[m | (1 << i)];
}
// OR-convolution: h[m] = sum_{a|b = m} f[a] g[b]      -> zetaSub, multiply, mobiusSub
// AND-convolution: h[m] = sum_{a&b = m} f[a] g[b]     -> zetaSuper, multiply, mobiusSuper
vector<Mint> orConv(vector<Mint> f, vector<Mint> g, int n) {
    zetaSub(f, n), zetaSub(g, n);
    for (int m = 0; m < (1 << n); m++) f[m] *= g[m];
    return mobiusSub(f, n), f;
}
vector<Mint> andConv(vector<Mint> f, vector<Mint> g, int n) {
    zetaSuper(f, n), zetaSuper(g, n);
    for (int m = 0; m < (1 << n); m++) f[m] *= g[m];
    return mobiusSuper(f, n), f;
}

// SUBSET (disjoint) CONVOLUTION: h[m] = sum_{a|b = m, a&b = 0} f[a] g[b].   O(2^n n^2).
// The ranked-zeta trick: index by popcount, convolve the ranks like polynomials.
vector<Mint> subsetConv(const vector<Mint>& f, const vector<Mint>& g, int n) {
    int N = 1 << n;
    vector<vector<Mint>> F(n + 1, vector<Mint>(N, 0)), G = F, H = F;
    for (int m = 0; m < N; m++) F[__builtin_popcount(m)][m] = f[m], G[__builtin_popcount(m)][m] = g[m];
    for (int i = 0; i <= n; i++) zetaSub(F[i], n), zetaSub(G[i], n);
    for (int i = 0; i <= n; i++)
        for (int j = 0; i + j <= n; j++)
            for (int m = 0; m < N; m++) H[i + j][m] += F[i][m] * G[j][m];
    for (int i = 0; i <= n; i++) mobiusSub(H[i], n);
    vector<Mint> h(N);
    for (int m = 0; m < N; m++) h[m] = H[__builtin_popcount(m)][m];
    return h;
}
// USES: exact set cover / "partition the ground set into k valid groups" (repeated subset
// convolution, or exponentiate the EGF-like series); counting perfect matchings by subsets;
// "for each mask, best way to split it into two disjoint valid halves".
// SUBMASK ENUMERATION over all masks is O(3^n):  for (s = m; s; s = (s-1) & m)
```

## 5.8 SpecialSequences

Named sequences that keep showing up, with what each one counts. Needs Mint + buildFact().

```
// MOTZKIN M(n): lattice paths (0,0)->(n,0) with steps U/D/FLAT
     staying >= 0; also the number of
// ways to draw non-crossing chords on n points on a circle. M:
     1,1,2,4,9,21,51,127
vector<Mint> motzkin(int n) {
    vector<Mint> m(n + 1, 0);
    m[0] = 1;
    if (n >= 1) m[1] = 1;
    for (int i = 2; i <= n; i++) {                          // M(i)
      = M(i-1) + sum M(k)M(i-2-k)
        m[i] = m[i - 1];
        for (int k = 0; k <= i - 2; k++) m[i] += m[k] * m[i - 2 -
      k];
    }
    return m;
}
// NARAYANA N(n,k): Dyck paths of length 2n with exactly k peaks.
     sum_k N(n,k) = Catalan(n).
Mint narayana(int n, int k) { return k < 1 || k > n ? Mint(0) : C
     (n, k) * C(n, k - 1) * inv_[n]; }
// SCHRODER: large R(n) counts lattice paths (0,0)->(n,n) with
     steps E/N/NE staying below y=x.
// R: 1,2,6,22,90,394.  R(n) = R(n-1) + sum_{k} R(k) R(n-1-k)  (
     little schroeder s(n) = R(n)/2 for n >= 1 only (s(0) = 1, not
      1/2))
vector<Mint> schroder(int n) {
    vector<Mint> r(n + 1, 0);
    r[0] = 1;
    for (int i = 1; i <= n; i++) { r[i] = r[i - 1]; for (int k =
     0; k < i; k++) r[i] += r[k] * r[i - 1 - k]; }
    return r;
}
// EULERIAN A(n,k): permutations of n with exactly k ascents.
// A(n,k) = (k+1) A(n-1,k) + (n-k) A(n-1,k-1).   sum_k A(n,k) = n!
vector<vector<Mint>> eulerian(int n) {
    vector<vector<Mint>> a(n + 1, vector<Mint>(n + 1, 0));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
        for (int k = 0; k < i; k++)
            a[i][k] = Mint(k + 1) * a[i - 1][k] + Mint(i - k) * (
     k ? a[i - 1][k - 1] : Mint(0));
    return a;
}
// q-BINOMIAL (Gaussian) coefficient: #k-dim subspaces of an n-dim
     space over F_q; also
// #partitions fitting in a k x (n-k) box, weighted by q^area.
// PRECONDITION: ord_p(q) > k. The denominator is prod (q^i - 1),
     which vanishes in F_p
// whenever ord_p(q) <= k - NOT only at q == 1. (q = -1 mod
     998244353 breaks every k >= 2.)
// Root-of-unity-safe fallback, O(nk): Pascal's rule [n,k]_q = [n
     -1,k-1]_q + q^k [n-1,k]_q.
Mint qBinom(int n, int k, Mint q) {                         //
     prod_{i=1..k} (q^(n-k+i)-1)/(q^i-1)
    if (k < 0 || k > n) return 0;
    if (q == Mint(1)) return C(n, k);                       // q=1
     is the 0/0 limit
    Mint num = 1, den = 1;
    for (int i = 1; i <= k; i++) num *= q.pow(n - k + i) - 1, den
      *= q.pow(i) - 1;
    return num / den;
}
// HOOK LENGTH FORMULA: #standard Young tableaux of shape lambda =
     n! / prod(hook lengths).
// hook(i,j) = (arm to the right) + (leg below) + 1.
Mint youngTableaux(const vector<int>& lam) {
    int n = 0;
    for (int x : lam) n += x;
    vector<int> conj(lam.empty() ? 0 : lam[0], 0);
    for (int i = 0; i < (int)lam.size(); i++) for (int j = 0; j <
      lam[i]; j++) conj[j]++;
    Mint den = 1;
    for (int i = 0; i < (int)lam.size(); i++)
        for (int j = 0; j < lam[i]; j++) den *= Mint(lam[i] - j +
      conj[j] - i - 1);
    return fact[n] / den;
}
// LGV LEMMA: for sources a_1..a_k and sinks b_1..b_k in a DAG,
     the number of NON-INTERSECTING
// path systems equals det(M) with M[i][j] = #paths a_i -> b_j (
     when only the identity
// permutation can be non-intersecting). Counts plane partitions,
     rhombus tilings, etc.
```

## 5.9 EulerTransformAndSymbolic

SYMBOLIC METHOD: spec -> generating function ==========

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01 - Mint.cpp).
/* ============ SYMBOLIC METHOD: spec -> generating function
     ============
 Decide labelled (EGF) or unlabelled (OGF), then translate the
     construction:
   UNLABELLED (OGF), A built from atoms with GF B(z):
     SEQ  (ordered list)     A = 1 / (1 - B)
     MSET (unordered, rep)   A = exp( sum_{k>=1} B(z^k) / k )
          <- Euler transform
     PSET (unordered, no rep)A = exp( sum_{k>=1} (-1)^(k-1) B(z^k)
     /k )
     CYC  (up to rotation)   A = sum_{k>=1} (phi(k)/k) * log(1/(1
     - B(z^k)))
   LABELLED (EGF):
     SEQ  A = 1/(1-B)        SET  A = exp(B)        CYC  A = log
     (1/(1-B))
 "SET of connected pieces = exp(connected)" is the exponential
     formula.
================================================================
      */

// EULER TRANSFORM: a[n] = #atoms of size n  ->  b[n] = #MULTISETS
      of atoms of total size n.
// prod_{n>=1} (1 - x^n)^(-a_n) = 1 + sum b_n x^n.     O(n log n)
     via the harmonic sum.
vector<Mint> eulerTransform(const vector<Mint>& a) {    // a[0]
     unused; returns b with b[0] = 1
    int n = a.size() - 1;
    vector<Mint> c(n + 1, 0), b(n + 1, 0);
    for (int d = 1; d <= n; d++)                            // c_n =
     sum_{d | n} d * a_d
        for (int m = d; m <= n; m += d) c[m] += Mint(d) * a[d];
    b[0] = 1;
    for (int i = 1; i <= n; i++) {                          // b_n = (
     c_n + sum_{k<n} c_k b_{n-k}) / n
        Mint s = c[i];
        for (int k = 1; k < i; k++) s += c[k] * b[i - k];
        b[i] = s * inv_[i];
    }
    return b;
}
int muSmall(int n) {                                        // Mobius
     of one small n, by trial division
    int r = 1;
    for (int p = 2; (ll)p * p <= n; p++) if (n % p == 0) {
        n /= p;
        if (n % p == 0) return 0;
        r = -r;
    }
    return n > 1 ? -r : r;
}
// INVERSE Euler transform: b -> a  (given multiset counts,
     recover the atom counts).
vector<Mint> eulerInverse(const vector<Mint>& b) {
    int n = b.size() - 1;
    vector<Mint> c(n + 1, 0), a(n + 1, 0);
    for (int i = 1; i <= n; i++) {                          // c_n = n
     *b_n - sum_{d<n} c_d b_{n-d}
        Mint s = Mint(i) * b[i];
        for (int d = 1; d < i; d++) s -= c[d] * b[i - d];
        c[i] = s;
    }
    for (int i = 1; i <= n; i++) {                          // a_n =
     (1/n) sum_{d|n} mu(n/d) c_d
        Mint s = 0;
        for (int d = 1; d <= i; d++) if (i % d == 0) s += Mint(
     muSmall(i / d)) * c[d];
        a[i] = s * inv_[i];
    }
    return a;
}
// UNLABELLED ROOTED TREES: r_1 = 1, and r_{n+1} = (Euler
     transform of r_1..r_n)[n]
// (a rooted tree = a root plus a MULTISET of rooted subtrees).
vector<Mint> rootedTrees(int n) {
    vector<Mint> r(n + 1, 0);
    if (n >= 1) r[1] = 1;
    for (int i = 1; i < n; i++) {
        vector<Mint> a(i + 1, 0);
        for (int j = 1; j <= i; j++) a[j] = r[j];
        auto b = eulerTransform(a);
        r[i + 1] = b[i];
    }
    return r;                                               //
     0,1,1,2,4,9,20,48,115,286,719,...
}
```

```
// PARTITIONS INTO PARTS FROM A SET S = eulerTransform with a[s] =
     1 for s in S.
// COMPOSITIONS (ordered) use SEQ instead: 1/(1 - sum x^s).
```

## 5.10 PermanentAndSymmetric

RYSER'S FORMULA - permanent of an n x n matrix in O(2^n * n).

```
// perm(A) counts PERFECT MATCHINGS of a bipartite graph (weighted
     : sum over matchings of the
// product of weights). Use for n <= ~20: "count ways to assign n
     tasks to n people",
// "count systems of distinct representatives", "count
     permutations avoiding forbidden cells".
Mint permanent(const vector<vector<Mint>>& a) {
    int n = a.size();
    vector<Mint> row(n, 0);                              // row[i]
     = sum_{j in S} a[i][j]
    Mint total = 0;
    int prev = 0;
    for (int g = 1; g < (1 << n); g++) {                 // Gray
     code: one column toggles per step
        int cur = g ^ (g >> 1), diff = cur ^ prev, j =
     __builtin_ctz(diff);
        if (cur & diff) for (int i = 0; i < n; i++) row[i] += a[i
     ][j];   // column j entered S
        else for (int i = 0; i < n; i++) row[i] -= a[i][j];
             // column j left S
        prev = cur;
        Mint p = 1;
        for (int i = 0; i < n; i++) p *= row[i];
        (__builtin_popcount(cur) & 1) ? total -= p : total += p;
    }
    return (n & 1) ? Mint(0) - total : total;            // perm =
     (-1)^n * sum_S (-1)^|S| prod_i rowSum
}
// PERMANENT vs DETERMINANT: identical formula without the signs.
     det is polynomial-time,
// perm is #P-hard - never expect an O(n^3) permanent.

// NEWTON'S IDENTITIES - convert between power sums p_k = sum x_i^
     k and elementary symmetric e_k.
// e_k = sum over k-subsets of the product; this is exactly "for
     every k, the sum over all
// k-subsets of the product of the chosen values".
//   e_0 = 1;   k * e_k = sum_{i=1..k} (-1)^(i-1) * e_(k-i) * p_i
vector<Mint> powerToElementary(const vector<Mint>& p, int n) {
     // p[1..n] -> e[0..n]
    vector<Mint> e(n + 1, 0);
    e[0] = 1;
    for (int k = 1; k <= n; k++) {
        Mint s = 0;
        for (int i = 1; i <= k; i++) {
            Mint t = e[k - i] * p[i];
            (i & 1) ? s += t : s -= t;
        }
        e[k] = s * inv_[k];
    }
    return e;
}
//   p_k = (-1)^(k-1) * k * e_k + sum_{i=1..k-1} (-1)^(k-1+i) * e_
     (k-i) * p_i
vector<Mint> elementaryToPower(const vector<Mint>& e, int n) {
     // e[0..n] -> p[1..n]
    vector<Mint> p(n + 1, 0);
    for (int k = 1; k <= n; k++) {
        Mint s = Mint(k) * e[k];
        if ((k - 1) & 1) s = Mint(0) - s;                // (-1)^(k
     -1) * k * e_k
        for (int i = 1; i < k; i++) {
            Mint t = e[k - i] * p[i];
            ((k - 1 + i) & 1) ? s -= t : s += t;
        }
        p[k] = s;
    }
    return p;
}
// e_k are also the coefficients of prod (x + x_i): prod_{i} (x +
     x_i) = sum_k e_k * x^(n-k).
// So "sum over k-subsets of the product" = coefficient extraction
     from that product, which you
// can also get in O(n log^2 n) by divide-and-conquer
     multiplication of the linear factors.
```

## 5.11 BernoulliFaulhaber

BERNOULLI NUMBERS and the CLOSED-FORM FAULHABER polynomial.

```
// Use this (not Lagrange interpolation) when you need the
     POLYNOMIAL COEFFICIENTS of
// sum_{k=1..n} k^p, or when p is large and many n are queried.
// Convention here: B_1 = -1/2  (the "B^-" convention that the
     formula below expects).
//   B_0 = 1 ;   sum_{k=0}^{m} C(m+1, k) * B_k = [m == 0]
//   =>  B_m = -(1/(m+1)) * sum_{k=0}^{m-1} C(m+1, k) * B_k
vector<Mint> bernoulli(int m) {                          // B[0..m
     ], O(m^2)
    vector<Mint> B(m + 1, 0);
    B[0] = 1;
    for (int i = 1; i <= m; i++) {
        Mint s = 0;
        for (int k = 0; k < i; k++) s += C(i + 1, k) * B[k];
        B[i] = Mint(0) - s * inv_[i + 1];
    }
    return B;                                            // 1,
     -1/2, 1/6, 0, -1/30, 0, 1/42, ...
}
// O(m log m) alternative: B_m / m! = [t^m] inv( sum_{i>=0} t^i /
     (i+1)! )  -- one Poly::inv.

// FAULHABER:  sum_{k=1}^{n} k^p = (1/(p+1)) * sum_{r=0}^{p} (-1)^
     r * C(p+1, r) * B_r * n^(p+1-r)
Mint powerSumClosed(ll n, int p, const vector<Mint>& B) {
    Mint s = 0, np = Mint(n).pow(p + 1);                 // n^(p+1-
     r), decreasing r
    vector<Mint> pw(p + 2);
    pw[0] = 1;
    for (int i = 1; i <= p + 1; i++) pw[i] = pw[i - 1] * Mint(n);
    for (int r = 0; r <= p; r++) {
        Mint t = C(p + 1, r) * B[r] * pw[p + 1 - r];
        (r & 1) ? s -= t : s += t;
    }
    return s * inv_[p + 1];
}
// The COEFFICIENTS of the Faulhaber polynomial in n (degree p+1):
//   [n^(p+1-r)] = (-1)^r * C(p+1, r) * B_r / (p+1)
vector<Mint> faulhaberPoly(int p, const vector<Mint>& B) {
    vector<Mint> c(p + 2, 0);
    for (int r = 0; r <= p; r++) {
        Mint t = C(p + 1, r) * B[r] * inv_[p + 1];
        c[p + 1 - r] = (r & 1) ? Mint(0) - t : t;
    }
    return c;                                            // c[i] =
     coefficient of n^i
}
// RELATED SUMS
//   sum_{k=0}^{n} C(k, r)      = C(n+1, r+1)
     (hockey stick)
//   sum_{k=1}^{n} k^p * r^k    : degree-p polynomial times r^n;
     solve by ansatz or by
//                                differentiating the geometric
     series p times.
//   Bernoulli also gives Euler-Maclaurin and the von Staudt-
     Clausen denominator facts.
```

## 5.12 CycleLemmaFussCatalan

CYCLE LEMMA (Dvoretzky-Motzkin) and the FUSS-CATALAN / rational-Catalan family.

```
// The general-k version of the ballot problem: it handles every "+1 / -k step" path problem.
//
// CYCLE LEMMA: a word with m a's and n b's, m >= k*n, has EXACTLY m - k*n cyclic shifts that
// are k-dominating (every non-empty prefix has #a > k * #b).
//   k = 1 ballot: p A's and q B's with p > q  ->  exactly p - q dominating shifts,
//                 so the probability that A leads the whole count is (p - q) / (p + q).

// #paths with p up-steps and q down-steps that stay STRICTLY positive after the first step.
Mint ballotStrict(ll p, ll q) {                          // p > q >= 0
    if (p <= q) return 0;
    return C(p + q, q) * Mint(p - q) * inv_[(int)(p + q)];
}
// FUSS-CATALAN: A_m(p, r) = (r / (m*p + r)) * C(m*p + r, m)
Mint fussCatalan(ll m, ll p, ll r) {
    if (m < 0) return 0;
    return C(m * p + r, m) * Mint(r) * Mint(m * p + r).inv();
}
// #p-ary trees with m internal nodes = A_m(p, 1) = C(m*p + 1, m) / (m*p + 1)
Mint pAryTrees(ll m, ll p) { return fussCatalan(m, p, 1); }
// Catalan is the case p = 2, r = 1.
// #sequences of m opens and (p-1)*m closes where every prefix keeps the stack non-negative
// is also A_m(p, 1).

// RATIONAL CATALAN: for gcd(a, b) = 1, the number of monotone lattice paths from (0,0) to
// (b, a) that never rise above the line a*x = b*y is  C(a + b, a) / (a + b).
Mint rationalCatalan(ll a, ll b) { return C(a + b, a) * Mint(a + b).inv(); }

// CHUNG-FELLER: among the C(2n,n) balanced +-1 paths of length 2n, the number having exactly
// 2r steps above the axis is Catalan(n) for EVERY r in 0..n.
// Hence C(2n,n) = (n+1)*Cat(n).
// Use it for "count sequences with exactly k flaws" - the answer does not depend on k.

// REFLECTION PRINCIPLE (the tool behind all of the above):
//   #paths (0,0)->(x,y) touching the line y = m  =  #paths (0,0)->(x, 2m - y)
//   so "#paths avoiding the line" = C(total, up) - C(total, up shifted by the reflection).
// PRECONDITION: the barrier must SEPARATE the start (height 0) from the end y, i.e. m != 0
// and 0, y strictly on the same side of m. Without it the reflection subtracts too much:
// (x,y,m) = (2,2,1) returns -1 where the true count is 0.
Mint pathsAvoidingLine(ll x, ll y, ll m) {
    if (m == 0 || (m > 0 && y >= m) || (m < 0 && y <= m)) return 0;          // steps +1/-1, must never reach y = m
    if (((x + y) & 1) || y > x) return 0;
    ll up = (x + y) / 2, up2 = (x + 2 * m - y) / 2;
    return C(x, up) - ((x + 2 * m - y) % 2 || up2 < 0 || up2 > x ? Mint(0) : C(x, up2));
}
```

## 5.13 LGVandPolya

LGV LEMMA and the POLYA CYCLE INDEX - the two theorems that were stated but never coded.

```
// Needs: Mint, determinant (06 - Math/04/02), phi (04 - NT/02/06), muSmall (09 - EulerTransform...).

// LINDSTROM-GESSEL-VIENNOT: with sources a_1..a_k and sinks b_1..b_k in a DAG,
//   det(M), M[i][j] = #paths a_i -> b_j
// counts NON-INTERSECTING path systems, PROVIDED the only permutation that admits a
// non-intersecting system is the identity (true for "sorted" sources/sinks in a grid).
// Grid instance: monotone lattice paths, so M[i][j] = C(dx + dy, dx).
Mint lgvGrid(const vector<pair<ll, ll>>& src, const vector<pair<ll, ll>>& dst) {
    int k = src.size();
    vector<vector<Mint>> M(k, vector<Mint>(k));
    for (int i = 0; i < k; i++)
        for (int j = 0; j < k; j++) {
            ll dx = dst[j].first - src[i].first, dy = dst[j].second - src[i].second;
            M[i][j] = (dx < 0 || dy < 0) ? Mint(0) : C(dx + dy, dx);
        }
    return determinant(M);                                   // needs GaussianElimination.cpp
}
// USES: k non-crossing monotone paths, rhombus/domino tilings of a region, plane partitions,
// "k tokens walk on a grid and must never share a cell".

// POLYA CYCLE INDEX. Z(G)(a_1..a_n) = (1/|G|) sum_{g in G} prod_k a_k^(#k-cycles of g).
// The UNWEIGHTED orbit count is Z(G)(m, m, ..., m) for m colours - that is what necklaces()
// and bracelets() already give. The reason to carry the cycle index is the WEIGHTED case:
// "necklaces with exactly c_1 of colour 1, c_2 of colour 2, ..." which the numeric form cannot do.
//   Z(C_n) = (1/n) * sum_{d | n} phi(d) * a_d^(n/d)
//   Z(D_n) = (1/2) Z(C_n) + (1/2) a_1 a_2^((n-1)/2)        [n odd]
//   Z(D_n) = (1/2) Z(C_n) + (1/4) ( a_1^2 a_2^((n-2)/2) + a_2^(n/2) )   [n even]
//   Z(S_n) = (1/n) * sum_{l=1..n} a_l * Z(S_(n-l)),   Z(S_0) = 1
// Weighted PET: substitute a_k -> (t_1^k + t_2^k + ... + t_m^k) and read off the coefficient
// of t_1^c1 * ... * t_m^cm.
// Necklaces with a PRESCRIBED colour multiset, n beads, counts c[] summing to n (rotations only):
Mint necklacesWithCounts(int n, const vector<int>& c) {
    Mint total = 0;
    for (int d = 1; d <= n; d++) {
        if (n % d) continue;                                 // rotations of order n/d have d cycles
        bool ok = true;
        for (int x : c) if (x % (n / d)) ok = false;         // each colour must split evenly
        if (!ok) continue;
        Mint ways = fact[d];                                 // multinomial over the d cycle slots
        for (int x : c) ways *= invFact[x / (n / d)];
        total += Mint(phi(n / d)) * ways;
    }
    return total * Mint(n).inv();
}
// APERIODIC necklaces (Lyndon words) with k colours: M(n,k) = (1/n) sum_{d|n} mu(d) k^(n/d).
Mint lyndonCount(ll n, ll k) {
    Mint s = 0;
    for (ll d = 1; d <= n; d++) if (n % d == 0) s += Mint(muSmall(d)) * Mint(k).pow(n / d);
    return s * Mint(n).inv();
}

// TRANSFER MATRIX: count length-L walks in a finite automaton. Build A[s][t] = #transitions,
// then (A^L)[s][t]. The generating function sum_L (A^L)[i][j] x^L = ((I - xA)^-1)[i][j] is
// RATIONAL with denominator det(I - xA) of degree <= #states, so the sequence satisfies a linear
// recurrence of that order - feed a prefix to Berlekamp-Massey and finish in O(k^2 log L)
// instead of O(states^3 log L).
```

# 6 Math

## 6.1 Modular Arithmetic

### 6.1.1 Mint

Modular integer (MOD must be PRIME for inv/division) + O(N) factorial tables.

```
struct Mint {
    int v;
    Mint(ll x = 0) { v = int(x % MOD); if (v < 0) v += MOD; }
    Mint& operator+=(Mint o) { if ((v += o.v) >= MOD) v -= MOD;
     return *this; }
    Mint& operator-=(Mint o) { if ((v -= o.v) < 0) v += MOD;
     return *this; }
    Mint& operator*=(Mint o) { v = int(1LL * v * o.v % MOD);
     return *this; }
    Mint& operator/=(Mint o) { return *this *= o.inv(); }
    friend Mint operator+(Mint a, Mint b) { return a += b; }
    friend Mint operator-(Mint a, Mint b) { return a -= b; }
    friend Mint operator*(Mint a, Mint b) { return a *= b; }
    friend Mint operator/(Mint a, Mint b) { return a /= b; }
    friend bool operator==(Mint a, Mint b) { return a.v == b.v; }
    Mint pow(ll e) const { Mint r = 1, b = *this; for (; e > 0; e
      >>= 1, b *= b) if (e & 1) r *= b; return r; }
    Mint inv() const { return pow(MOD - 2); }           // Fermat
     : a^(p-2) = a^-1 (mod p)
    friend ostream& operator<<(ostream& o, Mint m) { return o <<
     m.v; }
    friend istream& operator>>(istream& i, Mint& m) { ll x; i >>
     x; m = Mint(x); return i; }
};

const int N = 200005;                                   // ALSO
     in 01 - Template.cpp: keep one
Mint fact[N], invFact[N], inv_[N];
void buildFact() {                                      // O(N)
     total, not O(N log MOD)
    fact[0] = invFact[0] = inv_[1] = 1;
    for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i;
    invFact[N - 1] = fact[N - 1].inv();
    for (int i = N - 1; i > 0; i--) invFact[i - 1] = invFact[i] *
      i;
    for (int i = 2; i < N; i++) inv_[i] = Mint(MOD - MOD / i) *
     inv_[MOD % i];
}
Mint C(ll n, ll r) { if (r < 0 || r > n) return 0; return fact[n]
      * invFact[r] * invFact[n - r]; }
Mint Perm(ll n, ll r) { if (r < 0 || r > n) return 0; return fact
     [n] * invFact[n - r]; }
// C(n, r) for HUGE n and small r (no tables): prod_{i<r}(n-i) / r
     !
Mint Cbig(ll n, ll r) {
    if (r < 0 || n < 0 || r > n) return 0;
    Mint num = 1;
    for (ll i = 0; i < r; i++) num *= Mint(n - i);
    return num * invFact[r];
}
```

## 6.2 Multiplicative Functions

### 6.2.1 EulerTotientPhi

EULER TOTIENT SIEVE - phiArr[i] = #{1<=k<=i : gcd(k,i)=1} for all i < N in O(N log log N).

```
// Needed for: a^b mod m with gcd(a,m)=1 (Euler's theorem),
     counting coprime pairs, Mobius identities
// (sum_{d|n} phi(d) = n), and the order of the multiplicative
     group mod n.
// NAME: phiArr, not phi - a global 'phi' collides with the phi()
     FUNCTION in
// 04 - Number Theory/02 - Primes and Divisors/06 -
     MillerRabinPollard.cpp.
int phiArr[N];

void calculatePhi() {
    for (int i = 0; i < N; ++i) phiArr[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < N; i += 2)
        if (phiArr[i] == i)
            for (int j = i; j < N; j += i) phiArr[j] -= phiArr[j]
     / i;
}
```

### 6.2.2 MobiusFunction

Mobius Function mu(n) Sieve - Time: O(N), Space: O(N)

```
// mu(n) = 1 if n is square-free with an even number of prime
     factors
// mu(n) = -1 if n is square-free with an odd number of prime
     factors
// mu(n) = 0 if n has a squared prime factor
struct Mobius {
    int n;
    vector<int> mu, primes;
    vector<bool> is_prime;

    Mobius(int n = 1e7) : n(n), mu(n + 1, 0), is_prime(n + 1,
     true) {
        is_prime[0] = is_prime[1] = false;
        mu[1] = 1;

        for (int i = 2; i <= n; i++) {
            if (is_prime[i]) {
                primes.pb(i);
                mu[i] = -1;
            }
            for (int p : primes) {
                if ((ll)i * p > n) break;
                is_prime[i * p] = false;
                if (i % p == 0) {
                    mu[i * p] = 0;
                    break;
                } else {
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};
```

## 6.3 Matrices

### 6.3.1 MatrixExponentiation

Matrix over Mint. mul O(n^3) (cache-friendly i-k-j order), pow O(n^3 log e).

```
// Needs: Mint (01 - Modular Arithmetic/01 - Mint.cpp).
// Fix the size at compile time (array<array<Mint,K>,K>) if you
     need the last 2x speedup.
typedef vector<vector<Mint>> Mat;

Mat operator*(const Mat& a, const Mat& b) {
    int n = a.size(), k = b.size(), m = b[0].size();
    Mat c(n, vector<Mint>(m));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) if (a[i][t].v)
            for (int j = 0; j < m; j++) c[i][j] += a[i][t] * b[t
     ][j];
    return c;
}
Mat eye(int n) { Mat r(n, vector<Mint>(n)); for (int i = 0; i < n
     ; i++) r[i][i] = 1; return r; }
Mat mpow(Mat a, ll e) {
    Mat r = eye(a.size());
    for (; e > 0; e >>= 1, a = a * a) if (e & 1) r = r * a;
    return r;
}
// RECIPES
//  Linear recurrence f(n) = c1*f(n-1) + ... + ck*f(n-k):
//    companion matrix, first row = (c1..ck), subdiagonal = 1.  f(
     n) = (M^(n-k) * [f(k-1)..f(0)]^T)[0]
//  Add a "+d" constant term: enlarge by one row/col with a 1 on
     the diagonal, put d in the first row.
//  Count walks of length L between u and v: (A^L)[u][v] where A
     is the adjacency matrix.
//  Shortest walk with exactly L edges: same, but replace (+,*)
     with (min,+) - "min-plus product".
```

## 6.4 Linear Algebra

### 6.4.1 LinearXorBasis

XOR basis (vector space over GF(2)). insert / query O(B).

```
// span size = 2^sz. Use MERGE to keep a basis in each segment-tree node -> max-xor over a RANGE.
struct Basis {
    static const int B = 62;      // 60 silently drops bits 61-62 of an ll up to 9e18
    ll b[B + 1] = {};                                   // b[i] has leading bit i
    int sz = 0;

    bool insert(ll x) {                                 // false if x already in the span
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) { b[i] = x, sz++; return true; }
            x ^= b[i];
        }
        return false;
    }
    bool contains(ll x) const {
        for (int i = B; i >= 0; i--) if (x >> i & 1) { if (!b[i]) return false; x ^= b[i]; }
        return true;
    }
    ll maxXor(ll r = 0) const { for (int i = B; i >= 0; i--) r = max(r, r ^ b[i]); return r; }
    ll minXor(ll r = 0) const { for (int i = B; i >= 0; i--) if (r >> i & 1) r ^= b[i]; return r; }
    void reduce() {                                     // reduced row echelon form
        for (int i = B; i >= 0; i--) if (b[i])
            for (int j = i - 1; j >= 0; j--) if (b[i] >> j & 1) b[i] ^= b[j];
    }
    ll kth(ll k) {                                      // k-th SMALLEST value in the span, 0-indexed
        reduce();                                       // kth(0) == 0; -1 if k >= 2^sz
        ll r = 0;
        for (int i = 0; i <= B; i++) if (b[i]) { if (k & 1) r ^= b[i]; k >>= 1; }
        return k ? -1 : r;
    }
    ll spanSize() const { return sz >= 63 ? -1 : 1LL << sz; }
    friend Basis merge(Basis a, const Basis& c) {
        for (int i = 0; i <= B; i++) if (c.b[i]) a.insert(c.b[i]);
        return a;
    }
};
```

### 6.4.2 GaussianElimination

Gaussian elimination mod a PRIME. Returns rank; solves A x = b; also det and inverse.

```
// Needs: Mint (01 - Modular Arithmetic/01 - Mint.cpp) for the modular routines.
// O(n^2 m). Pass the augmented matrix (n rows, m+1 cols) to solve().
int gaussMod(vector<vector<Mint>>& a, int m) {          // reduce first m columns to RREF
    int n = a.size(), rank = 0;
    for (int col = 0; col < m && rank < n; col++) {
        int piv = -1;
        for (int i = rank; i < n; i++) if (a[i][col].v) { piv = i; break; }
        if (piv < 0) continue;
        swap(a[rank], a[piv]);
        Mint iv = a[rank][col].inv();
        for (auto& x : a[rank]) x *= iv;
        for (int i = 0; i < n; i++) if (i != rank && a[i][col].v) {
            Mint f = a[i][col];
            for (size_t j = 0; j < a[i].size(); j++) a[i][j] -= f * a[rank][j];
        }
        rank++;
    }
    return rank;
}
// Solve A x = b. Returns {} if inconsistent, else one solution (free vars = 0).
// `free` receives the indices of the free variables (solution space has MOD^|free| points).
vector<Mint> solveLinear(vector<vector<Mint>> a, vector<Mint> b, vector<int>* freeIdx = nullptr) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) a[i].push_back(b[i]);
    int rank = gaussMod(a, m);
    for (int i = rank; i < n; i++) if (a[i][m].v) return {};   // 0 = nonzero
    vector<Mint> x(m, 0);
    int r = 0;
    for (int c = 0; c < m; c++) {                       // must run over EVERY column: stopping at
        if (r < rank && a[r][c].v) x[c] = a[r][m], r++;   // r == rank drops the free columns
        else if (freeIdx) freeIdx->push_back(c);          // that come after the last pivot
    }
    return x;
}
Mint determinant(vector<vector<Mint>> a) {              // O(n^3), destroys a
    int n = a.size();
    Mint det = 1;
    for (int col = 0; col < n; col++) {
        int piv = -1;
        for (int i = col; i < n; i++) if (a[i][col].v) { piv = i; break; }
        if (piv < 0) return 0;
        if (piv != col) swap(a[col], a[piv]), det = Mint(0) - det;
        det *= a[col][col];
        Mint iv = a[col][col].inv();
        for (int i = col + 1; i < n; i++) if (a[i][col].v) {
            Mint f = a[i][col] * iv;
            for (int j = col; j < n; j++) a[i][j] -= f * a[col][j];
        }
    }
    return det;
}
vector<vector<Mint>> inverse(vector<vector<Mint>> a) {   // {} if singular
    int n = a.size();
    for (int i = 0; i < n; i++) { a[i].resize(2 * n, 0); a[i][n + i] = 1; }
    if (gaussMod(a, n) < n) return {};
    vector<vector<Mint>> r(n, vector<Mint>(n));
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) r[i][j] = a[i][n + j];
    return r;
}
// MATRIX-TREE: #spanning trees = determinant of the Laplacian (deg on diagonal, -1 per edge)
// with any one row AND the matching column deleted.
```

### 6.4.3 GaussXorAndReal

XOR-equation systems over GF(2) with bitset - O(n*m/64). n equations, m variables,

```
// row[i][m] holds the RHS. Returns rank, or -1 if inconsistent.
template <int M>
int gaussXor(vector<bitset<M + 1>>& a, int m, bitset<M>* sol =
    nullptr) {
    int n = a.size(), rank = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && rank < n; col++) {
        int piv = -1;
        for (int i = rank; i < n; i++) if (a[i][col]) { piv = i;
    break; }
        if (piv < 0) continue;
        swap(a[rank], a[piv]);
        for (int i = 0; i < n; i++) if (i != rank && a[i][col]) a
    [i] ^= a[rank];
        where[col] = rank++;
    }
    for (int i = rank; i < n; i++) if (a[i][m]) return -1;
    if (sol) { sol->reset(); for (int c = 0; c < m; c++) if (
    where[c] >= 0) (*sol)[c] = a[where[c]][m]; }
    return rank;                                    // #
    solutions = 2^(m - rank)
}
// USES: "light switch" puzzles, xor-equation counting, rank of a
    set of vectors over GF(2),
//       linear independence of xor values (dense sibling of the
    XOR basis).

// ---- Gaussian elimination over the reals, with partial pivoting
     ----
// Returns rank; a is n x (m+1) augmented. Solution in x (free
    vars = 0). eps-based.
int gaussReal(vector<vector<ld>> a, int m, vector<ld>& x) {
    int n = a.size(), rank = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && rank < n; col++) {
        int piv = rank;
        for (int i = rank; i < n; i++) if (fabsl(a[i][col]) >
    fabsl(a[piv][col])) piv = i;
        if (fabsl(a[piv][col]) < 1e-12) continue;
        swap(a[rank], a[piv]);
        for (int i = 0; i < n; i++) if (i != rank) {
            ld f = a[i][col] / a[rank][col];
            for (int j = col; j <= m; j++) a[i][j] -= f * a[rank
    ][j];
        }
        where[col] = rank++;
    }
    x.assign(m, 0);
    for (int c = 0; c < m; c++) if (where[c] >= 0) x[c] = a[where
    [c]][m] / a[where[c]][c];
    return rank;
}
// The classic use is EXPECTED-VALUE DP with cyclic state
    dependencies: write one equation per
// state, E[s] = 1 + sum p(s->t) E[t], and solve. n <= ~500 fits O
    (n^3).
```

## 6.5 Convolution

### 6.5.1 FFT

FFT over complex doubles. conv() is exact while |result coefficient| < ~1e15.

```
// Prefer NTT when the answer is taken modulo an NTT prime; use
    convMod for arbitrary moduli.
typedef complex<double> C;
void fft(vector<C>& a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n), rt.resize(n);
        auto x = polar(1.0L, acosl(-1.0L) / k);
        for (int i = k; i < 2 * k; i++) rt[i] = R[i] = i & 1 ? R[
    i / 2] * x : R[i / 2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) <<
     L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[
    i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                C z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
}
// Real convolution. Result is rounded to the nearest integer.
vector<ll> conv(const vector<ll>& a, const vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, n = 1;
    while (n < rs) n <<= 1;
    vector<C> in(n), out(n);
    for (size_t i = 0; i < a.size(); i++) in[i].real((double)a[i
    ]);
    for (size_t i = 0; i < b.size(); i++) in[i].imag((double)b[i
    ]);
    fft(in);
    for (C& x : in) x *= x;                          // (a+bi
    )^2 = a^2-b^2 + 2abi
    for (int i = 0; i < n; i++) out[i] = in[-i & (n - 1)] - conj(
    in[i]);
    fft(out);
    vector<ll> res(rs);
    for (int i = 0; i < rs; i++) res[i] = llround(imag(out[i]) /
    (4 * n));
    return res;
}
// Convolution modulo an ARBITRARY M, splitting coefficients at
    sqrt(M) to keep precision.
vector<ll> convMod(const vector<ll>& a, const vector<ll>& b, ll M
    ) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, B = 32 - __builtin_clz(rs),
     n = 1 << B;
    ll cut = (ll)sqrtl((ld)M);
    vector<C> L(n), R(n), os(n), ol(n);
    for (size_t i = 0; i < a.size(); i++) L[i] = C((double)(a[i]
    / cut), (double)(a[i] % cut));
    for (size_t i = 0; i < b.size(); i++) R[i] = C((double)(b[i]
    / cut), (double)(b[i] % cut));
    fft(L), fft(R);
    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        ol[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        os[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / C(0, 1);
    }
    fft(ol), fft(os);
    vector<ll> res(rs);
    for (int i = 0; i < rs; i++) {
        ll av = llround(real(ol[i])), cv = llround(imag(os[i]));
        ll bv = llround(imag(ol[i])) + llround(real(os[i]));
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

### 6.5.2 NTT

NTT modulo 998244353 (= 119*2^23 + 1, primitive root 3). O(n log n).

```
// multiply() is the entry point. MAX LENGTH 2^23 for 998244353 (see the assert in ntt()).
// For an ARBITRARY modulus: run this under three of the primes below and Garner them together
// (06 - Garner.cpp is written for exactly that), or split the coefficients at sqrt(M) and use
// convMod in 01 - FFT.cpp.
const ll NMOD = 998244353, NROOT = 3;
ll npow(ll a, ll e, ll m = NMOD) { ll r = 1; a %= m; for (; e; e >>= 1, a = a * a % m) if (e & 1) r = r * a % m; return r; }

void ntt(vector<ll>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        assert((NMOD - 1) % len == 0);   // 998244352 = 119*2^23: len > 2^23 truncates the division
        ll w = npow(NROOT, (NMOD - 1) / len);   // and returns GARBAGE with no other symptom
        if (inv) w = npow(w, NMOD - 2);
        for (int i = 0; i < n; i += len) {
            ll cur = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len / 2] * cur % NMOD;
                a[i + j] = (u + v) % NMOD;
                a[i + j + len / 2] = (u - v + NMOD) % NMOD;
                cur = cur * w % NMOD;
            }
        }
    }
    if (inv) { ll ninv = npow(n, NMOD - 2); for (ll& x : a) x = x * ninv % NMOD; }
}
vector<ll> multiply(vector<ll> a, vector<ll> b) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, n = 1;
    while (n < rs) n <<= 1;
    a.resize(n), b.resize(n);
    ntt(a, false), ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % NMOD;
    ntt(a, true);
    a.resize(rs);
    return a;
}
// ARBITRARY MODULUS: split each coefficient as a = hi*2^15 + lo and combine three products
// (or run NTT under three NTT-friendly primes and CRT). Values must stay below ~1e18.
// COMMON NTT PRIMES: 998244353 (g=3), 167772161 (g=3), 469762049 (g=3), 1004535809 (g=3).
// USES: polynomial multiplication; "count pairs/triples with a given sum"; convolving
// distributions; string matching with wildcards; sum of C(a_i, k) over all i for every k.
```

### 6.5.3 FWHT

Fast Walsh-Hadamard Transform (FWHT) - Time: O(N log N)

```
// Computes Bitwise Convolutions: XOR, AND, OR for array of size N = 2^k
struct FWHT {
    enum Type { XOR, AND, OR };

    static void transform(vector<ll>& a, bool invert, Type type = XOR) {
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    ll u = a[i + j], v = a[i + len + j];
                    if (type == XOR) {
                        a[i + j] = u + v;
                        a[i + len + j] = u - v;
                    } else if (type == AND) {
                        a[i + j] = invert ? u - v : u + v;
                    } else if (type == OR) {
                        a[i + len + j] = invert ? v - u : u + v;
                    }
                }
            }
        }
        if (type == XOR && invert) {
            for (ll &x : a) x /= n;
        }
    }

    // Computes C[k] = sum_{i op j = k} (A[i] * B[j]) where op in {XOR, AND, OR}
    static vector<ll> multiply(vector<ll> a, vector<ll> b, Type type = XOR) {
        int n = 1;
        while (n < max(a.size(), b.size())) n <<= 1;
        a.resize(n); b.resize(n);

        transform(a, false, type);
        transform(b, false, type);
        for (int i = 0; i < n; i++) a[i] *= b[i];
        transform(a, true, type);
        return a;
    }
};
```

### 6.5.4 Polynomial

FORMAL POWER SERIES over NMOD = 998244353, on top of multiply() from 02 - NTT.cpp.

```
// Every routine works "mod x^k". inv/log/exp/sqrt/pow are all Newton iteration: each step
// doubles the number of correct coefficients, so the cost is the cost of the last multiply.
typedef vector<ll> Poly;

Poly cut(Poly a, int k) { a.resize(k, 0); return a; }
Poly operator+(Poly a, const Poly& b) {
    a.resize(max(a.size(), b.size()), 0);
    for (size_t i = 0; i < b.size(); i++) a[i] = (a[i] + b[i]) % NMOD;
    return a;
}
Poly operator-(Poly a, const Poly& b) {
    a.resize(max(a.size(), b.size()), 0);
    for (size_t i = 0; i < b.size(); i++) a[i] = (a[i] - b[i] + NMOD) % NMOD;
    return a;
}
Poly operator*(const Poly& a, ll c) { Poly r = a; for (ll& x : r) x = x * (c % NMOD) % NMOD; return r; }

Poly deriv(const Poly& a) {
    if (a.size() <= 1) return {0};
    Poly r(a.size() - 1);
    for (size_t i = 1; i < a.size(); i++) r[i - 1] = a[i] * i % NMOD;
    return r;
}
Poly integ(const Poly& a) {
    Poly r(a.size() + 1, 0);
    for (size_t i = 0; i < a.size(); i++) r[i + 1] = a[i] * npow(i + 1, NMOD - 2) % NMOD;
    return r;
}
// 1/a mod x^k. Requires a[0] != 0.   g_{2m} = g_m * (2 - a * g_m)
Poly inv(const Poly& a, int k) {
    Poly g = {npow(a[0], NMOD - 2)};
    for (int m = 1; m < k; m <<= 1) {
        Poly t = multiply(cut(a, 2 * m), g);
        t = cut(t, 2 * m);
        for (ll& x : t) x = (NMOD - x) % NMOD;
        t[0] = (t[0] + 2) % NMOD;
        g = cut(multiply(g, t), 2 * m);
    }
    return cut(g, k);
}
// log(a) mod x^k. Requires a[0] == 1.   log a = integral(a' / a)
Poly log_(const Poly& a, int k) { return cut(integ(cut(multiply(deriv(a), inv(a, k)), k - 1)), k); }
// exp(a) mod x^k. Requires a[0] == 0.   g_{2m} = g_m * (1 + a - log g_m)
Poly exp_(const Poly& a, int k) {
    Poly g = {1};
    for (int m = 1; m < k; m <<= 1) {
        Poly t = cut(a, 2 * m) - log_(cut(g, 2 * m), 2 * m);
        t[0] = (t[0] + 1) % NMOD;
        g = cut(multiply(g, t), 2 * m);
    }
    return cut(g, k);
}
// sqrt(a) mod x^k, for a[0] == 1 (general a[0] needs a modular square root of a[0]).
Poly sqrt_(const Poly& a, int k) {
    Poly g = {1};
    ll i2 = npow(2, NMOD - 2);
    for (int m = 1; m < k; m <<= 1) {
        Poly t = cut(a, 2 * m) + multiply(g, g);
        t = cut(t, 2 * m);
        g = cut(multiply(t, inv(cut(g, 2 * m) * 2, 2 * m)), 2 * m);
    }
    return cut(g, k);
}
// a^e mod x^k, for a[0] == 1:  exp(e * log a).  For a[0] != 1 factor out c*x^d first.
Poly pow_(const Poly& a, ll e, int k) { return exp_(log_(a, k) * (e % NMOD), k); }

// Polynomial DIVISION with remainder: a = q*b + r, deg r < deg b.   O(n log n)
pair<Poly, Poly> divmod(Poly a, Poly b) {
    while (a.size() > 1 && !a.back()) a.pop_back();    // trim BOTH, or the reversal is wrong
    while (b.size() > 1 && !b.back()) b.pop_back();
    int n = a.size(), m = b.size();
    if (n < m) return {{0}, a};
    Poly ra(a.rbegin(), a.rend()), rb(b.rbegin(), b.rend());
    Poly q = cut(multiply(cut(ra, n - m + 1), inv(cut(rb, n - m + 1), n - m + 1)), n - m + 1);
    reverse(all(q));
    Poly r = cut(a - multiply(q, b), max(1, m - 1));    // cut AFTER subtracting, not before
    return {q, r};
}
// USES: counting with generating functions (exp of a "connected" series = all structures),
// partition numbers via prod 1/(1-x^i) = exp(sum ...), Catalan-style algebraic equations via
// sqrt, k-th power of a GF, linear recurrence coefficients via inv, and Lagrange inversion.
```

## 6.6 Interpolation

### 6.6.1 LagrangeAndFaulhaber

Lagrange interpolation + power sums. Needs Mint + buildFact().

```
// Needs: Mint (01 - Modular Arithmetic/01 - Mint.cpp).

// Given y[0..n] = f(0), f(1), ..., f(n) for a polynomial f of degree <= n, evaluate f(x). O(n).
// The consecutive-points case is the one you actually want; it kills a whole class of problems
// whose answer is "a polynomial in n of degree d" (guess d, sample d+1 values, interpolate).
Mint lagrangeConsecutive(const vector<Mint>& y, ll x) {
    int n = y.size() - 1;
    if (0 <= x && x <= n) return y[x];          // x < 0 would index y[negative]
    vector<Mint> pre(n + 2, 1), suf(n + 2, 1);
    for (int i = 0; i <= n; i++) pre[i + 1] = pre[i] * Mint(x - i);
    for (int i = n; i >= 0; i--) suf[i] = suf[i + 1] * Mint(x - i);
    Mint r = 0;
    for (int i = 0; i <= n; i++) {
        Mint t = y[i] * pre[i] * suf[i + 1] * invFact[i] * invFact[n - i];
        ((n - i) & 1) ? r -= t : r += t;
    }
    return r;
}
// Arbitrary sample points, O(n^2).
Mint lagrange(const vector<Mint>& xs, const vector<Mint>& ys, Mint x) {
    int n = xs.size();
    Mint r = 0;
    for (int i = 0; i < n; i++) {
        Mint num = ys[i], den = 1;
        for (int j = 0; j < n; j++) if (j != i) num *= x - xs[j], den *= xs[i] - xs[j];
        r += num / den;
    }
    return r;
}
// FAULHABER: sum_{i=1..n} i^k  is a polynomial in n of degree k+1 -> sample k+2 points. O(k log k).
Mint powerSum(ll n, int k) {
    vector<Mint> y(k + 2, 0);
    for (int i = 1; i <= k + 1; i++) y[i] = y[i - 1] + Mint(i).pow(k);
    return lagrangeConsecutive(y, n);
}
// Same trick works for: sum of C(i,k), sum of i^k * r^i (degree k in n after dividing by r^n),
// and any DP whose answer is eventually polynomial.
```

### 6.6.2 BerlekampMassey

Berlekamp-Massey: find the shortest linear recurrence fitting a sequence, O(n^2).

```
// Needs: Mint (01 - Modular Arithmetic/01 - Mint.cpp).
// Then jump to the k-th term in O(len^2 log k). THE weapon for "
    brute force small n, find the
// pattern, extrapolate" - works on any sequence that satisfies a
    constant-coefficient recurrence.
// Feed it ~2*len terms (60-100 is usually plenty).
vector<Mint> berlekampMassey(vector<Mint> s) {
    int n = s.size(), L = 0, m = 0;
    vector<Mint> C(n), B(n), T;
    C[0] = B[0] = 1;
    Mint b = 1;
    for (int i = 0; i < n; i++) {
        m++;
        Mint d = s[i];
        for (int j = 1; j <= L; j++) d += C[j] * s[i - j];
        if (d.v == 0) continue;
        T = C;
        Mint coef = d / b;
        for (int j = m; j < n; j++) C[j] -= coef * B[j - m];
        if (2 * L > i) continue;
        L = i + 1 - L, B = T, b = d, m = 0;
    }
    C.resize(L + 1), C.erase(C.begin());
    for (auto& x : C) x = Mint(0) - x;
    return C;                                          // s[i] =
      sum_j C[j] * s[i-1-j]
}
// k-th term (0-indexed) of the recurrence tr with initial values
    S. O(|tr|^2 log k).
Mint linearRec(const vector<Mint>& S, const vector<Mint>& tr, ll
    k) {
    if (tr.empty()) return 0;                  // e[1] would be
     out of bounds
    int n = tr.size();
    auto comb = [&](vector<Mint> a, vector<Mint> b) {
        vector<Mint> r(2 * n + 1, 0);
        for (int i = 0; i <= n; i++) if (a[i].v)
            for (int j = 0; j <= n; j++) r[i + j] += a[i] * b[j];
        for (int i = 2 * n; i > n; i--)
            for (int j = 0; j < n; j++) r[i - 1 - j] += r[i] * tr
     [j];
        r.resize(n + 1);
        return r;
    };
    vector<Mint> pol(n + 1, 0), e(pol);
    pol[0] = e[1] = 1;
    for (++k; k; k /= 2) {
        if (k % 2) pol = comb(pol, e);
        e = comb(e, e);
    }
    Mint r = 0;
    for (int i = 0; i < n; i++) r += pol[i + 1] * S[i];
    return r;
}
```

## 6.7 Numerical

### 6.7.1 Integration

ADAPTIVE SIMPSON - numeric integral of a black-box f on [a,b] to a requested absolute error.

```
// Simpson approximates f by a parabola through the two ends and
    the midpoint; "adaptive" means it
// splits an interval only where the parabola does not yet agree
    with the two half-parabolas.
// Reach for it when the answer is an integral you cannot do in
    closed form: area of a union of
// circles/ellipses cut by a line, area swept by a moving shape,
    probability over a continuous
// parameter, arc lengths. Cost is proportional to how wiggly f is
    , not to (b-a).
template <class F> ld simpson(F f, ld a, ld b) {
    return (b - a) * (f(a) + 4 * f((a + b) / 2) + f(b)) / 6;
}
template <class F> ld asr(F f, ld a, ld b, ld eps, ld S, int dep)
     {
    ld c = (a + b) / 2, L = simpson(f, a, c), R = simpson(f, c, b
    );
    if (dep <= 0 || fabsl(L + R - S) <= 15 * eps) return L + R +
    (L + R - S) / 15;
    return asr(f, a, c, eps / 2, L, dep - 1) + asr(f, c, b, eps /
     2, R, dep - 1);
}
template <class F> ld integrate(F f, ld a, ld b, ld eps = 1e-9) {
    return asr(f, a, b, eps, simpson(f, a, b), 50);
}
// TRAPS. (1) If f has a KINK or a jump, split the interval at
    that point by hand and integrate the
// pieces - adaptive Simpson will burn its depth budget trying to
    resolve it. (2) The (L+R-S)/15
// term is Richardson extrapolation; keeping it makes the result
    much more accurate for free.
// (3) A too-loose eps on a nearly flat region can stop early and
    miss a spike: if f can be zero on
// most of the range and huge somewhere, integrate the pieces
    separately.
//
// OTHER NUMERICAL TOOLS WORTH THE PAGE:
// TERNARY SEARCH on a strictly unimodal f over reals: 100
    iterations of
//   m1 = l + (r-l)/3, m2 = r - (r-l)/3;  f(m1) < f(m2) ? r = m2 :
     l = m1;   (minimising)
//   fixes the answer to ~1e-30 relative. On INTEGERS use binary
    search on the difference
//   f(m+1) - f(m) instead - the three-way split has off-by-one
    traps at the last two points.
// NEWTON'S METHOD for a root of g: x -= g(x) / g'(x). Doubles the
     correct digits per step but
//   needs a good start; bisect first if the sign changes, then
    Newton.
// BISECTION on a monotone predicate: 100 iterations of (l+r)/2 is
     always safe and never diverges.
// GOLDEN-SECTION SEARCH: like ternary search but one function
    evaluation per step instead of two -
//   worth it only when f is expensive.
// SOLVING f(x) = 0 WHERE f IS A POLYNOMIAL: find all real roots
    by recursing on the derivative
//   (roots of f' split the line into monotone pieces; bisect
    inside each).
// SIMULATED ANNEALING / HILL CLIMBING when the objective is not
    unimodal: start from several
//   random points, take a step of size T, accept worse solutions
    with probability exp(-dE/T),
//   cool T geometrically (T *= 0.995). Good for geometric-median
    / facility-location style
//   problems where an exact method is out of reach; always seed
    the RNG from the clock.
// WEISZFELD for the geometric median (minimise sum of Euclidean
    distances): iterate
//   x <- (sum p_i / |x - p_i|) / (sum 1 / |x - p_i|), guarding
    the case x == p_i. Converges
//   linearly; nested ternary search is the safer contest answer (
    the objective is convex).
```

### 6.7.2 LinearSolvers

SPECIAL-SHAPE LINEAR SOLVERS. Gaussian elimination is O(n^3); these are the shapes where the

```
// structure buys you a factor of n or removes a precondition.

// --- THOMAS ALGORITHM: TRIDIAGONAL system in O(n). ---
//   a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i],  with a[0] = c[
    n-1] = 0.
// This is THE closer for random walks on a path ("expected steps
    to escape [0, n]"), 1-D heat /
// diffusion DPs, and any DP whose state depends on its two
    neighbours. Gaussian elimination on
// the same system is O(n^3) and will TLE at n = 1e5; this is 8
    lines.
// STABLE without pivoting when the system is diagonally dominant
    (|b| > |a| + |c|), which every
// hitting-time system is.
vector<ld> thomas(vector<ld> a, vector<ld> b, vector<ld> c,
    vector<ld> d) {
    int n = b.size();
    for (int i = 1; i < n; i++) {
        ld m = a[i] / b[i - 1];
        b[i] -= m * c[i - 1], d[i] -= m * d[i - 1];
    }
    vector<ld> x(n);
    x[n - 1] = d[n - 1] / b[n - 1];
    for (int i = n - 2; i >= 0; i--) x[i] = (d[i] - c[i] * x[i +
    1]) / b[i];
    return x;
}
// BANDED systems of half-width w (state depends on w neighbours)
    generalise this at O(n w^2) -
// the shape of "robot on a grid with m columns", where w = m.
// CYCLIC tridiagonal (x[0] and x[n-1] also coupled): Sherman-
    Morrison - solve two tridiagonal
// systems and combine, still O(n).

// --- DETERMINANT MODULO AN ARBITRARY m (composite allowed), O(n
    ^3 log m). ---
// Gaussian elimination inverts the pivot, so it needs a PRIME
    modulus. This version never
// divides: it runs the Euclidean algorithm between two rows,
    swapping them, until the lower
// entry is 0. Use it when the modulus is 2^k, 1e9, or anything
    you cannot invert in.
ll detMod(vector<vector<ll>> a, ll m) {
    int n = a.size();
    ll det = 1 % m;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++)
            while (a[j][i]) {                                   //
     Euclid on the two rows
                ll t = a[i][i] / a[j][i];
                for (int k = i; k < n; k++) a[i][k] = (a[i][k] -
    t * a[j][k]) % m;
                swap(a[i], a[j]), det = -det;
            }
        det = det * a[i][i] % m;
        if (!a[i][i]) return 0;
    }
    return (det % m + m) % m;
}
// MATRIX RANK over a field: the usual elimination, counting non-
    zero pivot rows.
// RANK over GF(2): use bitsets - O(n^3 / 64), see 04 - Linear
    Algebra/03.
// INTEGER determinant with no modulus: compute it modulo several
    primes and CRT. Size it with
// HADAMARD'S BOUND |det| <= prod of the row 2-norms <= n^(n/2) *
    max|a_ij|^n, and take enough
// primes that the product exceeds 2x that.
// SPARSE determinant (n = 1e5 with few non-zeros): Wiedemann -
    pick random u, v, feed the Krylov
// sequence u^T A^k v to Berlekamp-Massey to get the minimal
    polynomial, whose constant term is
// +-det after a random diagonal preconditioner. O(n * nnz).
// CHARACTERISTIC POLYNOMIAL in O(n^3): reduce to upper HESSENBERG
     form by similarity transforms
// (so det(xI - H) satisfies a short recurrence in the leading
    principal minors), then read the
// coefficients off. With Cayley-Hamilton that gives A^k in O(n^3
    + n^2 log k) instead of
// O(n^3 log k) - worth it only for large k and small n.
```

## 6.8 Linear Programming

### 6.8.1 Simplex

TWO-PHASE SIMPLEX. maximise c.x subject to A x <= b, x >= 0.

```
// Returns the optimum, or +inf if unbounded, or -inf if
    infeasible; x is filled with a solution.
// Exponential in theory, O(n*m) per pivot and instant at contest
    sizes (n, m up to a few hundred).
// REACH FOR IT when the problem is "optimise a linear objective
    under linear constraints" and no
// combinatorial structure presents itself - resource allocation,
    blending, fractional covering,
// zero-sum matrix games, and any min-cost-flow-with-side-
    constraints that flow cannot express.
typedef vector<ld> vd;
struct Simplex {
    int m, n;
    vector<int> B, N;
    vector<vd> D;
    Simplex(const vector<vd>& A, const vd& b, const vd& c)
        : m(b.size()), n(c.size()), B(m), N(n + 1), D(m + 2, vd(n
     + 2, 0)) {
        for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) D
    [i][j] = A[i][j];
        for (int i = 0; i < m; i++) B[i] = n + i, D[i][n] = -1, D
    [i][n + 1] = b[i];
        for (int j = 0; j < n; j++) N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        for (int i = 0; i < m + 2; i++) if (i != r && fabsl(D[i][
    s]) > 1e-9) {
            ld f = D[i][s] * inv;                               //
     the pivot COLUMN is fixed up by
            for (int j = 0; j < n + 2; j++) if (j != s) D[i][j]
    -= f * D[r][j];   // the loop below
        }
        for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] *=
    inv;
        for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] *= -
    inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            for (int j = 0; j <= n; j++)
                if (N[j] != -phase && (s < 0 || make_pair(D[x][j
    ], N[j]) < make_pair(D[x][s], N[s]))) s = j;
            if (D[x][s] >= -1e-9) return true;
            int r = -1;
            for (int i = 0; i < m; i++) {
                if (D[i][s] <= 1e-9) continue;
                if (r < 0 || make_pair(D[i][n + 1] / D[i][s], B[i
    ]) < make_pair(D[r][n + 1] / D[r][s], B[r])) r = i;
            }
            if (r < 0) return false;                            //
     unbounded
            pivot(r, s);
        }
    }
    ld solve(vd& x) {
        int r = 0;
        for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n +
    1]) r = i;
        if (D[r][n + 1] < -1e-9) {                              //
     phase 1: find any feasible point
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -1e-9) return
    -1.0L / 0.0L;   // infeasible
            for (int i = 0; i < m; i++) if (B[i] == -1) {
                int s = 0;
                for (int j = 1; j <= n; j++)
                    if (make_pair(D[i][j], N[j]) < make_pair(D[i
    ][s], N[s])) s = j;
                pivot(i, s);
            }
        }
        bool ok = simplex(1);
        x.assign(n, 0);
        for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = D[i][
    n + 1];
        return ok ? D[m][n + 1] : 1.0L / 0.0L;                  //
     +inf = unbounded
    }
};
/* MODELLING NOTES - the part that actually costs you time
 MINIMISE instead: negate c and negate the answer.
```

```
>= CONSTRAINT: multiply the row by -1.   EQUALITY: add it twice,
    as <= and as >=.
FREE VARIABLE (no x >= 0): substitute x = u - v with u, v >= 0.
    Doubles that column.
UPPER BOUND x_i <= U: just another row; simplex has no special
    handling and does not need any.
LINEAR-FRACTIONAL objective (c.x + a) / (d.x + b): CHARNES-COOPER
     - substitute y = x/(d.x+b),
  t = 1/(d.x+b) and maximise c.y + a t subject to A y <= b t, d.y
     + b t = 1, t >= 0. Linear again.
INTEGER answers are NOT guaranteed. If the constraint matrix is
    TOTALLY UNIMODULAR (network
  matrices, interval matrices, bipartite incidence) the optimum
    IS integral - that is exactly why
  flow problems have integral optima. Otherwise you need ILP,
    which is NP-hard.
ZERO-SUM MATRIX GAME: shift A so every entry is > 0, then min sum
    (y) s.t. A y >= 1, y >= 0 gives
  value v = 1/sum(y) and the column player's mixed strategy y*v;
    the DUAL gives the row player's.
  Von Neumann: max_x min_y x^T A y = min_y max_x x^T A y.
DUALITY, which is often the actual solution: max c.x, Ax <= b, x
    >= 0  is dual to
  min b.y, A^T y >= c, y >= 0; the two optima are equal.
    COMPLEMENTARY SLACKNESS at the optimum:
  x_j > 0 => the j-th dual constraint is tight, and y_i > 0 =>
    the i-th primal row is tight.
  The combinatorial specialisations you already know are max-flow
    /min-cut and Konig.
PRECISION: this is floating point. If the data is integral and
    small, and you need an exact
  answer, prefer a combinatorial model (flow, matching) over
    simplex. */
```

# 7 Strings

## 7.1 String Matching

### 7.1.1 KMP

KMP. pi[i] = length of the longest proper border of s[0..i] (prefix that is also a suffix).

```
// Gives O(n+m) substring search, the period of a prefix (i+1 - pi
    [i]), and the automaton for DP.
vector<int> compute_pi(const string &s) {
    int n = s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> find_matches(const string &text, const string &pat) {
    int n = pat.length(), m = text.length();
    string s = pat + "#" + text;
    vector<int> pi = compute_pi(s), ans;
    for (int i = n + 1; i <= n + m; i++) { /* n + 1 is where the
     text starts */
        if (pi[i] == n) {
            ans.push_back(i - 2 * n); /* i - (n - 1) - (n + 1) */
        }
    }
    return ans;
}

void compute_automaton(string s, vector<vector<int>> &aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = compute_pi(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i - 1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

### 7.1.2 RabinKarp

RABIN-KARP - rolling-hash substring search in O(n+m) expected. Prefer 05 - Hashing for anything

```
// serious (this uses a FIXED base, which is hackable); keep this
    for one-off matching.
vector<int> rabin_karp(string const &s, string const &t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<ll> p_pow(max({S, T, 1}));            // max(0,0)
     would leave it empty
    p_pow[0] = 1;
    for (int i = 1; i < (int) p_pow.size(); i++)
        p_pow[i] = (p_pow[i - 1] * p) % m;

    vector<ll> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i + 1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
    ll h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        ll cur_h = (h[i + S] + m - h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}
```

### 7.1.3 ZFunction

z[i] is the length of the longest substring starting from s[i] which is also a prefix of s

```
vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

### 7.1.4 WildcardMatching

STRING MATCHING WITH WILDCARDS in O(n log n). A wildcard matches any character, on EITHER side.

```
// Needs: conv (06 - Math/05 - Convolution/01 - FFT.cpp).
// Map every real character to a positive value and every wildcard
     to 0. Then position i matches iff
//        sum_j  ( p[j] - t[i+j] )^2 * p[j] * t[i+j]  ==  0,
// because each term is >= 0 and vanishes exactly when the
    characters agree or one of them is 0.
// Expanding gives three correlations: sum p^3 t - 2 sum p^2 t^2 +
     sum p t^3, and a correlation is a
// convolution with the pattern reversed.
vector<int> wildcardMatch(const string& t, const string& p, char
    wild = '?') {
    int n = t.size(), m = p.size();
    if (m > n) return {};
    // The identity needs values >= 0 and wildcard == 0. Compress
     the characters that actually
    // occur to 1..k - with a raw 'c - 'a' + 1' any character
     below 'a' is NEGATIVE and the terms
    // can cancel, which reports matches that do not exist (e.g.
     text "_b" against pattern "Z]").
    map<char, ll> id;
    for (char c : t) if (c != wild) id.emplace(c, 0);
    for (char c : p) if (c != wild) id.emplace(c, 0);
    ll k = 0;
    for (auto& [c, v] : id) v = ++k;
    auto val = [&](char c) { return c == wild ? 0LL : id[c]; };
    vector<ll> a(m), b(n);
    for (int i = 0; i < m; i++) a[i] = val(p[m - 1 - i]);
     // reversed pattern
    for (int i = 0; i < n; i++) b[i] = val(t[i]);
    auto pw = [](vector<ll> v, int k) { for (ll& x : v) { ll y =
     1; for (int i = 0; i < k; i++) y *= x; x = y; } return v; };
    auto c1 = conv(pw(a, 3), b), c2 = conv(pw(a, 2), pw(b, 2)),
     c3 = conv(a, pw(b, 3));
    vector<int> res;
    for (int i = 0; i + m <= n; i++)
        if (c1[i + m - 1] - 2 * c2[i + m - 1] + c3[i + m - 1] ==
     0) res.push_back(i);
    return res;                                                //
      0-based start positions
}
// PRECISION: the largest intermediate is about k^4 * m, where k
    is the number of DISTINCT real
// characters (which is why compressing is free). With k = 26 and
    m = 1e5 that is
// ~4.6e10, comfortable for double FFT. For a big alphabet,
    compress the characters that actually
// occur first, or run the three convolutions under two NTT primes
     and CRT.
// COUNT MISMATCHES at every shift (no wildcards, small alphabet):
     for each letter c convolve the
// 0/1 indicator of c in p (reversed) with the indicator of c in t
    ; summing gives the number of
// MATCHES per shift, and mismatches = m - matches. O(|sigma| n
    log n).
// k-MISMATCH: same counts, keep shifts with matches >= m - k.
// SUBSET / "at most one character class" matching: encode each
    position as a bitmask over the
// alphabet and convolve the complement indicators; a shift is
    valid iff the total is 0.
```

## 7.2 Hashing

### 7.2.1 StringHashing

Double-mod polynomial hashing. RANDOM bases => not anti-hackable.

```
// No modular inverses: h(l,r) is compared already scaled, so get
    () is 2 mults.
ll HB1, HB2;
struct Hash {
    static const ll M1 = 1000000007, M2 = 998244353;
    int n;
    vector<ll> h1, h2, p1, p2;

    Hash(const string& s) : n(s.size()), h1(n + 1, 0), h2(n + 1,
     0), p1(n + 1, 1), p2(n + 1, 1) {
        if (!HB1) {
            mt19937_64 rng(chrono::steady_clock::now().
     time_since_epoch().count());
            HB1 = 256 + rng() % (M1 - 300), HB2 = 256 + rng() % (
     M2 - 300);   // FULL range: a base drawn
            // from only 1e6 values gives collision probability n/1e6
     per modulus (0.1 at n = 1e5)
        }
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * HB1 + s[i]) % M1, p1[i + 1] = p1
     [i] * HB1 % M1;
            h2[i + 1] = (h2[i] * HB2 + s[i]) % M2, p2[i + 1] = p2
     [i] * HB2 % M2;
        }
    }
    pair<ll, ll> get(int l, int r) const {              // [l, r]
      inclusive, 0-based
        ll a = (h1[r + 1] - h1[l] * p1[r - l + 1]) % M1;
        ll b = (h2[r + 1] - h2[l] * p2[r - l + 1]) % M2;
        return {a < 0 ? a + M1 : a, b < 0 ? b + M2 : b};
    }
    bool eq(int l1, int r1, int l2, int r2) const {
        return r1 - l1 == r2 - l2 && get(l1, r1) == get(l2, r2);
    }
    // concat(A over lenA, B): h = hA * base^lenB + hB   (works
     per modulus)
};
// USES: substring equality O(1) * longest common prefix of two
     suffixes by binary search
//       * dedup of substrings * 2D grid patterns (hash rows, then
      hash the row-hashes)
//       * tree isomorphism (hash the sorted multiset of child
     hashes)
```

### 7.2.2 Hashing2D

2D HASHING - O(1) hash of any submatrix, for grid pattern matching.

```
// 2D prefix hash with random bases b1 (down) and b2 (right),
     single 62-bit modulus via __int128.
ll HB2D1 = 0, HB2D2 = 0;                                    // ONE base
     pair per RUN, at file scope.
struct Hash2D {                                             // Per-
     object bases would make two grids
    static const ll M = (ll)1e18 + 9;                       //
     incomparable - and "find this pattern in
    int n, m;                                               // that grid
     " needs exactly two objects.
    ll b1, b2;
    vector<ll> P1, P2;
    vector<vector<ll>> h;

    static ll mul(ll a, ll b) { return (ll)((__int128)a * b % M);
      }
    Hash2D(const vector<string>& g) : n(g.size()), m(n ? g[0].
     size() : 0),
                                      P1(n + 1, 1), P2(m + 1, 1),
      h(n + 1, vector<ll>(m + 1, 0)) {
        if (!HB2D1) {
            mt19937_64 rng(chrono::steady_clock::now().
     time_since_epoch().count());
            HB2D1 = 256 + rng() % (M - 300), HB2D2 = 256 + rng()
     % (M - 300);
        }
        b1 = HB2D1, b2 = HB2D2;
        for (int i = 1; i <= n; i++) P1[i] = mul(P1[i - 1], b1);
        for (int j = 1; j <= m; j++) P2[j] = mul(P2[j - 1], b2);
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++) {
                __int128 v = (__int128)mul(h[i - 1][j], b1) + mul
     (h[i][j - 1], b2)
                           - mul(mul(h[i - 1][j - 1], b1), b2) +
     g[i - 1][j - 1];
                h[i][j] = (ll)((v % M + M) % M);
            }
    }
    // hash of the submatrix with top-left (r, c) and size dr x dc
     , 0-based
    ll get(int r, int c, int dr, int dc) const {
        int r2 = r + dr, c2 = c + dc;
        __int128 v = (__int128)h[r2][c2] - mul(h[r][c2], P1[dr])
     - mul(h[r2][c], P2[dc])
                   + mul(mul(h[r][c], P1[dr]), P2[dc]);
        return (ll)((v % M + M) % M);
    }
};
// USES: count occurrences of a pattern grid * largest common
     square submatrix (binary search the
// side, hash all windows of both grids) * count distinct k x k
     submatrices.
// CHEAPER when the pattern is fixed: 1D-hash each row of the
     pattern, then run KMP /
// Aho-Corasick over the rows treated as single symbols.
```

## 7.3 Aho Corasick

### 7.3.1 AhoCorasick

AHO-CORASICK - a trie of all patterns plus suffix (fail) links, so one pass over the text finds

```
// every occurrence of every pattern in O(|text| + total pattern
     length + #matches).
// The goto-automaton form (nxt filled in by BFS) is also the
     transition table for DP over strings.
struct Mapper {
    int operator()(char c) const { return c - 'a'; }
};

template <int ALPHABET, typename Mapper>
struct Aho {
    struct Node {
        array<int, ALPHABET> nxt;
        int link = 0;
        int terminal_idx = -1; // -1 if not a terminal, else idx
     of string insertion
        int exit_link = 0; // Points to the nearest terminal
     suffix
        int depth = 0; // Size of the prefix the node represents
        Node() { nxt.fill(0); }
    };

    vector<Node> t;
    vector<int> bfs_order;
    Mapper get_idx; // Instance of the mapping function
    int word_count = 0; // Tracks the order of inserted strings

    // Constructor optionally accepts a custom mapper instance
    Aho(Mapper mapper = Mapper()) : get_idx(mapper) {
        t.emplace_back();
    }

    int insert(const string& p) {
        int u = 0;
        for (char c : p) {
            int v = get_idx(c);
            if (!t[u].nxt[v]) {
                t[u].nxt[v] = t.size();
                t.emplace_back();
                t.back().depth = t[u].depth + 1; // New node's
     depth is parent's depth + 1
            }
            u = t[u].nxt[v];
        }

        // If this exact string hasn't been inserted before,
     assign it the current index
        if (t[u].terminal_idx == -1) {
            t[u].terminal_idx = word_count;
        }
        word_count++; // Increment for the next insertion

        return u;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < ALPHABET; ++c) {
            if (t[0].nxt[c]) {
                q.push(t[0].nxt[c]);
            }
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            bfs_order.push_back(u);

            for (int c = 0; c < ALPHABET; ++c) {
                int v = t[u].nxt[c];
                if (v) {
                    t[v].link = t[t[u].link].nxt[c];

                    // Compute the exit link:
                    // If the immediate suffix link is a terminal,
     point to it.
                    // Otherwise, copy its exit link.
                    t[v].exit_link = (t[t[v].link].terminal_idx
     != -1) ? t[v].link : t[t[v].link].exit_link;

                    q.push(v);
                }
                else {
                    t[u].nxt[c] = t[t[u].link].nxt[c];
                }
            }
        }
```

```
    }

    int advance(int u, char c) {
        return t[u].nxt[get_idx(c)];
    }
};
```

### 7.3.2 AhoCorasickDP

DP OVER THE AHO-CORASICK AUTOMATON - the high-value use of Aho, more than matching itself.

```
// Needs: Mint (06 - Math/01) and AhoCorasick (01 - AhoCorasick.cpp).
// After build(), nxt[] is a total transition function, so the automaton is a DFA over states
// 0..S-1 and you can run any DP on (position, state, extra).
// "bad" marks states that contain a forbidden pattern as a suffix; propagate it along exit links.
struct AhoDP {
    static const int A = 26;                        // trie fan-out
    int alpha = 26;                                 // alphabet the DP counts over (set smaller!)
    vector<array<int, A>> nxt;
    vector<int> link, term;
    vector<char> bad;
    vector<int> order;                              // BFS order of states

    AhoDP() { newNode(); }
    int newNode() { nxt.push_back({}); nxt.back().fill(0); link.push_back(0); term.push_back(0); bad.push_back(0); return nxt.size() - 1; }
    void insert(const string& p, int id = 1) {
        int u = 0;
        for (char c : p) {
            int v = c - 'a';
            if (!nxt[u][v]) nxt[u][v] = newNode();
            u = nxt[u][v];
        }
        term[u] = id, bad[u] = 1;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < A; c++) if (nxt[0][c]) q.push(nxt[0][c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            order.push_back(u);
            bad[u] = bad[u] || bad[link[u]];         // a state is bad if any SUFFIX is a pattern
            for (int c = 0; c < A; c++) {
                int v = nxt[u][c];
                if (v) link[v] = nxt[link[u]][c], q.push(v);
                else nxt[u][c] = nxt[link[u]][c];    // total transition function
            }
        }
    }
    // #strings of length L over the first `alpha` letters containing NO pattern. O(L*S*alpha).
    Mint countAvoiding(int L) {
        int S = nxt.size();
        vector<Mint> dp(S, 0), nd;
        dp[0] = 1;
        for (int i = 0; i < L; i++) {
            nd.assign(S, 0);
            for (int u = 0; u < S; u++) if (!bad[u] && dp[u].v)
                for (int c = 0; c < alpha; c++) {
                    int v = nxt[u][c];
                    if (!bad[v]) nd[v] += dp[u];
                }
            dp = nd;
        }
        Mint r = 0;
        for (int u = 0; u < S; u++) if (!bad[u]) r += dp[u];
        return r;
    }
    // #occurrences of every pattern in a text: walk the text, count visits per state, then push
    // the counts DOWN the suffix-link tree (reverse BFS order).
    vector<ll> occurrences(const string& s) {
        vector<ll> cnt(nxt.size(), 0);
        int u = 0;
        for (char c : s) u = nxt[u][c - 'a'], cnt[u]++;
        for (int i = order.size() - 1; i >= 0; i--) cnt[link[order[i]]] += cnt[order[i]];
        return cnt;                                  // cnt[state of pattern p] = #occurrences
    }
};
// SHORTEST / LEXICOGRAPHICALLY SMALLEST string avoiding all patterns: BFS over (state) with
// the alphabet in order. If a cycle among good states is reachable, arbitrarily long strings
// exist - detect it with a cycle check on the good-state subgraph.
```

## 7.4 Suffix Structures

### 7.4.1 SuffixArray

SUFFIX ARRAY + LCP + sparse table for O(1) LCE. Build is O(n log n) (prefix doubling with a radix

```
// sort), Kasai gives the LCP array in O(n). Everything you can
     ask about substrings reduces to a
// range on this array - see 03 - SuffixArrayApplications.cpp.
struct SuffixArray {
    string S;
    // sa is the suffix array with the empty suffix being sa[0]
    // lcp[i] holds the lcp between sa[i], sa[i - 1]
    vector<int> logs, sa, lcp, rank;
    vector<vector<int>> table;

    SuffixArray() {};

    SuffixArray(string &s, int lim = 256) {
        S = s;
        int n = s.size() + 1, k = 0, a, b;
        vector<int> c(s.begin(), s.end() + 1), tmp(n), frq(max(n,
     lim));
        c.back() = 0; //0 is less than any character
        sa = lcp = rank = tmp, iota(sa.begin(), sa.end(), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p)
     {
            p = j, iota(tmp.begin(), tmp.end(), n - j);
            for (int i = 0; i < n; i++) {
                if (sa[i] >= j)
                    tmp[p++] = sa[i] - j;
            }

            fill(frq.begin(), frq.end(), 0);

            for (int i = 0; i < n; i++) frq[c[i]]++;
            for (int i = 1; i < lim; i++) frq[i] += frq[i - 1];
            for (int i = n; i--;) sa[--frq[c[tmp[i]]]] = tmp[i];

            swap(c, tmp), p = 1, c[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                a = sa[i - 1], b = sa[i], c[b] = (tmp[a] == tmp[b
     ] && tmp[a + j] == tmp[b + j]) ? p - 1 : p++;
        }

        for (int i = 1; i < n; i++) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
            for (k &&k--, j = sa[rank[i] - 1];
                    s[i + k] == s[j + k];
        k++);
    }

    void preLcp() {
        int n = S.size() + 1;
        logs = vector<int>(n + 5);
        for (int i = 2; i < n + 5; ++i) {
            logs[i] = logs[i / 2] + 1;
        }
        // level-major: sized by logs[n], not a hard-coded 20 (
     which is OUT OF BOUNDS at
        // |s| >= 2^20 - and Codeforces routinely gives |s| up to
     1e6), and no dead columns.
        table = vector<vector<int>>(logs[n] + 1, vector<int>(n));
        for (int i = 0; i < n; ++i) table[0][i] = lcp[i];
        for (int j = 1; j <= logs[n]; ++j)
            for (int i = 0; i + (1 << j) <= n; ++i)
                table[j][i] = min(table[j - 1][i], table[j - 1][i
      + (1 << (j - 1))]);
    }

    // ARGUMENTS ARE SUFFIX-ARRAY INDICES, not string positions.
     For two string positions
    // p and q use  lce(p, q)  below, which is what every
     application actually wants.
    int queryLcp(int i, int j) {
        if (i == j) return (int)S.size() - sa[i];
        if (i > j) swap(i, j);
        i++;
        int len = logs[j - i + 1];
        return min(table[len][i], table[len][j - (1 << len) + 1])
     ;
    }
    int lce(int p, int q) { return queryLcp(rank[p], rank[q]); }
       // longest common extension
};
```

### 7.4.2 SuffixAutomaton

SUFFIX AUTOMATON - O(n) states/transitions, accepts exactly the substrings of s.

```
// st[v].len   = longest string in v's endpos class      link =
     suffix link (v's parent)
// cnt[v]      = |endpos(v)| = #occurrences of every string in v
// distinct substrings = sum over v != root of (len[v] - len[link[
     v]])
struct SAM {
    struct S { int len = 0, link = -1; array<int, 26> nxt; S() {
     nxt.fill(-1); } };
    vector<S> st;
    vector<ll> cnt;                                  //
     occurrence counts (after countOcc)
    vector<int> firstPos;                            // end
     position of the first occurrence
    int last = 0;
    bool counted = false;                            // countOcc
     () is idempotent

    SAM(int reserve = 0) { st.reserve(2 * reserve), st.push_back(
     S()), cnt.push_back(0), firstPos.push_back(0); }

    void extend(char ch) {
        int c = ch - 'a', cur = st.size();
        st.push_back(S()), cnt.push_back(1), firstPos.push_back
     (0);
        st[cur].len = st[last].len + 1, firstPos[cur] = st[cur].
     len - 1;
        int p = last;
        while (p != -1 && st[p].nxt[c] == -1) st[p].nxt[c] = cur,
      p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].nxt[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                int cl = st.size();
                st.push_back(st[q]), cnt.push_back(0), firstPos.
     push_back(firstPos[q]);
                st[cl].len = st[p].len + 1;
                while (p != -1 && st[p].nxt[c] == q) st[p].nxt[c]
      = cl, p = st[p].link;
                st[q].link = st[cur].link = cl;
            }
        }
        last = cur;
    }
    void build(const string& s) { for (char c : s) extend(c); }

    vector<int> byLen() const {                      // states
     sorted by len (counting sort)
        int n = st.size(), mx = st[0].len;
        for (auto& x : st) mx = max(mx, x.len);
        vector<int> c(mx + 2, 0), ord(n);
        for (auto& x : st) c[x.len]++;
        for (int i = 1; i <= mx; i++) c[i] += c[i - 1];
        for (int v = n - 1; v >= 0; v--) ord[--c[st[v].len]] = v;
        return ord;                                  //
     increasing len; reverse for link-tree order
    }
    void countOcc() {                                // push
     counts down the suffix-link tree
        if (counted) return;                         // NOT
     idempotent without this guard
        counted = true;
        auto ord = byLen();
        for (int i = (int)ord.size() - 1; i > 0; i--) {
            int v = ord[i];
            if (st[v].link >= 0) cnt[st[v].link] += cnt[v];
        }
        cnt[0] = 0;
    }
    ll distinctSubstrings() const {                  // number of
      DISTINCT non-empty substrings
        ll r = 0;
        for (int v = 1; v < (int)st.size(); v++) r += st[v].len -
      st[st[v].link].len;
        return r;
    }
    ll totalOccurrences() {                          // sum over
     distinct substrings of #occurrences
        countOcc();
        ll r = 0;
        for (int v = 1; v < (int)st.size(); v++) r += cnt[v] * (
     st[v].len - st[st[v].link].len);
        return r;
    }
    int countOf(const string& p) {                   // #
     occurrences of p in s (call countOcc first)
```

```
        int v = 0;
        for (char ch : p) { v = st[v].nxt[ch - 'a']; if (v < 0) return 0; }
        return cnt[v];
    }
    // K-th SMALLEST distinct substring (k is 1-indexed). Needs `paths` = #substrings from v.
    vector<ll> paths;
    void buildPaths() {
        paths.assign(st.size(), 1);                    // 1 counts the empty continuation
        auto ord = byLen();
        for (int i = (int)ord.size() - 1; i >= 0; i--) {
            int v = ord[i];
            paths[v] = 1;
            for (int c = 0; c < 26; c++) if (st[v].nxt[c] >= 0) paths[v] += paths[st[v].nxt[c]];
        }
    }
    string kthSubstring(ll k) {                        // "" if fewer than k distinct substrings
        string r;
        int v = 0;
        while (k > 0) {
            int c = 0;
            for (; c < 26; c++) {
                int u = st[v].nxt[c];
                if (u < 0) continue;
                if (k <= paths[u]) { r += char('a' + c), v = u, k--; break; }
                k -= paths[u];
            }
            if (c == 26) break;
        }
        return r;
    }
};
// LONGEST COMMON SUBSTRING of s and t: build the SAM of s, then walk t keeping the current
// length; on a missing transition follow suffix links and clamp the length to st[v].len.
int longestCommonSubstring(SAM& A, const string& t) {
    int v = 0, l = 0, best = 0;
    for (char ch : t) {
        int c = ch - 'a';
        while (v && A.st[v].nxt[c] < 0) v = A.st[v].link, l = A.st[v].len;
        if (A.st[v].nxt[c] >= 0) v = A.st[v].nxt[c], l++;
        best = max(best, l);
    }
    return best;
}
// GENERALISED SAM (several strings): reset `last = 0` before each string and, in extend(),
// if a transition already exists reuse/clone it instead of creating a new state.
```

### 7.4.3 SuffixArrayApplications

WHAT YOU ACTUALLY EXTRACT FROM A SUFFIX ARRAY.

```
// Convention of 01 - SuffixArray.cpp: sa has n+1 entries (sa[0] = the empty suffix),
// and lcp[i] = LCP(sa[i], sa[i-1]).  Everything below assumes that.

// 1) NUMBER OF DISTINCT SUBSTRINGS = sum over i of (len of suffix i) - lcp with the previous.
ll distinctSubstrings(const SuffixArray& S) {
    ll n = S.S.size(), r = 0;
    for (ll i = 1; i <= n; i++) r += (n - S.sa[i]) - S.lcp[i];
    return r;
}
// 2) LONGEST REPEATED SUBSTRING = max lcp; the substring starts at sa[argmax].
pair<int, int> longestRepeated(const SuffixArray& S) {          // {length, start position}
    int n = S.S.size(), bi = 0, best = 0;
    for (int i = 1; i <= n; i++) if (S.lcp[i] > best) best = S.lcp[i], bi = S.sa[i];
    return {best, bi};
}
// 3) OCCURRENCES of a pattern = the contiguous SA block whose suffixes start with it.
//    Two binary searches, O(|P| log n).
pair<int, int> patternRange(const SuffixArray& S, const string& p) {   // [lo, hi], empty if lo>hi
    int n = S.S.size();
    auto cmp = [&](int idx, const string& q) { return S.S.compare(idx, q.size(), q) < 0; };
    int lo = 1, hi = n, L = n + 1, R = n;
    while (lo <= hi) { int m = (lo + hi) / 2; cmp(S.sa[m], p) ? lo = m + 1 : (L = m, hi = m - 1); }
    lo = L, hi = n, R = L - 1;                                                  //
      R MUST be reset here:
                                                                                //
      otherwise an absent pattern
                                                                                //
      returns the range [L, n]
    while (lo <= hi) {
        int m = (lo + hi) / 2;
        S.S.compare(S.sa[m], p.size(), p) == 0 ? (R = m, lo = m + 1) : hi = m - 1;
    }
    return {L, R};                                                              //
      count = R - L + 1
}
// 4) LONGEST COMMON SUBSTRING of two strings (named lcsViaSuffixArray to avoid clashing
//    with the SAM version in 02 - SuffixAutomaton.cpp): build SA of a + sep + b (sep smaller than every
//    real character), then take the max lcp over ADJACENT pairs coming from different sides.
int lcsViaSuffixArray(const string& a, const string& b) {
    string t = a + char(1) + b;
    SuffixArray S(t);
    int n = t.size(), best = 0, cut = a.size();
    auto side = [&](int p) { return p < cut ? 0 : (p == cut ? -1 : 1); };
    for (int i = 2; i <= n; i++) {
        int s1 = side(S.sa[i]), s2 = side(S.sa[i - 1]);
        if (s1 >= 0 && s2 >= 0 && s1 != s2) best = max(best, S.lcp[i]);
    }
    return best;
}
// 5) K-TH SMALLEST DISTINCT SUBSTRING: SA entry i contributes (n - sa[i]) - lcp[i] new
//    substrings, all sharing the prefix of length lcp[i]. Prefix-sum those counts and binary
//    search for k; inside the block the answer is a prefix of suffix sa[i].
// 6) LCS OF K STRINGS: concatenate with distinct separators, then slide a window over the SA
//    that contains at least one suffix of every string; the answer is the max over windows of
//    the minimum lcp inside (monotonic deque), O(n) after the SA.
// 7) COMPARE TWO SUBSTRINGS in O(1): let l = S.lce(i, j) (STRING positions - queryLcp takes
//    SUFFIX-ARRAY indices, do not mix them up); if l >= both lengths they are equal, otherwise compare s[i+l] with s[j+l].
```

### 7.4.4 GeneralizedSAM

GENERALIZED SUFFIX AUTOMATON - one automaton accepting every substring of a SET of strings.

```
// Same size bound O(total length). The only change vs a plain SAM
     : when extending, the transition
// may ALREADY exist (a previous string walked here), so handle
     that case instead of blindly adding.
// Feeding strings one after another into a plain SAM is WRONG -
     it creates junk states.
struct GenSAM {
    struct S { int len = 0, link = -1; array<int, 26> nxt; S() {
     nxt.fill(-1); } };
    vector<S> st;
    GenSAM() { st.push_back(S()); }
    int clone(int q, int len) { int c = st.size(); st.push_back(
     st[q]), st[c].len = len; return c; }
    int extend(int last, int c) {                                //
      returns the new "last"
        if (st[last].nxt[c] != -1) {                             //
      the edge is already there
            int q = st[last].nxt[c];
            if (st[q].len == st[last].len + 1) return q;
            int cl = clone(q, st[last].len + 1);
            for (int p = last; p != -1 && st[p].nxt[c] == q; p =
     st[p].link) st[p].nxt[c] = cl;
            st[q].link = cl;
            return cl;
        }
        int cur = st.size();
        st.push_back(S()), st[cur].len = st[last].len + 1;
        int p = last;
        while (p != -1 && st[p].nxt[c] == -1) st[p].nxt[c] = cur,
     p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].nxt[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                int cl = clone(q, st[p].len + 1);
                for (; p != -1 && st[p].nxt[c] == q; p = st[p].
     link) st[p].nxt[c] = cl;
                st[q].link = st[cur].link = cl;
            }
        }
        return cur;
    }
    void add(const string& s) { int last = 0; for (char ch : s)
     last = extend(last, ch - 'a'); }
    vector<int> byLen() const {                                  //
      states in increasing len
        int n = st.size(), mx = 0;
        for (auto& x : st) mx = max(mx, x.len);
        vector<int> c(mx + 2, 0), ord(n);
        for (auto& x : st) c[x.len]++;
        for (int i = 1; i <= mx; i++) c[i] += c[i - 1];
        for (int v = n - 1; v >= 0; v--) ord[--c[st[v].len]] = v;
        return ord;
    }
    // HOW MANY OF THE k STRINGS CONTAIN each substring: mark the
     states each string walks through,
    // then push the marks up the link tree. k <= 64 here; for
     larger k use small-to-large sets, or
    // a bitset<K>, or process the strings in batches of 64 and
     add up the popcounts.
    vector<int> countStringsContaining(const vector<string>& v) {
        // requires v.size() <= 64
        vector<unsigned long long> mask(st.size(), 0);
        for (int i = 0; i < (int)v.size(); i++)
            for (int p = 0, j = 0; j < (int)v[i].size(); j++)
                p = st[p].nxt[v[i][j] - 'a'], mask[p] |= 1ULL <<
     i;
        auto ord = byLen();
        for (int i = ord.size() - 1; i > 0; i--)
            mask[st[ord[i]].link] |= mask[ord[i]];
        vector<int> r(st.size());
        for (size_t i = 0; i < st.size(); i++) r[i] =
     __builtin_popcountll(mask[i]);
        return r;                                                //
      r[v] for every state v
    }
};
// LONGEST COMMON SUBSTRING OF k STRINGS: build the generalized
     SAM, take the deepest state with
// count == k; its longest string is len[v] (walk suffix links for
      the actual string).
// DISTINCT SUBSTRINGS OF THE UNION = sum over v != 0 of (len[v] -
      len[link[v]]), same formula.
// SUBSTRING PRESENT IN EXACTLY ONE STRING / IN AT LEAST k: filter
      by the same counts.
// ALTERNATIVE when the strings are few: concatenate with distinct
      separators and use one SUFFIX
// ARRAY + a sliding window over lcp. Shorter to write, O(n log n)
     , but no online extension.
```

## 7.5 Palindromes

### 7.5.1 Manacher

MANACHER - the radius of the longest palindrome centred at every position, in O(n).

```
// Answers "is s[l..r] a palindrome" in O(1) and counts all
     palindromic substrings in one pass.
struct Manacher {
    vector<int> p;

    // Returns a vector where p[i] is the radius of the longest
     palindrome around center i
    // Centers are interleaved: 0 is before string, 1 is first
     char, 2 is between 1st & 2nd, etc.
    Manacher(string s) {
        string t = "#";
        for (char c : s) {
            t += c;
            t += "#";
        }
        int n = t.size();
        p.assign(n, 0);
        int c = 0, r = 0;
        for (int i = 0; i < n; i++) {
            int mirror = 2 * c - i;
            if (i < r) p[i] = min(r - i, p[mirror]);
            while (i - 1 - p[i] >= 0 && i + 1 + p[i] < n && t[i -
      1 - p[i]] == t[i + 1 + p[i]]) {
                p[i]++;
            }
            if (i + p[i] > r) {
                c = i;
                r = i + p[i];
            }
        }
    }

    // Check if substring [l, r] (0-based) is a palindrome in O(1)
    bool is_palindrome(int l, int r) {
        int len = r - l + 1;
        int center = l + r + 1;
        return p[center] >= len;
    }
};
```

### 7.5.2 PalindromicTree

PALINDROMIC TREE (eertree) - O(n) nodes, one per DISTINCT palindromic substring, built online in

```
// O(n). Gives the count of distinct palindromes, occurrence
     counts, and the longest palindromic
// suffix at every prefix - which is what palindromic-
     factorization DP needs.
struct PalindromeTree {
    int n, id, cur, tot;
    vector<array<int, 26>> go;

    // Core structural links
    vector<int> suflink, len, cnt;

    // Extentions for Hard DP (Palindrome Partitioning)
    vector<int> diff;   // Difference in length to the suffix link
    vector<int> slink;  // Series link (skips arithmetic
     progressions of suffix links)

    // Extensions for counting
    vector<int> num;    // Number of palindromic suffixes ending
     at this node
    long long total_overlapping; // Running sum of total
     palindromes

    PalindromeTree() {}

    PalindromeTree(const string &s) {
        build(s);
    }

    void init(int max_len) {
        n = max_len;
        go.assign(n + 2, {});
        suflink.assign(n + 2, 0);
        len.assign(n + 2, 0);
        cnt.assign(n + 2, 0);
        diff.assign(n + 2, 0);
        slink.assign(n + 2, 0);
        num.assign(n + 2, 0);

        suflink[0] = suflink[1] = 1;
        len[1] = -1;
        id = 2;
        cur = 0;
        tot = 0;
        total_overlapping = 0;
    }

    int get(const string &s, int i, int v) {
        while (i - len[v] - 1 < 0 || s[i - len[v] - 1] != s[i]) {
            v = suflink[v];
        }
        return v;
    }

    void add(const string &s, int i) {
        int ch = s[i] - 'a';
        cur = get(s, i, cur);

        if (go[cur][ch] == 0) {
            len[id] = 2 + len[cur];
            suflink[id] = go[get(s, i, suflink[cur])][ch];

            // 1. Number of palindromes ending exactly here
            num[id] = num[suflink[id]] + 1;

            // 2. Arithmetic Progression / Series Links for DP
            diff[id] = len[id] - len[suflink[id]];
            if (diff[id] == diff[suflink[id]]) {
                slink[id] = slink[suflink[id]];
            } else {
                slink[id] = suflink[id];
            }

            tot++;
            go[cur][ch] = id++;
        }
        cur = go[cur][ch];
        cnt[cur]++;
        total_overlapping += num[cur];
    }

    // Standard build
    void build(const string &s) {
        init(s.length());
        for (int i = 0; i < n; i++) {
            add(s, i);
        }
        countAll();
    }

    // Call this to push frequencies down the suffix links
    void countAll() {
        for (int i = id - 1; i >= 2; --i) {
            cnt[suflink[i]] += cnt[i];
        }
    }

    // --- Black-Box Solvers ---

    // 1. Returns number of unique palindromes
    int cntDistinct() {
        return tot;
    }

    // 2. Returns total number of palindromes (including overlaps/
     duplicates)
    long long cntTotal() {
        return total_overlapping;
    }

    // 3. Solves Minimum Palindrome Partitioning in O(N log N)
    // Rebuilds the tree internally to compute the DP concurrently
     .
    int min_partitions(const string &s) {
        init(s.length());
        const int INF = 1e9;
        vector<int> dp(s.length() + 1, INF);
        vector<int> series_ans(s.length() + 2, INF);

        dp[0] = 0;

        for (int i = 0; i < s.length(); ++i) {
            add(s, i);

            for (int v = cur; v > 1; v = slink[v]) {
                series_ans[v] = dp[i + 1 - (len[slink[v]] + diff[
     v])];

                if (diff[v] == diff[suflink[v]]) {
                    series_ans[v] = min(series_ans[v], series_ans
     [suflink[v]]);
                }

                dp[i + 1] = min(dp[i + 1], series_ans[v] + 1);
            }
        }
        return dp[s.length()];
    }

    // 4. Returns an array of frequencies for string B's
     palindromes inside this tree (String A)
    // Note: You must `build()` the tree with string A before
     calling this with B.
    vector<int> count_string(const string& B) {
        vector<int> out_cnt(id, 0);
        int curr = 0;

        for (int i = 0; i < B.length(); ++i) {
            int ch = B[i] - 'a';

            while (true) {
                if (i - len[curr] - 1 >= 0 && B[i - len[curr] -
     1] == B[i]) {
                    if (go[curr][ch]) {
                        curr = go[curr][ch];
                        break;
                    }
                }
                if (curr == 1) {
                    curr = 0;
                    break;
                }
                curr = suflink[curr];
            }
            out_cnt[curr]++;
        }

        for (int i = id - 1; i >= 2; --i) {
            out_cnt[suflink[i]] += out_cnt[i];
        }

        return out_cnt;
    }
};
```

### 7.5.3 PalindromicTreeRollback

PALINDROMIC TREE WITH ROLLBACK - same structure, but every extend() can be undone, so you can DFS

```
// NOTE: 02 - PalindromicTree.cpp declares the same 'struct
     PalindromeTree'. Paste one.
// a trie/tree of characters and maintain the palindromic
     structure of the current root-to-node string.
struct PalindromeTree {
    int n, id, cur, tot;
    vector<array<int, 26>> go;

    // Core structural links
    vector<int> suflink, len, cnt;

    // Extensions for Hard DP (Palindrome Partitioning)
    vector<int> diff;   // Length difference to suffix link
    vector<int> slink;  // Series link

    // Extensions for counting
    vector<int> num;    // Number of palindromic suffixes ending
     at this node
    long long total_overlapping; // Running total of all
     palindromic substrings

    // --- ROLLBACK EXTENSIONS ---
    string S;           // Maintains current active string
    struct History {
        int old_cur;
        int added_node;  // ID of created node, or 0 if edge
     already existed
        int parent_node; // Node where go[parent_node][ch] was
     updated
        int ch;          // Character index (0..25)
    };
    vector<History> history;

    PalindromeTree() {}

    PalindromeTree(int max_len) {
        init(max_len);
    }

    void init(int max_len) {
        n = max_len;
        go.assign(n + 2, {});
        suflink.assign(n + 2, 0);
        len.assign(n + 2, 0);
        cnt.assign(n + 2, 0);
        diff.assign(n + 2, 0);
        slink.assign(n + 2, 0);
        num.assign(n + 2, 0);

        suflink[0] = suflink[1] = 1;
        len[1] = -1;
        id = 2;
        cur = 0;
        tot = 0;
        total_overlapping = 0;

        S.clear();
        history.clear();
    }

    int get(int i, int v) {
        while (i - len[v] - 1 < 0 || S[i - len[v] - 1] != S[i]) {
            v = suflink[v];
        }
        return v;
    }

    // 1. PUSH / ADD CHARACTER (WITH ROLLBACK HISTORY)
    void push_back(char c) {
        S.push_back(c);
        int i = (int)S.length() - 1;
        int ch = c - 'a';

        int old_cur = cur;
        cur = get(i, cur);

        int added_node = 0;
        int parent_node = 0;

        if (go[cur][ch] == 0) {
            added_node = id;
            parent_node = cur;

            len[id] = 2 + len[cur];
            suflink[id] = go[get(i, suflink[cur])][ch];
            num[id] = num[suflink[id]] + 1;

            diff[id] = len[id] - len[suflink[id]];
            if (diff[id] == diff[suflink[id]]) {
                slink[id] = slink[suflink[id]];
            } else {
                slink[id] = suflink[id];
            }

            tot++;
            go[cur][ch] = id++;
        }

        cur = go[cur][ch];
        cnt[cur]++;
        total_overlapping += num[cur];

        // Save state transition into history stack
        history.push_back({old_cur, added_node, parent_node, ch})
     ;
    }

    // 2. POP / ROLLBACK LAST CHARACTER
    void pop_back() {
        if (history.empty()) return;

        History h = history.back();
        history.pop_back();

        // Revert occurrence count & running palindrome count
        total_overlapping -= num[cur];
        cnt[cur]--;

        // If a new node was created during this push_back, delete
      it
        if (h.added_node != 0) {
            int u = h.added_node;
            go[h.parent_node][h.ch] = 0; // Disconnect child edge

            // Reset node u's properties
            go[u].fill(0);
            suflink[u] = len[u] = cnt[u] = diff[u] = slink[u] =
     num[u] = 0;

            id--;
            tot--;
        }

        // Restore previous active node and string length
        cur = h.old_cur;
        S.pop_back();
    }

    // --- Black-Box Solvers ---

    int cntDistinct() {
        return tot;
    }

    long long cntTotal() {
        return total_overlapping;
    }

    // Call this if you need to push end-occurrence counts down
     suffix links
    void countAll() {
        for (int i = id - 1; i >= 2; --i) {
            cnt[suflink[i]] += cnt[i];
        }
    }
};
```

## 7.6 Rotations

### 7.6.1 LyndonAndRotations

LYNDON FACTORISATION (Duval) and MINIMAL CYCLIC ROTATION (Duval), both O(n), O(1) memory.

```
// A Lyndon word is strictly smaller than all of its proper
     suffixes. Every string factors
// UNIQUELY as w1 >= w2 >= ... >= wk with each wi Lyndon.
vector<string> duval(const string& s) {
    int n = s.size(), i = 0;
    vector<string> f;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) s[k] < s[j] ? k = i : k++,
     j++;
        while (i <= k) f.push_back(s.substr(i, j - k)), i += j -
     k;
    }
    return f;
}
// Smallest cyclic rotation: run Duval on s+s and take the factor
     starting before n.
int minRotation(string s) {
    int n = s.size();
    s += s;
    int i = 0, ans = 0;
    while (i < n) {
        ans = i;
        int j = i + 1, k = i;
        while (j < (int)s.size() && s[k] <= s[j]) s[k] < s[j] ? k
      = i : k++, j++;
        while (i <= k) i += j - k;
    }
    return ans;                                  // index in the
     ORIGINAL string
}
// USES: canonical form of a cyclic string (necklace dedup ->
     pairs with Burnside);
//       "is A a rotation of B" (compare minRotation forms, or
     search A in B+B);
//       lexicographically smallest result of rotating; runs /
     repetitions via Lyndon roots.
```

## 7.7 Sequences

### 7.7.1 DeBruijnAndBitap

DE BRUIJN SEQUENCES and BIT-PARALLEL MATCHING.

```
// --- DE BRUIJN SEQUENCE B(k, n): a CYCLIC string over an
     alphabet of size k, of length k^n, in
// which every one of the k^n words of length n appears exactly
     once as a (cyclic) substring.
// USES: "open the safe with the fewest keypresses" (the classic),
      minimum-length probe sequences,
// and hashing tricks (a de Bruijn constant is how you find the
     index of the lowest set bit).
// CONSTRUCTION (FKM / Lyndon): concatenate, in lexicographic
     order, every Lyndon word whose
// length DIVIDES n. That is exactly what this O(k^n) recursion
     emits.
string deBruijn(int k, int n) {
    string s;
    vector<int> a(k * n, 0);
    function<void(int, int)> db = [&](int t, int p) {
        if (t > n) {
            if (n % p == 0) for (int i = 1; i <= p; i++) s +=
     char('0' + a[i]);
        } else {
            a[t] = a[t - p];
            db(t + 1, p);
            for (int j = a[t - p] + 1; j < k; j++) a[t] = j, db(t
     + 1, t);
        }
    };
    db(1, 1);
    return s;                                                  //
      length k^n, read cyclically
}
// The same object is an EULERIAN CIRCUIT in the de Bruijn graph (
     vertices = words of length n-1,
// edges = words of length n), so Hierholzer on that graph is the
     alternative construction - and
// the BEST theorem then counts how many distinct de Bruijn
     sequences exist: (k!)^(k^(n-1)) / k^n.

// --- BITAP / SHIFT-AND: exact matching in O(n*m/64) with bitsets
     , and the natural home for
// approximate matching. Build one bitmask per character marking
     where it occurs in the pattern;
// then one pass over the text ANDs shifted copies together.
// Beats KMP only when you need the extra flexibility (k
     mismatches, wildcards, "any of these
// patterns"), or when you want ALL occurrences as a bitset to
     intersect with a range.
vector<int> bitap(const string& t, const string& p) {
    int n = t.size(), m = p.size();
    if (!m || m > n) return {};
    vector<unsigned long long> mask(256, 0);
    // Works for m <= 64 with a single word; for longer patterns
     use bitset<M> and shift it.
    if (m > 64) return {};
    for (int i = 0; i < m; i++) mask[(unsigned char)p[i]] |= 1ULL
      << i;
    unsigned long long st = 0, hit = 1ULL << (m - 1);
    vector<int> res;
    for (int i = 0; i < n; i++) {
        st = ((st << 1) | 1) & mask[(unsigned char)t[i]];
        if (st & hit) res.push_back(i - m + 1);
    }
    return res;
}
/* WHAT BITAP GENERALISES TO, which is the reason to carry it
 k MISMATCHES: keep k+1 state words R0..Rk, where Rj = "matched a
     prefix with at most j errors".
   Rj_new = ((Rj << 1) & mask[c]) | Rj-1 | (Rj-1 << 1) | (Rj-1_old
      << 1) | 1
   covering substitution, insertion and deletion respectively -
     that is Wu-Manber, O(k n m / 64).
 WILDCARDS in the PATTERN: set mask[c] bits for every character
     the class accepts. A '?' just
   means "every mask has that bit". This is free - no FFT needed -
      and is the right tool when the
   alphabet is small and the pattern is short. Use the FFT method
     (01 - String Matching/04) when
   the pattern is long or wildcards appear in the TEXT too.
 MULTIPLE PATTERNS of the same length: OR their masks and check
     which one actually matched.
 OCCURRENCES INSIDE [l, r]: keep the whole occurrence set as a
     bitset and popcount a range - that
   is what makes "how many times does p occur in t[l..r]" O(n/64)
     per query with no suffix
   structure at all.
 MYERS' BIT-VECTOR ALGORITHM computes full edit distance in O(n m
     / 64) with the same idea
```

```
   (carry-save arithmetic on the DP's difference vectors); it is
     the fastest practical exact edit
   distance and the escape hatch when the O(nm) DP is 1e10.  */
```

# 8 Dynamic Programming (DP)

## 8.1 DP Optimizations

### 8.1.1 DivideAndConquerDP

Divide & Conquer DP Optimization - Time: O(K * N log N), Space: O(N)

```
// Applies when DP transition is dp[k][i] = min_{j < i} (dp[k-1][j
    ] + cost(j+1, i))
// Condition: opt(i, j) <= opt(i, j + 1) (Monotonicity of optimal
    split points)
const int N = 200005;                                  // ALSO in 01 -
    Template.cpp: keep one copy
int a[N];
ll curCost;
int curL = 0, curR = -1, freq[N];
pair<int, ll> pre[N], cur[N];
// Modify at will :
int lo(int ind) { return 1; }
int hi(int ind) { return ind; }

void add(int val) {
    curCost += freq[val]++;
}

void remove(int val) {
    curCost -= --freq[val];
}

ll Cost(int l, int r) {
    while (curR < r)
        add(a[++curR]);
    while (curL > l)
        add(a[--curL]);
    while (curR > r)
        remove(a[curR--]);
    while (curL < l)
        remove(a[curL++]);
    return curCost;
}

ll f(int ind, int k) { return pre[k - 1].second + Cost(k, ind); }
void store(int ind, int k, ll v) { cur[ind] = pair(k, v); }

void rec(int L, int R, int LO, int HI) {
    if (L > R) return;
    int mid = (L + R) >> 1;
    pair best(llinf, LO);
    for (int k = max(LO, lo(mid)); k <= min(HI, hi(mid)); ++k)
        best = min(best, pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid - 1, LO, best.second);
    rec(mid + 1, R, best.second, HI);
}

void solve(int L, int R) { rec(L, R, -inf, inf); }
```

### 8.1.2 CHT

DYNAMIC CONVEX HULL TRICK (UPPER hull -> MAXIMUM of m*x + b). Lines may be inserted in ANY order

```
// and queried at any x, O(log n) each. If your slopes AND queries
     are both monotone, the deque
// version at the bottom of this file is O(1) amortised and much
    shorter - prefer it.
// For a MINIMUM, insert (-m, -b) and negate the answer.
// To maximise over all x in [l, r], query only l and r: the upper
     envelope is convex.
struct Line {
    ll m, b;
    mutable function<const Line *()> succ;

    bool operator<(const Line &other) const {
        return m < other.m;
    }

    bool operator<(const ll &x) const {
        const Line *s = succ();
        if (!s)
            return 0;
        return (lll)(b - s->b) < (lll)(s->m - m) * x;   //
    __int128: |dm|*|x| overflows ll
    }
};

// will maintain upper hull for maximum
struct HullDynamic : public multiset<Line, less<>> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end())
                return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end())
            return y->m == x->m && y->b <= x->b;
        // __int128, NOT long double: each factor reaches ~2e18,
    the product needs ~123 bits,
        // and this test is consulted exactly when three lines are
     nearly collinear.
        return (lll)(x->b - y->b) * (z->m - y->m) >= (lll)(y->b -
     z->b) * (y->m - x->m);
    }

    void insert_line(ll m, ll b) {
//          for minimum
//        m *= -1;
//        b *= -1;
        auto y = insert({m, b});
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y);
    };
        if (bad(y)) {
            erase(y);
            return;
        }
        while (next(y) != end() && bad(next(y)))
            erase(next(y));
        while (y != begin() && bad(prev(y)))
            erase(prev(y));
    }

    ll query(ll x) {                                    // UB if
    empty - insert at least one line first
        auto l = *lower_bound(x);
//        for minimum
//        return -(l.m * x + l.b);
        return l.m * x + l.b;
    }
};
//
    ------------------------------------------------------------------

// MONOTONE CHT (deque) - the version you actually want when the
    DP feeds lines in slope order.
// MINIMISES m*x + b. Requires slopes added in DECREASING order;
    queries in increasing x are O(1)
// amortised (move the head), arbitrary x is O(log n) by binary
    search on the hull.
// For a MAXIMUM, negate m and b on the way in and negate the
    answer on the way out.
struct MonoCHT {
    vector<ll> M, B;
    int head = 0;
    bool bad(int a, int b, int c) {                    // is line b
     unnecessary between a and c?
        return (lll)(B[c] - B[a]) * (M[a] - M[b]) <= (lll)(B[b] -
```

```
      B[a]) * (M[a] - M[c]);
    }
    void add(ll m, ll b) {                          // m must be
      <= the last slope added
        M.push_back(m), B.push_back(b);
        while (M.size() >= 3 && bad(M.size() - 3, M.size() - 2, M
     .size() - 1))
            M.erase(M.end() - 2), B.erase(B.end() - 2);
        head = min(head, (int)M.size() - 1);
    }
    ll f(int i, ll x) { return M[i] * x + B[i]; }
    ll queryInc(ll x) {                             // x non-
     decreasing across calls
        while (head + 1 < (int)M.size() && f(head + 1, x) <= f(
     head, x)) head++;
        return f(head, x);
    }
    ll query(ll x) {                                // any x, O(
     log n)
        int lo = 0, hi = M.size() - 1;
        while (lo < hi) { int m = (lo + hi) / 2; f(m + 1, x) <= f
     (m, x) ? lo = m + 1 : hi = m; }
        return f(lo, x);
    }
};
// WHEN EACH ONE: monotone slopes + monotone queries -> MonoCHT::
     queryInc, O(n).
//               monotone slopes, arbitrary queries  -> MonoCHT::
     query, O(n log n).
//               arbitrary slopes                    ->
     HullDynamic above, or Li Chao (04),
//                                                      which
     needs no ordering at all.
//               lines inserted and removed          -> CHT with
     rollback (03) or Li Chao merge.
```

### 8.1.3 CHTRollback

Rollback Convex Hull Trick - Space: O(N), Add: O(log N), Query: O(log N), Rollback: O(1)

```
// Supports inserting monotonic lines with history stack rollback
     for tree/dsu DP
// Template parameters:
// IS_MAX: Set to true for Maximum queries, false for Minimum
     queries.
// IS_INC: Set to true if slopes added are strictly Increasing,
     false for Decreasing.
template<bool IS_MAX = false, bool IS_INC = false>
struct RollbackCHT {
    struct Line {
        ll m, c;
        Line() : m(0), c(llinf) {}
        Line(ll m, ll c) : m(m), c(c) {}
        ll eval(ll x) const { return m * x + c; }
    };

    struct History {
        int pos;
        int sz;
        Line old_line;
    };

    vector<Line> st;
    vector<History> hist;
    int sz;

    RollbackCHT(int max_nodes) {
        st.resize(max_nodes);
        sz = 0;
    }

    bool redundant(Line l1, Line l2, Line l3) {
        __int128 dx1 = l1.m - l2.m;
        __int128 dc1 = l2.c - l1.c;
        __int128 dx2 = l2.m - l3.m;
        __int128 dc2 = l3.c - l2.c;
        return dc1 * dx2 >= dc2 * dx1;
    }

    void add(ll m, ll c) {
        // The magic happens here: automatically formats internal
     state
        ll m_int = IS_INC ? -m : m;
        ll c_int = IS_MAX ? -c : c;

        Line l_new(m_int, c_int);
        int pos = sz;
        int low = 1, high = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (redundant(st[mid - 1], st[mid], l_new)) {
                pos = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        hist.push_back({pos, sz, st[pos]});
        st[pos] = l_new;
        sz = pos + 1;
    }

    void rollback() {
        if (hist.empty()) return;
        History h = hist.back();
        hist.pop_back();
        sz = h.sz;
        st[h.pos] = h.old_line;
    }

    ll query(ll x) {
        if (sz == 0) return IS_MAX ? -llinf : llinf;

        // The magic happens here: matches x to the internal
     geometry
        ll x_int = (IS_INC != IS_MAX) ? -x : x;

        int low = 0, high = sz - 2;
        int best = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (st[mid].eval(x_int) <= st[mid + 1].eval(x_int)) {
                best = mid;
```

```
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    ll ans = st[best].eval(x_int);
    return IS_MAX ? -ans : ans;
  }
};
```

### 8.1.4 LiChaoTree

LI CHAO TREE - MAXIMUM of a set of lines y = m*x + b at any query x, with NO order requirement

```
// on the slopes or on the queries (that is the whole point; CHT needs monotone slopes).
// O(log X) per line insert and per query, O(log^2 X) to insert a line only on a SEGMENT [lx, rx].
// Space O(n log X). Works for any family of functions where two members cross AT MOST ONCE -
// lines, and also a*x^2 + b*x + c with a FIXED a.
// EXACT INTEGER ARITHMETIC: never compute the crossing abscissa. Compare the two candidates at
// ns, mid and ne only - a division in double is off by ~1e3 in x when the coefficients reach 1e18,
// which silently sends the loser down the wrong half.
// SET the domain [X0, X1] before use; it must cover every x you will ever query.
struct LiChao {
    struct Line { ll m = 0, b = LLONG_MIN / 4; };               // neutral = -infinity
    ll X0, X1;
    vector<Line> t;
    vector<int> lf, rt;
    LiChao(ll X0, ll X1) : X0(X0), X1(X1) { newNode(); }
    int newNode() { t.push_back({}), lf.push_back(-1), rt.push_back(-1); return t.size() - 1; }
    static ll f(const Line& L, ll x) { return L.m == 0 && L.b == LLONG_MIN / 4 ? L.b : L.m * x + L.b; }
    void add(Line nw, int v, ll l, ll r) {                      // insert on the whole node range
        while (true) {
            ll m = l + (r - l) / 2;                             // NOT (l+r)/2: negative l+r truncates up
            bool lef = f(nw, l) > f(t[v], l), mid = f(nw, m) > f(t[v], m);
            if (mid) swap(t[v], nw);
            if (l == r) return;
            if (lef != mid) {
                if (lf[v] < 0) lf[v] = newNode();
                v = lf[v], r = m;
            } else {
                if (f(nw, r) > f(t[v], r)) {
                    if (rt[v] < 0) rt[v] = newNode();
                    v = rt[v], l = m + 1;
                } else return;
            }
        }
    }
    void addLine(ll m, ll b) { add({m, b}, 0, X0, X1); }
    void addSegment(ll m, ll b, ll lx, ll rx, int v = 0, ll l = LLONG_MIN, ll r = LLONG_MIN) {
        if (l == LLONG_MIN) l = X0, r = X1;
        if (rx < l || r < lx) return;
        if (lx <= l && r <= rx) { add({m, b}, v, l, r); return; }
        ll md = l + (r - l) / 2;
        if (lf[v] < 0) lf[v] = newNode();
        if (rt[v] < 0) rt[v] = newNode();
        addSegment(m, b, lx, rx, lf[v], l, md), addSegment(m, b, lx, rx, rt[v], md + 1, r);
    }
    ll query(ll x) {
        ll r = LLONG_MIN, l = X0, rr = X1;
        for (int v = 0; v >= 0;) {
            r = max(r, f(t[v], x));
            if (l == rr) break;
            ll m = l + (rr - l) / 2;
            if (x <= m) v = lf[v], rr = m;
            else v = rt[v], l = m + 1;
        }
        return r;                                               // LLONG_MIN/4 if no line covers x
    }
};
// FOR A MINIMUM: insert (-m, -b) and negate the answer.
// PERSISTENT / MERGEABLE: see 05 - PersistentLiChaoTree.cpp. MERGING two Li Chao trees costs
// O(n log X) amortised in total (push the loser line down into the other subtree, exactly like
// segment-tree merging) - that is how you do convex-hull DP on a tree without small-to-large.
```

### 8.1.5 PersistentLiChaoTree

Persistent Li Chao Tree - Space: O(N log X), Insert Line: O(log X), Query: O(log X)

```
// Version-persistent Li Chao Segment Tree for tree / history DP queries
const ll maxN = 1e7 + 5;

struct Line {
    ll m, c;

    Line() : m(0), c(llinf) {}

    Line(ll m, ll c) : m(m), c(c) {}
};

ll sub(ll x, Line l) {
    return x * l.m + l.c;
}

// Persistent Li Chao
struct Node {
// range I am responsible for
    Line line;
    Node *left, *right;

    Node() {
        left = right = NULL;
    }

    Node(ll m, ll c) {
        line = Line(m, c);
        left = right = NULL;
    }

    void extend(int l, int r) {
        if (left == NULL && l != r) {
            left = new Node();
            right = new Node();
        }
    }

    Node *copy(Node *node) {
        Node *newNode = new Node;
        newNode->left = node->left;
        newNode->right = node->right;
        newNode->line = node->line;
        return newNode;
    }

    Node *add(Line toAdd, int l, int r) {
        int mid = l + (r - l) / 2;     // NOT (l+r)/2: that truncates toward ZERO, so on a negative
                                       // interval the left child equals the parent -> infinite recursion
        Node *cur = copy(this);
        if (l == r) {
            if (sub(l, toAdd) < sub(l, cur->line))
                swap(toAdd, cur->line);
            return cur;
        }
        bool lef = sub(l, toAdd) < sub(l, cur->line);
        bool midE = sub(mid + 1, toAdd) < sub(mid + 1, cur->line);
        if (midE)
            swap(cur->line, toAdd);
        cur->extend(l, r);
        if (lef != midE) {
            cur->left = cur->left->add(toAdd, l, mid);
        } else {
            if (mid + 1 > r)
                return cur;
            cur->right = cur->right->add(toAdd, mid + 1, r);
        }
        return cur;
    }

    Node *add(Line toAdd) {
        return add(toAdd, -maxN + 1, maxN - 1);
    }

    ll query(ll x, int l, int r) {
        int mid = l + (r - l) / 2;     // NOT (l+r)/2: that truncates toward ZERO, so on a negative
                                       // interval the left child equals the parent -> infinite recursion
        if (l == r || left == NULL)
            return sub(x, line);
        extend(l, r);
        if (x <= mid)
            return min(sub(x, line), left->query(x, l, mid));
        else
            return min(sub(x, line), right->query(x, mid + 1, r));
    }

    ll query(ll x) {
        return query(x, -maxN + 1, maxN - 1);
    }

    void clear() {
        if (left != NULL) {
            left->clear();
            right->clear();
        }
        delete this;
    }
};

const int N = 200005;                          // #versions; also in 01 - Template.cpp
Node *tree[N];
```

### 8.1.6 KnuthDP

KNUTH OPTIMIZATION - interval DP dp[i][j] = min_{i<=k<j}(dp[i][k] + dp[k+1][j]) + cost(i,j)

```
// from O(n^3) to O(n^2), by restricting k to [opt[i][j-1], opt[i+1][j]].
// THE HYPOTHESIS IS ON cost, NOT ON opt. 'opt[i][j-1] <= opt[i][j] <= opt[i+1][j]' is the
// CONCLUSION - you cannot test it before running. What you must check is BOTH of:
//   (1) QUADRANGLE INEQUALITY (Monge):  cost(a,c) + cost(b,d) <= cost(a,d) + cost(b,c)
//       for all a <= b <= c <= d.  Equivalent local form, and the only one checkable by hand:
//       cost(a,c) + cost(a+1,c+1) <= cost(a+1,c) + cost(a,c+1).
//   (2) MONOTONE ON INTERVALS:  a <= b <= c <= d  =>  cost(b,c) <= cost(a,d).
// Both hold for: sum of a non-negative array over [i,j]; (S_j - S_i)^2; f(S_j - S_i) with f convex
// and a[] >= 0; "number of equal pairs inside [i,j]". They FAIL for anything built from max, and
// monotonicity fails as soon as a[] can be negative (QI can still hold - D&C needs only QI,
// Knuth needs both). Applied without them there is no crash and no TLE, just a wrong answer.
// Classic uses: optimal BST, merging n piles of stones, CF 1101F-style interval costs.
struct KnuthDP {
    int n;
    vector<vector<ll>> dp;
    vector<vector<int>> opt;

    KnuthDP(int n = 0) : n(n), dp(n + 2, vector<ll>(n + 2, 0)),
     opt(n + 2, vector<int>(n + 2, 0)) {}

    // User-defined cost function cost(i, j)
    ll solve(function<ll(int, int)> cost) {
        for (int i = 1; i <= n; i++) {
            opt[i][i] = i;
        }

        for (int len = 2; len <= n; len++) {
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = llinf;
                int L = opt[i][j - 1], R = opt[i + 1][j];
                for (int k = L; k <= min(R, j - 1); k++) {
                    ll cur = dp[i][k] + dp[k + 1][j] + cost(i, j);
                    if (cur < dp[i][j]) {
                        dp[i][j] = cur;
                        opt[i][j] = k;
                    }
                }
            }
        }
        return dp[1][n];
    }
};
```

### 8.1.7 AliensTrick

ALIENS TRICK (Lagrangian relaxation / wqs binary search) - O(n log(range)).

```
// Drops an "exactly k pieces" constraint: charge lambda per piece, solve the UNCONSTRAINED DP,
// and binary search lambda until the optimum uses k pieces. Answer = cost(lambda) - k*lambda.
// PRECONDITION: f(k), the optimum with exactly k pieces, must be CONVEX in k (for a minimum;
// concave for a maximum). NOT "strictly" - strictness is never needed and rarely true.
// THREE THINGS THAT MAKE IT CORRECT, all easy to get wrong:
//  (1) dp_solver MUST return the LARGEST optimal count for that lambda. With f convex, the
//      difference d(k) = f(k) - f(k-1) is non-decreasing; the search below finds the largest
//      lambda with count >= k, and then d(k) <= -lambda <= d(k+1), so k really is optimal at
//      that lambda and cost - k*lambda = f(k). Any other tie-break makes the last line wrong.
//  (2) An integer lambda suffices ONLY if all costs are integers.
//  (3) [min_penalty, max_penalty] must straddle every slope of f; assert it.
// CONVEX families: "k pairwise non-adjacent items", "exactly k segments with a Monge cost",
// "k disjoint intervals", "min-cost flow of value k" (flow cost is always convex in the value),
// matroid-rank-constrained optima. TRAPS: two coupled constraints, or a cost rewarding a parity
// of k. CHECK IT: brute-force f(k) for n <= 60 and assert f(k+1)-f(k) >= f(k)-f(k-1).
struct AliensTrick {
    // Binary searches optimal penalty lambda to choose exactly K items
    // Returns minimum cost to choose exactly K items
    static ll solve(int n, int k, ll min_penalty, ll max_penalty,
      function<pair<ll, int>(ll)> dp_solver) {
        ll l = min_penalty, r = max_penalty, best_lambda = 0;

        while (l <= r) {
            ll mid = l + (r - l) / 2;
            auto [cost, count] = dp_solver(mid);
            if (count >= k) {
                best_lambda = mid;
                l = mid + 1; // Increase penalty if we used too many items
            } else {
                r = mid - 1; // Decrease penalty if we used too few items
            }
        }

        // True cost = cost_with_penalty - (k * best_lambda)
        auto [cost, count] = dp_solver(best_lambda);
        return cost - (ll)k * best_lambda;
    }
};
```

### 8.1.8 AliensTrickExample

ALIENS TRICK (Lagrangian / wqs binary search) - drops the "exactly k pieces" constraint by charging

```
// a penalty lambda per piece, solving the unconstrained DP, and binary searching lambda until the
// optimal solution uses k. REQUIRES the optimum as a function of k to be CONVEX; check that first.
// 1. DP State Structure
struct DPState {
    long long cost;
    long long count; // Number of items/segments picked

    // TIE-BREAKER LOGIC:
    // If costs are equal, we maximize the count to handle collinear points on the convex hull.
    // If you are writing a MINIMIZATION problem with min(), change this to:
    // if (cost != other.cost) return cost > other.cost; (since priority queue / max naturally takes the "biggest")
    // Actually, when using standard >, < operators for min/max DP:
    bool operator<(const DPState& other) const {
        if (cost != other.cost) return cost < other.cost;
        return count < other.count; // Always prefer taking MORE items if costs are tied
    }
};

void solve_aliens() {
    int n, k;
    cin >> n >> k;
    // Read ABC 218 H input: A array is of size N-1
    vector<long long> A(n);
    for(int i = 1; i < n; i++) cin >> A[i];
    // If K > N / 2, swap colors so K <= N / 2 (the score is symmetric)
    if (k > n / 2) k = n - k;
    // Construct 1-indexed values array 'a' where a[i] = A[i-1] + A[i]
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) {
        a[i] = A[i - 1] + (i < n ? A[i] : 0);
    }
    // 2. The Unconstrained DP
    // Returns the optimal {cost, count} for a given penalty.
    auto check_dp = [&](long long penalty) -> DPState {
        // dp[i][0]: Max score in prefix i, ending completely outside a segment
        // dp[i][1]: Max score in prefix i, ending inside an active segment
        vector<vector<DPState>> dp(n + 1, vector<DPState>(2, {0, 0}));

        for (int i = 1; i <= n; i++) {
            // Transition 0: We don't pick item i
            dp[i][0] = max(dp[i-1][0], dp[i-1][1]);

            // Transition 1: We pick item i
            // Rule: ONLY apply penalty when STARTING a new pick/segment!
            // Type B logic: To ensure non-adjacency, we must transition from dp[i-1][0].

            // --- IF MAXIMIZING PROFIT ---
            DPState start_new = {dp[i-1][0].cost + a[i] - penalty, dp[i-1][0].count + 1};

            // Replaced max(start_new, continue_old) with just start_new
            dp[i][1] = start_new;
        }
        return max(dp[n][0], dp[n][1]);
    };

    // 3. WQS Binary Search
    // The bounds must be large enough to cover the maximum possible penalty.
    // FIXED: Dropped bounds to 2e9 to prevent k * mid_penalty from overflowing long long
    long long low = -2e9 - 7, high = 2e9 + 7;
    long long best_ans = 0;

    while (low <= high) {
        long long mid_penalty = low + (high - low) / 2;

        DPState res = check_dp(mid_penalty);

        // We want EXACTLY k items.
        // If we picked >= k items, it means the penalty wasn't
```

```
    harsh enough to stop us.
        // Even if res.count > k, we save the answer because of
    collinear points on the hull.
        if (res.count >= k) {

            // --- IF MAXIMIZING PROFIT ---
            // We subtracted the penalty K times, so we add it
    back.
            best_ans = res.cost + (1LL * k * mid_penalty);

            // --- IF MINIMIZING COST ---
            // best_ans = res.cost - (1LL * k * mid_penalty);

            // Penalty was too weak, increase it to force picking
    fewer items next time
            low = mid_penalty + 1;
        } else {
            // We picked too few items. The penalty was too harsh.
            high = mid_penalty - 1;
        }
    }

    // Output the final answer
    cout << best_ans << "\n";
}
```

### 8.1.9 MaxAverageSubarray

UPPER hull (maximum) and this one is the LOWER hull (minimum) - they are NOT

```
// NOTE: 02 - CHT.cpp also declares `struct Line` / `struct
    HullDynamic`, but that one is the
//       UPPER hull (maximum) and this one is the LOWER hull (
    minimum) - they are NOT
//       interchangeable. If you need both, rename one pair to
    UpperHull / LowerHull.
/*
    ================================================================

 * DYNAMIC CONVEX HULL TRICK (CHT) - AVERAGE SUBARRAY TEMPLATE
 *
    ================================================================

 *
 * HOW TO USE THE 4 VARIATIONS:
 *
 * 1. MAX Average, LONGEST length  (Default)
 *    - Code: Use `lhs < rhs` in Query::operator<
 *    - Call: hull.insert_line(i, S[i]); hull.query_tangent_index(
    i, S[i]);
 *
 * 2. MAX Average, SHORTEST length
 *    - Code: Use `lhs <= rhs` in Query::operator<
 *    - Call: hull.insert_line(i, S[i]); hull.query_tangent_index(
    i, S[i]);
 *
 * 3. MIN Average, LONGEST length
 *    - Code: Use `lhs < rhs` in Query::operator<
 *    - Call: hull.insert_line(i, -S[i]); hull.query_tangent_index
    (i, -S[i]); // Negated
 *
 * 4. MIN Average, SHORTEST length
 *    - Code: Use `lhs <= rhs` in Query::operator<
 *    - Call: hull.insert_line(i, -S[i]); hull.query_tangent_index
    (i, -S[i]); // Negated
 *
    ================================================================
     */

struct Query {
    ll i, Si;
};

struct Line {
    ll m, b;
    mutable function<const Line *()> succ;

    bool operator<(const Line &other) const {
        return m < other.m;
    }

    bool operator<(const Query &q) const {
        const Line *s = succ();
        if (!s) return false;

        // CORRECTED ALGEBRA: Direct cross-multiplication of
    slopes
        __int128_t lhs = (__int128_t)(q.Si - b) * (q.i - s->m);
        __int128_t rhs = (__int128_t)(q.Si - s->b) * (q.i - m);

        // --- TIE-BREAKING CONTROL ---
        // Change this return statement based on length
    requirements:
        // return lhs < rhs;  // <-- Use for LONGEST subarray
        // return lhs <= rhs; // <-- Use for SHORTEST subarray
        return lhs < rhs;
    }
};

struct HullDynamic : public multiset<Line, less<>> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end()) return false;
            // CORRECTED: Lower hull keeps smallest y
            return y->m == z->m && y->b >= z->b;
        }
        auto x = prev(y);
        if (z == end()) return y->m == x->m && y->b >= x->b;

        return (__int128_t)(x->b - y->b) * (z->m - y->m) <= (
    __int128_t)(y->b - z->b) * (y->m - x->m);
    }

    void insert_line(ll m, ll b) {
        auto y = insert({m, b});
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y);
```

```
    };
        if (bad(y)) {
            erase(y);
            return;
        }
        while (next(y) != end() && bad(next(y))) erase(next(y));
        while (y != begin() && bad(prev(y))) erase(prev(y));
    }

    int query_tangent_index(ll i, ll Si) {
        auto l = *lower_bound(Query{i, Si});
        return l.m;
    }
};

// USAGE (longest subarray with average >= 0, after subtracting
    the target from every element):
//   S[0..n] = prefix sums; the answer for each i is i - (tangent
    index from the hull of (j, S[j])).
//   HullDynamic hull; hull.insert_line(0, 0);
//   for (int i = 1; i <= n; i++) {
//       int k = hull.query_tangent_index(i, S[i]);          //
    best j < i
//       ans[i] = i - k;
//       hull.insert_line(i, S[i]);
//   }
// WHY IT WORKS: avg(j+1..i) >= 0  <=>  S[i] - S[j] >= 0, and
    among all valid j you want the
// SMALLEST, which is the point of tangency from (i, S[i]) to the
    lower hull of the points (j, S[j]).
// Binary search on the answer + prefix sums is the shorter O(n
    log n) alternative; this is O(n).
```

### 8.1.10 SlopeTrick

SLOPE TRICK - maintain a CONVEX piecewise-linear f(x) as two heaps of its breakpoints.

```
// L = left slopes (max-heap), R = right slopes (min-heap), minf =
     current minimum value.
// Every operation below is O(log n). Classic use: "minimum total
    |change| to make an array
// non-decreasing", scheduling with earliness/tardiness penalties,
     and any DP where
//   dp_i(x) = min over y<=x of dp_{i-1}(y)  + |x - a_i|.
struct SlopeTrick {
    priority_queue<ll> L;                                          //
     left breakpoints (max-heap)
    priority_queue<ll, vector<ll>, greater<ll>> R;                 //
     right breakpoints (min-heap)
    ll minf = 0, addL = 0, addR = 0;                               //
     lazy shifts of each side

    ll topL() { return L.top() + addL; }
    ll topR() { return R.top() + addR; }
    void pushL(ll x) { L.push(x - addL); }
    void pushR(ll x) { R.push(x - addR); }

    void addConst(ll c) { minf += c; }
    void addRampR(ll a) {                                          // f
     (x) += max(0, x - a)
        if (!L.empty()) minf += max(0LL, topL() - a);
        pushL(a);
        pushR(topL()); L.pop();
    }
    void addRampL(ll a) {                                          // f
     (x) += max(0, a - x)
        if (!R.empty()) minf += max(0LL, a - topR());
        pushR(a);
        pushL(topR()); R.pop();
    }
    void addAbs(ll a) { addRampL(a), addRampR(a); }                // f
     (x) += |x - a|
    void prefixMin() { while (!R.empty()) R.pop(); }               // f
     (x) = min_{y<=x} f(y)
    void suffixMin() { while (!L.empty()) L.pop(); }               // f
     (x) = min_{y>=x} f(y)
    void shift(ll dl, ll dr) { addL += dl, addR += dr; }           // f
     (x) -> min over [x-dr, x-dl]
    ll minVal() const { return minf; }
};
// RECIPE for "make a[] non-decreasing with min sum |b_i - a_i|":
//   for each a_i:  st.prefixMin();  st.addAbs(a_i);   answer = st
    .minVal()
// For "strictly increasing", first replace a_i by a_i - i.
```

### 8.1.11 MinPlusConvolution

MIN-PLUS / MAX-PLUS CONVOLUTION: c[k] = min_i ( a[i] + b[k-i] ).

```
// In general this is quadratic and CONJECTURED to stay that way (
    the MinConv hypothesis:
// MinConv, MaxConv, unbounded knapsack and 0/1 knapsack are all
    equivalent under subquadratic
// reductions). So the ONLY fast versions rely on convexity.

// BOTH CONVEX -> O(n + m). The difference sequence of the answer
    is the sorted MERGE of the two
// difference sequences: greedily always take the smaller next
    slope.
vector<ll> minPlusConvexConvex(const vector<ll>& a, const vector<
    ll>& b) {
    int n = a.size(), m = b.size();
    vector<ll> c(n + m - 1);
    c[0] = a[0] + b[0];
    int i = 0, j = 0;
    for (int k = 1; k < n + m - 1; k++) {
        ll da = (i + 1 < n) ? a[i + 1] - a[i] : LLONG_MAX;
        ll db = (j + 1 < m) ? b[j + 1] - b[j] : LLONG_MAX;
        if (da <= db) c[k] = c[k - 1] + da, i++;
        else c[k] = c[k - 1] + db, j++;
    }
    return c;
}
// MAX-PLUS with both CONCAVE: same code, take the LARGER slope
    first (or negate and reuse).
vector<ll> maxPlusConcaveConcave(vector<ll> a, vector<ll> b) {
    for (ll& x : a) x = -x;
    for (ll& x : b) x = -x;
    auto c = minPlusConvexConvex(a, b);
    for (ll& x : c) x = -x;
    return c;
}
// ONLY ONE CONVEX -> the matrix X[k][i] = a[i] + b[k-i] is
    totally monotone (Monge), so the row
// minima can be found with SMAWK in O(n + m), or with divide &
    conquer in O(n log n):
//   solve(kl, kr, il, ir): take k = mid, scan i in [il, ir] for
    the best, recurse with the
//   split at the optimum. Same skeleton as DivideAndConquerDP.
// `valid(k)` must return the legal range of i for output index k
    - for the convolution above
// that is [max(0, k-m+1), min(n-1, k)]. Intersecting with it is
    what keeps `best` well defined;
// without it an empty candidate set leaves best = -1 and poisons
    both recursive calls.
template <class F, class V>
void dcOpt(int kl, int kr, int il, int ir, vector<ll>& c, F cost,
     V valid) {
    if (kl > kr) return;
    int km = (kl + kr) / 2, best = -1;
    ll bv = LLONG_MAX;
    auto [lo, hi] = valid(km);
    for (int i = max(il, lo); i <= min(ir, hi); i++) {
        ll v = cost(km, i);
        if (v < bv) bv = v, best = i;
    }
    c[km] = bv;
    if (best < 0) best = max(il, lo);                  // no
    candidate: keep the bounds sane
    dcOpt(kl, km - 1, il, best, c, cost, valid);
    dcOpt(km + 1, kr, best, ir, c, cost, valid);
}
// NEITHER CONVEX -> O(n*m) is the best you should plan for, and
    that is not laziness: min-plus
// convolution, 0/1 knapsack, unbounded knapsack and tree knapsack
     are subquadratic-EQUIVALENT,
// so a general fast merge would break all four. Look for
    convexity, a small value range, or a
// bitset instead of a cleverer convolution.
// WHERE THIS APPEARS: merging two "cost as a function of how many
     you take" curves - group
// knapsack, tree DP where each child contributes a convex cost
    curve, and combining the
// per-part curves produced by Alien's trick.
```

### 8.1.12 OneDOneD

1D/1D DP OPTIMISATION - the self-referential case that divide & conquer structurally cannot do.

```
//     dp[i] = min over j < i of ( dp[j] + w(j, i) )
// D&C optimisation needs the LAYERED form dp[k][i] = min(dp[k-1][
    j] + w) so that the whole
// previous layer is already known. Here dp[j] is produced by the
    very loop that consumes it, so
// you must decide dp[i] before you may use it. That is what this
    file solves.
// PRECONDITION: w satisfies the QUADRANGLE INEQUALITY (Monge)
//     w(a,c) + w(b,d) <= w(a,d) + w(b,c)   for all a <= b <= c <=
     d
// checkable in the local form  w(a,c) + w(a+1,c+1) <= w(a+1,c) +
    w(a,c+1).
// Then the optimal j for i is NON-DECREASING in i, so the array
    of "who is currently best for
// each future i" is a sequence of INTERVALS, each owned by one j
    - and finalising dp[i] can only
// append one new interval and overwrite a suffix of the old ones.
     Keep them on a stack.
// O(n log n) (the binary search for the crossover); O(n) if the
    crossover is O(1) to find.
template <class F>
vector<ll> onedOned(int n, F w) {                                //
     w(j, i) for j < i
    vector<ll> dp(n + 1, llinf);
    dp[0] = 0;
    vector<array<int, 3>> st;                                    //
      {owner j, from, to} intervals
    st.push_back({0, 1, n});
    int head = 0;
    for (int i = 1; i <= n; i++) {
        while (head < (int)st.size() && st[head][2] < i) head++;
        dp[i] = dp[st[head][0]] + w(st[head][0], i);
    // the owner of position i
        st[head][1] = i;
        // now insert i as a candidate owner for the suffix it
    wins
        auto better = [&](int a, int b, int x) {
    // is a better than b at x?
            return dp[a] + w(a, x) <= dp[b] + w(b, x);
        };
        while (!st.empty() && st.back()[1] > i && better(i, st.
    back()[0], st.back()[1])) st.pop_back();
        if (st.empty() || head >= (int)st.size()) st.push_back({i
    , i + 1, n});
        else {
            auto [j, l, r] = st.back();
            if (better(i, j, r)) {
    // i wins part of this interval
                int lo = max(l, i + 1), hi = r, pos = r + 1;
                while (lo <= hi) {
    // first x where i beats j
                    int m = (lo + hi) / 2;
                    better(i, j, m) ? (pos = m, hi = m - 1) : lo
    = m + 1;
                }
                st.back()[2] = pos - 1;
                if (pos <= n) st.push_back({i, pos, n});
            } else if (r < n) st.push_back({i, r + 1, n});
        }
    }
    return dp;
}
/* WHEN YOU NEED WHICH OPTIMISATION - the whole family on one card
   dp[i] = min_j (dp[j] + w(j,i)),   w Monge, SELF-REFERENTIAL
        -> THIS FILE, O(n log n)
   dp[k][i] = min_j (dp[k-1][j] + w(j,i)),  w Monge, LAYERED
        -> D&C opt (01), O(n k log n)
   dp[i][j] = min_k (dp[i][k] + dp[k+1][j]) + w(i,j),  w Monge AND
             monotone on intervals
        -> Knuth (06), O(n^2)
   dp[i] = min_j (m_j * x_i + b_j)
        -> CHT (02) / Li Chao (04)
   "exactly k pieces", optimum convex in k
        -> Aliens (07)
   min-plus convolution of a CONVEX sequence with anything
        -> SMAWK / D&C (11)
   the query point moves monotonically and lines are range-updated
        -> kinetic segment tree
   VERIFY the quadrangle inequality numerically on random
     quadruples before trusting any of them:
   a DP optimisation applied without its precondition is a silent
     wrong answer, never a crash.
   COMMON MONGE COSTS: (S_j - S_i)^2; f(S_j - S_i) with f convex
     and a[] >= 0; c * (j - i);
   "number of equal pairs inside [i,j]"; a_i * b_j with a non-
     increasing and b non-decreasing.
   NOT MONGE: anything built from max, and anything with negative
     a[] where you also need the
   interval-monotonicity half (D&C only needs QI; Knuth needs both
     ). */
```

## 8.2 Bitmask DP

### 8.2.1 SOS_DP

Computes the sum of all subsets for every bitmask in O(N * 2^N)

```
vector<long long> SOS_DP(int n_bits, const vector<long long>& a)
     {
    int max_mask = 1 << n_bits;
    vector<long long> dp = a; // dp[mask] initially holds exactly
     the value for mask

    for (int i = 0; i < n_bits; ++i) {
        for (int mask = 0; mask < max_mask; ++mask) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
    return dp; // dp[mask] now contains sum of all submasks of
     mask
}
```

### 8.2.2 BitmaskDP

BITMASK DP - the n <= 20 toolkit.

```
// MEMORY: vector<vector<ll>> dp(1<<20, vector<ll>(20)) is ~193 MB
      (24 B of vector header per row).
// Flatten to ONE vector<ll> of size (1<<n)*n (168 MB) or vector<
     int> (84 MB), or iterate masks in
// increasing popcount and keep only two layers. At n = 20, 2^n *
     n^2 = 4e8 - usually not intended.

// TSP: shortest Hamiltonian cycle from 0 back to 0. O(2^n * n^2).
ll tsp(const vector<vector<ll>>& d) {
    int n = d.size(), N = 1 << n;
    vector<vector<ll>> dp(N, vector<ll>(n, llinf));
    dp[1][0] = 0;
    for (int m = 1; m < N; m++)
        for (int u = 0; u < n; u++) if ((m >> u & 1) && dp[m][u]
     < llinf)
            for (int v = 0; v < n; v++) if (!(m >> v & 1) && d[u
     ][v] < llinf)
                dp[m | 1 << v][v] = min(dp[m | 1 << v][v], dp[m][
     u] + d[u][v]);
    ll best = llinf;
    for (int u = 1; u < n; u++) if (dp[N - 1][u] < llinf && d[u
     ][0] < llinf)
        best = min(best, dp[N - 1][u] + d[u][0]);
    return n == 1 ? 0 : best;                          // drop the
     +d[u][0] for a Hamiltonian PATH
}

// MINIMUM SET COVER / partition into the fewest valid groups. O
     (3^n) via submask enumeration.
// ok[m] = true if the set m is a legal single group.
int minPartition(int n, const vector<char>& ok) {
    int N = 1 << n;
    vector<int> dp(N, inf);
    dp[0] = 0;
    for (int m = 1; m < N; m++) {
        int low = m & -m;                              // fix the
     lowest element: kills the n! factor
        for (int s = m; s; s = (s - 1) & m) if ((s & low) && ok[s
     ])
            dp[m] = min(dp[m], dp[m ^ s] + 1);
    }
    return dp[N - 1];
}
// COUNTING the partitions instead: same loop with dp[m] += dp[m ^
      s].
// If groups are UNORDERED, fixing the lowest element (above)
     already prevents double counting.

// ASSIGNMENT (n tasks to n workers, minimum cost), O(2^n * n). dp
     [m] = best cost using the
// first popcount(m) workers on exactly the task set m.
ll assignment(const vector<vector<ll>>& c) {
    int n = c.size(), N = 1 << n;
    vector<ll> dp(N, llinf);
    dp[0] = 0;
    for (int m = 0; m < N - 1; m++) if (dp[m] < llinf) {
        int i = __builtin_popcount(m);                 // next
     worker
        for (int j = 0; j < n; j++) if (!(m >> j & 1))
            dp[m | 1 << j] = min(dp[m | 1 << j], dp[m] + c[i][j])
     ;
    }
    return dp[N - 1];                                  // n > 20 ->
      use the Hungarian algorithm
}

// COUNTING HAMILTONIAN PATHS / permutations with adjacency
     constraints: same shape as tsp but
// dp[m][u] += dp[m ^ (1<<u)][v] over allowed v.
// SUBSET-SUM STYLE: when the state is "which items used" AND the
     order does not matter, prefer
// dp over masks; when only a total matters, use a knapsack - 2^n
     is not always necessary.
// MEET IN THE MIDDLE: n <= 40 -> split in half, enumerate 2^20
     each side, sort + two pointers.
```

## 8.3 Knapsack

### 8.3.1 BoundedKnapsackBitset

BOUNDED KNAPSACK, FEASIBILITY ONLY, with a bitset: O(n W / 64). Binary-group the counts

```
// (1,2,4,...) so "cnt copies of w" becomes O(log cnt) shift-ors.
     Use when you only need which sums
// are reachable, not the best value.
const int MAX_W = 100005;
bitset<MAX_W> dp;

void multiset_bitset_dp() {
    // Example: We have 13 items, each with weight 5
    int count = 13;
    int weight = 5;

    vector<int> bundled_weights;

    // Binary grouping logic
    int current_power = 1;
    while (count >= current_power) {
        bundled_weights.push_back(current_power * weight);
        count -= current_power;
        current_power *= 2;
    }
    // Add whatever is left over
    if (count > 0) {
        bundled_weights.push_back(count * weight);
    }

    // Now run the ultra-fast Bitset DP on the bundles
    dp[0] = 1;
    for (int bundle_w : bundled_weights) {
        dp |= (dp << bundle_w);
    }
}
```

### 8.3.2 KnapsackFamily

KNAPSACK FAMILY. W = capacity. All O(n*W) unless noted.

```
// 0/1: iterate capacity DOWNWARD so each item is used once.
ll knap01(const vector<int>& w, const vector<ll>& v, int W) {
    vector<ll> dp(W + 1, 0);
    for (size_t i = 0; i < w.size(); i++)
        for (int c = W; c >= w[i]; c--) dp[c] = max(dp[c], dp[c -
      w[i]] + v[i]);
    return dp[W];
}
// UNBOUNDED: iterate capacity UPWARD.
ll knapInf(const vector<int>& w, const vector<ll>& v, int W) {
    vector<ll> dp(W + 1, 0);
    for (size_t i = 0; i < w.size(); i++)
        for (int c = w[i]; c <= W; c++) dp[c] = max(dp[c], dp[c -
      w[i]] + v[i]);
    return dp[W];
}
// BOUNDED (item i available cnt[i] times) via BINARY GROUPING:
     split cnt into 1,2,4,...,rest
// and run 0/1 on the groups. O(W * sum log cnt).
ll knapBounded(const vector<int>& w, const vector<ll>& v, const
     vector<int>& cnt, int W) {
    vector<int> W2; vector<ll> V2;
    for (size_t i = 0; i < w.size(); i++) {
        int c = cnt[i];
        for (int k = 1; c > 0; k <<= 1) {
            int t = min(k, c); c -= t;
            W2.push_back(w[i] * t), V2.push_back(v[i] * t);
        }
    }
    return knap01(W2, V2, W);
}
// FEASIBILITY ONLY (which sums are reachable) -> bitset, O(n*W
     /64). Massively faster.
bitset<100001> reachableSums(const vector<int>& w) {
    bitset<100001> dp;
    dp[0] = 1;
    for (int x : w) dp |= dp << x;
    return dp;
}
// COUNTING ways (mod p): same loops, dp[c] += dp[c - w[i]].
// Order matters: item-outer/capacity-inner counts COMBINATIONS,
//                capacity-outer/item-inner counts PERMUTATIONS (
     ordered sequences).
```

## 8.4 Classics

### 8.4.1 DigitDP

DIGIT DP - count numbers in [L, R] with a digit property. f(R) - f(L-1).

```
// State: (position, tight, started, problem state). 'started' handles leading zeros.
// THIS CODE IS BASE 10 (to_string, digit cap 9, memo[20] positions). For another base, write the
// digits yourself, change the 9, and size memo by the digit count (base 2 needs 64 positions).
// L - 1 IS NOT ALWAYS AVAILABLE: if the bounds are given as huge decimal STRINGS you cannot
// subtract one, so carry two tight flags (tightLo, tightHi) and count [L, R] in a single pass.
string DIG;
ll memo[20][2][2][200];
bool vis[20][2][2][200];

ll go(int p, int tight, int started, int st) {
    if (p == (int)DIG.size()) return started ? /* ACCEPT? */ (st == 0) : 0;   // <- edit
    if (vis[p][tight][started][st]) return memo[p][tight][started][st];
    vis[p][tight][started][st] = true;
    ll res = 0;
    int hi = tight ? DIG[p] - '0' : 9;
    for (int d = 0; d <= hi; d++) {
        int nStarted = started || d > 0;
        int nSt = nStarted ? (st + d) % 10 : 0;
                  // <- edit
        res += go(p + 1, tight && d == hi, nStarted, nSt);
    }
    return memo[p][tight][started][st] = res;
}
bool qualifiesZero = 1;                          // does the number 0 satisfy the predicate?
ll countUpTo(ll x) {
    if (x < 0) return 0;
    DIG = to_string(x);
    memset(vis, 0, sizeof vis);
    return go(0, 1, 0, 0) + qualifiesZero;       // set qualifiesZero = 1 only if 0 itself counts
}
ll countRange(ll L, ll R) { return countUpTo(R) - countUpTo(L - 1); }
// VARIANTS: track (sum of digits), (value mod k), (last digit), (a bitmask of used digits),
// (whether we already saw "13"), (count so far) - anything with a small state.
// For "k-th number with the property", binary search on countUpTo.
```

### 8.4.2 LIS

LIS in O(n log n), with reconstruction. STRICT increasing by default.

```
// Non-decreasing: change lower_bound -> upper_bound.
// Longest DEcreasing: negate the array. Longest non-increasing: negate + upper_bound.
vector<int> lis(const vector<ll>& a) {
    int n = a.size();
    vector<ll> tail;                             // tail[k] = smallest possible tail of an LIS of length k+1
    vector<int> pos(n), prv(n, -1), idx;
    for (int i = 0; i < n; i++) {
        int k = lower_bound(all(tail), a[i]) - tail.begin();
        if (k == (int)tail.size()) tail.push_back(a[i]), idx.push_back(i);
        else tail[k] = a[i], idx[k] = i;
        pos[i] = k;
        prv[i] = k ? idx[k - 1] : -1;
    }
    vector<int> res;
    for (int i = idx.empty() ? -1 : idx.back(); i >= 0; i = prv[i]) res.push_back(i);
    reverse(all(res));
    return res;                                  // indices of one LIS; length = tail.size()
}
// SAME SHAPE SOLVES: longest chain of pairs (sort by x, LIS on y); maximum non-overlapping
// intervals; "minimum number of non-increasing subsequences covering the array" = LIS length
// (Dilworth). For LIS with an extra weight, replace 'tail' with a max-BIT over compressed values.
```

### 8.4.3 IntervalDP

INTERVAL DP - dp over [l, r], processed by increasing length. O(n^3) (O(n^2) with Knuth).

```
// This is the base pattern that Knuth's optimisation speeds up; write this first, optimise later.

// 1) SPLIT form: dp[l][r] = min over k of dp[l][k] + dp[k+1][r] + cost(l, r)
//    matrix chain multiplication, merging stones, optimal BST, "cost to combine a range"
template <class Cost> ll intervalSplit(int n, Cost cost) {
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++) {
            dp[l][r] = llinf;
            for (int k = l; k < r; k++) dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]);
            dp[l][r] += cost(l, r);
        }
    return dp[0][n - 1];
}

// 2) ENDPOINT form: decide about a[l] or a[r] and shrink. Palindromes, "remove from either end".
// Minimum insertions to make s a palindrome:
int minInsertPalindrome(const string& s) {
    int n = s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++)
            dp[l][r] = s[l] == s[r] ? dp[l + 1][r - 1] : 1 + min(dp[l + 1][r], dp[l][r - 1]);
    return n ? dp[0][n - 1] : 0;
}

// 3) "BURST BALLOONS" form: choose the LAST element removed in the range, so the neighbours are
//    the (still intact) boundaries l-1 and r+1.
ll burst(vector<ll> a) {
    int n = a.size();
    a.insert(a.begin(), 1), a.push_back(1);
    vector<vector<ll>> dp(n + 2, vector<ll>(n + 2, 0));
    for (int len = 1; len <= n; len++)
        for (int l = 1, r = len; r <= n; l++, r++)
            for (int k = l; k <= r; k++)
                dp[l][r] = max(dp[l][r], dp[l][k - 1] + a[l - 1] * a[k] * a[r + 1] + dp[k + 1][r]);
    return dp[1][n];
}
// WHEN KNUTH APPLIES (form 1): cost is monotone on intervals and satisfies the quadrangle
// inequality; then opt[l][r-1] <= opt[l][r] <= opt[l+1][r] and the k-loop becomes amortised O(1).
// COUNTING variants (e.g. #ways to parenthesise) replace min with + and cost with 1.
```

### 8.4.4 BrokenProfileDP

BROKEN PROFILE DP - sweep the grid cell by cell carrying a bitmask of the "broken" frontier.

```
// O(n * m * 2^m); always make m the SMALLER dimension.
// mask bit j = "cell (i, j) is already covered by something placed earlier".

// Count tilings of an n x m grid by 1x2 dominoes.
ll dominoTilings(int n, int m) {
    if (m > n) swap(n, m);
    if (((ll)n * m) & 1) return 0;
    int M = 1 << m;
    vector<ll> cur(M, 0), nxt;
    cur[0] = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            nxt.assign(M, 0);
            for (int mask = 0; mask < M; mask++) if (cur[mask]) {
                if (mask >> j & 1) nxt[mask ^ (1 << j)] += cur[mask];          // already covered
                else {
                    if (i + 1 < n) nxt[mask | (1 << j)] += cur[mask];          // vertical domino
                    if (j + 1 < m && !(mask >> (j + 1) & 1))
                        nxt[mask | (1 << (j + 1))] += cur[mask];               // horizontal domino
                }
            }
            cur = nxt;
        }
    return cur[0];
}
// BLOCKED CELLS: keep a grid `bad[i][j]`; a blocked cell must arrive already "covered"
// (skip it by clearing bit j and forbidding any placement on it).
// OTHER PIECES: add one transition per shape; the mask must record every cell the shape
// occupies in a LATER position of the sweep.
// COUNTING INDEPENDENT SETS / no-two-adjacent colourings: mask = which cells of the previous
// row are used; transition over masks with (a & b) == 0 and no two adjacent bits in b.
// AS WRITTEN this is O(n * 4^m) (nested mask loops), not O(n * m * 2^m): at m = 14 that is 2.7e8
// per row. To get O(n * 2^m), note sum over a with (a & b) == 0 of cur[a] = zetaSub(cur)[~b],
// so one subset-zeta pass per row replaces the inner loop (see 05 - Combinatorics/07).
ll independentSets(int n, int m) {                                  // #ways to pick cells, none orthogonally adjacent
    if (m > n) swap(n, m);
    int M = 1 << m;
    vector<ll> cur(M, 0), nxt;
    for (int b = 0; b < M; b++) if (!(b & (b << 1))) cur[b] = 1;
    for (int i = 1; i < n; i++) {
        nxt.assign(M, 0);
        for (int a = 0; a < M; a++) if (cur[a])
            for (int b = 0; b < M; b++) if (!(b & (b << 1)) && !(a & b)) nxt[b] += cur[a];
        cur = nxt;
    }
    ll r = 0;
    for (ll x : cur) r += x;
    return r;
}
// CONNECTED-COMPONENT DP ("plug DP") is the heavy version: the state stores, for each frontier
// cell, WHICH connected component its plug belongs to (a bracket-like labelling). Use it for
// "count Hamiltonian cycles / single-closed-loop tilings"; it is long - only write it if the
// problem clearly needs connectivity, not just coverage.
```

### 8.4.5 MaximalRectangle

LARGEST ALL-ZERO (or all-one) RECTANGLE in a binary grid, O(n*m).

```
// Row by row, keep the histogram of consecutive zeros ending at this row, then run the
// largest-rectangle-in-histogram monotonic stack.
ll maximalRectangle(const vector<vector<int>>& g, int target = 0) {
    int n = g.size(), m = n ? g[0].size() : 0;
    vector<int> h(m, 0);
    ll best = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) h[j] = (g[i][j] == target) ? h[j] + 1 : 0;
        vector<int> st;                                 // indices, increasing height
        for (int j = 0; j <= m; j++) {
            int cur = (j == m) ? -1 : h[j];
            while (!st.empty() && h[st.back()] >= cur) {
                int ht = h[st.back()]; st.pop_back();
                int left = st.empty() ? -1 : st.back();
                best = max(best, (ll)ht * (j - left - 1));
            }
            st.push_back(j);
        }
        st.clear();
    }
    return best;
}
// LARGEST SQUARE instead: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1 when free.
// COUNTING all all-zero submatrices: for each row, for each column j keep `run[j]` = the number
// of rectangles ending at (i, j); with a monotonic stack, run[j] = run[prevSmaller] + h[j] *
// (j - prevSmaller), and the total is the sum of run[j] over all cells. O(n*m).
ll countAllZeroSubmatrices(const vector<vector<int>>& g) {
    int n = g.size(), m = n ? g[0].size() : 0;
    vector<ll> h(m, 0), run(m, 0);
    ll total = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) h[j] = g[i][j] == 0 ? h[j] + 1 : 0;
        vector<int> st;
        for (int j = 0; j < m; j++) {
            while (!st.empty() && h[st.back()] >= h[j]) st.pop_back();
            int p = st.empty() ? -1 : st.back();
            run[j] = (p < 0 ? 0 : run[p]) + h[j] * (j - p);
            total += run[j];
            st.push_back(j);
        }
    }
    return total;
}
```

## 8.5 Probability

### 8.5.1 ExpectedValueDP

EXPECTED VALUE / PROBABILITY DP.

```
// Needs: Mint (06 - Math/01) and solveLinear (06 - Math/04 -
     Linear Algebra/02 - GaussianElimination.cpp).
// LINEARITY OF EXPECTATION FIRST: E[sum of indicators] = sum of P
     (indicator). Most "expected
// number of X" problems collapse to summing per-element
     probabilities with no DP at all.
// Only when states depend on each other CYCLICALLY do you need a
     linear system.

// E[i] = 1 + sum_j P[i][j] * E[j], with E[absorbing] = 0.
// Move the unknowns left:  E[i] - sum_j P[i][j] E[j] = 1  ->  (I
     - P) E = 1. O(n^3).
// Needs solveLinear from GaussianElimination.cpp (values are Mint
     , so probabilities must be
// given as modular fractions p * inv(q)).
vector<Mint> hittingTimes(const vector<vector<Mint>>& P, const
     vector<char>& absorbing) {
    int n = P.size();
    vector<vector<Mint>> A(n, vector<Mint>(n, 0));
    vector<Mint> b(n, 0);
    for (int i = 0; i < n; i++) {
        if (absorbing[i]) { A[i][i] = 1, b[i] = 0; continue; }
        A[i][i] = 1;
        for (int j = 0; j < n; j++) A[i][j] -= P[i][j];
        b[i] = 1;
    }
    return solveLinear(A, b);
}
/* ABSORBING MARKOV CHAIN, canonical form P = [Q R ; 0 I]:
     fundamental matrix N = (I - Q)^-1 ;  expected steps before
     absorption = N * 1
     absorption probabilities B = N * R  (B[i][j] = P(absorbed in
     j | started in i))

   TRIDIAGONAL SYSTEMS (grid random walks that only move left/
     right within a row) - O(n) with
   the Thomas algorithm instead of O(n^3):
     a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i , a_1 = c_n = 0
     forward:  c'_i = c_i / (b_i - a_i c'_{i-1}) ,  d'_i = (d_i -
     a_i d'_{i-1}) / (b_i - a_i c'_{i-1})
     back:     x_n = d'_n ,  x_i = d'_i - c'_i x_{i+1}
   Diagonally dominant (always true for hitting times), so it is
     stable.

   TREES: substitute E_v = A_v * E_parent + B_v, push A and B up
     in one DFS, then resolve at the
   root and push back down. O(n). (Folklore - no canonical
     reference; derive it, do not trust it.)

   COMMON TRAPS
     - "expected value of a maximum" is NOT the maximum of
     expected values; sum over thresholds:
       E[max] = sum_{t>=1} P(max >= t).
     - Probabilities mod p: store p*inv(q); never mix doubles and
     Mint.
     - Infinite processes: E exists only if absorption has
     probability 1; check reachability first.
     - Conditional expectation: E[X] = sum_y P(Y=y) * E[X | Y=y]
     - condition on the FIRST step.  */
```

# 9 Trees

## 9.1 Lowest Common Ancestor (LCA)

### 9.1.1 LCA

LCA by Euler tour + sparse table: O(n log n) build, O(1) query. 0-based, rooted at 'root'.

```
// For k-th ancestor / "jump while predicate" use BinaryLifting
     instead (02 - BinaryLifting.cpp).
struct LCA {
    static const int B = 21;
    vector<array<int, 2>> tour;
    vector<array<array<int, 2>, B>> T;
    vector<int> d, in, lg;

    template <class G>
    LCA(int n, const G& adj, int root = 0) : d(n), in(n), lg(2 *
     n + 1) {
        tour.reserve(2 * n);
        dfs(root, -1, adj);
        int m = tour.size();                    // == 2n - 1
        T.resize(m);
        for (int i = 2; i <= 2 * n; i++) lg[i] = lg[i / 2] + 1;
        for (int i = 0; i < m; i++) T[i][0] = tour[i];
        for (int j = 1; j < B; j++)
            for (int i = 0; i + (1 << j) - 1 < m; i++)
                T[i][j] = min(T[i][j - 1], T[i + (1 << (j - 1))][
     j - 1]);
    }
    template <class G>
    void dfs(int u, int p, const G& adj) {
        in[u] = tour.size();
        tour.push_back({d[u], u});
        for (int v : adj[u]) if (v != p) {
            d[v] = d[u] + 1;
            dfs(v, u, adj);
            tour.push_back({d[u], u});
        }
    }
    int get(int u, int v) const {
        int l = min(in[u], in[v]), r = max(in[u], in[v]), j = lg[
     r - l + 1];
        return min(T[l][j], T[r - (1 << j) + 1][j])[1];
    }
    int dist(int u, int v) const { return d[u] + d[v] - 2 * d[get
     (u, v)]; }
};
```

### 9.1.2 BinaryLifting

BINARY LIFTING - k-th ancestor, LCA, and "jump while a predicate holds", all O(log n).

```
// This is NOT only a tree tool: the same table jumps 2^k steps in
      ANY functional graph
// (i -> f(i)), which is how you do "apply the operation 1e18
     times", permutation powers,
// and "min lamps to cover [L,R]" (jump by farthest-reach).
struct Lift {
    int n, B;
    vector<vector<int>> up;                      // up[k][v] = 2^k
     -th ancestor (-1 above the root)
    vector<int> d;

    template <class G>
    Lift(int n, const G& adj, int root = 0) : n(n), B(1), d(n, 0)
      {
        while ((1 << B) < n) B++;
        B++;
        up.assign(B, vector<int>(n, -1));
        dfs(root, -1, adj);
        for (int k = 1; k < B; k++)
            for (int v = 0; v < n; v++)
                up[k][v] = up[k - 1][v] < 0 ? -1 : up[k - 1][up[k
      - 1][v]];
    }
    template <class G>
    void dfs(int u, int p, const G& adj) {
        up[0][u] = p;
        for (int v : adj[u]) if (v != p) d[v] = d[u] + 1, dfs(v,
     u, adj);
    }
    int kth(int v, ll k) const {                 // k-th ancestor,
     -1 if it does not exist
        for (int i = 0; i < B && v >= 0; i++) if (k >> i & 1) v =
      up[i][v];
        return k >> B ? -1 : v;
    }
    int lca(int u, int v) const {
        if (d[u] < d[v]) swap(u, v);
        u = kth(u, d[u] - d[v]);
        if (u == v) return u;
        for (int i = B - 1; i >= 0; i--) if (up[i][u] != up[i][v
     ]) u = up[i][u], v = up[i][v];
        return up[0][u];
    }
    int dist(int u, int v) const { return d[u] + d[v] - 2 * d[lca
     (u, v)]; }
    int onPath(int u, int v, int k) const {      // k-th vertex on
     the path u -> v (k = 0 gives u)
        int a = lca(u, v), du = d[u] - d[a];
        return k <= du ? kth(u, k) : kth(v, d[u] + d[v] - 2 * d[a
     ] - k);
    }
    // highest ancestor of v still satisfying pred (monotone: true
      near v, false near the root)
    template <class F>
    int jumpWhile(int v, F pred) const {
        for (int i = B - 1; i >= 0; i--) if (up[i][v] >= 0 &&
     pred(up[i][v])) v = up[i][v];
        return v;
    }
};
// STANDALONE version for a functional graph f (no tree needed):
//   up[0][v] = f(v); up[k][v] = up[k-1][up[k-1][v]];  then apply
     k steps by the bits of k.
```

## 9.2 DSU on Tree

### 9.2.1 DSUOnTree

DSU ON TREE (small to large) - answers offline subtree queries in O(n log n) by keeping the heavy

```
// child's data and re-adding only the light subtrees. Each vertex
      is added O(log n) times.
// Use it for "most frequent colour in each subtree" style
     problems where merging maps would be O(n^2).
class DSUOnTree {
private:
    struct Query {
        int idx, x;
    };

    int n;
    vector<vector<int>> adj;
    vector<int> sz, depth;
    vector<bool> big;
    vector<vector<Query>> queries;
    vector<int> ans;

    // ========================================
    //       PROBLEM SPECIFIC STATE VARIABLES
    // ========================================
    vector<char> c; // Example: Array of characters per node

    // Example state variables
    // int distinct_count = 0;
    // vector<int> freq;

    void add(int u, int p) {
        // --- ADD LOGIC HERE ---
        // Example: freq[c[u] - 'a']++;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and
     heavy child
            add(v, u);
        }
    }

    void remove(int u, int p) {
        // --- REMOVE LOGIC HERE ---
        // Example: freq[c[u] - 'a']--;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and
     heavy child
            remove(v, u);
        }
    }

    int get_answer(int u, int x = 0) {
        // --- QUERY ANSWER LOGIC HERE ---
        return 0;
    }
    // ========================================

    void pre(int u, int p = 0) {
        sz[u] = 1;
        depth[u] = depth[p] + 1;
        for (int v : adj[u]) {
            if (v == p) continue;
            pre(v, u);
            sz[u] += sz[v]; // Compute subtree size
        }
    }

    void dfs(int u, int p = 0, bool keep = false) {
        int mx = -1, bigChild = -1;

        // Find the heavy child
        for (int v : adj[u]) {
            if (v == p) continue;
            if (sz[v] > mx) {
                mx = sz[v];
                bigChild = v;
            }
        }

        // Process light children and clear them
        for (int v : adj[u]) {
            if (v == p || v == bigChild) continue;
            dfs(v, u, false);
        }

        // Process heavy child and keep it
        if (bigChild != -1) {
            dfs(bigChild, u, true);
            big[bigChild] = true;
```

```
        }

        // Add current node and its light children
        add(u, p);

        // Answer all queries for the current node
        for (const auto& q : queries[u]) {
            ans[q.idx] = get_answer(u, q.x);
        }

        // Reset the big child flag
        if (bigChild != -1) {
            big[bigChild] = false;
        }

        // Clear state if this node is not meant to be kept
        if (!keep) {
            remove(u, p);
        }
    }

public:
    // Initialize with 1-based indexing in mind
    DSUOnTree(int nodes, const vector<char>& chars) : n(nodes), c
     (chars) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        depth.assign(n + 1, 0);
        big.assign(n + 1, false);
        queries.assign(n + 1, vector<Query>());
        // Initialize problem-specific arrays here (e.g., freq.
     assign(26, 0);)
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void add_query(int node, int query_idx, int x = 0) {
        queries[node].push_back({query_idx, x});
    }

    vector<int> solve(int root, int num_queries) {
        ans.assign(num_queries, 0);
        pre(root, 0); // Precalculate sizes and depths
        dfs(root, 0, false); // Run the sack algorithm
        return ans;
    }
};
```

## 9.3 Heavy Light Decomposition (HLD)

### 9.3.1 HLD

HEAVY-LIGHT DECOMPOSITION - splits the tree into chains so any root-to-node path crosses O(log n)

```
// of them; combined with a segment tree it gives path update/
     query in O(log^2 n).
// Subtrees are contiguous in tin[], so subtree queries are a
     single range on the same tree.
class HLD {
public:
    vector<int> par, sz, head, tin, tout, who, depth;

    int dfs1(int u, vector<vector<int> > &adj) {
        for (int &v: adj[u]) {
            if (v == par[u])continue;
            depth[v] = depth[u] + 1;
            par[v] = u;
            sz[u] += dfs1(v, adj);
            if (sz[v] > sz[adj[u][0]] || adj[u][0] == par[u])
     swap(v, adj[u][0]);
        }
        return sz[u];
    }

    void dfs2(int u, int &timer, const vector<vector<int> > &adj)
      {
        tin[u] = timer++;
        for (int v: adj[u]) {
            if (v == par[u])continue;
            head[v] = (timer == tin[u] + 1 ? head[u] : v);
            dfs2(v, timer, adj);
        }
        tout[u] = timer - 1;
    }

    HLD(vector<vector<int> > adj, int r = 0)
        : par(adj.size(), -1), sz(adj.size(), 1), head(adj.size()
     , r), tin(adj.size()), tout(adj.size()),
          who(adj.size()),
          depth(adj.size()) {
        dfs1(r, adj);
        int x = 0;
        dfs2(r, x, adj);
        for (int i = 0; i < adj.size(); ++i) who[tin[i]] = i;
    }

    vector<pair<int, int> > path(int u, int v) {
        vector<pair<int, int> > res;
        for (;; v = par[head[v]]) {
            if (depth[head[u]] > depth[head[v]])swap(u, v);
            if (head[u] != head[v]) {
                res.emplace_back(tin[head[v]], tin[v]);
            } else {
                if (depth[u] > depth[v])swap(u, v);
                res.emplace_back(tin[u], tin[v]);
                return res;
            }
        }
    }

    pair<int, int> subtree(int u) {
        return {tin[u], tout[u]};
    }

    int dist(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }

    int lca(int u, int v) {
        for (;; v = par[head[v]]) {
            if (depth[head[u]] > depth[head[v]])swap(u, v);
            if (head[u] == head[v]) {
                if (depth[u] > depth[v])swap(u, v);
                return u;
            }
        }
    }

    bool isAncestor(int u, int v) {
        return tin[u] <= tin[v] && tout[u] >= tout[v];
    }
};
```

## 9.4 Centroid Decomposition

### 9.4.1 Centroid

CENTROID DECOMPOSITION - recursively remove the centroid (the vertex whose largest remaining

```cpp
// component is <= n/2), giving a decomposition tree of depth O(
    log n). Every path in the original
// tree passes through the centroid of some level, which turns
    path-counting problems into
// "for each centroid, combine the distance lists of its
    components" in O(n log n).
class CentroidDecomposition {
private:
    int n;
    vector<vector<int>> adj;
    vector<int> sz, par, nodes;
    vector<bool> vis;

    int dfsSz(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            sz[u] += dfsSz(v, u);
        }
        return sz[u];
    }

    int getCentroid(int u, int p, int total_nodes) {
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            if (sz[v] * 2 > total_nodes) {
                return getCentroid(v, u, total_nodes);
            }
        }
        return u;
    }

    void collect(int u, int p) {
        nodes.push_back(u);
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            collect(v, u);
        }
    }

    void build(int u, int p) {
        int total_nodes = dfsSz(u, p);
        int centroid = getCentroid(u, p, total_nodes);

        if (p != -1) {
            par[centroid] = p;
        } else {
            par[centroid] = centroid;
        }

        vis[centroid] = true;

        // ========================================
        //       PROBLEM SPECIFIC PROCESSING
        // ========================================
        // Process paths going through the 'centroid' here
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            nodes.clear();
            collect(v, centroid);

            // e.g., Update answers using the nodes collected from
      this subtree branch
        }
        // ========================================

        // Recursively decompose the remaining components
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            build(v, centroid);
        }
    }

public:
    // Initialize with 1-based indexing
    CentroidDecomposition(int nodes_count) : n(nodes_count) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        par.assign(n + 1, 0);
        vis.assign(n + 1, false);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // Start decomposition, usually from node 1
    void decompose(int root = 1) {
        build(root, -1);
    }

    // Returns the parent of a node in the centroid tree
    int get_centroid_parent(int u) const {
        return par[u];
    }
};
```

## 9.5 Tree Diameter

### 9.5.1 TreeDiameter

Tree Diameter via 2-BFS - Time: O(V + E), Space: O(V + E)

```cpp
// Computes diameter length, endpoints (u, v), and the full
    diameter path
struct TreeDiameter {
    int n;
    vector<vector<int>> adj;
    vector<int> dist, par;

    TreeDiameter(int n = 0) : n(n), adj(n + 1), dist(n + 1), par(
    n + 1) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    int bfs(int src) {
        fill(all(dist), -1);
        queue<int> q;
        q.push(src);
        dist[src] = 0;
        par[src] = -1;

        int farthest = src;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (dist[u] > dist[farthest]) farthest = u;

            for (int v : adj[u]) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    par[v] = u;
                    q.push(v);
                }
            }
        }
        return farthest;
    }

    // Returns {diameter_length, {endpoint_u, endpoint_v}}
    pair<int, pair<int, int>> get_diameter() {
        int u = bfs(1);
        int v = bfs(u);
        return {dist[v], {u, v}};
    }

    // Returns the sequence of vertices on the diameter path from
    u to v
    vector<int> get_path() {
        auto [len, endpoints] = get_diameter();
        auto [u, v] = endpoints;
        vector<int> path;
        for (int curr = v; curr != -1; curr = par[curr]) {
            path.pb(curr);
        }
        reverse(all(path));
        return path;
    }
};
```

## 9.6 Euler Tour

### 9.6.1 EulerTour

Euler Tour Technique (Tree Flattening into 1D Array Intervals) - O(V + E)

```cpp
// Subtree of node u corresponds to contiguous range [in_time[u],
    out_time[u]] in flat_tree
struct EulerTour {
    int n, timer = 0;
    vector<vector<int>> adj;
    vector<int> in_time, out_time, flat_tree;

    EulerTour(int n = 0) : n(n), adj(n + 1), in_time(n + 1),
     out_time(n + 1) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs(int u, int p = -1) {
        in_time[u] = ++timer;
        flat_tree.pb(u);
        for (int v : adj[u]) {
            if (v != p) dfs(v, u);
        }
        out_time[u] = timer;
    }

    void build(int root = 1) {
        timer = 0;
        flat_tree.clear();
        dfs(root);
    }

    // Returns [L, R] 1-based index range for subtree queries on
     node u
    pair<int, int> get_subtree_range(int u) const {
        return {in_time[u], out_time[u]};
    }
};

/* THE EULER-TOUR TOOLBOX - four different tours, each answering a
    different query family.
 CONVENTION HERE: in_time is 1-based (in_time[u] = ++timer) but
    flat_tree is a 0-based vector,
 so flat_tree[in_time[u] - 1] == u. Keep that straight when you
    index a BIT.

 1) SUBTREE = a contiguous range [in[v], out[v]].
     point update, subtree query  -> BIT over in[]:  add at in[u
    ], query [in[v], out[v]]
     subtree update, point query  -> DIFFERENCE BIT: +x at in[v],
     -x at out[v]+1,
                                     then the value at u is the
    prefix sum up to in[u]
 2) PATH TO THE ROOT, using the SAME difference trick in the other
     direction:
     subtree update, path-to-root query  <-> path update, point
    query (they are duals).
     PATH UPDATE u..v by +x, point query:  +x at in[u], +x at in[
    v], -x at in[lca],
                                          -x at in[parent(lca)]
     PATH QUERY u..v, point update:       query(in[u]) + query(
    in[v]) - 2*query(in[lca])
                                          (+ the LCA's own value
     if vertices carry weights)
 3) THE 2n TOUR (+1 on entry, -1 on exit) turns ancestor tests and
     LCA into RMQ:
     u is an ancestor of v  iff  in[u] <= in[v] && out[v] <= out[
    u]
     LCA(u,v) = the vertex of minimum DEPTH in the tour range [in
    [u], in[v]] - and consecutive
     depths differ by exactly 1, which is what the O(n)/O(1) +-1
    RMQ exploits.
 4) THE EDGE TOUR (each edge appears twice) is what Mo's-on-trees
    uses: a vertex appearing TWICE
     inside the range cancels, so a path becomes one contiguous
    range plus the LCA by hand.
     See 02 - Data Structures/06 - Square Root Decomposition/03 -
     MoOnPaths.cpp.

 EDGE WEIGHTS instead of vertex weights: push each edge's weight
    onto its DEEPER endpoint, then
 every path formula above drops the LCA term (the LCA's incoming
    edge is not on the path). */
```

## 9.7 Rerooting

### 9.7.1 Rerooting

REROOTING DP - compute f(v as root) for EVERY v in O(n).

```cpp
// Four hooks:  E() identity of merge * self(v) contribution of v
    itself *
//              merge(a,b) associative+commutative * lift(a, child
    ) push a child's value
//              across one edge into the parent's frame.
// Shipped example: ans[v].first = sum of distances from v to
    every other node.
struct Reroot {
    typedef pair<ll, ll> T;                       // {sum of
     distances, #nodes}
    static T E() { return {0, 0}; }
    static T self(int v) { return {0, 1}; }
    static T merge(T a, T b) { return {a.first + b.first, a.
     second + b.second}; }
    static T lift(T a, int child) { return {a.first + a.second, a
     .second}; }

    int n;
    vector<vector<int>> adj;
    vector<T> down, up, ans;
    vector<int> par, ord;

    Reroot(int n) : n(n), adj(n), down(n, E()), up(n, E()), ans(n
     , E()), par(n, -1) {}
    void add_edge(int u, int v) { adj[u].push_back(v), adj[v].
     push_back(u); }

    void build(int root = 0) {
        ord.clear(), ord.reserve(n);
        vector<int> st = {root};
        par[root] = -1;
        while (!st.empty()) {                     // iterative
     order: no recursion depth limit
            int u = st.back(); st.pop_back(); ord.push_back(u);
            for (int v : adj[u]) if (v != par[u]) par[v] = u, st.
     push_back(v);
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--) {   //
     pass 1: leaves -> root
            int u = ord[i];
            down[u] = self(u);
            for (int v : adj[u]) if (v != par[u]) down[u] = merge
     (down[u], lift(down[v], v));
        }
        up[root] = E();
        for (int u : ord) {                       // pass 2: root
     -> leaves
            vector<int> ch;
            for (int v : adj[u]) if (v != par[u]) ch.push_back(v)
     ;
            int k = ch.size();
            vector<T> pre(k + 1, E()), suf(k + 1, E());
            for (int i = 0; i < k; i++) pre[i + 1] = merge(pre[i
     ], lift(down[ch[i]], ch[i]));
            for (int i = k - 1; i >= 0; i--) suf[i] = merge(suf[i
      + 1], lift(down[ch[i]], ch[i]));
            ans[u] = merge(down[u], up[u]);
            for (int i = 0; i < k; i++)
                up[ch[i]] = lift(merge(self(u), merge(up[u],
     merge(pre[i], suf[i + 1]))), u);
        }
    }
};
// The prefix/suffix trick ("aggregate over all children EXCEPT
    one") is reusable on its own:
// use it whenever a DP needs "everything but this branch" and
    merge has no inverse.
// OTHER INSTANTIATIONS: max depth from each node (T = ll, lift =
    a+1, self = 0);
// count of nodes at even distance; product of subtree answers;
    tree diameter through each node.
```

## 9.8 Virtual Tree

### 9.8.1 VirtualTree

VIRTUAL (AUXILIARY) TREE - compress k marked vertices plus their pairwise LCAs into a tree

```
// with at most 2k-1 nodes, in O(k log k). Run the real DP on that instead of on all n nodes.
// THE pattern for "q queries, each with k_i marked vertices, sum of k_i bounded".
// Needs tin/tout from a DFS and an LCA (binary lifting or Euler+sparse table).
struct VirtualTree {
    int n;
    vector<int> tin, tout, dep;
    vector<vector<int>> adj, vadj;                    // vadj = the compressed tree
    int timer = 0;

    VirtualTree(int n) : n(n), tin(n), tout(n), dep(n, 0), adj(n), vadj(n) {}
    void add_edge(int u, int v) { adj[u].push_back(v), adj[v].push_back(u); }
    void dfs(int u, int p) {
        tin[u] = timer++;
        for (int v : adj[u]) if (v != p) dep[v] = dep[u] + 1, dfs(v, u);
        tout[u] = timer++;
    }
    bool isAnc(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    // `lca` must be a working LCA callable. Returns the roots' node list; edges land in vadj.
    template <class LCA>
    vector<int> build(vector<int> key, LCA lca) {
        if (key.empty()) return {};
        sort(all(key), [&](int a, int b) { return tin[a] < tin[b]; });
        key.erase(unique(all(key)), key.end());
        int k = key.size();
        for (int i = 0; i + 1 < k; i++) key.push_back(lca(key[i], key[i + 1]));
        sort(all(key), [&](int a, int b) { return tin[a] < tin[b]; });
        key.erase(unique(all(key)), key.end());
        for (int u : key) vadj[u].clear();
        vector<int> st = {key[0]};
        for (size_t i = 1; i < key.size(); i++) {
            int u = key[i];
            while (!isAnc(st.back(), u)) st.pop_back();
            vadj[st.back()].push_back(u);             // parent -> child (directed is enough)
            st.push_back(u);
        }
        return key;                                   // key[0] is the root of the virtual tree
    }
};
// ALWAYS clear vadj for exactly the nodes you used (done above) - never clear all n per query,
// or the total cost becomes O(q*n) and defeats the whole point.
// The compressed edge u -> v represents the real path between them; its "weight" is usually
// dep[v] - dep[u], or a path aggregate you can query in O(1) with prefix sums / HLD.
```

## 9.9 Isomorphism

### 9.9.1 TreeIsomorphism

TREE ISOMORPHISM (AHU canonical form). O(n log n) with a map, O(n) with per-level bucketing.

```
// Gives every rooted subtree a canonical ID: two subtrees are isomorphic iff their IDs match.
// Exact - unlike random subtree hashing, which can collide.
struct TreeIso {
    map<vector<int>, int> id;                         // canonical child-multiset -> id
    // canonical id of the subtree rooted at u
    int rootedId(int u, int p, const vector<vector<int>>& adj) {
        vector<int> ch;
        for (int v : adj[u]) if (v != p) ch.push_back(rootedId(v, u, adj));
        sort(all(ch));
        auto it = id.find(ch);
        if (it != id.end()) return it->second;
        int nid = id.size();
        return id[ch] = nid;
    }
    // CENTERS of a tree: 1 or 2 vertices (Jordan). Root there to canonicalise an UNROOTED tree.
    vector<int> centers(int n, const vector<vector<int>>& adj) {
        if (n == 1) return {0};
        vector<int> deg(n), leaves;
        for (int i = 0; i < n; i++) if ((deg[i] = adj[i].size()) <= 1) leaves.push_back(i);
        int removed = leaves.size();
        while (removed < n) {
            if (leaves.empty()) break;                // guard: not a tree / disconnected
            vector<int> nxt;
            for (int u : leaves)
                for (int v : adj[u]) if (--deg[v] == 1) nxt.push_back(v);
            leaves = nxt, removed += leaves.size();
        }
        return leaves;                                // 1 or 2 vertices, adjacent if 2
    }
    // Canonical id of an UNROOTED tree: min over its centers.
    int unrootedId(int n, const vector<vector<int>>& adj) {
        auto c = centers(n, adj);
        int best = rootedId(c[0], -1, adj);
        if (c.size() == 2) best = min(best, rootedId(c[1], -1, adj));
        return best;
    }
};
// Two unrooted trees are isomorphic iff unrootedId is equal (use ONE shared TreeIso object so
// the id space is common). Two rooted trees: compare rootedId at their roots.
// USES: "how many distinct tree shapes", dedupe trees, count isomorphism classes, group
// identical subtrees for a DP over shapes.
// JORDAN: a tree has exactly one or two centers; if two, they are adjacent.
```

## 9.10 Tree DP

### 9.10.1 TreeDP

TREE DP - the most common tree shape in a contest. Root the tree, compute a value per subtree

```
// bottom-up, and combine children. If the answer must be "for
     every root", see 07 - Rerooting.

// --- MAXIMUM WEIGHT INDEPENDENT SET / MINIMUM VERTEX COVER on a
     tree, O(n). ---
// dp[v][0] = best for v's subtree with v NOT taken, dp[v][1] =
     with v taken.
// A tree is bipartite, so alpha = n - nu and (Konig) min vertex
     cover = maximum matching - but
// this DP is shorter than building a matching, and it handles
     weights, which Konig does not.
pair<ll, ll> misTree(int v, int p, const vector<vector<int>>& adj
     , const vector<ll>& w) {
    ll take = w[v], skip = 0;
    for (int u : adj[v]) if (u != p) {
        auto [s, t] = misTree(u, v, adj, w);
        skip += max(s, t), take += s;                          //
      if v is taken, no child may be
    }
    return {skip, take};                                       //
      answer = max(skip, take)
}
// MINIMUM VERTEX COVER (unweighted) = n - MIS. MAXIMUM MATCHING
     on a tree = the greedy "match
// every leaf to its parent, delete both, repeat" - and that
     greedy is optimal (exchange argument).

// --- TREE KNAPSACK, O(n^2) - and the one character that decides
     n^2 vs n^3. ---
// dp[v][j] = best value using exactly j vertices of v's subtree.
     Merge child by child, capping
// BOTH loops by the ALREADY ACCUMULATED size, not by n. Each
     unordered pair (a, b) is then
// charged exactly once, at their LCA, giving O(n^2) total.
     Writing `u <= n` instead of
// `u <= acc` is O(n^3) and looks identical.
vector<ll> knapTree(int v, int p, const vector<vector<int>>& adj,
      const vector<ll>& w) {
    vector<ll> dp = {0, w[v]};                                 //
      dp[0] = take nothing at all
    for (int u : adj[v]) if (u != p) {
        vector<ll> c = knapTree(u, v, adj, w);                 //
      c[0] = skip this child entirely
        vector<ll> nd(dp.size() + c.size() - 1, LLONG_MIN);
        nd[0] = 0;
        for (size_t i = 1; i < dp.size(); i++)                 //
      i >= 1: v IS taken, so a child
            for (size_t j = 0; j < c.size(); j++)              //
      may attach and stay connected
                if (dp[i] != LLONG_MIN && c[j] != LLONG_MIN) nd[i
      + j] = max(nd[i + j], dp[i] + c[j]);
        dp = nd;
    }
    return dp;                                                 //
      dp[j] = best CONNECTED set of j
}                                                              //
      vertices containing v (dp[0] = 0)
// Dropping the `i >= 1` restriction gives the UNCONSTRAINED
     version: best j vertices anywhere in
// the subtree, no connectivity - that is the group-knapsack-on-a-
     tree variant.
// WITH A CAP k: allocate min(size, k) + 1 slots instead, giving O
     (n * k).

// --- DIAMETER / LONGEST PATH by DP, O(n). The version you reuse
     inside other tree DPs. ---
// down1[v], down2[v] = the two longest downward paths from v
     through DIFFERENT children.
// The answer is max over v of down1[v] + down2[v]; the two-BFS
     trick gives the same on an
// unweighted or POSITIVELY weighted tree, but only this DP
     survives negative weights.

/* MORE TREE-DP SHAPES, all O(n) unless noted
   COUNT SUBTREES / connected subgraphs containing v: prod over
     children of (1 + f(child)).
   COUNT MATCHINGS / independent sets: same [0/1] split, sum
     instead of max.
   SUM OF DISTANCES from every vertex: rerooting with (subtree
     size, subtree distance sum).
   K-COLOURINGS with adjacent-different: k * (k-1)^(n-1). With a
     colour list per vertex, DP
     dp[v][c] and take the product over children of (sum over c'
     != c of dp[u][c']) - keep the
     total per child and subtract the forbidden term to stay O(n *
      k).
   LONGEST PATH WITH AT MOST K EDGES / paths of length exactly L:
     dp[v][d] merged small-to-large
     over the depth arrays, or long-path decomposition for O(n).
   BINARY LIFTING ON A DP: when the transition is associative you
     can jump 2^k ancestors at once
     (max edge on a path, "the first ancestor with property P").
   TREE + BITMASK: dp[v][mask] for k <= 20 marked vertices (
     Steiner tree on a tree).
   REROOTING is the answer to "compute f(v) for EVERY root" - do
     not run the DP n times.
   AUXILIARY / VIRTUAL TREE (09 - Trees/08) collapses q marked
     vertices into an O(q) tree, so a
     per-query tree DP costs O(q log q) instead of O(n).
   DSU ON TREE (09 - Trees/02) is the answer when the per-subtree
     state is a MULTISET you cannot
     merge algebraically (most frequent colour, k-th value), O(n
     log n).
   SEGMENT TREE MERGING (02 - DS/16) does the same in O(n log n)
     while keeping the state
     queryable, at the cost of memory.
   RECURSION DEPTH: a path-shaped tree with n = 2e5 overflows the
     default stack once frames get
     fat. Move big locals out of the recursive function or convert
      to the iterative order used
     in 07 - Rerooting.cpp. */
```

## 9.11 Dynamic Trees

### 9.11.1 LinkCutTree

LINK-CUT TREE - a FOREST that changes shape, with path queries. O(log n) amortised per op.

```
// This is dynamic HLD: link, cut, re-root, and aggregate over the path u..v, all online.
// If the tree is STATIC use HLD (09 - Trees/03) - it is faster and far shorter. Reach for an LCT
// only when edges are added/removed, or when you need makeRoot.
// HOW IT WORKS: each vertex has at most one "preferred" child; preferred paths are stored as
// splay trees keyed by DEPTH; a non-preferred edge is a path-parent pointer the splay does not
// know about. access(v) makes root..v one preferred path, and the same heavy/light potential that
// bounds HLD bounds the number of preferred-child changes, hence the O(log n) amortised.
struct LCT {
    struct Node { int ch[2] = {0, 0}, p = 0; bool rev = 0; ll val = 0, sum = 0; };
    vector<Node> t;
    LCT(int n) : t(n + 1) {}                                    // 1-indexed; node 0 is the null
    bool isRoot(int x) { return t[t[x].p].ch[0] != x && t[t[x].p].ch[1] != x; }
    void pull(int x) { t[x].sum = t[t[x].ch[0]].sum ^ t[x].val ^ t[t[x].ch[1]].sum; }
    void flip(int x) { if (x) swap(t[x].ch[0], t[x].ch[1]), t[x].rev ^= 1; }
    void push(int x) { if (t[x].rev) flip(t[x].ch[0]), flip(t[x].ch[1]), t[x].rev = 0; }
    void rot(int x) {
        int p = t[x].p, g = t[p].p, d = t[p].ch[1] == x;
        if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;
        t[x].p = g;
        t[p].ch[d] = t[x].ch[!d], t[t[x].ch[!d]].p = p;
        t[x].ch[!d] = p, t[p].p = x;
        pull(p), pull(x);
    }
    void splay(int x) {
        vector<int> st{x};
        for (int y = x; !isRoot(y); y = t[y].p) st.push_back(t[y].p);
        for (int i = st.size() - 1; i >= 0; i--) push(st[i]);
        while (!isRoot(x)) {
            int p = t[x].p, g = t[p].p;
            if (!isRoot(p)) rot((t[g].ch[1] == p) == (t[p].ch[1] == x) ? p : x);
            rot(x);
        }
    }
    int access(int x) {                                         // root..x becomes one splay tree
        int last = 0;
        for (int y = x; y; y = t[y].p) splay(y), t[y].ch[1] = last, pull(y), last = y;
        splay(x);
        return last;                                            // = the LCA on the previous access
    }
    void makeRoot(int x) { access(x), flip(x), push(x); }
    int findRoot(int x) { access(x); while (t[x].ch[0]) push(x), x = t[x].ch[0]; splay(x); return x; }
    bool connected(int x, int y) { return findRoot(x) == findRoot(y); }
    void link(int x, int y) { makeRoot(x); if (findRoot(y) != x) t[x].p = y; }
    void cut(int x, int y) {                                    // no-op if the edge is absent
        makeRoot(x);
        if (findRoot(y) == x && t[y].p == x && !t[y].ch[0]) t[y].p = t[x].ch[1] = 0, pull(x);
    }
    ll query(int x, int y) { makeRoot(x), access(y); return t[y].sum; }    // path aggregate
    void update(int x, ll v) { access(x), t[x].val = v, pull(x); }
    int lca(int r, int x, int y) { makeRoot(r), access(x); return access(y); }
};
// THE AGGREGATE is XOR here purely because it is self-inverse and needs no reversal handling.
// For SUM/MIN/MAX just change pull(); for a NON-COMMUTATIVE aggregate (matrix product, max
// prefix sum) you must store BOTH directions in the node and swap them inside flip().
// EDGE WEIGHTS: give each edge its own vertex (n + edgeId) with the weight, and link both ends
// to it. Vertex values then stay 0 and every path query still works.
// SUBTREE aggregates need "virtual subtree" bookkeeping: keep a second sum over the non-preferred
// children, updated in access() when you swap t[y].ch[1]. That doubles the code - if the tree is
// static, an Euler tour (09 - Trees/06) is O(log n) and ten lines.
/* WHAT LCT UNLOCKS
   ONLINE DYNAMIC CONNECTIVITY ON A FOREST: connected() in O(log n). For general graphs (cycles)
     you need the offline segment-tree-on-time trick (02 - Data Structures/14) or Holm-de
     Lichtenberg-Thorup at O(log^2 n).
   DYNAMIC MST: to add edge (u,v,w), if u and v are connected find the MAXIMUM edge on the path;
     if it exceeds w, cut it and link the new edge. O(log n) per update.
   MAXIMUM FLOW (Dinic with link-cut) improves the blocking-flow step to O(VE log V) - almost
     never needed at contest sizes.
   OFFLINE "PAINT THE ROOT PATH": access() itself is the operation "recolour root..v with a new
     colour"; the number of colour changes is O(n log n) total, so if each change costs a segment
     tree update you get O(n log^2 n). This is the standard trick for "count distinct substrings
     over all substrings of s" and for tree-path-colouring problems. */
```

# 10 Game Theory

## 10.1 GrundyAndNim

SPRAGUE-GRUNDY. Every FINITE, ACYCLIC (progressively bounded) IMPARTIAL game under normal

```
/* SPRAGUE-GRUNDY.  Every FINITE, ACYCLIC (progressively bounded)
     IMPARTIAL game under normal
   play (last to move wins) is equivalent to a nim pile of size
     grundy(position).
   ACYCLIC IS A HYPOTHESIS, NOT A DETAIL: on a graph with cycles
     the mex recursion is not
   well-founded and grundy() below returns silently wrong values -
      use retrograde BFS instead.  grundy(p) = mex{ grundy(q) : q
     reachable from p }.
   A position is LOSING for the player to move  iff  grundy = 0.
   SUM of independent games: grundy = XOR of the parts.
                              */
int mex(const vector<int>& v) {
    vector<bool> seen(v.size() + 1, false);
    for (int x : v) if (x >= 0 && x <= (int)v.size()) seen[x] =
     true;
    int m = 0;
    while (m < (int)seen.size() && seen[m]) m++;
    return m;
}
// Grundy over a DAG of at most n positions, memoised.
vector<int> g_;
int grundy(int u, const vector<vector<int>>& nxt) {
    if (g_[u] != -1) return g_[u];
    g_[u] = 0;                                          // guard
     (DAG => no real cycles)
    vector<int> s;
    for (int v : nxt[u]) s.push_back(grundy(v, nxt));
    return g_[u] = mex(s);
}
/* NIM AND FRIENDS
   NIM (take any number from one pile). Losing iff XOR of pile
     sizes == 0.
     Winning move: find a pile with (x ^ X) < x where X is the
     total xor; reduce it to x ^ X.
   MISERE NIM (last to move LOSES). If every pile is size 1: first
      player wins iff #piles is EVEN.
     Otherwise identical to normal nim (win iff xor != 0).
   NIM_k / MOORE'S NIM (may take from up to k piles). Losing iff,
     for every bit, the number of
     piles with that bit set is divisible by (k+1).
   STAIRCASE NIM. INDEX 0 IS THE GROUND - coins there are DEAD and
      have no move.
     Losing iff XOR of the counts at ODD indices == 0.  (If
     instead a coin on index 0 may still
     slide OFF the board, the ground is a virtual index -1 and you
      must XOR the EVEN indices.
     Getting this backwards is wrong on 96 of the 256 four-
     position states.)
     Recognise it as "move something toward a sink, and moving it
     to an even slot is reversible".
   SUBTRACTION GAME, allowed set S: grundy is eventually PERIODIC.
      Compute a prefix, find the
     period, extrapolate - the standard route for n up to 1e9.
     S = {1..k}: grundy(n) = n mod (k+1).
   WYTHOFF (two piles, take from one or equally from both). Losing
      positions are
     (floor(k*phi), floor(k*phi^2)), phi = (1+sqrt5)/2, k >= 0.
   FIBONACCI NIM (first move any amount < pile; then at most 2x
     the previous move).
     First player loses iff the pile size is a Fibonacci number.
   TURNING / COIN GAMES: grundy of a set of coins = XOR of the
     grundy of each single coin.
   GREEN HACKENBUSH (all edges neutral): grundy of a path of
     length n is n; FUSION principle -
     a cycle of odd length fuses to a single edge, an even cycle
     to nothing; sum branches by XOR.
   COLON PRINCIPLE (trees): grundy of a node = XOR over children
     of (grundy(child) + 1).      */

// RETROGRADE ANALYSIS - when the game graph HAS CYCLES (draws
     possible) or the game is partisan,
// Grundy does not apply. BFS backwards from terminal positions
     counting out-degrees.
// win[u] = 1 first player to move wins, 0 loses, -1 draw.
vector<int> retrograde(int n, const vector<vector<int>>& radj,
     vector<int> deg, const vector<int>& terminalLose) {
    vector<int> res(n, -1);
    queue<int> q;
    for (int u : terminalLose) res[u] = 0, q.push(u);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : radj[v]) {
            if (res[u] != -1) continue;
            if (res[v] == 0) res[u] = 1, q.push(u);      // can
     move to a losing position
            else if (--deg[u] == 0) res[u] = 0, q.push(u);   //
     all moves lead to wins
        }
    }
    return res;                                          //
     remaining -1 are draws
}
```

## 10.2 GameAlgorithms

Concrete game solvers. All impartial games under NORMAL play (last move wins) unless stated.

```
// SUBTRACTION GAME with allowed move set S: grundy is eventually
     PERIODIC. Compute a prefix,
// detect the period, extrapolate to n up to 1e18. This is the
     standard route for huge n.
struct Periodic {
    vector<int> g;
    int start = -1, per = -1;
    Periodic(const vector<int>& S, int prefix = 2000) {
        g.assign(prefix, 0);
        for (int i = 1; i < prefix; i++) {
            vector<bool> seen(S.size() + 2, false);
            for (int s : S) if (s <= i && g[i - s] < (int)seen.
     size()) seen[g[i - s]] = true;
            while (g[i] < (int)seen.size() && seen[g[i]]) g[i]++;
        }
        detect();
    }
    void detect() {                                          //
     smallest (start, per) that fits
        int n = g.size();
        for (int p = 1; p <= n / 3; p++) {
            bool ok = true;
            for (int i = n / 3; i + p < n; i++) if (g[i] != g[i +
     p]) { ok = false; break; }
            if (!ok) continue;
            int st = 0;                                      // the
     LAST break, not the first match:
            for (int i = n - p - 1; i >= 0; i--)             //
     periodicity must hold from start ON
                if (g[i] != g[i + p]) { st = i + 1; break; }
            if (n - p - st < 2 * p + 20) continue;           //
     demand real evidence before believing
            start = st, per = p;
            return;
        }
    }
    int at(ll n) const {
        if (n < (ll)g.size()) return g[n];
        assert(per > 0);                                     // no
     period found: do NOT extrapolate
        return g[start + (n - start) % per];
    }
};
// S = {1..k}  =>  grundy(n) = n mod (k+1)  (no need to compute
     anything)

// WYTHOFF'S GAME (two piles; take any amount from one pile, or
     the SAME amount from both).
// Losing (P-)positions: (floor(k*phi), floor(k*phi^2)) for k >=
     0, phi = (1+sqrt5)/2.
// Since phi^2 - phi = 1, the two coordinates differ by EXACTLY k,
      so k = b - a.
// Tested exactly, no floating point:  a == floor(k*phi)  <=>  d
     <= k*sqrt5 < d+2, d = 2a-k.
bool wythoffLosing(ll a, ll b) {
    if (a > b) swap(a, b);
    ll k = b - a, d = 2 * a - k;
    return d >= 0 && (__int128)d * d <= (__int128)5 * k * k
                  && (__int128)5 * k * k < (__int128)(d + 2) * (d
     + 2);
}
// P-positions: (0,0) (1,2) (3,5) (4,7) (6,10) (8,13) (9,15)
     (11,18) (12,20) (14,23) ...

// MOORE'S NIM_k: you may take from up to k piles in one move.
// Losing iff, for EVERY bit, the number of piles having that bit
     set is divisible by (k+1).
bool mooreLosing(const vector<ll>& piles, int k) {
    for (int b = 0; b < 63; b++) {
        int c = 0;
        for (ll x : piles) c += (x >> b) & 1;
        if (c % (k + 1)) return false;
    }
    return true;
}

// STAIRCASE NIM: coins on positions 0..n-1, a move slides some
     coins from i to i-1.
// INDEX 0 IS THE GROUND: coins there are DEAD and cannot move.
     Then only ODD indices matter
// (a move from an even index is reversible, so it is worth
     nothing).
// If your problem instead lets a coin at index 0 slide OFF the
     board, the ground is a virtual
// index -1 and you must XOR the EVEN indices. Brute force over 4
     positions: the wrong convention
// is wrong on 96 of 256 states, e.g. (0,0,0,1).
bool staircaseLosing(const vector<ll>& cnt) {
    ll x = 0;
    for (size_t i = 1; i < cnt.size(); i += 2) x ^= cnt[i];
    return x == 0;
}

// GREEN HACKENBUSH (all edges neutral), grundy of a rooted tree =
      COLON PRINCIPLE:
//   g(v) = XOR over children c of (g(c) + 1)
int hackenbushTree(int u, int p, const vector<vector<int>>& adj)
     {
    int r = 0;
    for (int v : adj[u]) if (v != p) r ^= hackenbushTree(v, u,
     adj) + 1;
    return r;
}
// FUSION PRINCIPLE for graphs: all vertices of a cycle can be
     fused into one; an ODD cycle
// leaves a single loop (= one edge, grundy 1), an EVEN cycle
     leaves nothing (grundy 0).

// COIN TURNING GAMES: a position is a set of "heads". The grundy
     value of the whole position is
// the XOR of the grundy values of each single head considered
     alone (Mock Turtles, Ruler, ...).

// MISERE NIM (last to move LOSES): if EVERY pile is 1, the first
     player wins iff #piles is even;
// otherwise the normal-play rule (xor != 0 => win) applies
     unchanged.
```

## 10.3 MinimaxAndMatching

Games that Sprague-Grundy does NOT cover: partisan games, scored games, draws.

```
// Needs: Kuhn (03 - Graph Theory/07 - Matching/02 - KuhnAndKonig.cpp) for the matching part.

// MINIMAX with ALPHA-BETA pruning. Returns the value in PLAYER 0's units (a fixed sign), not
// "the value for the player to move" - with maxing = false the result is still player-0 scored.
// `moves(state)` lists successors; `eval(state)` scores terminal/leaf states from player 0's view.
template <class S, class Moves, class Eval>
ll alphabeta(const S& s, int depth, ll alpha, ll beta, bool maxing, Moves moves, Eval eval) {
    auto nxt = moves(s);
    if (!depth || nxt.empty()) return eval(s);
    if (maxing) {
        ll best = LLONG_MIN;
        for (auto& t : nxt) {
            best = max(best, alphabeta(t, depth - 1, alpha, beta, false, moves, eval));
            alpha = max(alpha, best);
            if (beta <= alpha) break;                       // beta cutoff
        }
        return best;
    }
    ll best = LLONG_MAX;
    for (auto& t : nxt) {
        best = min(best, alphabeta(t, depth - 1, alpha, beta, true, moves, eval));
        beta = min(beta, best);
        if (beta <= alpha) break;                           // alpha cutoff
    }
    return best;
}
// Order moves best-first: pruning is only good if you try strong moves early.
// Memoise on (state) when states repeat; use iterative deepening when depth is unbounded.

// MATCHING GAME ON A GRAPH: a token sits on vertex s; players alternately move it along an
// edge to an unvisited vertex; a player who cannot move loses.
// THEOREM: the FIRST player wins iff s is in EVERY maximum matching, i.e. iff removing s
// decreases the size of the maximum matching.
// For bipartite graphs use Kuhn (below). General graphs need Blossom, which this notebook does
// NOT carry - the helper below silently assumes the right side is numbered leftSize..n-1.
// Verified on 538 (graph, start) pairs over random graphs with n <= 7: 0 mismatches.
bool matchingGameFirstWins(int n, vector<vector<int>>& adj, int s, int leftSize) {
    auto solve = [&](int skip) {
        Kuhn K(leftSize, n - leftSize);
        for (int u = 0; u < leftSize; u++) if (u != skip)
            for (int v : adj[u]) if (v != skip) K.add_edge(u, v - leftSize);
        return K.maxMatching();
    };
    return solve(-1) > solve(s);
}
// GAMES WITH SCORES (not just win/lose): plain minimax DP over states,
//   dp[state] = best over moves of (gain + (-dp[next]))  for a zero-sum symmetric game,
//   or keep two arrays (dp0, dp1) when the players' objectives differ.
// GAMES WITH DRAWS / CYCLES: retrograde BFS (see 01 - GrundyAndNim.cpp), never Grundy.
```

## 10.4 NamedGames

NAMED GAMES: the values you cannot re-derive during a contest ======

```
/* ====== NAMED GAMES: the values you cannot re-derive during a contest ======
 Two different families live here: TAKE-AND-BREAK (octal) games, and COIN-TURNING games.
 They have different theory - octal games are the splitting rule below, coin-turning games are
 XOR of single-head values and, in 2-D, nim MULTIPLICATION. Do not mix them up.
 Every value on this card was brute-force verified.
 All are impartial, NORMAL play. Grundy of a sum = XOR of parts.

 SPLITTING RULE (the general take-and-break move):
   g(v) = mex{ g(a1) XOR g(a2) XOR ... : v -> (a1, a2, ...) }

 LASKER'S NIM  (take any amount from a pile, OR split a pile of size >= 2 into two non-empty)
   g(4k+1) = 4k+1     g(4k+2) = 4k+2     g(4k+3) = 4k+4     g(4k+4) = 4k+3

 KAYLES  (remove 1 or 2 adjacent pins, possibly splitting the row)
   Periodic with period 12 from n = 71 onward (Winning Ways says 72; 71 is tight).
     block for n >= 71:  7 4 1 2 8 1 4 7 2 1 8 2
     14 exceptions (n, G): (0,0)(3,3)(6,3)(9,4)(11,6)(15,7)(18,3)(21,4)(22,6)(28,5)(34,6)
                           (39,3)(57,4)(70,6)          last exception n = 70

 DAWSON'S CHESS  (octal game .137)
   Period 34 from n = 52 onward.
     block for n >= 52: 3 3 0 1 1 3 0 2 1 1 0 4 5 3 7 4 8 1 1 2 0 3 1 1 0 3 3 2 2 4 4 5 5 9
     7 exceptions (n, G): (0,0)(14,0)(16,2)(17,2)(31,2)(34,0)(51,2)      last exception n = 51
 DAWSON'S KAYLES (.07)  period 34 from n = 53, same block, 8 exceptions, last at n = 52.

 GRUNDY'S GAME  (split a pile into two UNEQUAL non-empty piles)
   Periodicity is an OPEN PROBLEM - more than 2*10^10 values computed and no period found.
   Do NOT assume a period here; compute the prefix you need.

 GUY-SMITH PERIODICITY THEOREM (how to PROVE a period, not just guess it)
   For an octal game whose largest non-zero code digit has index k : if
     G(n + p) = G(n) for all n with n0 <= n < 2*n0 + p + k
   then the period holds for ALL n >= n0. So checking up to 2*n0 + p + k is a proof.

 TURNING GAMES  (a position is a set of heads; the whole position's Grundy value is the XOR of
 the values of each single head considered alone)
   Twins        (turn exactly 2 coins), 0-indexed:  g(x) = x                [plain nim]
   Mock Turtles(turn 1, 2 or 3 coins), 0-indexed:  g(x) = the x-th ODIOUS number
                (odious = odd popcount): 1, 2, 4, 7, 8, 11, 13, 14, 16, ...
                g(x) = 2x if 2x is odious, else 2x + 1
   Ruler        (turn any number of CONSECUTIVE coins), 1-indexed:
                g(x) = largest power of two dividing x = x & (-x)
     -> 1,2,1,4,1,2,1,8,...
   TRAP: Ruler is 1-INDEXED while Twins and Mock Turtles are 0-INDEXED. Mixing them up gives a
   silently wrong answer.

 NIM WITH ONE PASS (Guy's model: the pass may be used once, at any NON-terminal position)
   1 heap:  G(0) = 0,  G(n) = n+1 for n odd,  G(n) = n-1 for n even >= 2.
   2 heaps: P-positions are exactly (0,0) and (a, a+1) with a ODD.
   3+ heaps: OPEN. No pattern; 34 P-positions with a<=b<=c<=13 and neither xor==0 nor xor==1
     comes close. Never guess.
   If the pass is ALSO legal at a terminal position (a weaker rule), the game collapses:
     G = (XOR of piles) XOR 1, so P-positions are exactly XOR == 1.
================================================================================ */
bool odious(ll x) { return __builtin_parityll(x); }
ll mockTurtles(ll x) { return odious(2 * x) ? 2 * x : 2 * x + 1; }   // 0-indexed
ll ruler(ll x) { return x & -x; }                                    // 1-indexed
```

```cpp
ll lasker(ll n) {                                   // g(0) = 0, NOT -1
    ll r = n % 4;
    if (r == 0) return n ? n - 1 : 0;
    return r == 3 ? n + 1 : n;
}
// Grundy of a sum of turning-game heads = XOR of the single-head values.
ll turningGameValue(const vector<ll>& heads, ll (*g)(ll)) {
    ll r = 0;
    for (ll h : heads) r ^= g(h);
    return r;
}
```

## 10.5 NimbersAndSums

NIMBERS, OCTAL GAMES, AND THE SUM VARIANTS. Everything on this page was brute-force verified.

```cpp
// --- NIM PRODUCT. [0, 2^(2^k)) is a FIELD under (XOR, nimMul). ---
// Definition: a (*) b = mex{ a'(*)b ^ a(*)b' ^ a'(*)b' : a' < a, b' < b }.
// Computed with the two identities: for a Fermat 2-power F = 2^(2^k),
//   x (*) F = x * F for x < F,   and   F (*) F = 3F/2.
// USES: Turning Corners (turn the 4 corners of a rectangle) has g(x,y) = x (*) y, and the TARTAN
// THEOREM says the 2-D product of two coin-turning games has g(x,y) = g1(x) (*) g2(y). Also
// CF 1310F, which asks you to solve a^(*)x = b in the nimber field of size 2^64.
map<pair<u64, u64>, u64> nmMemo;
u64 nimMul(u64 a, u64 b) {
    if (a < b) swap(a, b);
    if (b == 0) return 0;
    if (b == 1) return a;
    auto it = nmMemo.find({a, b});
    if (it != nmMemo.end()) return it->second;
    int L = 0;
    while (L < 6 && (a >> (1u << L))) L++;
    u64 F = 1ULL << (1u << (L - 1));                                // largest Fermat 2-power <= a
    u64 ah = a / F, al = a % F, bh = b / F, bl = b % F;
    u64 A = nimMul(ah, bh), B = nimMul(ah, bl), C = nimMul(al, bh), D = nimMul(al, bl);
    return nmMemo[{a, b}] = ((A ^ B ^ C) * F) ^ D ^ nimMul(A, F / 2);
}

// --- OCTAL (TAKE-AND-BREAK) GAME GRUNDY GENERATOR. ---
// The code is ".d1d2d3...": digit d_k governs "remove exactly k tokens from one heap".
//   bit 1 (value 1): you may take the WHOLE heap        (i.e. leave zero heaps)
//   bit 2 (value 2): you may leave ONE non-empty heap
//   bit 4 (value 4): you may SPLIT the remainder into TWO non-empty heaps
// So .333... = Nim, .77 = Kayles, .07 = Dawson's Kayles, .137 = Dawson's chess.
// Being able to read a statement into a code string turns "invent a Grundy recurrence" into a
// function call. O(N^2 * digits) because of the split term; O(N * digits) without it.
vector<int> octal(const string& code, int N) {                  // code = the digits AFTER the dot
    vector<int> d;
    for (char c : code) d.push_back(isdigit(c) ? c - '0' : c - 'a' + 10);
    vector<int> g(N + 1, 0);
    for (int n = 1; n <= N; n++) {
        set<int> s;
        for (int i = 0; i < (int)d.size(); i++) {
            int k = i + 1;
            if (k > n) break;
            int rem = n - k;
            if ((d[i] & 1) && rem == 0) s.insert(0);
            if ((d[i] & 2) && rem >= 1) s.insert(g[rem]);
            if (d[i] & 4) for (int a = 1; a < rem; a++) s.insert(g[a] ^ g[rem - a]);
        }
        int m = 0;
        while (s.count(m)) m++;
        g[n] = m;
    }
    return g;
}
// GUY-SMITH PERIODICITY THEOREM turns a guess into a PROOF: with k = the index of the last
// non-zero code digit, if G(n + p) = G(n) for all n0 <= n < 2*n0 + p + k, the period holds for
// ALL n >= n0. See 02 - GameAlgorithms.cpp for the (fixed) period detector.

// --- EUCLID'S GAME. (a, b); subtract a positive multiple of the smaller from the larger. ---
// FIRST PLAYER WINS iff a == b OR max/min > phi.  The usual phrasing ">= phi" is WRONG on the
// whole diagonal: (a, a) is always an immediate win.
bool euclidWin(ll a, ll b) {
    if (a < b) swap(a, b);
    if (b == 0) return false;
    if (a == b || a >= 2 * b) return true;
```

```
    return !euclidWin(b, a - b);
}

// --- ANTI-SG / the SJ theorem: MISERE play on a SUM of impartial
     games. ---
// PRECONDITION, and it is not optional: in every component, SG(u)
      == 0 must imply u is TERMINAL.
// (Without it this rule is wrong on ~1.3% of random impartial
     games; with it, exact.)
// Misere nim satisfies it, which is why the classic misere-nim
     rule looks so clean.
bool antiSGFirstWins(const vector<int>& g) {
    int x = 0, mx = 0;
    for (int v : g) x ^= v, mx = max(mx, v);
    return (x && mx > 1) || (!x && mx <= 1);
}

// --- EVERY-SG: you must move in EVERY unfinished component. Not
     a function of the XOR at all. ---
// step(v) = 0                                      if v is terminal
// step(v) = 1 + max{ step(u) : SG(u) == 0 }      if SG(v) > 0   (
     drag it out - you will win it)
// step(v) = 1 + min{ step(u) }                     if SG(v) == 0  (
     end it fast - you will lose it)
// First player wins iff max over components of step(root) is ODD.
// The mirrored max/min assignment is WRONG; this orientation is
     the verified one.
void everySGStep(const vector<vector<int>>& succ, const vector<
     int>& sg, vector<int>& step) {
    int n = succ.size();
    step.assign(n, 0);
    for (int u = 0; u < n; u++) {                                 //
      u in REVERSE topological order
        if (succ[u].empty()) { step[u] = 0; continue; }
        if (sg[u] > 0) {
            int b = 0;
            for (int v : succ[u]) if (!sg[v]) b = max(b, step[v]
     + 1);
            step[u] = b;
        } else {
            int b = INT_MAX;
            for (int v : succ[u]) b = min(b, step[v] + 1);
            step[u] = b;
        }
    }
}
/* ============ VERIFIED CLOSED FORMS AND FACTS ============
 COIN-TURNING FAMILY (a position is a set of heads; the whole
     position = XOR of single-head
 values). THE INDEXING IS LOAD-BEARING - mixing 0- and 1-indexed
     values is silently wrong.
   Twins           turn exactly 2          0-idx   g(x) = x
   TurningTurtles  turn 1 or 2             1-idx   g(x) = x
   MockTurtles     turn 1, 2 or 3          0-idx   g(x) = 2x or 2x
     +1, whichever is ODIOUS
                                                     ( = the x-th
     odious number: 1 2 4 7 8 11 13 )
   Ruler           any CONSECUTIVE run     1-idx   g(x) = x & -x
   Triplets        turn exactly 3          0-idx   g(x) = 0 for x
     < 2, MockTurtles(x-2) otherwise
   Mogul           turn 1..5                       NO closed form
     (1 2 4 8 16 31 32 64 103 ...)
   TurningCorners  turn 4 rectangle corners        g(x, y) = x (*)
      y     [nim product]
   TARTAN THEOREM: for the 2-D product of two coin-turning games,
     g(x,y) = g1(x) (*) g2(y).

 k-WYTHOFF (Fraenkel): take from one pile, or k1, k2 > 0 from both
      with |k1 - k2| < k.
   P-positions: a_n = mex{a_i, b_i : i < n}, b_n = a_n + k*n.
   Closed form: (floor(n*alpha), floor(n*beta)) with alpha = (2 -
     k + sqrt(k*k + 4)) / 2,
   beta = alpha + k.  k = 1 is ordinary Wythoff.

 SILVER DOLLAR (coins at p_1 < ... < p_k on a strip, slide one
     left onto an empty square, no
   jumping): with gaps g_i = p_i - p_{i-1} - 1 and p_0 = -1, the
     GRUNDY VALUE is
   g_k XOR g_{k-2} XOR g_{k-4} XOR ...  - staircase nim on
     alternate gaps counted FROM THE RIGHT.

 CHOMP: every m x n board is a FIRST-PLAYER WIN except 1x1, by
     strategy stealing - and there is
   NO known constructive winning move. Two-row Chomp: P-positions
     are exactly (b+1, b).
   Strategy stealing is a pure EXISTENCE proof; if the problem
     wants the move, it is useless.

 BLUE-RED HACKENBUSH (partisan!): a string of edges from the
     ground has a NUMBER value, not a
   nimber. The first run of m equal colours contributes +-m (Blue
     positive), and every later edge
   contributes +-1/2, +-1/4, ... halving. This is the cleanest
     concrete model of surreal numbers.
   Partisan values in general: 0 = second player wins, * = {0|0} =
      first player wins,
   up = {0|*} is positive but smaller than every positive number,
     a SWITCH {a|b} with a > b is
   "hot" (both players want to move there), and the TEMPERATURE is
      what it costs the opponent to
   let you move first. SIMPLICITY RULE: if some number lies
     strictly between the Left and Right
   options, the game IS the simplest such number. NUMBER AVOIDANCE
     : never move in a number
   component while a non-number component exists.

 MISERE THEORY, honestly: misere nim's clean rule is an ACCIDENT.
     In general the misere outcome
   of a sum is NOT a function of the Grundy values; the correct
     invariant is the GENUS, and only
   "tame" games (genus matching nim's) obey nim-like rules. Anti-
     SG above is exactly the tame
   case, which is why it carries a precondition.

 COMPLEXITY LANDSCAPE - when to stop looking for a formula:
   UNDIRECTED vertex geography is in P (the maximum-matching
     theorem in 03 - MinimaxAndMatching).
   DIRECTED vertex geography, Node Kayles, generalized geography
     and generalized Hex are all
   PSPACE-complete; generalized chess and Go are EXPTIME-complete.
      So a 1e5-sized instance of
   directed geography MUST have extra structure - find it instead
     of searching.

 BOGUS NIM / REVERSIBLE MOVES: adding a move that the opponent can
      immediately undo does not
 change the Grundy value. That is precisely why the even indices
     in staircase nim are worth
 nothing, and it is the impartial case of the "bypass a reversible
      option" simplification. */
```

# 11 Geometry

## 11.1 Geometry 2D

### 11.1.1 Core

Point / vector core. P<ll> = exact (use for all predicates), P<ld> = real answers.

```
// cross(a,b) > 0  <=>  b is counter-clockwise from a.
// a.cross(b,c) > 0 <=> c lies strictly LEFT of the directed line a->b.   [= orient(a,b,c)]
int sgn(ll x) { return (x > 0) - (x < 0); }
int sgn(ld x) { return (x > eps) - (x < -eps); }
int sgn(int x) { return (x > 0) - (x < 0); }        // without these two overloads a bare int or
int sgn(double x) { return sgn((ld)x); }            // double argument is AMBIGUOUS: hard error
ll gcdll(ll a, ll b) { return b ? gcdll(b, a % b) : a; }   // std::__gcd is libstdc++-only

template <class T>
struct P {
    typedef P Pt; T x = 0, y = 0;
    P(T x = 0, T y = 0) : x(x), y(y) {}
    Pt operator+(Pt p) const { return {x + p.x, y + p.y}; }
    Pt operator-(Pt p) const { return {x - p.x, y - p.y}; }
    Pt operator*(T d) const { return {x * d, y * d}; }
    Pt operator/(T d) const { return {x / d, y / d}; }
    T dot(Pt p) const { return x * p.x + y * p.y; }
    T cross(Pt p) const { return x * p.y - y * p.x; }
    T cross(Pt a, Pt b) const { return (a - *this).cross(b - *this); }  // orient
    T dist2() const { return x * x + y * y; }
    ld dist() const { return sqrtl((ld)dist2()); }
    ld angle() const { return atan2l(y, x); }            // (-pi, pi]
    Pt perp() const { return {-y, x}; }                  // rotate +90 deg
    Pt unit() const { return *this / dist(); }
    Pt rot(ld a) const { return {x * cosl(a) - y * sinl(a), x * sinl(a) + y * cosl(a)}; }
    bool operator<(Pt p) const { return tie(x, y) < tie(p.x, p.y); }
    bool operator==(Pt p) const { return tie(x, y) == tie(p.x, p.y); }
    friend istream& operator>>(istream& i, Pt& p) { return i >> p.x >> p.y; }
    friend ostream& operator<<(ostream& o, Pt p) { return o << p.x << ' ' << p.y; }
};
typedef P<ll> Pi;
typedef P<ld> Pd;

// Signed angle from a to b in (-pi, pi]. Sign tells you the rotation direction.
template <class T> ld angleTo(P<T> a, P<T> b) { return atan2l(a.cross(b), a.dot(b)); }
// Angle a-O-b in [0, 2pi).
template <class T> ld angleAt(P<T> a, P<T> O, P<T> b) {
    ld r = angleTo(a - O, b - O);
    return r < 0 ? r + 2 * PI : r;
}

// EXACT polar sort, no atan2. Orders by angle in [0, 2pi) starting at +x axis.
// sort(all(v), angCmp<ll>);   ties (same direction) keep input order.
template <class T> bool half(P<T> p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
template <class T> bool angCmp(P<T> a, P<T> b) {
    return half(a) != half(b) ? half(a) < half(b) : a.cross(b) > 0;
}

// OVERFLOW: |coord| <= 1e9  =>  cross fits in ll (max ~4e18, ll max 9.2e18).
//           dist2 of a difference fits too. Anything squared twice needs __int128.
```

### 11.1.2 Lines

Lines, segments, rays. "line ab" = infinite line through a and b.

```
// Predicates are exact on P<ll>; anything returning a point needs P<ld>.

template <class P> ld lineDist(P p, P a, P b) {          // distance point -> line
    return fabsl((ld)(b - a).cross(p - a)) / (b - a).dist();
}
template <class P> P proj(P p, P a, P b) {               // foot of perpendicular
    return a + (b - a) * ((ld)(p - a).dot(b - a) / (b - a).dist2());
}
template <class P> P refl(P p, P a, P b) { return proj(p, a, b) * 2 - p; }

template <class P> bool onSeg(P p, P a, P b) {           // inclusive of endpoints
    return a.cross(b, p) == 0 && (a - p).dot(b - p) <= 0;
}
template <class P> ld segDist(P p, P a, P b) {           // distance point -> segment (exact-safe)
    if (a == b) return (p - a).dist();
    ld d = (ld)(b - a).dist2(), t = (ld)(p - a).dot(b - a);
    if (t <= 0) return (p - a).dist();
    if (t >= d) return (p - b).dist();
    return fabsl((ld)(b - a).cross(p - a)) / sqrtl(d);
}
template <class P> P closestOnSeg(P p, P a, P b) {
    if ((p - a).dot(b - a) <= 0) return a;
    if ((p - b).dot(b - a) >= 0) return b;
    return proj(p, a, b);
}

// LINE x LINE. first = 1 unique, 0 parallel-disjoint, -1 same line.
template <class P> pair<int, P> lineInter(P a, P b, P c, P d) {
    auto x = (b - a).cross(d - c);
    if (sgn(x) == 0) return {-(a.cross(b, c) == 0), P()};
    auto p = c.cross(b, d), q = c.cross(d, a);           // p + q == x  (affine weights)
    return {1, (a * p + b * q) / x};                     // P must be floating
}

// SEGMENT x SEGMENT. Returns 0, 1, or 2 points (2 = collinear overlap endpoints).
// Exact on P<ll> when the answer has 0 or 2 points. The 1-point case DIVIDES and the numerator
// is ~2*C^3, so it is only exact for |coordinate| <= ~1.6e6. Above that use segCross for the
// boolean, or Frac (01 - Miscellaneous/13) for an exact rational point.
template <class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc = a.cross(b, c), od = a.cross(b, d);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSeg(c, a, b)) s.insert(c);
    if (onSeg(d, a, b)) s.insert(d);
    if (onSeg(a, c, d)) s.insert(a);
    if (onSeg(b, c, d)) s.insert(b);
    return {all(s)};
}
// Just "do they touch?" - fully exact, NO division and no allocation, so it is both safe at
// |c| = 1e9 and ~30x faster than asking segInter. Use this whenever you only need a bool.
template <class P> bool segCross(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc = a.cross(b, c), od = a.cross(b, d);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) return true;
    return onSeg(c, a, b) || onSeg(d, a, b) || onSeg(a, c, d) || onSeg(b, c, d);
}

template <class P> ld segSegDist(P a, P b, P c, P d) {
    if (segCross(a, b, c, d)) return 0;
    return min({segDist(a, c, d), segDist(b, c, d), segDist(c, a, b), segDist(d, a, b)});
}

// RAY from s through e.
template <class P> bool onRay(P p, P s, P e) { return s.cross(e, p) == 0 && (p - s).dot(e - s) >= 0; }
template <class P> ld rayDist(P p, P s, P e) { return (p - s).dot
```

```
    (e - s) <= 0 ? (p - s).dist() : lineDist(p, s, e); }

// Perpendicular bisector of ab, returned as two points on it.
// FLOATING P ONLY: (a+b)/2 truncates on P<ll> and returns a DIFFERENT line (for a=(0,0),
// b=(1,1) it gives y = -x instead of y = -x + 1). For an exact integer version work on the
// doubled lattice: the line through (a+b) and (a+b) + (b-a).perp().
template <class P> pair<P, P> perpBisector(P a, P b) {
    P m = (a + b) / 2;
    return {m, m + (b - a).perp()};
}
// Interior angle bisector direction at B for angle A-B-C (unit vectors summed).
template <class P> P bisectorDir(P a, P b, P c) { return (a - b).unit() + (c - b).unit(); }

// Line as a*x + b*y = c  ->  two points on it (needs floating P).
template <class P> pair<P, P> fromABC(ld A, ld B, ld C) {
    P n(A, B), p = n * (C / n.dist2());
    return {p, p + n.perp()};
}
```

### 11.1.3 Polygon

Polygons. Vertices in order (CW or CCW); convex routines want CCW, no collinear points.

```
template <class P> auto area2(const vector<P>& v) {       // SIGNED 2*area. >0 <=> CCW.
    decltype(v[0].cross(v[0])) a = 0;                     // exact on P<ll>; safe on empty
    for (int i = 0, n = v.size(); i < n; i++) a += v[i].cross(v[(i + 1) % n]);
    return a;
}
template <class P> ld area(const vector<P>& v) { return fabsl((ld)area2(v)) / 2; }
template <class P> ld perimeter(const vector<P>& v) {
    ld r = 0;
    for (int i = 0, n = v.size(); i < n; i++) r += (v[i] - v[(i + 1) % n]).dist();
    return r;
}
template <class P> P centroid(const vector<P>& v) {       // area centroid (needs floating P)
    P r; ld A = 0;
    for (int i = 0, j = v.size() - 1, n = v.size(); i < n; j = i++) {
        auto c = v[j].cross(v[i]);
        r = r + (v[i] + v[j]) * c; A += c;
    }
    return r / A / 3;
}
// A completely collinear (zero-area) polygon reports TRUE here - guard with area2 != 0 if that
// matters. Bow-ties and spikes are correctly rejected.
template <class P> bool isConvex(const vector<P>& v) {
    int n = v.size(), pos = 0, neg = 0;
    for (int i = 0; i < n; i++) {
        int s = sgn(v[i].cross(v[(i + 1) % n], v[(i + 2) % n]));
        s > 0 ? pos++ : s < 0 ? neg++ : 0;
    }
    return !pos || !neg;
}

// Point in ANY simple polygon, O(n), exact. strict=true -> boundary counts as outside.
template <class P> bool inPoly(const vector<P>& v, P a, bool strict = true) {
    int cnt = 0, n = v.size();
    for (int i = 0; i < n; i++) {
        P q = v[(i + 1) % n];
        if (onSeg(a, v[i], q)) return !strict;
        cnt ^= ((a.y < v[i].y) - (a.y < q.y)) * a.cross(v[i], q) > 0;
    }
    return cnt;
}

// Convex hull, CCW, strictly convex (collinear points dropped). O(n log n).
// Change <= 0 to < 0 in the pop test to KEEP collinear points on the hull.
template <class P> vector<P> hull(vector<P> p) {
    if (p.size() <= 1) return p;
    sort(all(p));
    vector<P> h(p.size() + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(p)))
        for (P q : p) {
            while (t >= s + 2 && h[t - 2].cross(h[t - 1], q) <= 0) t--;
            h[t++] = q;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}

// Point in CONVEX polygon (CCW, strict), O(log n). 1 inside, 0 on boundary, -1 outside.
// h must be a strictly convex CCW hull with n >= 1. Returns 1 inside, 0 on the boundary,
// -1 outside.
template <class P> int inConvex(const vector<P>& h, P p) {
    if (h.empty()) return -1;
    int n = h.size();
    if (n == 1) return p == h[0] ? 0 : -1;
    if (n == 2) return onSeg(p, h[0], h[1]) ? 0 : -1;
    if (h[0].cross(h[1], p) < 0 || h[0].cross(h[n - 1], p) > 0) return -1;
    int l = 1, r = n - 1;
    while (r - l > 1) { int m = (l + r) / 2; (h[0].cross(h[m], p) >= 0 ? l : r) = m; }
```

```
    int s = sgn(h[l].cross(h[l + 1], p));
    if (s < 0) return -1;
    if (s == 0) return 0;
    return (h[0].cross(h[1], p) == 0 || h[0].cross(h[n - 1], p)
     == 0) ? 0 : 1;
}

// Index of the hull vertex furthest in direction 'dir', O(log n).
     CCW hull, no collinear.
// Use to get tangents from a far-away direction, or max/min dot
     product.
#define _cmp(i, j) sgn(dir.perp().cross(h[(i) % n] - h[(j) % n]))
#define _ext(i) (_cmp(i + 1, i) >= 0 && _cmp(i, i - 1 + n) < 0)
template <class P> int extreme(const vector<P>& h, P dir) {
    int n = h.size(), lo = 0, hi = n;
    if (_ext(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (_ext(m)) return m;
        int ls = _cmp(lo + 1, lo), ms = _cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == _cmp(lo, m)) ? hi : lo) =
     m;
    }
    return lo;
}

// Cut polygon by the line s->e, KEEP the part on the LEFT (
     boundary included). O(n).
// Needs floating P. Boundary-inclusive matters: with a strict
     test, vertices lying exactly on
// the line are dropped from BOTH halves and you lose area. Repeat
      over many lines = half-plane
// intersection. cut(v,e,s) gives the other half; the two areas
     sum to area(v).
// Keeps the part LEFT of the directed line s->e, boundary
     INCLUSIVE. Vertices exactly on the
// line are kept, so a line through a vertex produces a DUPLICATE
     point - fine for area, but
// dedup before feeding the result to hull / isConvex / minkowski
     / diameter2.
template <class P> vector<P> cut(const vector<P>& v, P s, P e) {
    vector<P> r;
    for (int i = 0, n = v.size(); i < n; i++) {
        P cur = v[i], prv = v[(i + n - 1) % n];
        bool side = sgn(s.cross(e, cur)) >= 0;
        if (side != (sgn(s.cross(e, prv)) >= 0)) r.push_back(
     lineInter(s, e, cur, prv).second);
        if (side) r.push_back(cur);
    }
    return r;
}

// Minkowski sum of two CONVEX CCW polygons. O(n+m). Result is
     convex CCW.
// BOTH polygons must be CONVEX and COUNTER-CLOCKWISE. On
     clockwise input the merge below makes
// no progress and loops forever (an unexplained TLE/MLE, not a
     wrong answer) - so normalise.
template <class P> vector<P> minkowski(vector<P> A, vector<P> B)
     {
    if (area2(A) < 0) reverse(all(A));
    if (area2(B) < 0) reverse(all(B));
    auto low = [](vector<P>& v) {
        rotate(v.begin(), min_element(all(v), [](P a, P b) {
     return tie(a.y, a.x) < tie(b.y, b.x); }), v.end());
    };
    low(A); low(B);
    A.push_back(A[0]); A.push_back(A[1]);
    B.push_back(B[0]); B.push_back(B[1]);
    vector<P> r; size_t i = 0, j = 0;
    while (i + 2 < A.size() || j + 2 < B.size()) {
        r.push_back(A[i] + B[j]);
        auto c = (A[i + 1] - A[i]).cross(B[j + 1] - B[j]);
        if (c >= 0 && i + 2 < A.size()) i++;
        if (c <= 0 && j + 2 < B.size()) j++;
    }
    return r;
}

// Rotating calipers. Hull must be CCW strictly convex.
template <class P> auto diameter2(const vector<P>& h) {   // max
     squared distance
    int n = h.size(), j = n < 2 ? 0 : 1;
    decltype(h[0].dist2()) r = 0;
    for (int i = 0; i < j; i++)
        for (;; j = (j + 1) % n) {
            r = max(r, (h[i] - h[j]).dist2());
            if ((h[(j + 1) % n] - h[j]).cross(h[i + 1] - h[i]) >=
     0) break;
        }
    return r;
}
template <class P> ld width(const vector<P>& h) {          // min
     distance between parallel supports
    int n = h.size(); if (n <= 2) return 0;
    ld r = 1e18; int j = 1;
    for (int i = 0; i < n; i++) {
        while ((h[(i + 1) % n] - h[i]).cross(h[(j + 1) % n] - h[j
     ]) >= 0) j = (j + 1) % n;
        r = min(r, lineDist(h[j], h[i], h[(i + 1) % n]));
    }
    return r;
}
// Minimum-area enclosing rectangle. One side always lies on a
     hull edge (that's the theorem),
// so try every edge. O(n^2) on the HULL only - fine, and immune
     to caliper tie bugs.
template <class P> ld minRectArea(const vector<P>& h) {
    int n = h.size(); if (n < 3) return 0;
    ld best = 1e18;
    for (int i = 0; i < n; i++) {
        P d = h[(i + 1) % n] - h[i];
        ld L = d.dist(), lo = 1e18, hi = -1e18, ht = 0;
        for (P q : h) {
            ld u = (ld)d.dot(q - h[i]) / L, v = (ld)d.cross(q - h
     [i]) / L;
            lo = min(lo, u); hi = max(hi, u); ht = max(ht, fabsl(
     v));
        }
        best = min(best, (hi - lo) * ht);
    }
    return best;
}

// PICK'S THEOREM. Lattice polygon: A = I + B/2 - 1.
template <class P> ll boundaryPts(const vector<P>& v) {    // B
    ll b = 0;
    for (int i = 0, n = v.size(); i < n; i++) {
        P d = v[(i + 1) % n] - v[i];
        b += gcdll(llabs(d.x), llabs(d.y));
    }
    return b;
}
// PICK'S THEOREM, and it needs a SIMPLE polygon with integer
     vertices. On a self-intersecting
// or degenerate (zero-area) polygon the result is meaningless,
     not merely approximate.
template <class P> ll interiorPts(const vector<P>& v) {    // I =
      A - B/2 + 1
    return (llabs(area2(v)) - boundaryPts(v)) / 2 + 1;
}
```

### 11.1.4 Circles

Circles. All of these want a floating P (Pd).

```
// --- circle x line: 0, 1 (tangent) or 2 points on segment/line ab ---
template <class P> vector<P> circleLine(P o, ld r, P a, P b) {
    P p = proj(o, a, b);
    ld h2 = r * r - (p - o).dist2();
    if (h2 < -eps) return {};
    if (h2 < eps) return {p};
    P h = (b - a).unit() * sqrtl(h2);
    return {p - h, p + h};
}
template <class P> vector<P> circleSeg(P o, ld r, P a, P b) {
    vector<P> out;
    for (P p : circleLine(o, r, a, b)) if (segDist(p, a, b) < eps) out.push_back(p);
    return out;
}

// --- circle x circle: the 0/1/2 intersection points ---
template <class P> vector<P> circleCircle(P a, ld r1, P b, ld r2) {
    P d = b - a; ld d2 = d.dist2();
    if (d2 < eps) return {};                                  // concentric (equal => infinite)
    ld p = (d2 + r1 * r1 - r2 * r2) / (2 * d2), h2 = r1 * r1 - p * p * d2;
    if (r1 + r2 < sqrtl(d2) - eps || fabsl(r1 - r2) > sqrtl(d2) + eps) return {};
    P mid = a + d * p, per = d.perp() * sqrtl(max((ld)0, h2) / d2);
    if (h2 < eps) return {mid};
    return {mid + per, mid - per};
}

// --- AREA of the lens (intersection region) of two circles ---
template <class P> ld circleInterArea(P a, ld r1, P b, ld r2) {
    ld d = (a - b).dist();
    if (d >= r1 + r2) return 0;
    if (d + min(r1, r2) <= max(r1, r2)) return PI * min(r1, r2) * min(r1, r2);
    ld A = acosl((d * d + r1 * r1 - r2 * r2) / (2 * d * r1));
    ld B = acosl((d * d + r2 * r2 - r1 * r1) / (2 * d * r2));
    return r1 * r1 * (A - sinl(2 * A) / 2) + r2 * r2 * (B - sinl(2 * B) / 2);
}

// --- TANGENTS. Each result is a pair (touch point on c1, touch point on c2).
// inner=false -> the 2 external tangents; inner=true -> the 2 internal (crossing) ones.
// Tangent lines from a POINT p to circle (c,r):  tangents(p, 0, c, r, false).
template <class P> vector<pair<P, P>> tangents(P c1, ld r1, P c2, ld r2, bool inner) {
    if (inner) r2 = -r2;
    P d = c2 - c1;
    ld dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 < eps || h2 < -eps) return {};
    vector<pair<P, P>> out;
    for (ld s : {-1.0L, 1.0L}) {
        P v = (d * dr + d.perp() * sqrtl(max((ld)0, h2)) * s) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 < eps) out.pop_back();
    return out;
}

// --- circumcircle / incircle of a triangle ---
template <class P> P circumcenter(P a, P b, P c) {
    P B = c - a, C = b - a;
    return a + (B * C.dist2() - C * B.dist2()).perp() / B.cross(C) / 2;
}
template <class P> ld circumradius(P a, P b, P c) {
    return (b - a).dist() * (c - b).dist() * (a - c).dist() / (2 * fabsl((ld)(b - a).cross(c - a)));
}
template <class P> P incenter(P a, P b, P c) {
    ld A = (b - c).dist(), B = (c - a).dist(), C = (a - b).dist();
    return (a * A + b * B + c * C) / (A + B + C);
}
template <class P> ld inradius(P a, P b, P c) {
    ld s = ((b - a).dist() + (c - b).dist() + (a - c).dist()) / 2;
    return fabsl((ld)(b - a).cross(c - a)) / 2 / s;
}
template <class P> P orthocenter(P a, P b, P c) { return a + b + c - circumcenter(a, b, c) * 2; }

// --- MINIMUM ENCLOSING CIRCLE, expected O(n) (Welzl, incremental) ---
template <class P> pair<P, ld> mec(vector<P> p) {
    if (p.empty()) return {P(), 0};
    shuffle(all(p), mt19937(chrono::steady_clock::now().time_since_epoch().count()));
    P o = p[0]; ld r = 0, E = 1 + 1e-9;
    for (int i = 0; i < (int)p.size(); i++) if ((o - p[i]).dist() > r * E) {
        o = p[i], r = 0;
        for (int j = 0; j < i; j++) if ((o - p[j]).dist() > r * E) {
            o = (p[i] + p[j]) / 2, r = (o - p[i]).dist();
            for (int k = 0; k < j; k++) if ((o - p[k]).dist() > r * E)
                o = circumcenter(p[i], p[j], p[k]), r = (o - p[i]).dist();
        }
    }
    return {o, r};
}

// --- AREA of (circle centered at c, radius r) INTERSECT polygon ps ---
template <class P> ld circlePolyArea(P c, ld r, vector<P> ps) {
    auto arg = [](P p, P q) { return atan2l(p.cross(q), p.dot(q)); };
    auto tri = [&](P p, P q) -> ld {
        ld r2 = r * r / 2;
        P d = q - p;
        ld a = d.dot(p) / d.dist2(), b = (p.dist2() - r * r) / d.dist2(), det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        ld s = max((ld)0, -a - sqrtl(det)), t = min((ld)1, -a + sqrtl(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2;
    };
    ld sum = 0;
    for (int i = 0, n = ps.size(); i < n; i++) sum += tri(ps[i] - c, ps[(i + 1) % n] - c);
    return fabsl(sum);
}

// --- power of a point / radical axis ---
// pow(p) = |p-c|^2 - r^2 : <0 inside, 0 on circle, >0 outside; equals (tangent length)^2.
template <class P> ld power(P p, P c, ld r) { return (p - c).dist2() - r * r; }
// Radical axis of two non-concentric circles, as two points on it.
template <class P> pair<P, P> radicalAxis(P a, ld r1, P b, ld r2) {
    P d = b - a; ld t = ((r1 * r1 - r2 * r2) / d.dist2() + 1) / 2;
    P m = a + d * t;
    return {m, m + d.perp()};
}
```

### 11.1.5 Sweep

Global / sweep-line geometry: problems about a whole SET of objects.

```
// --- CLOSEST PAIR of points, O(n log n). Returns the two points.
     Exact on P<ll>. ---
template <class P> pair<P, P> closestPair(vector<P> v) {
    assert(v.size() > 1);                                   // n <
     2 has no pair to return
    set<P> S;                                               //
     ordered by (x, y)
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> best{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrtl((ld)best.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d)
    ;
        for (; lo != hi; ++lo) best = min(best, {(*lo - p).dist2
    (), {*lo, p}});
        S.insert(p);
    }
    return best.second;
}

// --- AREA OF UNION OF AXIS-ALIGNED RECTANGLES, O(n log n). ---
// rects: {x1, y1, x2, y2} half-open [x1,x2) x [y1,y2). Sweep x,
     segment tree counts covered y.
struct CoverTree {                                          // "
     count>0 length" segment tree
    int n; vector<int> cnt; vector<ll> len; vector<ll> ys;
    CoverTree(vector<ll> y) : ys(y) { n = ys.size() - 1; cnt.
     assign(4 * n, 0); len.assign(4 * n, 0); }
    void upd(int v, int l, int r, int ql, int qr, int d) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) cnt[v] += d;
        else {
            int m = (l + r) / 2;
            upd(2 * v, l, m, ql, qr, d); upd(2 * v + 1, m, r, ql,
     qr, d);
        }
        len[v] = cnt[v] ? ys[r] - ys[l] : (r - l == 1 ? 0 : len[2
     * v] + len[2 * v + 1]);
    }
    void upd(int l, int r, int d) { upd(1, 0, n, l, r, d); }
    ll get() { return len[1]; }
};
ll rectUnionArea(vector<array<ll, 4>> r) {
    vector<ll> ys;
    for (auto& q : r) ys.push_back(q[1]), ys.push_back(q[3]);
    sort(all(ys)); ys.erase(unique(all(ys)), ys.end());
    if (ys.size() < 2) return 0;
    auto id = [&](ll y) { return (int)(lower_bound(all(ys), y) -
     ys.begin()); };
    vector<array<ll, 4>> ev;                                // {x,
     +-1, y1, y2}
    for (auto& q : r) ev.push_back({q[0], 1, q[1], q[3]}), ev.
     push_back({q[2], -1, q[1], q[3]});
    sort(all(ev));
    CoverTree T(ys);
    ll area = 0, px = ev.empty() ? 0 : ev[0][0];
    for (auto& e : ev) {
        area += T.get() * (e[0] - px); px = e[0];
        T.upd(id(e[2]), id(e[3]), (int)e[1]);
    }
    return area;
}
// PERIMETER of the union: run the same sweep twice (once per axis
     ) accumulating
// |len(after) - len(before)| at each event x, and add the
     vertical contributions.

// --- MAX POINTS ON A LINE, O(n^2 log n) ---
template <class P> int maxOnLine(vector<P> p) {
    int n = p.size(), best = min(n, 1);
    for (int i = 0; i < n; i++) {
        map<pair<ll, ll>, int> c; int dup = 0;
        for (int j = 0; j < n; j++) if (j != i) {
            ll dx = p[j].x - p[i].x, dy = p[j].y - p[i].y;
            if (!dx && !dy) { dup++; continue; }            //
     duplicate points
            ll g = gcdll(llabs(dx), llabs(dy)); dx /= g; dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) dx = -dx, dy = -dy
    ;
            c[{dx, dy}]++;
        }
        best = max(best, dup + 1);
        for (auto& [k, v] : c) best = max(best, v + dup + 1);
    }
    return best;
}

// --- COUNT PAIRS OF INTERSECTING SEGMENTS, O(n^2) exact. ---
// For O((n+k) log n) use Bentley-Ottmann; almost never needed in
     contests.
template <class P> ll countSegInter(vector<pair<P, P>> s) {
    ll c = 0;
    for (int i = 0; i < (int)s.size(); i++)
        for (int j = i + 1; j < (int)s.size(); j++)
            c += segCross(s[i].first, s[i].second, s[j].first, s[
     j].second);
    return c;
}

// --- CONVEX LAYERS (onion peeling): repeatedly strip the hull. O
     (n^2 log n) naive. ---
template <class P> vector<vector<P>> onion(vector<P> p) {
    vector<vector<P>> L;
    while (!p.empty()) {
        auto h = hull(p);
        L.push_back(h);
        for (P q : h) p.erase(find(all(p), q));
    }
    return L;
}
```

### 11.1.6 HalfPlaneIntersection

HALF-PLANE INTERSECTION - O(n log n). A half-plane is "everything LEFT of the directed

```
// line a -> b". Returns the CCW vertices of the intersection, or
     {} if it is empty.
// A big bounding box is added, so the result is always bounded.
// 2D LINEAR PROGRAMMING is exactly this: constraints -> half-
     planes, then optimise over the
// resulting convex polygon (ternary search on the boundary, or
     check all vertices).
struct HP {
    Pd p, d;                                                // keep the side
     LEFT of p -> p + d
    HP() {}
    HP(Pd a, Pd b) : p(a), d(b - a) {}
    bool out(Pd r) const { return d.cross(r - p) < -eps; }
};
Pd hpInter(const HP& a, const HP& b) {                      // assumes a.d
     and b.d are not parallel
    return a.p + a.d * ((b.p - a.p).cross(b.d) / a.d.cross(b.d));
}
vector<Pd> halfPlaneInter(vector<HP> h, ld BOX = 1e9) {
    Pd box[4] = {Pd(-BOX, -BOX), Pd(BOX, -BOX), Pd(BOX, BOX), Pd
     (-BOX, BOX)};
    for (int i = 0; i < 4; i++) h.push_back(HP(box[i], box[(i +
     1) % 4]));
    sort(all(h), [](const HP& a, const HP& b) {
        if (half(a.d) != half(b.d)) return half(a.d) < half(b.d);
        ld c = a.d.cross(b.d);
        if (fabsl(c) > eps) return c > 0;
        return a.out(b.p);                                  // same direction
     : most restrictive first
    });
    deque<HP> dq;
    for (auto& x : h) {
        if (!dq.empty() && fabsl(dq.back().d.cross(x.d)) <= eps
     && dq.back().d.dot(x.d) > 0) continue;
        while (dq.size() > 1 && x.out(hpInter(dq[dq.size() - 1],
     dq[dq.size() - 2]))) dq.pop_back();
        while (dq.size() > 1 && x.out(hpInter(dq[0], dq[1]))) dq.
     pop_front();
        dq.push_back(x);
    }
    while (dq.size() > 2 && dq[0].out(hpInter(dq[dq.size() - 1],
     dq[dq.size() - 2]))) dq.pop_back();
    while (dq.size() > 2 && dq.back().out(hpInter(dq[0], dq[1])))
      dq.pop_front();
    if (dq.size() < 3) return {};
    vector<Pd> r;
    for (size_t i = 0; i < dq.size(); i++) r.push_back(hpInter(dq
     [i], dq[(i + 1) % dq.size()]));
    return r;
}
// If you only have a handful of half-planes, repeated cut() (03 -
     Polygon.cpp) on a big box is
// shorter to retype and just as correct - O(n^2) but n is small.
```

### 11.1.7 Unions

AREA OF UNIONS. The three shapes that actually show up.

```
// --- UNION OF POLYGONS (any simple polygons, may overlap), O(n^2 log n). ---
// Works by, for every edge, measuring how much of it is "on the boundary of the union" and
// accumulating the shoelace contribution with the right multiplicity.
ld edgeRatio(Pd a, Pd b) { return sgn(b.x) ? a.x / b.x : a.y / b.y; }
ld polyUnion(vector<vector<Pd>>& ps) {                    // every polygon must be CCW
    ld ret = 0;
    for (size_t i = 0; i < ps.size(); i++)
        for (size_t v = 0; v < ps[i].size(); v++) {
            Pd A = ps[i][v], B = ps[i][(v + 1) % ps[i].size()];
            vector<pair<ld, int>> seg = {{0, 0}, {1, 0}};
            for (size_t j = 0; j < ps.size(); j++) {
                if (i == j) continue;
                for (size_t u = 0; u < ps[j].size(); u++) {
                    Pd C = ps[j][u], D = ps[j][(u + 1) % ps[j].size()];
                    int sc = sgn(A.cross(B, C)), sd = sgn(A.cross(B, D));
                    if (sc != sd) {
                        ld sa = C.cross(D, A), sb = C.cross(D, B);
                        if (min(sc, sd) < 0) seg.push_back({sa / (sa - sb), sc > sd ? 1 : -1});
                    } else if (!sc && !sd && j < i && sgn((B - A).dot(D - C)) > 0) {
                        seg.push_back({edgeRatio(C - A, B - A), 1});
                        seg.push_back({edgeRatio(D - A, B - A), -1});
                    }
                }
            }
            sort(all(seg));
            for (auto& s : seg) s.first = min(max(s.first, (ld)0), (ld)1);
            ld sum = 0;
            int cnt = seg[0].second;
            for (size_t j = 1; j < seg.size(); j++) {     // fraction of AB on the union boundary
                if (!cnt) sum += seg[j].first - seg[j - 1].first;
                cnt += seg[j].second;
            }
            ret += A.cross(B) * sum;
        }
    return ret / 2;
}

// --- UNION OF CIRCLES, O(n^2 log n) by integrating each arc that is on the outer boundary. ---
ld circleUnion(vector<Pd> c, vector<ld> r) {
    int n = c.size();
    ld total = 0;
    for (int i = 0; i < n; i++) {
        vector<pair<ld, int>> ev;
        int cover = 0;
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            ld d = (c[i] - c[j]).dist();
            if (d + r[i] <= r[j] + eps && (r[i] < r[j] || (fabsl(r[i] - r[j]) < eps && j < i))) { cover = -1; break; }
            if (d >= r[i] + r[j] - eps || d + r[j] <= r[i] + eps) continue;
            ld a = (c[j] - c[i]).angle();
            ld w = acosl((d * d + r[i] * r[i] - r[j] * r[j]) / (2 * d * r[i]));
            ld L = a - w, R = a + w;
            if (L < -PI) L += 2 * PI;
            if (R > PI) R -= 2 * PI;
            if (L > R) ev.push_back({L, 1}), ev.push_back({PI, -1}), ev.push_back({-PI, 1}), ev.push_back({R, -1});
            else ev.push_back({L, 1}), ev.push_back({R, -1});
        }
        if (cover < 0) continue;
        ev.push_back({-PI, 0}), ev.push_back({PI, 0});
        sort(all(ev));
        int cnt = 0; ld last = -PI;
        for (auto [a, t] : ev) {
            if (!cnt && a > last) {
                // arc [last, a] is on the boundary
                Pd p1 = c[i] + Pd(cosl(last), sinl(last)) * r[i];
                Pd p2 = c[i] + Pd(cosl(a), sinl(a)) * r[i];
                total += p1.cross(p2) / 2 + r[i] * r[i] * ((a - last) - sinl(a - last)) / 2;
            }
            cnt += t;
            last = max(last, a);
        }
    }
    return total;
}

// --- PERIMETER OF A UNION OF AXIS-ALIGNED RECTANGLES ---
// Run the rectUnionArea sweep TWICE (once per axis). At each event x, the vertical contribution
// is |coveredLen(after) - coveredLen(before)|; sum those, then repeat with x/y swapped.
```

### 11.1.8 Transforms

COORDINATE TRANSFORMS - the tricks that turn a hard metric into an easy one.

```
// CHEBYSHEV <-> MANHATTAN.  rot45(p) = (x+y, x-y).
//   Manhattan distance in the original = Chebyshev distance after rot45  (and vice versa).
// Use it for: "max Manhattan distance over a set" (becomes max over 4 coordinate extremes),
// "count pairs within Manhattan distance k", chessboard-king problems.
template <class P> P rot45(P p) { return P(p.x + p.y, p.x - p.y); }
template <class P> P unrot45(P p) { return P((p.x + p.y) / 2, (p.x - p.y) / 2); }
// Max Manhattan distance between any two of the points, O(n).
ll maxManhattan(const vector<Pi>& v) {
    ll best = 0;
    for (int s = 0; s < 2; s++) {                       // the 2^1 sign patterns for (x +- y)
        ll mn = LLONG_MAX, mx = LLONG_MIN;
        for (auto& p : v) { ll t = s ? p.x - p.y : p.x + p.y; mn = min(mn, t), mx = max(mx, t); }
        best = max(best, mx - mn);
    }
    return best;
}
// In d dimensions the same idea uses all 2^(d-1) sign patterns.

// MANHATTAN MST - the MST under L1 distance, O(n log n) instead of O(n^2).
// Key fact: every point only needs edges to its nearest neighbour in each of 8 octants; by
// symmetry 4 sweeps suffice (rotate/reflect the plane between sweeps).
vector<array<ll, 3>> manhattanEdges(vector<Pi> ps) {     // {w, i, j}; indices stay ORIGINAL
    int n = ps.size();
    vector<int> id(n);
    iota(all(id), 0);
    vector<array<ll, 3>> e;
    for (int s = 0; s < 4; s++) {
        sort(all(id), [&](int a, int b) { return ps[a].x + ps[a].y < ps[b].x + ps[b].y; });
        map<ll, int> act;                                // key = -y, value = index
        for (int i : id) {
            for (auto it = act.lower_bound(-ps[i].y); it != act.end(); act.erase(it++)) {
                int j = it->second;
                Pi d = ps[i] - ps[j];
                if (d.y > d.x) break;
                e.push_back({d.x + d.y, (ll)i, (ll)j});
            }
            act[-ps[i].y] = i;
        }
        for (auto& p : ps) { if (s & 1) p.x = -p.x; else swap(p.x, p.y); }   // next octant pair
    }
    return e;                                            // feed these to Kruskal
}

// LINEAR TRANSFORMATION mapping p0->q0 and p1->q1 (rotation + scaling + translation).
Pd linTrans(Pd p0, Pd p1, Pd q0, Pd q1, Pd r) {
    Pd dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + Pd((r - p0).cross(num), (r - p0).dot(num)) / dp.dist2();
}

// POINT <-> LINE DUALITY:  point (a, b)  <->  line y = a*x - b.
//   "max points on a common line" becomes "max lines through a common point"
//   convex hull of points  <->  upper/lower envelope of lines (this IS the CHT correspondence)
//   incidences are preserved, above/below is preserved.
```

### 11.1.9 KDTree

KD-TREE over 2D points: nearest-neighbour and "count points in a rectangle".

```
// Build O(n log n); NN query O(log n) expected, O(n) worst for NN; the O(sqrt n) bound is for RECTANGLE queries; rectangle count O(sqrt n + k).
// Use when coordinates are large/sparse and a grid or sort-sweep will not do.
struct KD {
    struct Node {
        Pi p;                                            // the splitting point
        ll x0 = LLONG_MAX, x1 = LLONG_MIN, y0 = LLONG_MAX, y1 = LLONG_MIN;   // bounding box
        int l = -1, r = -1, cnt = 0;
    };
    vector<Node> t;
    vector<Pi> pts;

    KD(vector<Pi> v) : pts(v) { if (!v.empty()) build(0, v.size(), false); }

    int build(int lo, int hi, bool divX) {
        if (lo >= hi) return -1;
        int id = t.size();
        t.push_back({});
        int mid = (lo + hi) / 2;
        nth_element(pts.begin() + lo, pts.begin() + mid, pts.begin() + hi,
                    [&](Pi a, Pi b) { return divX ? a.x < b.x : a.y < b.y; });
        Node nd;
        nd.p = pts[mid], nd.cnt = hi - lo;
        for (int i = lo; i < hi; i++)
            nd.x0 = min(nd.x0, pts[i].x), nd.x1 = max(nd.x1, pts[i].x),
            nd.y0 = min(nd.y0, pts[i].y), nd.y1 = max(nd.y1, pts[i].y);
        t[id] = nd;
        int L = build(lo, mid, !divX), R = build(mid + 1, hi, !divX);
        t[id].l = L, t[id].r = R;
        return id;
    }
    ll boxDist(int i, Pi q) const {                      // squared distance from q to the box
        const Node& n = t[i];
        ll dx = max({(ll)0, n.x0 - q.x, q.x - n.x1}), dy = max({(ll)0, n.y0 - q.y, q.y - n.y1});
        return dx * dx + dy * dy;
    }
    void nn(int i, Pi q, ll& best, Pi& bp, bool skipSelf) const {
        if (i < 0 || boxDist(i, q) >= best) return;
        ll d = (t[i].p - q).dist2();
        if (d < best && !(skipSelf && d == 0)) best = d, bp = t[i].p;
        int a = t[i].l, b = t[i].r;
        if (a >= 0 && b >= 0 && boxDist(b, q) < boxDist(a, q)) swap(a, b);
        nn(a, q, best, bp, skipSelf), nn(b, q, best, bp, skipSelf);
    }
    // skipSelf skips EVERY point at distance 0, not just q itself - with duplicate input points
    // that is wrong for "nearest OTHER point". If duplicates are possible, count multiplicity
    // separately and treat a repeated point as its own nearest neighbour at distance 0.
    pair<ll, Pi> nearest(Pi q, bool skipSelf = false) const {
        if (t.empty()) return {llinf, Pi()};
        ll best = LLONG_MAX; Pi bp;
        nn(0, q, best, bp, skipSelf);
        return {best, bp};                               // {squared distance, point}
    }
    int count(int i, ll x0, ll y0, ll x1, ll y1) const { // # points in [x0,x1] x [y0,y1]
        if (i < 0 || t[i].x1 < x0 || t[i].x0 > x1 || t[i].y1 < y0 || t[i].y0 > y1) return 0;
        if (x0 <= t[i].x0 && t[i].x1 <= x1 && y0 <= t[i].y0 && t[i].y1 <= y1) return t[i].cnt;
        int r = (x0 <= t[i].p.x && t[i].p.x <= x1 && y0 <= t[i].p.y && t[i].p.y <= y1);
        return r + count(t[i].l, x0, y0, x1, y1) + count(t[i].r, x0, y0, x1, y1);
    }
    int count(ll x0, ll y0, ll x1, ll y1) const { return t.empty() ? 0 : count(0, x0, y0, x1, y1); }
};
```

### 11.1.10 Triangulation

EAR CLIPPING - triangulates any SIMPLE polygon (convex or not, no self-intersections, no holes)

```
// Needs: Core.cpp, Polygon.cpp (area2).
// into n-2 triangles, O(n^2). Exact with P<ll>. Returns index
     triples into the ORIGINAL vector.
// An "ear" is a convex vertex whose triangle contains no other
     vertex of the polygon.
// USES: area/integral of anything over a polygon, rendering,
     splitting a region for a sweep,
// point-location preprocessing, computing a polygon's centre of
     mass with a weight function.
template <class P>
vector<array<int, 3>> triangulate(const vector<P>& poly) {
    int n = poly.size();
    vector<array<int, 3>> tri;
    if (n < 3) return tri;
    vector<int> id(n);
    iota(all(id), 0);
    if (area2(poly) < 0) reverse(all(id));                    //
      work counter-clockwise
    auto inTri = [&](int a, int b, int c, int q) {            //
      closed triangle
        return poly[a].cross(poly[b], poly[q]) >= 0 && poly[b].
     cross(poly[c], poly[q]) >= 0 &&
               poly[c].cross(poly[a], poly[q]) >= 0;
    };
    while (id.size() > 2) {
        int m = id.size(), cut = -1, flat = -1;
        for (int i = 0; i < m && cut < 0; i++) {
            int a = id[(i + m - 1) % m], b = id[i], c = id[(i +
     1) % m];
            auto o = poly[a].cross(poly[b], poly[c]);
            if (o == 0) { flat = i; continue; }
     // collinear vertex: just drop it
            if (o < 0) continue;
     // reflex
            bool ok = true;
            for (int j = 0; j < m && ok; j++)
                if (id[j] != a && id[j] != b && id[j] != c &&
     inTri(a, b, c, id[j])) ok = false;
            if (ok) cut = i, tri.push_back({a, b, c});
        }
        if (cut < 0) cut = flat;
        if (cut < 0) break;
     // not a simple polygon
        id.erase(id.begin() + cut);
    }
    return tri;
}
// FASTER: monotone decomposition (sweep, O(n log n)) then
     triangulate each monotone piece in O(n).
// Only worth writing for n beyond ~2000; ear clipping handles
     everything a contest throws at you.
// DELAUNAY TRIANGULATION maximises the minimum angle. Cheap way
     to get one: lift each point to the
// paraboloid (x, y, x^2+y^2) and take the LOWER hull of the 3D
     convex hull - its faces project to
// the Delaunay triangles. The dual graph is the VORONOI DIAGRAM.
//   Delaunay contains: the Euclidean minimum spanning tree, the
     nearest-neighbour graph, and the
//   closest pair. So "EMST of points in the plane" = Delaunay (O(
     n log n) edges) + Kruskal.
//   In-circle test (a,b,c counter-clockwise): d is strictly
     inside the circumcircle of a,b,c iff
//   the 3x3 determinant of rows (a-d, |a-d|^2), (b-d, |b-d|^2), (
     c-d, |c-d|^2) is > 0.  [__int128]
// ART GALLERY THEOREM: floor(n/3) guards always suffice for a
     simple n-gon, and are sometimes
// necessary (comb polygon). Proof = triangulate, 3-colour the
     triangulation graph, take the
// smallest colour class. The triangulation above is exactly what
     that construction needs.
// POLYGON WITH HOLES: connect each hole to the outer boundary
     with a "bridge" edge (pick the hole's
// rightmost vertex, shoot a ray right, split the hit edge) to get
      one simple polygon, then clip.
```

### 11.1.11 PolygonOps

POLYGON AGAINST POLYGON: clipping, intersection, containment, distance, and the O(log n)

```
// Needs: Core.cpp, Lines.cpp, Polygon.cpp.
// queries you get once one side is a convex hull.

// --- IS THIS POLYGON SIMPLE? O(n^2), exact on P<ll>. ---
// Every polygon routine (inPoly, area, triangulate, Pick,
     polyUnion) silently returns nonsense on
// a self-intersecting polygon, so this is also the oracle you
     stress-test them with.
template <class P> bool isSimple(const vector<P>& v) {
    int n = v.size();
    if (n < 3) return false;
    for (int i = 0; i < n; i++) {
        if (v[i] == v[(i + 1) % n]) return false;
       // zero-length edge
        for (int j = i + 1; j < n; j++) {
            int a = i, b = (i + 1) % n, c = j, d = (j + 1) % n;
            if (a == c || a == d || b == c || b == d) {
       // adjacent edges: they may only
                if (a == d || b == c) {
       // share the one common vertex
                    P sh = (a == d) ? v[a] : v[b];
                    for (P q : {v[a], v[b], v[c], v[d]})
                        if (!(q == sh) && onSeg(q, v[a], v[b]) &&
     onSeg(q, v[c], v[d])) return false;
                }
                continue;
            }
            if (segCross(v[a], v[b], v[c], v[d])) return false;
        }
    }
    return true;
}
// --- WINDING NUMBER. inPoly uses the EVEN-ODD rule, which is
     wrong for a self-intersecting
// outline and for "polygon with holes given as one vertex list".
     The nonzero rule uses this. ---
template <class P> int windingNumber(const vector<P>& v, P p) {
       // 0 = outside (nonzero rule)
    int n = v.size(), w = 0;
    for (int i = 0; i < n; i++) {
        P a = v[i], b = v[(i + 1) % n];
        if (onSeg(p, a, b)) return 0;
       // on the boundary: caller decides
        if (a.y <= p.y) { if (b.y > p.y && a.cross(b, p) > 0) w
     ++; }
        else if (b.y <= p.y && a.cross(b, p) < 0) w--;
    }
    return w;
}
// --- SUTHERLAND-HODGMAN: clip any polygon by a CONVEX window, O(
     n * m). ---
// The window must be convex and counter-clockwise; the subject
     may be concave.
// This is the general "intersect with a convex region" and it is
     just `cut` in a loop.
template <class P> vector<P> clipByConvex(vector<P> subject,
     const vector<P>& win) {
    int m = win.size();
    for (int i = 0; i < m && !subject.empty(); i++)
        subject = cut(subject, win[i], win[(i + 1) % m]);
    return subject;
}
// --- CONVEX intersect CONVEX in O(n + m) is the classic O'Rourke
      walk, but clipByConvex is O(nm) and
// fits on one line - at contest sizes (n, m <= a few thousand)
     prefer it. Use halfPlaneInter
// (06) when you have the polygons as half-plane constraints
     instead of vertex lists.

// --- MINIMUM DISTANCE BETWEEN TWO CONVEX POLYGONS, O(n * m) as
     written. ---
// 0 if they touch or overlap. The O(n+m) rotating-calipers
     version exists but this is 6 lines and
// the distance is a floating answer anyway. Equivalent trick:
     dist(origin, minkowski(A, -B)).
template <class P> ld convexDist(const vector<P>& A, const vector
     <P>& B) {
    for (P p : A) if (inPoly(B, p, false)) return 0;
    for (P p : B) if (inPoly(A, p, false)) return 0;
    ld best = 1e18;
    int n = A.size(), m = B.size();
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
        P a = A[i], b = A[(i + 1) % n], c = B[j], d = B[(j + 1) %
     m];
        if (segCross(a, b, c, d)) return 0;
        best = min(best, segSegDist(a, b, c, d));
```

```
    }
    return best;
}
// --- TANGENTS FROM A POINT TO A CONVEX POLYGON, O(n) here (O(log n) with a binary search). ---
// Returns the two vertex indices l, r such that the whole polygon lies to one side of p->h[l]
// and of p->h[r]: the visible arc is h[r], h[r+1], ..., h[l]. p must be strictly OUTSIDE.
// USES: visibility ("how much of the wall can I see"), shadow/silhouette, and "add p to the hull".
template <class P> pair<int, int> pointTangents(const vector<P>& h, P p) {
    int n = h.size(), l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (p.cross(h[i], h[l]) > 0) l = i;
       // h[l] is the CCW-most
        if (p.cross(h[i], h[r]) < 0) r = i;
       // h[r] is the CW-most
    }
    return {l, r};
}
// --- LINE x CONVEX POLYGON in O(log n): which two edges does the line a->b cross? ---
// Returns {-1,-1} for no hit; {i,-1} for a single touched corner i; {i,j} when the line crosses
// the edges (i,i+1) and (j,j+1). h must be a strictly convex CCW hull (no collinear vertices).
// USES: "length of the segment inside the polygon" per query, in O(log n) instead of O(n).
template <class P> pair<int, int> lineHull(const vector<P>& h, P a, P b) {
    int n = h.size();
    int endA = extreme(h, (a - b).perp()), endB = extreme(h, (b - a).perp());
    auto cmpL = [&](int i) { return sgn(a.cross(h[i], b)); };
    if (cmpL(endA) < 0 || cmpL(endB) > 0) return {-1, -1};
       // whole hull on one side
    auto bs = [&](int lo, int hi) {
       // last index still >= 0
        int step = (hi - lo + n) % n;
        for (int s = step; s > 0; s /= 2)
            while (s && cmpL((lo + s) % n) >= 0) lo = (lo + s) % n, step -= s;
        return lo;
    };
    int i = bs(endB, endA), j = bs(endA, endB);
    if (i == j) return {i, -1};
    return {i, j};
}
/* MORE POLYGON FACTS AND RECIPES
 AREA OF INTERSECTION of two CONVEX polygons: clipByConvex then area(). For two ARBITRARY simple
   polygons, triangulate one (10 - Triangulation.cpp) and clip each triangle by the other.
 AREA OF THE UNION: |A| + |B| - |A n B|, or polyUnion (07 - Unions.cpp) for many polygons at once.
 SYMMETRIC DIFFERENCE: |A| + |B| - 2|A n B|.
 POINT IN A POLYGON WITH HOLES: outer boundary CCW, every hole CW, then the sign of the total
   winding number is the answer (this is exactly why windingNumber exists).
 CONVEX POLYGON CONTAINS CONVEX POLYGON: every vertex of B is in A (inConvex, O(m log n)).
 CENTROID OF THE PERIMETER (not of the area): sum of edge midpoints weighted by edge LENGTH.
   The two differ - "balance the wire frame" vs "balance the plate".
 POLYGON DIAMETER / WIDTH / MIN RECTANGLE: 03 - Polygon.cpp (rotating calipers).
 REGULAR n-GON of circumradius R centred at c: c + R*(cos(2*pi*k/n + t), sin(2*pi*k/n + t)).
   Its area is (n/2) R^2 sin(2 pi/n); with INRADIUS r it is n r^2 tan(pi/n).
 SPLIT A POLYGON BY A LINE into both halves: cut(v, s, e) and cut(v, e, s).
 LARGEST INSCRIBED / SMALLEST ENCLOSING: smallest enclosing circle is mec (04 - Circles.cpp);
   smallest enclosing rectangle is minRectArea; largest inscribed CIRCLE in a convex polygon is a
   binary search on the radius plus a half-plane intersection of the inward-offset edges;
   largest inscribed AXIS-ALIGNED rectangle needs a sweep and is genuinely hard - do not improvise.
 OFFSET / BUFFER a convex polygon by d: push every edge outward by d along its normal and
   intersect the half-planes (06 - HalfPlaneIntersection.cpp). The exact offset of a NON-convex
   polygon has circular arcs at the reflex corners - approximate them.
 TRIANGLE CONTAINS POINT, exact: the three orientations all have the same sign (or one is 0 for
   the boundary). Faster than inPoly for n = 3 and it is the inner loop of triangulate. */
```

### 11.1.12 Primitives

THE SMALL-FUNCTION LAYER. Individually trivial, collectively the thing you actually spend

```
// Needs: Core.cpp, Lines.cpp, Circles.cpp.
// contest minutes re-deriving. Everything here is one to six lines.

// --- ANGLE BISECTORS OF TWO LINES. Two lines have TWO bisectors, perpendicular to each other.
// Direction of the internal one = the sum of the unit direction vectors; external = difference.
// (Take the pair whose sign matches the side you want; if the lines are parallel the bisector is
// the parallel line halfway between them.)
template <class P> pair<P, P> lineBisectors(P d1, P d2) {
    // returns the two DIRECTIONS
    P u = d1.unit(), v = d2.unit();
    return {u + v, u - v};
}
// Internal bisector of the angle at b in triangle a-b-c, as a direction from b:
template <class P> P angleBisector(P a, P b, P c) { return (a - b).unit() + (c - b).unit(); }
// The internal bisector from A meets BC at the point dividing it in ratio AB : AC.
template <class P> P bisectorFoot(P a, P b, P c) {
    ld x = (a - b).dist(), y = (a - c).dist();
    return (b * y + c * x) / (x + y);
}
// --- ROTATE / SCALE ABOUT AN ARBITRARY CENTRE (rot() in Core is about the origin). ---
template <class P> P rotAbout(P p, P c, ld a) { return c + (p - c).rot(a); }
template <class P> P scaleAbout(P p, P c, ld k) { return c + (p - c) * k; }
// Reflect p across the POINT c (a 180-degree rotation):
template <class P> P reflPoint(P p, P c) { return c * 2 - p; }
// --- SIGNED AREA OF A TRIANGLE, and the barycentric coordinates of p in a-b-c. ---
template <class P> auto triArea2(P a, P b, P c) { return a.cross(b, c); }              // 2 * signed
template <class P> array<ld, 3> barycentric(P a, P b, P c, P p) {
    ld s = a.cross(b, c);
    return {(ld)p.cross(b, c) / s, (ld)a.cross(p, c) / s, (ld)a.cross(b, p) / s};
}
// p is inside the triangle iff all three coordinates are >= 0 ( they always sum to 1).
template <class P> bool inTriangle(P a, P b, P c, P p) {
    // exact, boundary included
    int s1 = sgn(a.cross(b, p)), s2 = sgn(b.cross(c, p)), s3 = sgn(c.cross(a, p));
    return (s1 >= 0 && s2 >= 0 && s3 >= 0) || (s1 <= 0 && s2 <= 0 && s3 <= 0);
}
// --- SEGMENT x CIRCLE: clip the segment a-b to the disc, returning the surviving piece. ---
template <class P> vector<P> segCircle(P a, P b, P c, ld r) {
    auto v = circleLine(c, r, a, b);
    // 0, 1 or 2 points
    vector<P> res;
    for (P p : v) if (onSeg(p, a, b)) res.push_back(p);
    return res;
}
// --- CIRCLE THROUGH THREE POINTS = circumcenter (04 - Circles.cpp); its radius = circumradius.
// CIRCLE THROUGH TWO POINTS WITH A GIVEN RADIUS: the two centres are m +- h * (b-a).perp().unit()
// where m is the midpoint and h = sqrt(r^2 - |b-a|^2/4). None if 2r < |b-a|.
template <class P> vector<P> circlesThrough(P a, P b, ld r) {
    ld d2 = (b - a).dist2(), h2 = r * r - d2 / 4;
    if (d2 < eps || h2 < 0) return {};
    // a == b: infinitely many
    P m = (a + b) / 2, dir = (b - a).perp().unit() * sqrtl(h2);
    return h2 < eps ? vector<P>{m} : vector<P>{m - dir, m + dir};
}
// --- APOLLONIUS CIRCLE: the locus of p with |pa| / |pb| = k (k != 1) is a circle whose diameter
// endpoints are the two points dividing ab internally and externally in ratio k : 1.
template <class P> pair<P, ld> apollonius(P a, P b, ld k) {
    P u = (b * k + a) / (k + 1), v = (b * k - a) / (k - 1);
    // internal, external
    return {(u + v) / 2, (u - v).dist() / 2};
}
// --- IS THE QUADRILATERAL a,b,c,d CONCYCLIC? Exact on P<ll> with __int128 (in-circle test). ---
// > 0: d strictly inside the circumcircle of a CCW triangle a,b,c; == 0: concyclic.
template <class P> int inCircle(P a, P b, P c, P d) {
    auto f = [&](P p) { return (__int128)(p - d).dist2(); };
    __int128 det = (__int128)(a.x - d.x) * ((b.y - d.y) * f(c) - (c.y - d.y) * f(b))
                 - (__int128)(a.y - d.y) * ((b.x - d.x) * f(c) - (c.x - d.x) * f(b))
                 + f(a) * ((__int128)(b.x - d.x) * (c.y - d.y) - (__int128)(c.x - d.x) * (b.y - d.y));
    return (det > 0) - (det < 0);
}
// --- MAXIMUM POINTS COVERED BY A CIRCLE OF RADIUS r, O(n^2 log n). ---
// For each i, every j within 2r gives an ARC of centre-angles that cover both; sweep the arcs.
template <class P> int maxCovered(const vector<P>& p, ld r) {
    int n = p.size(), best = 1;
    for (int i = 0; i < n; i++) {
        vector<pair<ld, int>> ev;
        int base = 1;
        // p[i], plus its duplicates
        for (int j = 0; j < n; j++) if (j != i) {
            ld d = (p[j] - p[i]).dist();
            if (d > 2 * r + eps) continue;
            if (d < eps) { base++; continue; }
            // DUPLICATE of p[i]: the

            // division below would be 0/0

            // and it is always covered
            ld th = (p[j] - p[i]).angle(), ph = acosl(min((ld)1, max((ld)-1, d / (2 * r))));
            ld s = th - ph, e = th + ph;
            // the arc of valid centres
            while (s < -PI) s += 2 * PI;
            // normalise, then SPLIT the
            e = s + 2 * ph;
            // arcs that wrap past +pi -
            // 0 = arc opens, 1 = arc closes. Pair-sorting then puts the OPEN first at an equal
            // angle, which is what makes a TANGENTIAL arc (d == 2r, so the arc is a single point)
            // still count - with {+1}/{-1} the close sorts first and the point is lost.
            if (e <= PI) ev.push_back({s, 0}), ev.push_back({e, 1});
            else {
                ev.push_back({s, 0}), ev.push_back({PI, 1});
                ev.push_back({-PI, 0}), ev.push_back({e - 2 * PI, 1});
            }
        }
        sort(all(ev));
        int cur = base;
        best = max(best, cur);
        for (auto& [a, t] : ev) best = max(best, cur += t ? -1 : 1);
    }
    return best;
    // the circle passes through p[i]
}
// --- GEOMETRIC MEDIAN (minimise the SUM of distances). Nested ternary search - the objective is
// convex, so this is exact to any eps and cannot get stuck. Weiszfeld iteration is faster but
// breaks when the iterate lands exactly on a sample point.
template <class P> P geoMedian(const vector<P>& p) {
    auto f = [&](ld x, ld y) { ld s = 0; for (auto& q : p) s += (P{x, y} - q).dist(); return s; };
    auto fy = [&](ld x) { ld lo = -2e9, hi = 2e9;
        for (int it = 0; it < 100; it++) { ld a = lo + (hi - lo) / 3, b = hi - (hi - lo) / 3;
            f(x, a) < f(x, b) ? hi = b : lo = a; } return (lo + hi) / 2; };
    ld lo = -2e9, hi = 2e9;
    for (int it = 0; it < 100; it++) {
        ld a = lo + (hi - lo) / 3, b = hi - (hi - lo) / 3;
        f(a, fy(a)) < f(b, fy(b)) ? hi = b : lo = a;
    }
    ld x = (lo + hi) / 2;
    return {x, fy(x)};
}
/* THE REST OF THE SMALL LAYER, as one-liners you can reconstruct instantly
 COLLINEAR a, b, c            a.cross(b, c) == 0
 PARALLEL / PERPENDICULAR     d1.cross(d2) == 0   /   d1.dot(d2) == 0
 SAME SIDE OF A LINE          sgn(a.cross(b, p)) == sgn(a.cross(b, q))
```

```
POINT ON A RAY / SEGMENT     onRay, onSeg (02 - Lines.cpp)
DISTANCE point-line/segment  lineDist, segDist    ray: rayDist
    seg-seg: segSegDist
PROJECT / REFLECT over a line  proj, refl (02 - Lines.cpp)
ANGLE between two vectors    atan2(cross, dot) -> signed, in (-pi
    , pi]. NEVER acos(dot/|a||b|):
                             it loses all precision near 0 and pi
    and cannot give you a sign.
ROTATE 90 DEGREES            p.perp() = (-y, x). Twice = negation
    . This is the whole of "turn
                             left"; you almost never need a
    general rot() for a right angle.
SORT BY ANGLE                angCmp (01 - Core.cpp), NOT atan2 -
    exact and 5x faster.
AREA OF A TRIANGLE           |cross| / 2. Heron only if you are
    given the three SIDES, and then
                             use the sorted stable form: sort a
    >= b >= c and
                             A = sqrt((a+(b+c))(c-(a-b))(c+(a-b))
    (a+(b-c))) / 4.
TRIANGLE CENTRES             centroid (a+b+c)/3, circumcenter,
    incenter, orthocenter (04).
                             EULER LINE: O, G, N, H are collinear
    with OG:GN:NH = 2:1:3, and
                             H = A + B + C - 2O. Nine-point
    centre = midpoint of OH, radius R/2.
TRIANGLE IDENTITIES          A = rs = abc/(4R);  R = abc/(4A);  r
    = A/s;  r_a = A/(s-a)
                             law of cosines c^2 = a^2 + b^2 - 2ab
    cos C
                             law of sines a/sin A = 2R
                             OI^2 = R(R - 2r)  =>  EULER'S
    INEQUALITY R >= 2r
                             Stewart (cevian d to BC split m + n)
    : b^2 m + c^2 n = a(d^2 + mn)
                             median m_a = sqrt(2b^2 + 2c^2 - a^2)
    /2
CYCLIC QUADRILATERAL         PTOLEMY: AC*BD = AB*CD + BC*AD (in
    cyclic order). For ANY four
                             points AC*BD <= AB*CD + BC*AD,
    equality iff concyclic in that order.
                             BRAHMAGUPTA area = sqrt((s-a)(s-b)(s
    -c)(s-d)).
                             General quadrilateral (BRETSCHNEIDER
    , angle-free):
                             A = sqrt(4p^2q^2 - (b^2+d^2-a^2-c^2)
    ^2)/4 with p, q the diagonals.
POWER OF A POINT             pow(p) = |pc|^2 - r^2; for ANY line
    through p meeting the circle at
                             A and B the signed product PA*PB
    equals it. Outside: = tangent^2.
                             RADICAL AXIS = equal power, a line
    perpendicular to the centre line.
                             RADICAL CENTRE of three circles with
    non-collinear centres: the
                             three pairwise axes are concurrent.
DESCARTES CIRCLE THEOREM     four mutually tangent circles,
    curvatures k_i = +-1/r_i (negative
                             for the enclosing one):  (k1+k2+k3+
    k4)^2 = 2(k1^2+k2^2+k3^2+k4^2),
                             so k4 = k1+k2+k3 +- 2 sqrt(k1k2 +
    k2k3 + k3k1).
LATTICE POINTS               on a segment: gcd(|dx|, |dy|) + 1
    endpoints included.
                             inside a polygon: PICK, I = A - B/2
    + 1 (simple polygons only).
                             under a line, in O(log): floorSum
    (04 - Number Theory/01/03).
HELLY (2D)                   if every THREE of a family of convex
    sets meet, all of them meet.
RADON (2D)                   any FOUR points split into two sets
    whose hulls intersect.
CARATHEODORY (2D)            a point in the hull of S is in the
    hull of at most THREE points of S
                             - which is exactly why the minimum
    enclosing circle needs only 3.
ART GALLERY                  floor(n/3) guards always suffice for
    a simple n-gon (triangulate,
                             3-colour, take the smallest class).
    Orthogonal polygon: floor(n/4).
EULER, PLANAR                V - E + F = 1 + C. n lines in
    general position cut the plane into
                             1 + n + C(n,2) regions, C(n-1,2) of
    them bounded; n circles in
                             general position give n^2 - n + 2
    regions.
DUALITY                      point (a,b) <-> line y = ax - b.
    Incidence and above/below are
                             preserved, so "max collinear points"
    becomes "max concurrent lines"
                             and a convex hull becomes an upper
    envelope. */
```

## 11.2 Geometry 3D

### 11.2.1 Geometry3D

3D point / vector, planes, and the few 3D things that actually appear.

```
template <class T>
struct P3 {
    typedef P3 Pt; T x = 0, y = 0, z = 0;
    P3(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
    Pt operator+(Pt p) const { return {x + p.x, y + p.y, z + p.z
     }; }
    Pt operator-(Pt p) const { return {x - p.x, y - p.y, z - p.z
     }; }
    Pt operator*(T d) const { return {x * d, y * d, z * d}; }
    Pt operator/(T d) const { return {x / d, y / d, z / d}; }
    T dot(Pt p) const { return x * p.x + y * p.y + z * p.z; }
    Pt cross(Pt p) const { return {y * p.z - z * p.y, z * p.x - x
      * p.z, x * p.y - y * p.x}; }
    T dist2() const { return x * x + y * y + z * z; }
    ld dist() const { return sqrtl((ld)dist2()); }
    Pt unit() const { return *this / dist(); }
    ld phi() const { return atan2l(y, x); }
      // longitude, (-pi, pi]
    ld theta() const { return atan2l(sqrtl((ld)(x * x + y * y)),
     z); }  // colatitude, [0, pi]
    Pt rot(ld a, Pt ax) const {
      // Rodrigues, ccw around ax
        Pt u = ax.unit(); ld s = sinl(a), c = cosl(a);
        return u * dot(u) * (1 - c) + (*this) * c + u.cross(*this
     ) * s;
    }
    bool operator<(Pt p) const { return tie(x, y, z) < tie(p.x, p
     .y, p.z); }
    bool operator==(Pt p) const { return tie(x, y, z) == tie(p.x,
      p.y, p.z); }
    friend istream& operator>>(istream& i, Pt& p) { return i >> p
     .x >> p.y >> p.z; }
    friend ostream& operator<<(ostream& o, Pt p) { return o << p.
     x << ' ' << p.y << ' ' << p.z; }
};
typedef P3<ll> Pi3;
typedef P3<ld> Pd3;

// PLANE  n.dot(X) = d.   Build from 3 points, or from a normal
     and a point.
template <class P> struct Plane {
    P n; ld d;
    Plane(P n = P(), ld d = 0) : n(n), d(d) {}
    Plane(P a, P b, P c) : n((b - a).cross(c - a)), d(n.dot(a))
     {}
    ld side(P p) const { return n.dot(p) - d; }
      // sign = which side
    ld dist(P p) const { return fabsl(side(p)) / n.dist(); }
    P proj(P p) const { return p - n * (side(p) / n.dist2()); }
    P refl(P p) const { return p - n * (2 * side(p) / n.dist2());
      }
};
// LINE a->b vs plane. {0 parallel-disjoint, 1 unique, -1 line
     lies in plane}
template <class P> pair<int, P> planeLine(const Plane<P>& pl, P a
     , P b) {
    ld da = pl.side(a), db = pl.side(b);
    if (fabsl(da - db) < eps) return {fabsl(da) < eps ? -1 : 0, P
     ()};
    return {1, (b * da - a * db) / (da - db)};
}
// PLANE x PLANE -> a line, given as {point on it, direction}. dir
     ==0 means parallel.
template <class P> pair<P, P> planePlane(const Plane<P>& A, const
      Plane<P>& B) {
    P dir = A.n.cross(B.n);
    if (dir.dist2() < eps) return {P(), P()};
    P p = (B.n * A.d - A.n * B.d).cross(dir) / dir.dist2();
    return {p, dir};
}

// VOLUME of the tetrahedron abcd (signed; x6 form is exact on P3<
     ll> only up to |coordinate| ~ 5.7e5 (|vol6| <= 48*C^3);
// above that make the return type __int128).
template <class P> auto vol6(P a, P b, P c, P d) { return (b - a)
     .cross(c - a).dot(d - a); }
template <class P> ld tetraVolume(P a, P b, P c, P d) { return
     fabsl((ld)vol6(a, b, c, d)) / 6; }

// SPHERE x LINE: points where line a->b meets the sphere (o, r).
template <class P> vector<P> sphereLine(P o, ld r, P a, P b) {
    P d = b - a, f = a - o;
    ld A = d.dist2(), B = 2 * f.dot(d), C = f.dist2() - r * r, D
     = B * B - 4 * A * C;
    if (D < -eps) return {};
    D = sqrtl(max((ld)0, D));
    if (D < eps) return {a + d * (-B / (2 * A))};
    return {a + d * ((-B - D) / (2 * A)), a + d * ((-B + D) / (2
     * A))};
}

// GREAT-CIRCLE distance on a sphere of radius R, inputs in
     DEGREES (haversine).
ld greatCircle(ld lat1, ld lon1, ld lat2, ld lon2, ld R) {
    auto rad = [](ld d) { return d * PI / 180; };
    ld dl = rad(lat2 - lat1) / 2, dg = rad(lon2 - lon1) / 2;
    ld h = sinl(dl) * sinl(dl) + cosl(rad(lat1)) * cosl(rad(lat2)
     ) * sinl(dg) * sinl(dg);
    return 2 * R * asinl(sqrtl(min((ld)1, h)));
}
```

# 12 Problem Solving

## 12.1 Reframing

REFRAMING: CHANGE THE PROBLEM, NOT THE EFFORT
==========

```
/* ================= REFRAMING: CHANGE THE PROBLEM, NOT THE EFFORT
     =================
 This chapter has no algorithms - it has the moves you make BEFORE
     you know which template to
 paste. Stuck for 10 minutes? Do not re-read the statement a
    fourth time. Walk the six questions,
 then walk the TRIGGERS below and try each against your problem.

 THE SIX QUESTIONS
   1. What object am I counting / optimising over? Say it in one
     sentence with no input format.
   2. What is INVARIANT under the allowed operation?
   3. What does the ANSWER look like (a prefix? one of the inputs?
     a threshold? always <= 2?)
   4. Can I CHECK a candidate faster than I can find one?
     -> binary search the answer
   5. What do the constraints forbid, and what does the leftover
     budget exactly equal?
   6. What would the brute force print for n = 1..8?

 --- REVERSE THE PROCESS ---
   Run time backwards: deletions become insertions, splits become
     merges. Every structure is
   add-friendly and delete-hostile (DSU above all), so reversing
     turns impossible into trivial.
   TRIGGER: things are removed one at a time and you must answer
     after each removal.
   EXAMPLE: CF 722C Destroying Array - process destructions in
     reverse, union with restored
     neighbours.  Also CSES 1163 Traffic Lights.
   TRAP: emit answers in reverse too, and check the off-by-one at
     both ends with n = 1.

 --- WORK BACKWARDS FROM THE GOAL ---
   Define the answer at terminal states and propagate backwards.
     Standard for games and for
   "what must have been true one step earlier".
   TRIGGER: few terminal states, huge forward branching, small
     backward branching.
   TRAP: on a CYCLIC game graph this is NOT a plain DP - you need
     retrograde BFS with out-degree
     counters (a position is losing only once ALL successors are
     known winning). See 10 - Game
     Theory. Writing win[v] = OR over children on a cyclic graph
     silently mislabels draws.

 --- COMPLEMENTARY COUNTING ---
   Count the bad ones and subtract; count P(X <= x) instead of P(X
      = x).
   TRIGGER: "at least one", "there exists", "not all equal", "some
      pair".
   EXAMPLE: E[max] = sum over x of P(max >= x), with P(max <= x) =
     (x/n)^k.
   TRAP: the complement is easy for AT MOST / AT LEAST, not for
     EXACTLY. Getting from one to the
     other is binomial inversion: exactly_k = sum_{j>=k} (-1)^(j-k
     ) C(j,k) atleast_j.

 --- THE CONTRIBUTION TECHNIQUE ---
   Instead of "for each subarray, compute f", ask "for each
     element, in how many objects does it
   contribute, and how much". Turns O(n^2) enumeration into O(n).
   TRIGGER: "sum over all subarrays / pairs / subsets of [min |
     max | gcd | #distinct | bottleneck]".
   EXAMPLE: sum over subarrays of the minimum -> monotonic stack
     gives each element's span.
     Sum over pairs of the bottleneck -> Kruskal, each merge
     contributes w * |A| * |B| pairs.
   TRAP: ties. With equal values the span must be strict on ONE
     side and non-strict on the other,
     or every duplicated minimum is counted twice. Decide which
     side BEFORE coding.

 --- SWAP THE ORDER OF SUMMATION ---
   sum_i sum_{d | i} f(d) = sum_d f(d) * floor(n/d). Move the
     outer loop to the variable whose
   inner count has a closed form.
   TRIGGER: a double sum whose inner range depends on the outer
     index; divisors, multiples,
     ancestors, intervals containing a point.
   EXAMPLE: sum of sigma(i) for i <= n = sum_d d * floor(n/d),
     then divisor blocks -> O(sqrt n).

 --- LINEARITY OF EXPECTATION ---
   E[sum X_i] = sum E[X_i], with NO independence needed. Decompose
      into indicators.
   TRIGGER: "expected number of ...", or an expectation over a
     process you cannot untangle.
   TRAP: linearity does not survive max, min, or division. E[max]
     != max E, E[1/X] != 1/E[X].
     For an expected maximum use the complementary-counting tail
     sum above.

 --- FIX THE EXTREMUM ---
   Add an artificial outer loop "for each choice of the maximum /
     the leftmost / the last move".
   Everything else becomes bounded relative to the fixed thing.
   TRIGGER: the statement mentions max or min over a range; a
     Cartesian-tree recursion is available.
   EXAMPLE: CF 1156E - fix the position of the maximum, then
     enumerate the SMALLER side.
   TRAP: always iterate the smaller side (small-to-large, O(n log
     n) total). Iterating the left
     side unconditionally is O(n^2) on a sorted permutation - and
     it passes the samples.

 --- DUALIZE ---
   max-flow = min-cut; max bipartite matching = min vertex cover (
     Konig); min path cover of a DAG
   = n - matching; longest antichain = min chain cover (Dilworth);
      "a perfect matching exists" =
   "no Hall violator".
   TRIGGER: the statement asks for a MINIMUM removal / blocking
     set / number of chains - that is
     almost always the dual of a maximum you can compute.
   TRAP: Konig gives the SIZE for free, but recovering the cover
     needs the alternating-BFS
     construction (L \ Z) union (R and Z). And Konig is BIPARTITE
     ONLY.

 --- MODEL IT AS A GRAPH ---
   Objects -> nodes, relations -> edges. States can be tuples: (
     cell, fuel), (vertex, parity),
   (position, mask).
   TRIGGER: "can I get from A to B", "minimum number of operations
     ", "are these constraints
     consistent", "each x points at exactly one y" (functional
     graph).
   TRAP: compute |V| * |E| explicitly before coding. In a layered
     graph the answer is usually
     "any layer", not layer 0.

 --- MODEL IT AS INTERVALS / A TIMELINE ---
   Recast objects as [start, end] and use a sweep, greedy-by-
     endpoint, or a difference array.
   Tree subtrees become intervals via the Euler tour.
   TRIGGER: "from time a to time b", "covers l..r", "is available
     during", subtree aggregates.
   TRAP: decide ONCE whether [1,3] and [3,5] overlap, and encode
     it in exactly one place.

 --- MODEL IT AS A POLYNOMIAL ---
   "For every possible sum, count the ways" is a convolution.
     Multiplying GFs = combining
   independent choices; x^k = "used k of the budget".
   TRIGGER: modulus 998244353 (NTT is intended); independent parts
      whose sizes add.
   TRAP: convolving an array with itself double-counts i != j and
     mis-counts i = j.

 --- MODEL IT AS LINEAR ALGEBRA OVER GF(2) ---
   Each element is a bit-vector; "can I make X" = "is X in the
     span"; the number of subsets giving
   a representable Y is 2^(n - rank).
   TRIGGER: XOR anywhere; "product is a perfect square" (exponent
     parity vector over primes);
     switch-toggling puzzles; "each constraint satisfied an even
     number of times".
   TRAP: 2^(n-rank) counts solutions only if the system is
     CONSISTENT - check first. And subtract
     the empty subset when the problem wants a non-empty one.

 --- MODEL IT AS THE SUBSET LATTICE ---
   Bitmask DP plus zeta/Mobius (SOS) to aggregate over submasks in
      O(2^n n) instead of O(3^n).
   TRIGGER: n <= 20 with a "set of chosen items" state, or a <=
     22-bit value domain with queries
     "over all y contained in x".
   TRAP: the bit loop goes OUTSIDE the mask loop. Swapping them
     gives a wrong-but-plausible array.
     Submask enumeration must be  for (s = m; ; s = (s-1) & m) {
     ...; if (!s) break; }.

 --- THINK ABOUT WHAT THE ANSWER LOOKS LIKE ---
```

```
  Characterise the optimum's SHAPE first: it is a prefix; it is
    one of the inputs; it is at most
  2; it has a threshold structure. Then search only that shape.
  TRIGGER: the output is one small number regardless of n; the
    problem screams "constructive".
  TRAP: "the answer is at most k" needs a matching lower-bound
    example, or you print k-1.

--- CHANGE OF VARIABLES ---
  Difference array (range add -> two point updates; "make all
    equal" -> zero the differences),
  prefix sums (subarray -> pair of points), Chebyshev <->
    Manhattan via (x+y, x-y), logs
  (products -> sums).
  TRIGGER: operations act on ranges; the objective is max(|dx|,|
    dy|) or |dx|+|dy|.
  TRAP: C++ % is negative for negative operands - use ((x % n) +
    n) % n. Seed the empty prefix.

--- PER-BIT / PER-PRIME INDEPENDENCE ---
  If the objective decomposes across bits (XOR/AND/OR sums,
    popcount weights) or across primes
  (gcd/lcm exponent-wise), solve 30 tiny problems and recombine.
  EXAMPLE: sum over pairs of (a_i XOR a_j) = sum_b 2^b * cnt0_b *
     cnt1_b.
  TRAP: bit-independence holds for SUMS, not for COMPARISONS. "
    Count pairs with a_i^a_j < k" is
    not separable - that needs a binary trie walked high bit
    first.

--- REDUCE TO SOMETHING YOU ALREADY HAVE ---
  "minimum adjacent swaps to sort" = inversions; "maximum
    pairwise-compatible set" = matching;
  "minimise the maximum edge on a path" = MST bottleneck /
    Kruskal reconstruction tree; "longest
  chain of pairs" = LIS; "pairwise gap constraints" = difference
    constraints + Bellman-Ford;
  "each person points at one other" = functional graph.
  TRIGGER: you can state the problem without the story. Do that,
    then match it.
  TRAP: the disguise is usually ALMOST the classic - one extra
    constraint breaks the reduction.
    Check the reduction on the samples by hand before trusting it
    .
================================================================================
     */
```

## 12.2 SearchAndProof

SEARCH: TURN "FIND" INTO "CHECK" ==========

```
/* ============ SEARCH: TURN "FIND" INTO "CHECK" ============

 BINARY SEARCH ON THE ANSWER. If feasible(x) is monotone, stop
    optimising and find the boundary.
   TRIGGER: "minimise the maximum", "maximise the minimum", "
     minimum time such that", "possible
     within k". Also: the answer is an integer in a huge range and
      no formula presents itself.
   TRAPS: (1) the check itself overflows - break early once a
     running count passes the target;
     (2) `hi` must be PROVABLY feasible - seed it with the trivial
      answer, not 1e9;
     (3) if the predicate is not monotone you get a plausible
     wrong answer and no crash.

 BINARY SEARCH THE VALUE, COUNT WITH A SECOND SEARCH. For "k-th
    smallest X" over a set you cannot
   materialise (pairs, products, distances, subarray sums): binary
      search x, count how many are
   <= x. The counting routine is the real problem.
   TRAP: use firstTrue(count(x) >= k) so the returned x actually
     occurs.

 PARAMETRIC SEARCH / LAGRANGIAN (Aliens, wqs). Replace "exactly k"
     by a penalty per item and
   binary search the penalty. Requires the optimum to be CONVEX in
     k. See 08 - DP/01/07.
   TRAP: verify convexity by brute-forcing f(k) for small n; and
     the inner DP must break ties
     toward the largest count or the final subtraction is off by
     an arbitrary amount.

 TERNARY SEARCH. Unimodal f: cut a third each step. Reals: fixed
    ~200 iterations (never an eps
   loop). Integers: while (hi - lo > 2), then scan the survivors.
   TRAP: PLATEAUS KILL IT. f(m1) == f(m2) on a flat region can
     discard the true optimum. For any
     discretely convex f use binary search on the difference
     instead:
       argmin = firstTrue(lo, hi-1, [&](ll x){ return f(x+1) >= f(
     x); });

 TWO POINTERS - and why it is legal. Legal precisely when validity
     is MONOTONE IN l for fixed r:
   if [l, r] is valid then so is [l', r] for every l' >= l.
   TRAP: "subarray with sum <= S" is two-pointerable with positive
     values and WRONG with negative
     ones. Every time you reach for two pointers, say out loud "
     shrinking can only help". If it
     cannot, stop.

 SWEEP LINE. Sort events by one coordinate, maintain a structure
    over the other.
   TRAP: event ordering at TIES is the whole problem - for closed
     intervals process opens first;
     for touching rectangles process closes first. Getting it
     wrong is off-by-exactly-the-ties.

 COORDINATE COMPRESSION. Only the order matters; map <= 2n values
    to 0..m-1.
   TRAP: if the problem cares about GAPS (lengths, areas, "how
     many integers strictly between"),
     keep the original values to weight each compressed cell.
     Insert both l and r+1.

 DIVIDE AND CONQUER ON TIME. Put each object's lifetime on a
    segment tree over the timeline, DFS
   it with a ROLLBACK structure, answer at the leaves. This is how
     you delete from a DSU.
   TRAP: rollback DSU must use union by size only - path
     compression makes undo impossible, so
     find is O(log n), not alpha. Budget accordingly.

 MEET IN THE MIDDLE. Split in half, enumerate 2^(n/2) per side,
    recombine by sorting + binary
   search / two pointers / hash map.
   TRIGGER: n <= 40..44 with a subset-flavoured question.
   TRAP: memory (2^20 ll = 8 MB per array); and in "mod m"
     variants you must also consider the
     wrap-around pair. Put the extra element of an odd n in the
     CHEAPER half.

 SQRT DECOMPOSITION OF THE PROBLEM (not the data structure). Two
    complementary bad algorithms,
   O(x) and O(n/x): threshold at sqrt and use each where it wins.
     Heavy vs light vertices, small
```

vs large divisors, big vs small queries, "sum a_i <= 1e5 so at most sqrt distinct values".
TRAP: tune the threshold to the ACTUAL costs; if updates are 10x cheaper the optimum is far from sqrt(n). Measure.

OFFLINE REORDERING. If nothing forces you online, sort the queries: by right endpoint (+ BIT), by threshold (+ DSU), by Mo's ordering, by time. The highest-yield contest move for "q queries on a static array".
TRIGGER: no "xor with the previous answer" marker; all queries readable up front.
TRAP: store answers by the ORIGINAL query id and print in input order.

SMALL TO LARGE. Always insert the smaller container into the larger: each element moves O(log n) times. SWAP THE HANDLES, never copy - copying the big one in is O(n^2) and looks identical.

AMORTIZATION / POTENTIAL. An occasionally-expensive operation is cheap on average if each expensive step DESTROYS something expensive to create. Justifies monotonic stacks, ODT, beats.
TRAP: ODT's amortization needs assign-heavy input; it is O(n) per op in the worst case. Monotonic-stack amortization dies if you ever re-push an index.

ADD ONE ELEMENT AT A TIME. Solve for prefixes and maintain the answer incrementally. Turns "for every prefix, output X" from O(n^2) into O(n log n) and usually reveals the structure.
TRAP: LIS - lower_bound gives strictly increasing, upper_bound gives non-decreasing. The tails array is NOT the subsequence; reconstruct with parent pointers.

PARALLEL BINARY SEARCH. q independent binary searches over one shared timeline: bucket queries by their current mid, sweep the timeline once, resolve one bit for every query.
TRAP: reset or rebuild the shared structure between LEVELS. Forgetting produces monotone-looking garbage.

============ PROOF TOOLS: THE TEN MINUTES THAT SAVE THE SUBMIT ============

EXCHANGE ARGUMENT (for greedy). Take any optimum; show that swapping two ADJACENT elements toward your greedy order never makes it worse. For a sorting greedy this DERIVES the comparator: compare cost(a then b) against cost(b then a).
TRAP: the derived comparator must be a STRICT WEAK ORDERING. Ties returning true, or an overflowing cross-multiplication, makes std::sort run off the array - an RE that looks like a logic bug. Test with all-equal input, and cross-multiply in __int128.

INVARIANTS. A quantity unchanged by every legal operation. If start and target differ on it the answer is "impossible" - and you have a proof instead of a guess.
TRAP: an invariant proves IMPOSSIBILITY only. You still owe a construction for the other side.

MONOVARIANTS. A quantity that strictly decreases and is bounded: proves termination and bounds the number of operations. (Euclid: a+b strictly decreases.)
TRAP: on reals a monovariant proves nothing - you need integrality or a floor.

PARITY AND COLOURING. Colour the board so every legal move changes the colour counts in a fixed way, then count. Try in order: checkerboard, columns mod k, (i+j) mod 3, a weight function.
EXAMPLE: the mutilated chessboard has 32/30 of the two colours, so no domino tiling exists.
TRAP: if no colouring produces an imbalance in five minutes, the obstruction is not parity.

THE EXTREMAL PRINCIPLE. Argue about the largest / smallest / deepest object - extremes have neighbours on one side only, which collapses the cases.
EXAMPLE: two-BFS tree diameter is correct because the farthest vertex from ANY start is an endpoint of SOME diameter.
TRAP: that theorem is TREE-ONLY. It is false on general graphs and on negative weights.

PIGEONHOLE. n+1 objects in n boxes. In practice: n+1 prefix sums but only n residues, so some subarray sum is divisible by n.
TRAP: the constructive version needs the FIRST repeat - store the earliest index per residue.

SYMMETRY AND WLOG. If the problem is invariant under swapping two roles, assume an order and divide by the symmetry factor at the end.
TRAP: handle the diagonal i == j separately: (total - diagonal)/2 + diagonal. Under a modulus, "divide by 2" means multiply by inv2.

STRATEGY STEALING. In a symmetric game where an extra move never hurts, the first player wins or draws: if the second player had a winning strategy the first could steal it.
TRAP: pure EXISTENCE. It gives you no winning move, and it fails under misere play.

ADVERSARY ARGUMENTS / QUERY LOWER BOUNDS. q binary queries distinguish at most 2^q inputs, so q >= log2(#inputs); comparison sorting needs log2(n!) ~ n log n.
TRAP: assume the interactor is ADAPTIVE unless told otherwise - it fixes nothing in advance, only stays consistent. Strategies relying on "the answer was fixed before I started" lose.

============ CONSTRAINTS ARE A MESSAGE FROM THE SETTER ============

| | |
|---|---|
| n <= 10 | permutations n!, or backtracking with pruning |
| n <= 20 | 2^n bitmask DP, SOS, 3^n submask partition |
| n <= 24..28 | 2^n with a small factor |
| n <= 40..44 | MEET IN THE MIDDLE, 2^(n/2) |
| n <= 100 | O(n^3)/O(n^4): Floyd, Gauss, interval DP, min-cost flow, Hungarian |
| n <= 500 | O(n^3) or O(n^2 log n); flows on n^2 edges |
| n <= 5000 | O(n^2): LCS, tree knapsack, or O(n^2/64) with a bitset |
| n <= 1e5, log^2 slack | segtree of segtrees, merge-sort tree, parallel binary search, HLD |
| n <= 2e5 | O(n log n). This is the default and it means "find the n log n" |
| n <= 1e6 | O(n) or O(n log n) with a SMALL constant: no map, no set, fast IO mandatory |
| n,q <= 1e5, offline | Mo's (n sqrt q) is in budget, ~3e7 pointer moves |
| value <= 1e9, n small | coordinate compression, or binary search the value |
| a_i <= 1e18 | Miller-Rabin + Pollard rho, __int128, no sieve |
| sum n <= 2e5 over tests | per-test O(n log n); NEVER memset a MAXN global per test |
| t <= 1e4, no sum bound | the intended solution is O(1) or O(log) per test |
| mod 998244353 | NTT / polynomial methods intended |
| mod 1e9+7 | counting DP / combinatorics, NOT NTT |
| real output, 1e-6 | geometry, ternary/binary search on reals, or expectation |
| <= 20-30 | oracle queries binary search (2^20 > 1e6) or bit-by-bit determination |
| TL 3-5 s at n <= 1e5 | the intended solution really is n sqrt n or n log^2 n |
| ML 32-64 MB | offline/streaming, rolling DP arrays, no persistence |

THE /64 ARGUMENT. An "impossible" O(n^2) or O(n^3) over BOOLEANS becomes n^2/64 - fine up to ~1e10 raw ops. Reachability, subset sum, LCS, string matching, set intersection.
TRAP: memory. n bitsets of n bits is n^2/8 bytes: at n = 5e4 that is 312 MB. Process in blocks of ~1000 and reuse. bitset needs a compile-time size; a vector<u64> hand-roll is ~30% slower and always available.

THE "ANSWER IS SMALL" ARGUMENT. Prove the answer is <= a tiny constant, then brute force over it. Each operation halves something / clears a bit / merges two components; "OR of a range equals its SUM" means the values are bit-disjoint so at most 30 are nonzero; sum a_i <= 1e5 means at most ~632 distinct values; a_i <= 1e6 means at most 7 distinct primes, 240 divisors;

```
   gcd over an expanding range takes only O(log V) distinct values
     .
   TRAP: prove the bound, do not observe it on samples, and handle
      "impossible" explicitly.
================================================================================
      */
```

## 12.3 ContestCraft

CONSTRUCTIVE, INTERACTIVE, RANDOMISED, EMPIRICAL ============

```
/* ============ CONSTRUCTIVE, INTERACTIVE, RANDOMISED, EMPIRICAL
     ============

BUILD GREEDILY AND PROVE AS YOU GO. Emit the answer one decision
    at a time, always taking the
  best local choice that KEEPS THE REMAINDER FEASIBLE - that
    check is the whole problem.
  TRAP: for LEXICOGRAPHICALLY SMALLEST, greedy alone is wrong.
    You need "pick the smallest
    candidate such that the rest is still completable", and the
    completability test must be
    exact. If you cannot write that test, you cannot do
    lexicographic greedy.

BUILD RECURSIVELY. Solve n from n/2 (or n-1, or four quadrants)
    and glue with a fixed pattern.
  TRIGGER: n is a power of two, or the object is self-similar (
    Gray code, Hanoi, de Bruijn).
  TRAP: base cases are usually n = 1 AND n = 2. Recursive output
    of 2^n lines needs buffered
    output - endl will TLE.

EXTEND FROM A KNOWN SMALL CASE. Brute force n = 1..8, find the
    pattern of CONSTRUCTION (not just
  the count), then prove the inductive step.
  TRAP: the small cases are exactly where the pattern fails. Find
     the smallest n where it first
    works and special-case everything below. Nearly every
    constructive WA is a small-n exception.

CERTIFICATE CONSTRUCTION (prove the NO). Output the witness: the
    negative cycle, the odd cycle,
  the Hall violator, the parity mismatch. Producing it also
    validates your YES logic.
  TRAP: after n Bellman-Ford rounds you have a vertex AFFECTED BY
     a negative cycle, not one ON
    it. Walk n parent pointers back first, then follow parents
    until a vertex repeats.

INTERACTIVE: BUDGET FIRST, ALGORITHM SECOND. Compare 2^Q against
    the number of possibilities.
  Q ~ log n -> binary search;  Q ~ n -> one query per element;  Q
     ~ n log n -> sorting/merging.
  TRAPS: (1) FLUSH after every query (endl, or fflush) or you get
     "idleness limit exceeded",
    which looks nothing like the real bug; (2) count your queries
     locally and assert the limit;
    (3) assume the interactor is ADAPTIVE - it only has to stay
    consistent with past answers.

RANDOM SHUFFLE to destroy adversarial order: Kuhn's matching,
    quickselect, incremental convex
  hull, minimum enclosing circle (which is O(n) only after a
    shuffle), KD-tree splits.
  TRAP: seed from the clock (mt19937_64 with steady_clock), never
     srand(time(0)) + rand().
    unordered_map needs the splitmix64 custom hash or it is O(n
    ^2) on a crafted test.

RANDOM SAMPLING. If a target occupies more than a 1/c fraction, k
     samples miss it with
  probability (1 - 1/c)^k.
  TRIGGER: "majority element of a range", "a value shared by at
    least half".
  TRAP: sampling gives a CANDIDATE - you must verify it exactly.
    And the failure probability
    compounds over q queries: with q = 1e5 you need per-query
    failure below 1e-12, so ~40
    samples, not 20.

RANDOM WEIGHTS FOR UNIQUENESS (Zobrist / XOR hashing). Give each
    value a random 64-bit label;
  then multiset equality, "every value appears a multiple of k
    times", and "this subtree shape
  occurred before" become single-number comparisons.
  TRAP: one random assignment has a real false-positive rate -
    repeat ~30 times with FRESH
    randoms. A fixed small set of randoms is breakable by
    Gaussian elimination.

HASHING AS O(1) EQUALITY. Polynomial hashing for substrings, 2D
    for submatrices.
  TRAP: RANDOM base, and either double-mod or one 61-bit Mersenne
     mod. Fixed base 31 with mod
    1e9+7 is anti-hashed by a Thue-Morse construction. Birthday
    bound: comparing m hashes
```

```
    pairwise collides with probability ~m^2/(2*MOD), so m = 1e6
    against a single 1e9+7 mod is a
    coin flip.

THE REPETITION MATH. k independent trials fail with p^k. Budget
    PER QUERY, not per test, then
  multiply by 100 for safety. For primality use the DETERMINISTIC
     witness set
  {2,3,5,7,11,13,17,19,23,29,31,37} (valid below 3.3e24) - zero
    failure probability, same cost.

STRESS TESTING AS A THINKING TOOL. gen + brute + a diff loop;
    then SHRINK the counterexample
  until it fits on one line and stare at it.
  TRAPS: (1) the generator must produce DEGENERATE cases - all
    equal, n = 1, all zero, maximum
    values, duplicate coordinates. A uniform generator over [1, 1
    e9] never produces a tie and
    will never find your tie bug. (2) If the problem says "print
    any", diffing is useless -
    write a CHECKER. (3) Stress the brute force against the
    samples first.

BRUTE FORCE SMALL CASES, THEN LOOK IT UP. Compute n = 1..12
    exactly, then search OEIS, or feed
  the sequence to Berlekamp-Massey, or fit a polynomial by
    Lagrange interpolation.
  TRAP: an OEIS match on 4 terms is noise - get 8, then confirm
    the 9th. BM needs terms computed
    under the SAME prime you will use, and at least 2x the order.

PRINT THE DP TABLE AND LOOK FOR STRUCTURE. Dump the n x n table (
    and the argmin table) for
  n = 20 and look for monotone rows, monotone opt[i][j], convex
    differences, sparsity.
  That is how you decide between Knuth, divide & conquer, and
    Aliens.
  TRAP: monotonicity that holds at n = 20 can fail at n = 200.
    Also verify the quadrangle
    inequality numerically on random quadruples.

INSTRUMENT, DO NOT GUESS (for TLE). Print a counter of inner-loop
     iterations on your largest
  local test and compare against 1e8-1e9. That distinguishes "
    wrong complexity" from "wrong
  constant", which need completely different fixes. Compile
    locally with -O2.

============ CONTEST CRAFT ============

READING THE SET
  Minute 0-15: all three read DIFFERENT problems, front to back.
    Nobody codes. On the board, per
    problem: a one-line restatement, n, and a guess at the family
    .
  Order by (confidence x 1/effort), not by letter. Setters mis-
    order ~30% of the time.
  Use the scoreboard from minute 30 - it is the best difficulty
    signal you have.
  ONE coder at the keyboard, always. A problem is not "started"
    until it has a written plan with
    its complexity and its main data structure.

WHEN TO ABANDON - when any TWO hold:
  * 25+ minutes with no NEW idea (the idea count stopped growing,
     not the code).
  * The complexity is 10x over budget with no plan to close it.
  * Two teammates failed to find the bug and 5 minutes of stress
    testing found nothing
    => you are testing the wrong thing or misread the statement.
    Re-read it OUT LOUD.
  * Another unattempted problem has more solves.
  Write the state on paper before switching, and set a hard
    return time.

DEBUGGING UNDER TIME PRESSURE - in this order:
  1. RE-READ THE STATEMENT: output format, the modulus, 1-based
    vs 0-based, multi-test.
     About a quarter of "impossible" bugs are misreads.
  2. Re-read your input parsing (n and m swapped; looping to n
    over an array sized m).
  3. Run ALL the samples, including the one you skipped.
  4. Run n = 1, n = 2, and the all-equal case.
  5. Stress test. This beats reading code whenever you have a
    brute force.
  6. Only then read the code - out loud, to a teammate, for
    INTENT. Silent re-reading just
     re-hallucinates the same thing.
  7. Nothing after 15 minutes: REWRITE the suspicious function
     from scratch.

PRE-SUBMIT CHECKLIST (40 seconds, every submit)
  [ ] Overflow: every product of two inputs; inside comparators;
    inside prefix sums.
  [ ] Modulus: right constant, +MOD after subtraction, inverse
    instead of division, non-negative.
  [ ] n = 1, m = 1, k = 0, single-element array.
  [ ] Degenerate input: empty string, zero edges, one-node tree,
    all equal, all zero, negatives.
  [ ] STATE RESET between test cases: every global, vector,
    counter, visited array, the answer
      variable, and any function-local static. Clear only what
    you touched if sum n is bounded.
  [ ] Right bound read: MAXN vs the actual limit; the sum over
    queries can exceed the per-test n.
  [ ] Uninitialised: dp arrays that need -INF not 0; DSU parent[i
    ] = i.
  [ ] Recursion depth: DFS on a path with n = 2e5 is 2e5 frames -
     move big locals out.
  [ ] Output format: newline vs space, YES vs Yes, -1 vs
    IMPOSSIBLE, setprecision.
  [ ] endl removed from any loop running more than ~1e4 times;
    debug output removed.
  [ ] The `cin >> tc` line matches whether the problem is multi-
    test.

VERDICT TRIAGE
  WA  -> which test? Test 1 means a misread or a format error; a
    late test means an edge case or
         overflow. Then: modulus, overflow, ties/duplicates (
    strict vs non-strict), stress test,
         unreset state, floating-point comparison.
  TLE -> is the complexity really what you think (count the loops
     on paper)? endl in a loop /
         missing sync_with_stdio. map/set in a hot loop.
    Reallocation and pass-by-value. memset
         of a huge global per test. Infinite loop: a binary
    search with l = mid.
         Only then change algorithm. A /64 win never fixes a
    wrong complexity class.
  RE  -> array bounds first: rebuild with -fsanitize=address,
    undefined -D_GLIBCXX_DEBUG and run
         the samples; it finds it in seconds. Then stack overflow
    , division by zero, a comparator
         that is not a strict weak ordering (crashes INSIDE std::
    sort), .back()/.top() on an
         empty container, 0- vs 1-indexed access.
  MLE -> n x n arrays (5000^2 ints = 100 MB); vector<vector<int>>
     for a graph (5-8x the raw
         data - use CSR); persistent structures (count the nodes
    before allocating); recursion
         frames; go offline and stream instead of storing
    versions.
  Idleness on an interactive -> a missing flush, or debug output
    on stdout.

TEAM PROTOCOL
  Never submit without the checklist: a rejection costs 20
    penalty minutes plus morale.
  One person owns the printed notebook and fetches pages; the
    coder does not browse.
  Paste, adapt, then read once - the #1 template bug is an index-
    convention mismatch.
  Two solutions disagree? Trust the brute force. Always.
  Last 30 minutes: stop starting new problems; finish the nearest
     one and re-check the submits.

THE TEN MOST EXPENSIVE RECURRING BUGS (by contest-time cost)
  1. Overflow in an intermediate product, invisible because the
    final answer is small.
  2. State not reset between test cases - passes locally, fails
    on test 2.
  3. Wrong strictness (< vs <=) in a comparator, a binary-search
    predicate, or a tie-break.
  4. Two pointers on a non-monotone predicate: plausible output,
    silently wrong.
  5. Binary search bounds: hi not provably feasible, or l = mid
    causing an infinite loop.
  6. 0- vs 1-based off-by-one when pasting a template.
  7. endl in an output loop, or a missing sync_with_stdio, at n =
     1e6.
  8. Negative modulo in C++ (-3 % 5 == -3).
  9. A greedy that was never proved and passes the samples.
 10. Recursion depth on a path-shaped tree with n = 2e5.
======================================================================
     */
```

## Index of every routine (alphabetical)